

TYCampport3

3

Generated by Doxygen 1.8.17

1 Main Page	1
1.1 Note	1
2 Class Index	3
2.1 Class List	3
3 File Index	7
3.1 File List	7
4 Class Documentation	51
4.1 AlgVersion Struct Reference	51
4.1.1 Detailed Description	51
4.2 DepthEnhanceParameters Struct Reference	51
4.2.1 Detailed Description	52
4.3 DepthInpainterParameters Struct Reference	52
4.3.1 Detailed Description	52
4.4 DepthSpeckleFilterParameters Struct Reference	52
4.4.1 Detailed Description	52
4.5 f16 Struct Reference	52
4.5.1 Detailed Description	53
4.6 gz_header_s Struct Reference	53
4.6.1 Detailed Description	53
4.7 gzFile_s Struct Reference	53
4.7.1 Detailed Description	53
4.8 nsimd_aarch64_vf16 Struct Reference	54
4.8.1 Detailed Description	54
4.9 nsimd_aarch64_vlf16 Struct Reference	54
4.9.1 Detailed Description	54
4.10 nsimd_avx2_vf16 Struct Reference	54
4.10.1 Detailed Description	54
4.11 nsimd_avx2_vlf16 Struct Reference	55
4.11.1 Detailed Description	55
4.12 nsimd_avx512_knl_vf16 Struct Reference	55
4.12.1 Detailed Description	55
4.13 nsimd_avx512_knl_vlf16 Struct Reference	55
4.13.1 Detailed Description	55
4.14 nsimd_avx512_skylake_vf16 Struct Reference	56
4.14.1 Detailed Description	56
4.15 nsimd_avx512_skylake_vlf16 Struct Reference	56
4.15.1 Detailed Description	56
4.16 nsimd_avx_vf16 Struct Reference	56
4.16.1 Detailed Description	56
4.17 nsimd_avx_vlf16 Struct Reference	57

4.17.1 Detailed Description	57
4.18 nsimd_cpu_vf16 Struct Reference	57
4.18.1 Detailed Description	57
4.19 nsimd_cpu_vf32 Struct Reference	57
4.19.1 Detailed Description	58
4.20 nsimd_cpu_vf64 Struct Reference	58
4.20.1 Detailed Description	58
4.21 nsimd_cpu_vi16 Struct Reference	58
4.21.1 Detailed Description	58
4.22 nsimd_cpu_vi32 Struct Reference	59
4.22.1 Detailed Description	59
4.23 nsimd_cpu_vi64 Struct Reference	59
4.23.1 Detailed Description	59
4.24 nsimd_cpu_vl8 Struct Reference	59
4.24.1 Detailed Description	60
4.25 nsimd_cpu_vlf16 Struct Reference	60
4.25.1 Detailed Description	60
4.26 nsimd_cpu_vlf32 Struct Reference	60
4.26.1 Detailed Description	60
4.27 nsimd_cpu_vlf64 Struct Reference	61
4.27.1 Detailed Description	61
4.28 nsimd_cpu_vli16 Struct Reference	61
4.28.1 Detailed Description	61
4.29 nsimd_cpu_vli32 Struct Reference	61
4.29.1 Detailed Description	62
4.30 nsimd_cpu_vli64 Struct Reference	62
4.30.1 Detailed Description	62
4.31 nsimd_cpu_vli8 Struct Reference	62
4.31.1 Detailed Description	63
4.32 nsimd_cpu_vlu16 Struct Reference	63
4.32.1 Detailed Description	63
4.33 nsimd_cpu_vlu32 Struct Reference	63
4.33.1 Detailed Description	63
4.34 nsimd_cpu_vlu64 Struct Reference	64
4.34.1 Detailed Description	64
4.35 nsimd_cpu_vlu8 Struct Reference	64
4.35.1 Detailed Description	64
4.36 nsimd_cpu_vu16 Struct Reference	65
4.36.1 Detailed Description	65
4.37 nsimd_cpu_vu32 Struct Reference	65
4.37.1 Detailed Description	65
4.38 nsimd_cpu_vu64 Struct Reference	65

4.38.1 Detailed Description	66
4.39 nsimd_cpu_vu8 Struct Reference	66
4.39.1 Detailed Description	66
4.40 nsimd_neon128_vf16 Struct Reference	66
4.40.1 Detailed Description	67
4.41 nsimd_neon128_vf64 Struct Reference	67
4.41.1 Detailed Description	67
4.42 nsimd_neon128_vlf16 Struct Reference	67
4.42.1 Detailed Description	67
4.43 nsimd_neon128_vlf64 Struct Reference	67
4.43.1 Detailed Description	68
4.44 nsimd_sse2_vf16 Struct Reference	68
4.44.1 Detailed Description	68
4.45 nsimd_sse2_vlf16 Struct Reference	68
4.45.1 Detailed Description	68
4.46 nsimd_sse42_vf16 Struct Reference	68
4.46.1 Detailed Description	69
4.47 nsimd_sse42_vlf16 Struct Reference	69
4.47.1 Detailed Description	69
4.48 nsimd_vmx_vf16 Struct Reference	69
4.48.1 Detailed Description	69
4.49 nsimd_vmx_vf64 Struct Reference	69
4.49.1 Detailed Description	70
4.50 nsimd_vmx_vf64 Struct Reference	70
4.50.1 Detailed Description	70
4.51 nsimd_vmx_vlf16 Struct Reference	70
4.51.1 Detailed Description	70
4.52 nsimd_vmx_vlf64 Struct Reference	70
4.52.1 Detailed Description	71
4.53 nsimd_vmx_vli64 Struct Reference	71
4.53.1 Detailed Description	71
4.54 nsimd_vmx_vlu64 Struct Reference	71
4.54.1 Detailed Description	71
4.55 nsimd_vmx_vu64 Struct Reference	71
4.55.1 Detailed Description	72
4.56 nsimd_vsx_vf16 Struct Reference	72
4.56.1 Detailed Description	72
4.57 nsimd_vsx_vlf16 Struct Reference	72
4.57.1 Detailed Description	72
4.58 OperatorInfo Struct Reference	73
4.58.1 Detailed Description	73
4.59 ParamDetail Struct Reference	73

4.59.1 Detailed Description	74
4.60 pattern_bin_param Struct Reference	74
4.60.1 Detailed Description	74
4.61 pattern_gray_param Struct Reference	74
4.61.1 Detailed Description	74
4.62 pattern_sine_param Struct Reference	74
4.62.1 Detailed Description	75
4.63 TY_PHC_GROUP_ATTR::phc_group_attr Struct Reference	75
4.63.1 Detailed Description	75
4.64 TY_ACC_BIAS Struct Reference	75
4.64.1 Detailed Description	75
4.65 TY_ACC_MISALIGNMENT Struct Reference	76
4.65.1 Detailed Description	76
4.66 TY_ACC_SCALE Struct Reference	76
4.66.1 Detailed Description	76
4.67 TY_AEC_ROI_PARAM Struct Reference	77
4.67.1 Detailed Description	77
4.68 TY_BYTEARRAY_ATTR Struct Reference	77
4.68.1 Detailed Description	78
4.68.2 Member Data Documentation	78
4.68.2.1 unit_size	78
4.68.2.2 valid_size	78
4.69 TY_CAMERA_CALIB_INFO Struct Reference	78
4.69.1 Detailed Description	79
4.70 TY_CAMERA_DISTORTION Struct Reference	79
4.70.1 Detailed Description	79
4.71 TY_CAMERA_EXTRINSIC Struct Reference	79
4.71.1 Detailed Description	79
4.72 TY_CAMERA_INTRINSIC Struct Reference	80
4.72.1 Detailed Description	80
4.73 TY_CAMERA_ROTATION Struct Reference	81
4.73.1 Detailed Description	81
4.74 TY_CAMERA_STATISTICS Struct Reference	81
4.74.1 Detailed Description	81
4.75 TY_CAMERA_TO_IMU Struct Reference	82
4.75.1 Detailed Description	82
4.76 TY_DEVICE_BASE_INFO Struct Reference	82
4.76.1 Detailed Description	83
4.77 TY_DEVICE_NET_INFO Struct Reference	83
4.77.1 Detailed Description	84
4.78 TY_DEVICE_USB_INFO Struct Reference	84
4.78.1 Detailed Description	84

4.79 TY_DI_WORKMODE Struct Reference	84
4.79.1 Detailed Description	84
4.80 TY_DO_WORKMODE Struct Reference	85
4.80.1 Detailed Description	85
4.81 TY_ENUM_ENTRY Struct Reference	85
4.81.1 Detailed Description	85
4.82 TY_EVENT_INFO Struct Reference	86
4.82.1 Detailed Description	86
4.83 TY_FEATURE_INFO Struct Reference	86
4.83.1 Detailed Description	86
4.84 TY_FLOAT_RANGE Struct Reference	87
4.84.1 Detailed Description	87
4.85 TY_FRAME_DATA Struct Reference	87
4.85.1 Detailed Description	88
4.86 TY_GYRO_BIAS Struct Reference	88
4.86.1 Detailed Description	88
4.87 TY_GYRO_MISALIGNMENT Struct Reference	88
4.87.1 Detailed Description	89
4.88 TY_GYRO_SCALE Struct Reference	89
4.88.1 Detailed Description	89
4.89 TY_IMAGE_DATA Struct Reference	90
4.89.1 Detailed Description	90
4.90 TY_IMU_DATA Struct Reference	90
4.90.1 Detailed Description	91
4.91 TY_INT_RANGE Struct Reference	91
4.91.1 Detailed Description	91
4.92 TY_INTERFACE_INFO Struct Reference	91
4.92.1 Detailed Description	92
4.93 TY_LASER_PARAM Struct Reference	92
4.93.1 Detailed Description	92
4.94 TY_LASER_PATTERN_PARAM Struct Reference	92
4.94.1 Detailed Description	93
4.95 TY_PHC_GROUP_ATTR Struct Reference	93
4.95.1 Detailed Description	94
4.96 TY_PIXEL_COLOR_DESC Struct Reference	94
4.96.1 Detailed Description	94
4.97 TY_PIXEL_DESC Struct Reference	94
4.97.1 Detailed Description	94
4.98 TY_TEMP_DATA Struct Reference	95
4.98.1 Detailed Description	95
4.99 TY_TOF_FREQ Struct Reference	95
4.99.1 Detailed Description	95

4.100 TY_TRIGGER_PARAM Struct Reference	95
4.100.1 Detailed Description	95
4.101 TY_TRIGGER_PARAM_EX Struct Reference	96
4.101.1 Detailed Description	96
4.102 TY_TRIGGER_TIMER_LIST Struct Reference	96
4.102.1 Detailed Description	96
4.103 TY_TRIGGER_TIMER_PERIOD Struct Reference	97
4.103.1 Detailed Description	97
4.104 TY_VECT_3F Struct Reference	97
4.104.1 Detailed Description	97
4.105 TY_VERSION_INFO Struct Reference	97
4.105.1 Detailed Description	98
4.106 TYEnumEntry Struct Reference	98
4.106.1 Detailed Description	98
4.107 z_stream_s Struct Reference	98
4.107.1 Detailed Description	98
5 File Documentation	99
5.1 TYApi.h File Reference	99
5.1.1 Detailed Description	103
5.1.2 Function Documentation	103
5.1.2.1 TYCfgLogFile()	104
5.1.2.2 TYCfgLogServer()	104
5.1.2.3 TYClearBufferQueue()	105
5.1.2.4 TYCloseDevice()	105
5.1.2.5 TYCloseInterface()	106
5.1.2.6 TYDeinitLib()	107
5.1.2.7 TYDisableComponents()	107
5.1.2.8 TYDisableLog()	108
5.1.2.9 TYEnableComponents()	108
5.1.2.10 TYEnableLog()	109
5.1.2.11 TYEnqueueBuffer()	110
5.1.2.12 TYErrorString()	111
5.1.2.13 TYFetchFrame()	111
5.1.2.14 TYForceDeviceIP()	112
5.1.2.15 TYGetBool()	114
5.1.2.16 TYGetByteArray()	115
5.1.2.17 TYGetByteArrayAttr()	116
5.1.2.18 TYGetByteArraySize()	118
5.1.2.19 TYGetComponentIDs()	119
5.1.2.20 TYGetDeviceFeatureInfo()	120
5.1.2.21 TYGetDeviceFeatureNumber()	121

5.1.2.22 TYGetDeviceInfo()	122
5.1.2.23 TYGetDeviceInterface()	123
5.1.2.24 TYGetDeviceList()	124
5.1.2.25 TYGetDeviceNumber()	125
5.1.2.26 TYGetDeviceXML()	125
5.1.2.27 TYGetDeviceXMLSize()	126
5.1.2.28 TYGetEnabledComponents()	127
5.1.2.29 TYGetEnum()	128
5.1.2.30 TYGetEnumEntryCount()	129
5.1.2.31 TYGetEnumEntryInfo()	130
5.1.2.32 TYGetFeatureInfo()	132
5.1.2.33 TYGetFloat()	133
5.1.2.34 TYGetFloatRange()	134
5.1.2.35 TYGetFrameBufferSize()	135
5.1.2.36 TYGetInt()	136
5.1.2.37 TYGetInterfaceList()	138
5.1.2.38 TYGetInterfaceNumber()	138
5.1.2.39 TYGetIntRange()	139
5.1.2.40 TYGetString()	140
5.1.2.41 TYGetStringLength()	141
5.1.2.42 TYGetStruct()	144
5.1.2.43 TYHasDevice()	145
5.1.2.44 TYHasFeature()	146
5.1.2.45 TYHasInterface()	147
5.1.2.46 TYLibVersion()	148
5.1.2.47 TYOpenDevice()	148
5.1.2.48 TYOpenDeviceWithIP()	150
5.1.2.49 TYOpenInterface()	151
5.1.2.50 TYRegisterEventCallback()	152
5.1.2.51 TYRegisterImuCallback()	152
5.1.2.52 TYSendSoftTrigger()	153
5.1.2.53 TYSetBool()	154
5.1.2.54 TYSetByteArray()	155
5.1.2.55 TYSetEnum()	157
5.1.2.56 TYSetFloat()	158
5.1.2.57 TYSetInt()	160
5.1.2.58 TYSetLogLevel()	161
5.1.2.59 TYSetLogPrefix()	162
5.1.2.60 TYSetString()	162
5.1.2.61 TYSetStruct()	164
5.1.2.62 TYStartCapture()	165
5.1.2.63 TYStopCapture()	166

5.1.2.64 TYUpdateAllDeviceList()	167
5.1.2.65 TYUpdateDeviceList()	167
5.1.2.66 TYUpdateInterfaceList()	168
5.2 TYCoordinateMapper.h File Reference	168
5.2.1 Detailed Description	170
5.2.2 Function Documentation	170
5.2.2.1 TYInvertExtrinsic()	171
5.2.2.2 TYMapDepthImageToColorCoordinate()	171
5.2.2.3 TYMapDepthImageToPoint3d()	172
5.2.2.4 TYMapDepthToColorCoordinate()	172
5.2.2.5 TYMapDepthToPoint3d()	173
5.2.2.6 TYMapMono16ImageToDepthCoordinate()	173
5.2.2.7 TYMapMono8ImageToDepthCoordinate()	174
5.2.2.8 TYMapPoint3dToDepth()	175
5.2.2.9 TYMapPoint3dToDepthImage()	175
5.2.2.10 TYMapPoint3dToPoint3d()	176
5.2.2.11 TYMapRGB48ImageToDepthCoordinate()	176
5.2.2.12 TYMapRGBImageToDepthCoordinate()	177
5.2.2.13 TYMapRGBPixelsToDepthCoordinate()	178
5.3 TYDefs.h File Reference	179
5.3.1 Detailed Description	189
5.3.2 Macro Definition Documentation	189
5.3.2.1 TY_DECLARE_IMAGE_MODE1	189
5.3.3 Typedef Documentation	190
5.3.3.1 TY_ACC_BIAS	190
5.3.3.2 TY_ACC_MISALIGNMENT	190
5.3.3.3 TY_ACC_SCALE	190
5.3.3.4 TY_ACCESS_MODE_LIST	190
5.3.3.5 TY_BYTEARRAY_ATTR	191
5.3.3.6 TY_CAMERA_CALIB_INFO	191
5.3.3.7 TY_CAMERA_DISTORTION	191
5.3.3.8 TY_CAMERA_EXTRINSIC	191
5.3.3.9 TY_CAMERA_INTRINSIC	192
5.3.3.10 TY_CAMERA_ROTATION	192
5.3.3.11 TY_CAMERA_TO_IMU	193
5.3.3.12 TY_COMPONENT_ID	193
5.3.3.13 TY_DEVICE_BASE_INFO	193
5.3.3.14 TY_DEVICE_COMPONENT_LIST	193
5.3.3.15 TY_ENUM_ENTRY	194
5.3.3.16 TY_FEATURE_ID	194
5.3.3.17 TY_FLOAT_RANGE	194
5.3.3.18 TY_GYRO_BIAS	194

5.3.3.19 TY_GYRO_MISALIGNMENT	195
5.3.3.20 TY_GYRO_SCALE	195
5.3.3.21 TY_INTERFACE_INFO	195
5.3.3.22 TY_INTERFACE_TYPE_LIST	195
5.3.3.23 TY_PIXEL_BITS_LIST	196
5.3.3.24 TY_TRIGGER_MODE_LIST	196
5.3.4 Enumeration Type Documentation	196
5.3.4.1 TY_ACCESS_MODE_LIST	196
5.3.4.2 TY_DEVICE_COMPONENT_LIST	196
5.3.4.3 TY_FEATURE_ID_LIST	197
5.3.4.4 TY_INTERFACE_TYPE_LIST	200
5.3.4.5 TY_PIXEL_BITS_LIST	200
5.3.4.6 TY_PIXEL_FORMAT_LIST	200
5.3.4.7 TY_RESOLUTION_MODE_LIST	201
5.3.4.8 TY_TRIGGER_MODE_LIST	202
5.4 TYFeatureList.h File Reference	202
5.4.1 Detailed Description	211
5.4.2 Enumeration Type Documentation	211
5.4.2.1 TY_ACQ_MODE_LIST	211
5.4.2.2 TY_BIN_H_LIST	211
5.4.2.3 TY_BIN_V_LIST	212
5.4.2.4 TY_DEPTH_TOF_QUALITY_LIST	212
5.4.2.5 TY_DEV_CHARSET_LIST	212
5.4.2.6 TY_DEV_COMPRESS_LIST	213
5.4.2.7 TY_DEV_LINK_HB_MODE_LIST	213
5.4.2.8 TY_DEV_STREAM_ASYNC_LIST	213
5.4.2.9 TY_DEV_STREAM_CH_TYPE_LIST	214
5.4.2.10 TY_DEV_TEMP_SEL_LIST	214
5.4.2.11 TY_DEV_TIME_SYNC_LIST	214
5.4.2.12 TY_DEV_TYPE_LIST	215
5.4.2.13 TY_GAIN_SEL_LIST	215
5.4.2.14 TY_GEV_CCP_LIST	215
5.4.2.15 TY_LIGHT_SEL_LIST	215
5.4.2.16 TY_PIX_FMT_LIST	216
5.4.2.17 TY_SRC_SEL_LIST	216
5.4.2.18 TY_TRIG_ACT_LIST	217
5.4.2.19 TY_TRIG_MODE_LIST	217
5.4.2.20 TY_TRIG_SEL_LIST	217
5.4.2.21 TY_TRIG_SRC_LIST	218
5.4.2.22 TY_USER_SET_SEL_LIST	218
5.5 TYImageProc.h File Reference	219
5.5.1 Detailed Description	221

5.5.2 Function Documentation	221
5.5.2.1 TYDepthEnhenceFilter()	221
5.5.2.2 TYDepthImageInpainter()	221
5.5.2.3 TYDepthSpeckleFilter()	222
5.5.2.4 TYGetImageAlgorithmVersion()	222
5.5.2.5 TYImageProcesAcceEnable()	223
5.5.2.6 TYUndistortImage()	223
5.5.2.7 TYUndistortImage2()	224
Index	225

Chapter 1

Main Page

Copyright(C)2016-2023 Percipio All Rights Reserved

1.1 Note

Depth camera, called "device", consists of several components. Each component is a hardware module or virtual module, such as RGB sensor, depth sensor. Each component has its own features, such as image width, exposure time, etc..

NOTE: The component `TY_COMPONENT_DEVICE` is a virtual component that contains all features related to the whole device, such as trigger mode, device IP.

Each frame consists of several images. Normally, all the images have identical timestamp, means they are captured at the same time.

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AlgVersion	51
DepthEnhenceParameters	51
DepthInpainterParameters	52
DepthSpeckleFilterParameters	52
f16	52
gz_header_s	53
gzFile_s	53
nsimd_aarch64_vf16	54
nsimd_aarch64_vlf16	54
nsimd_avx2_vf16	54
nsimd_avx2_vlf16	55
nsimd_avx512_knl_vf16	55
nsimd_avx512_knl_vlf16	55
nsimd_avx512_skylake_vf16	56
nsimd_avx512_skylake_vlf16	56
nsimd_avx_vf16	56
nsimd_avx_vlf16	57
nsimd_cpu_vf16	57
nsimd_cpu_vf32	57
nsimd_cpu_vf64	58
nsimd_cpu_vi16	58
nsimd_cpu_vi32	59
nsimd_cpu_vi64	59
nsimd_cpu_vi8	59
nsimd_cpu_vlf16	60
nsimd_cpu_vlf32	60
nsimd_cpu_vlf64	61
nsimd_cpu_vli16	61
nsimd_cpu_vli32	61
nsimd_cpu_vli64	62
nsimd_cpu_vli8	62
nsimd_cpu_vlu16	63
nsimd_cpu_vlu32	63
nsimd_cpu_vlu64	64
nsimd_cpu_vlu8	64

nsimd_cpu_vu16	65
nsimd_cpu_vu32	65
nsimd_cpu_vu64	65
nsimd_cpu_vu8	66
nsimd_neon128_vf16	66
nsimd_neon128_vf64	67
nsimd_neon128_vlf16	67
nsimd_neon128_vlf64	67
nsimd_sse2_vf16	68
nsimd_sse2_vlf16	68
nsimd_sse42_vf16	68
nsimd_sse42_vlf16	69
nsimd_vmx_vf16	69
nsimd_vmx_vf64	69
nsimd_vmx_vf64	70
nsimd_vmx_vlf16	70
nsimd_vmx_vlf64	70
nsimd_vmx_vli64	71
nsimd_vmx_vlu64	71
nsimd_vmx_vu64	71
nsimd_vsx_vf16	72
nsimd_vsx_vlf16	72
OperatorInfo	73
ParamDetail	73
pattern_bin_param	74
pattern_gray_param	74
pattern_sine_param	74
TY_PHC_GROUP_ATTR::phc_group_attr	75
TY_ACC_BIAS	75
TY_ACC_MISALIGNMENT	76
TY_ACC_SCALE	76
TY_AEC_ROI_PARAM	77
TY_BYTEARRAY_ATTR	
Byte array data structure	77
TY_CAMERA_CALIB_INFO	78
TY_CAMERA_DISTORTION	79
TY_CAMERA_EXTRINSIC	79
TY_CAMERA_INTRINSIC	80
TY_CAMERA_ROTATION	81
TY_CAMERA_STATISTICS	81
TY_CAMERA_TO_IMU	82
TY_DEVICE_BASE_INFO	82
TY_DEVICE_NET_INFO	
Device network information	83
TY_DEVICE_USB_INFO	84
TY_DI_WORKMODE	84
TY_DO_WORKMODE	85
TY_ENUM_ENTRY	85
TY_EVENT_INFO	86
TY_FEATURE_INFO	86
TY_FLOAT_RANGE	
Float range data structure	87
TY_FRAME_DATA	87
TY_GYRO_BIAS	88
TY_GYRO_MISALIGNMENT	88
TY_GYRO_SCALE	89
TY_IMAGE_DATA	90
TY_IMU_DATA	90

TY_INT_RANGE	91
TY_INTERFACE_INFO	91
TY_LASER_PARAM	92
TY_LASER_PATTERN_PARAM	92
TY_PHC_GROUP_ATTR	93
TY_PIXEL_COLOR_DESC	94
TY_PIXEL_DESC	94
TY_TEMP_DATA	95
TY_TOF_FREQ	95
TY_TRIGGER_PARAM	95
TY_TRIGGER_PARAM_EX	96
TY_TRIGGER_TIMER_LIST	96
TY_TRIGGER_TIMER_PERIOD	97
TY_VECT_3F	97
TY_VERSION_INFO	97
TYEnumEntry	98
z_stream_s	98

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

arm/aarch64/abs.h	??
arm/neon128/abs.h	??
arm/sve/abs.h	??
arm/sve1024/abs.h	??
arm/sve128/abs.h	??
arm/sve2048/abs.h	??
arm/sve256/abs.h	??
arm/sve512/abs.h	??
cpu/cpu/abs.h	??
ppc/vmx/abs.h	??
ppc/vsx/abs.h	??
x86/avx/abs.h	??
x86/avx2/abs.h	??
x86/avx512_knl/abs.h	??
x86/avx512_skylake/abs.h	??
x86/sse2/abs.h	??
x86/sse42/abs.h	??
arm/aarch64/acos_u10.h	??
arm/neon128/acos_u10.h	??
arm/sve/acos_u10.h	??
arm/sve1024/acos_u10.h	??
arm/sve128/acos_u10.h	??
arm/sve2048/acos_u10.h	??
arm/sve256/acos_u10.h	??
arm/sve512/acos_u10.h	??
cpu/cpu/acos_u10.h	??
ppc/vmx/acos_u10.h	??
ppc/vsx/acos_u10.h	??
x86/avx/acos_u10.h	??
x86/avx2/acos_u10.h	??
x86/avx512_knl/acos_u10.h	??
x86/avx512_skylake/acos_u10.h	??
x86/sse2/acos_u10.h	??
x86/sse42/acos_u10.h	??
arm/aarch64/acos_u35.h	??

arm/neon128/acos_u35.h	??
arm/sve/acos_u35.h	??
arm/sve1024/acos_u35.h	??
arm/sve128/acos_u35.h	??
arm/sve2048/acos_u35.h	??
arm/sve256/acos_u35.h	??
arm/sve512/acos_u35.h	??
cpu/cpu/acos_u35.h	??
ppc/vmx/acos_u35.h	??
ppc/vsx/acos_u35.h	??
x86/avx/acos_u35.h	??
x86/avx2/acos_u35.h	??
x86/avx512_knl/acos_u35.h	??
x86/avx512_skylake/acos_u35.h	??
x86/sse2/acos_u35.h	??
x86/sse42/acos_u35.h	??
arm/aarch64/acosh_u10.h	??
arm/neon128/acosh_u10.h	??
arm/sve/acosh_u10.h	??
arm/sve1024/acosh_u10.h	??
arm/sve128/acosh_u10.h	??
arm/sve2048/acosh_u10.h	??
arm/sve256/acosh_u10.h	??
arm/sve512/acosh_u10.h	??
cpu/cpu/acosh_u10.h	??
ppc/vmx/acosh_u10.h	??
ppc/vsx/acosh_u10.h	??
x86/avx/acosh_u10.h	??
x86/avx2/acosh_u10.h	??
x86/avx512_knl/acosh_u10.h	??
x86/avx512_skylake/acosh_u10.h	??
x86/sse2/acosh_u10.h	??
x86/sse42/acosh_u10.h	??
arm/aarch64/add.h	??
arm/neon128/add.h	??
arm/sve/add.h	??
arm/sve1024/add.h	??
arm/sve128/add.h	??
arm/sve2048/add.h	??
arm/sve256/add.h	??
arm/sve512/add.h	??
cpu/cpu/add.h	??
ppc/vmx/add.h	??
ppc/vsx/add.h	??
x86/avx/add.h	??
x86/avx2/add.h	??
x86/avx512_knl/add.h	??
x86/avx512_skylake/add.h	??
x86/sse2/add.h	??
x86/sse42/add.h	??
arm/aarch64/adds.h	??
arm/neon128/adds.h	??
arm/sve/adds.h	??
arm/sve1024/adds.h	??
arm/sve128/adds.h	??
arm/sve2048/adds.h	??
arm/sve256/adds.h	??
arm/sve512/adds.h	??

cpu/cpu/adds.h	??
ppc/vmx/adds.h	??
ppc/vsx/adds.h	??
x86/avx/adds.h	??
x86/avx2/adds.h	??
x86/avx512_knl/adds.h	??
x86/avx512_skylake/adds.h	??
x86/sse2/adds.h	??
x86/sse42/adds.h	??
arm/aarch64/addv.h	??
arm/neon128/addv.h	??
arm/sve/addv.h	??
arm/sve1024/addv.h	??
arm/sve128/addv.h	??
arm/sve2048/addv.h	??
arm/sve256/addv.h	??
arm/sve512/addv.h	??
cpu/cpu/addv.h	??
ppc/vmx/addv.h	??
ppc/vsx/addv.h	??
x86/avx/addv.h	??
x86/avx2/addv.h	??
x86/avx512_knl/addv.h	??
x86/avx512_skylake/addv.h	??
x86/sse2/addv.h	??
x86/sse42/addv.h	??
arm/aarch64/all.h	??
arm/neon128/all.h	??
arm/sve/all.h	??
arm/sve1024/all.h	??
arm/sve128/all.h	??
arm/sve2048/all.h	??
arm/sve256/all.h	??
arm/sve512/all.h	??
cpu/cpu/all.h	??
ppc/vmx/all.h	??
ppc/vsx/all.h	??
x86/avx/all.h	??
x86/avx2/all.h	??
x86/avx512_knl/all.h	??
x86/avx512_skylake/all.h	??
x86/sse2/all.h	??
x86/sse42/all.h	??
arm/aarch64/andb.h	??
arm/neon128/andb.h	??
arm/sve/andb.h	??
arm/sve1024/andb.h	??
arm/sve128/andb.h	??
arm/sve2048/andb.h	??
arm/sve256/andb.h	??
arm/sve512/andb.h	??
cpu/cpu/andb.h	??
ppc/vmx/andb.h	??
ppc/vsx/andb.h	??
x86/avx/andb.h	??
x86/avx2/andb.h	??
x86/avx512_knl/andb.h	??
x86/avx512_skylake/andb.h	??

x86/sse2/andb.h	??
x86/sse42/andb.h	??
arm/aarch64/andl.h	??
arm/neon128/andl.h	??
arm/sve/andl.h	??
arm/sve1024/andl.h	??
arm/sve128/andl.h	??
arm/sve2048/andl.h	??
arm/sve256/andl.h	??
arm/sve512/andl.h	??
cpu/cpu/andl.h	??
ppc/vmx/andl.h	??
ppc/vsx/andl.h	??
x86/avx/andl.h	??
x86/avx2/andl.h	??
x86/avx512_knl/andl.h	??
x86/avx512_skylake/andl.h	??
x86/sse2/andl.h	??
x86/sse42/andl.h	??
arm/aarch64/andnotb.h	??
arm/neon128/andnotb.h	??
arm/sve/andnotb.h	??
arm/sve1024/andnotb.h	??
arm/sve128/andnotb.h	??
arm/sve2048/andnotb.h	??
arm/sve256/andnotb.h	??
arm/sve512/andnotb.h	??
cpu/cpu/andnotb.h	??
ppc/vmx/andnotb.h	??
ppc/vsx/andnotb.h	??
x86/avx/andnotb.h	??
x86/avx2/andnotb.h	??
x86/avx512_knl/andnotb.h	??
x86/avx512_skylake/andnotb.h	??
x86/sse2/andnotb.h	??
x86/sse42/andnotb.h	??
arm/aarch64/andnotl.h	??
arm/neon128/andnotl.h	??
arm/sve/andnotl.h	??
arm/sve1024/andnotl.h	??
arm/sve128/andnotl.h	??
arm/sve2048/andnotl.h	??
arm/sve256/andnotl.h	??
arm/sve512/andnotl.h	??
cpu/cpu/andnotl.h	??
ppc/vmx/andnotl.h	??
ppc/vsx/andnotl.h	??
x86/avx/andnotl.h	??
x86/avx2/andnotl.h	??
x86/avx512_knl/andnotl.h	??
x86/avx512_skylake/andnotl.h	??
x86/sse2/andnotl.h	??
x86/sse42/andnotl.h	??
arm/aarch64/any.h	??
arm/neon128/any.h	??
arm/sve/any.h	??
arm/sve1024/any.h	??
arm/sve128/any.h	??

arm/sve2048/any.h	??
arm/sve256/any.h	??
arm/sve512/any.h	??
cpu/cpu/any.h	??
ppc/vmx/any.h	??
ppc/vsx/any.h	??
x86/avx/any.h	??
x86/avx2/any.h	??
x86/avx512_knl/any.h	??
x86/avx512_skylake/any.h	??
x86/sse2/any.h	??
x86/sse42/any.h	??
arm/aarch64/asin_u10.h	??
arm/neon128/asin_u10.h	??
arm/sve/asin_u10.h	??
arm/sve1024/asin_u10.h	??
arm/sve128/asin_u10.h	??
arm/sve2048/asin_u10.h	??
arm/sve256/asin_u10.h	??
arm/sve512/asin_u10.h	??
cpu/cpu/asin_u10.h	??
ppc/vmx/asin_u10.h	??
ppc/vsx/asin_u10.h	??
x86/avx/asin_u10.h	??
x86/avx2/asin_u10.h	??
x86/avx512_knl/asin_u10.h	??
x86/avx512_skylake/asin_u10.h	??
x86/sse2/asin_u10.h	??
x86/sse42/asin_u10.h	??
arm/aarch64/asin_u35.h	??
arm/neon128/asin_u35.h	??
arm/sve/asin_u35.h	??
arm/sve1024/asin_u35.h	??
arm/sve128/asin_u35.h	??
arm/sve2048/asin_u35.h	??
arm/sve256/asin_u35.h	??
arm/sve512/asin_u35.h	??
cpu/cpu/asin_u35.h	??
ppc/vmx/asin_u35.h	??
ppc/vsx/asin_u35.h	??
x86/avx/asin_u35.h	??
x86/avx2/asin_u35.h	??
x86/avx512_knl/asin_u35.h	??
x86/avx512_skylake/asin_u35.h	??
x86/sse2/asin_u35.h	??
x86/sse42/asin_u35.h	??
arm/aarch64/asinh_u10.h	??
arm/neon128/asinh_u10.h	??
arm/sve/asinh_u10.h	??
arm/sve1024/asinh_u10.h	??
arm/sve128/asinh_u10.h	??
arm/sve2048/asinh_u10.h	??
arm/sve256/asinh_u10.h	??
arm/sve512/asinh_u10.h	??
cpu/cpu/asinh_u10.h	??
ppc/vmx/asinh_u10.h	??
ppc/vsx/asinh_u10.h	??
x86/avx/asinh_u10.h	??

x86/avx2/asinh_u10.h	??
x86/avx512_knl/asinh_u10.h	??
x86/avx512_skylake/asinh_u10.h	??
x86/sse2/asinh_u10.h	??
x86/sse42/asinh_u10.h	??
arm/aarch64/atan2_u10.h	??
arm/neon128/atan2_u10.h	??
arm/sve/atan2_u10.h	??
arm/sve1024/atan2_u10.h	??
arm/sve128/atan2_u10.h	??
arm/sve2048/atan2_u10.h	??
arm/sve256/atan2_u10.h	??
arm/sve512/atan2_u10.h	??
cpu/cpu/atan2_u10.h	??
ppc/vmx/atan2_u10.h	??
ppc/vsx/atan2_u10.h	??
x86/avx/atan2_u10.h	??
x86/avx2/atan2_u10.h	??
x86/avx512_knl/atan2_u10.h	??
x86/avx512_skylake/atan2_u10.h	??
x86/sse2/atan2_u10.h	??
x86/sse42/atan2_u10.h	??
arm/aarch64/atan2_u35.h	??
arm/neon128/atan2_u35.h	??
arm/sve/atan2_u35.h	??
arm/sve1024/atan2_u35.h	??
arm/sve128/atan2_u35.h	??
arm/sve2048/atan2_u35.h	??
arm/sve256/atan2_u35.h	??
arm/sve512/atan2_u35.h	??
cpu/cpu/atan2_u35.h	??
ppc/vmx/atan2_u35.h	??
ppc/vsx/atan2_u35.h	??
x86/avx/atan2_u35.h	??
x86/avx2/atan2_u35.h	??
x86/avx512_knl/atan2_u35.h	??
x86/avx512_skylake/atan2_u35.h	??
x86/sse2/atan2_u35.h	??
x86/sse42/atan2_u35.h	??
arm/aarch64/atan_u10.h	??
arm/neon128/atan_u10.h	??
arm/sve/atan_u10.h	??
arm/sve1024/atan_u10.h	??
arm/sve128/atan_u10.h	??
arm/sve2048/atan_u10.h	??
arm/sve256/atan_u10.h	??
arm/sve512/atan_u10.h	??
cpu/cpu/atan_u10.h	??
ppc/vmx/atan_u10.h	??
ppc/vsx/atan_u10.h	??
x86/avx/atan_u10.h	??
x86/avx2/atan_u10.h	??
x86/avx512_knl/atan_u10.h	??
x86/avx512_skylake/atan_u10.h	??
x86/sse2/atan_u10.h	??
x86/sse42/atan_u10.h	??
arm/aarch64/atan_u35.h	??
arm/neon128/atan_u35.h	??

arm/sve/atan_u35.h	??
arm/sve1024/atan_u35.h	??
arm/sve128/atan_u35.h	??
arm/sve2048/atan_u35.h	??
arm/sve256/atan_u35.h	??
arm/sve512/atan_u35.h	??
cpu/cpu/atan_u35.h	??
ppc/vmx/atan_u35.h	??
ppc/vsx/atan_u35.h	??
x86/avx/atan_u35.h	??
x86/avx2/atan_u35.h	??
x86/avx512_knl/atan_u35.h	??
x86/avx512_skylake/atan_u35.h	??
x86/sse2/atan_u35.h	??
x86/sse42/atan_u35.h	??
arm/aarch64/atanh_u10.h	??
arm/neon128/atanh_u10.h	??
arm/sve/atanh_u10.h	??
arm/sve1024/atanh_u10.h	??
arm/sve128/atanh_u10.h	??
arm/sve2048/atanh_u10.h	??
arm/sve256/atanh_u10.h	??
arm/sve512/atanh_u10.h	??
cpu/cpu/atanh_u10.h	??
ppc/vmx/atanh_u10.h	??
ppc/vsx/atanh_u10.h	??
x86/avx/atanh_u10.h	??
x86/avx2/atanh_u10.h	??
x86/avx512_knl/atanh_u10.h	??
x86/avx512_skylake/atanh_u10.h	??
x86/sse2/atanh_u10.h	??
x86/sse42/atanh_u10.h	??
c_adv_api.h	??
c_adv_api_functions.h	??
arm/aarch64/cbrt_u10.h	??
arm/neon128/cbrt_u10.h	??
arm/sve/cbrt_u10.h	??
arm/sve1024/cbrt_u10.h	??
arm/sve128/cbrt_u10.h	??
arm/sve2048/cbrt_u10.h	??
arm/sve256/cbrt_u10.h	??
arm/sve512/cbrt_u10.h	??
cpu/cpu/cbrt_u10.h	??
ppc/vmx/cbrt_u10.h	??
ppc/vsx/cbrt_u10.h	??
x86/avx/cbrt_u10.h	??
x86/avx2/cbrt_u10.h	??
x86/avx512_knl/cbrt_u10.h	??
x86/avx512_skylake/cbrt_u10.h	??
x86/sse2/cbrt_u10.h	??
x86/sse42/cbrt_u10.h	??
arm/aarch64/cbrt_u35.h	??
arm/neon128/cbrt_u35.h	??
arm/sve/cbrt_u35.h	??
arm/sve1024/cbrt_u35.h	??
arm/sve128/cbrt_u35.h	??
arm/sve2048/cbrt_u35.h	??
arm/sve256/cbrt_u35.h	??

arm/sve512/cbrt_u35.h	??
cpu/cpu/cbrt_u35.h	??
ppc/vmx/cbrt_u35.h	??
ppc/vsx/cbrt_u35.h	??
x86/avx/cbrt_u35.h	??
x86/avx2/cbrt_u35.h	??
x86/avx512_knl/cbrt_u35.h	??
x86/avx512_skylake/cbrt_u35.h	??
x86/sse2/cbrt_u35.h	??
x86/sse42/cbrt_u35.h	??
arm/aarch64/ceil.h	??
arm/neon128/ceil.h	??
arm/sve/ceil.h	??
arm/sve1024/ceil.h	??
arm/sve128/ceil.h	??
arm/sve2048/ceil.h	??
arm/sve256/ceil.h	??
arm/sve512/ceil.h	??
cpu/cpu/ceil.h	??
ppc/vmx/ceil.h	??
ppc/vsx/ceil.h	??
x86/avx/ceil.h	??
x86/avx2/ceil.h	??
x86/avx512_knl/ceil.h	??
x86/avx512_skylake/ceil.h	??
x86/sse2/ceil.h	??
x86/sse42/ceil.h	??
arm/aarch64/cos_u10.h	??
arm/neon128/cos_u10.h	??
arm/sve/cos_u10.h	??
arm/sve1024/cos_u10.h	??
arm/sve128/cos_u10.h	??
arm/sve2048/cos_u10.h	??
arm/sve256/cos_u10.h	??
arm/sve512/cos_u10.h	??
cpu/cpu/cos_u10.h	??
ppc/vmx/cos_u10.h	??
ppc/vsx/cos_u10.h	??
x86/avx/cos_u10.h	??
x86/avx2/cos_u10.h	??
x86/avx512_knl/cos_u10.h	??
x86/avx512_skylake/cos_u10.h	??
x86/sse2/cos_u10.h	??
x86/sse42/cos_u10.h	??
arm/aarch64/cos_u35.h	??
arm/neon128/cos_u35.h	??
arm/sve/cos_u35.h	??
arm/sve1024/cos_u35.h	??
arm/sve128/cos_u35.h	??
arm/sve2048/cos_u35.h	??
arm/sve256/cos_u35.h	??
arm/sve512/cos_u35.h	??
cpu/cpu/cos_u35.h	??
ppc/vmx/cos_u35.h	??
ppc/vsx/cos_u35.h	??
x86/avx/cos_u35.h	??
x86/avx2/cos_u35.h	??
x86/avx512_knl/cos_u35.h	??

x86/avx512_skylake/cos_u35.h	??
x86/sse2/cos_u35.h	??
x86/sse42/cos_u35.h	??
arm/aarch64/cosh_u10.h	??
arm/neon128/cosh_u10.h	??
arm/sve/cosh_u10.h	??
arm/sve1024/cosh_u10.h	??
arm/sve128/cosh_u10.h	??
arm/sve2048/cosh_u10.h	??
arm/sve256/cosh_u10.h	??
arm/sve512/cosh_u10.h	??
cpu/cpu/cosh_u10.h	??
ppc/vmx/cosh_u10.h	??
ppc/vsx/cosh_u10.h	??
x86/avx/cosh_u10.h	??
x86/avx2/cosh_u10.h	??
x86/avx512_knl/cosh_u10.h	??
x86/avx512_skylake/cosh_u10.h	??
x86/sse2/cosh_u10.h	??
x86/sse42/cosh_u10.h	??
arm/aarch64/cosh_u35.h	??
arm/neon128/cosh_u35.h	??
arm/sve/cosh_u35.h	??
arm/sve1024/cosh_u35.h	??
arm/sve128/cosh_u35.h	??
arm/sve2048/cosh_u35.h	??
arm/sve256/cosh_u35.h	??
arm/sve512/cosh_u35.h	??
cpu/cpu/cosh_u35.h	??
ppc/vmx/cosh_u35.h	??
ppc/vsx/cosh_u35.h	??
x86/avx/cosh_u35.h	??
x86/avx2/cosh_u35.h	??
x86/avx512_knl/cosh_u35.h	??
x86/avx512_skylake/cosh_u35.h	??
x86/sse2/cosh_u35.h	??
x86/sse42/cosh_u35.h	??
arm/aarch64/cospi_u05.h	??
arm/neon128/cospi_u05.h	??
arm/sve/cospi_u05.h	??
arm/sve1024/cospi_u05.h	??
arm/sve128/cospi_u05.h	??
arm/sve2048/cospi_u05.h	??
arm/sve256/cospi_u05.h	??
arm/sve512/cospi_u05.h	??
cpu/cpu/cospi_u05.h	??
ppc/vmx/cospi_u05.h	??
ppc/vsx/cospi_u05.h	??
x86/avx/cospi_u05.h	??
x86/avx2/cospi_u05.h	??
x86/avx512_knl/cospi_u05.h	??
x86/avx512_skylake/cospi_u05.h	??
x86/sse2/cospi_u05.h	??
x86/sse42/cospi_u05.h	??
arm/aarch64/cvt.h	??
arm/neon128/cvt.h	??
arm/sve/cvt.h	??
arm/sve1024/cvt.h	??

arm/sve128/cvt.h	??
arm/sve2048/cvt.h	??
arm/sve256/cvt.h	??
arm/sve512/cvt.h	??
cpu/cpu/cvt.h	??
ppc/vmx/cvt.h	??
ppc/vsx/cvt.h	??
x86/avx/cvt.h	??
x86/avx2/cvt.h	??
x86/avx512_knl/cvt.h	??
x86/avx512_skylake/cvt.h	??
x86/sse2/cvt.h	??
x86/sse42/cvt.h	??
arm/aarch64/div.h	??
arm/neon128/div.h	??
arm/sve/div.h	??
arm/sve1024/div.h	??
arm/sve128/div.h	??
arm/sve2048/div.h	??
arm/sve256/div.h	??
arm/sve512/div.h	??
cpu/cpu/div.h	??
ppc/vmx/div.h	??
ppc/vsx/div.h	??
x86/avx/div.h	??
x86/avx2/div.h	??
x86/avx512_knl/div.h	??
x86/avx512_skylake/div.h	??
x86/sse2/div.h	??
x86/sse42/div.h	??
arm/aarch64/downcvt.h	??
arm/neon128/downcvt.h	??
arm/sve/downcvt.h	??
arm/sve1024/downcvt.h	??
arm/sve128/downcvt.h	??
arm/sve2048/downcvt.h	??
arm/sve256/downcvt.h	??
arm/sve512/downcvt.h	??
cpu/cpu/downcvt.h	??
ppc/vmx/downcvt.h	??
ppc/vsx/downcvt.h	??
x86/avx/downcvt.h	??
x86/avx2/downcvt.h	??
x86/avx512_knl/downcvt.h	??
x86/avx512_skylake/downcvt.h	??
x86/sse2/downcvt.h	??
x86/sse42/downcvt.h	??
arm/aarch64/eq.h	??
arm/neon128/eq.h	??
arm/sve/eq.h	??
arm/sve1024/eq.h	??
arm/sve128/eq.h	??
arm/sve2048/eq.h	??
arm/sve256/eq.h	??
arm/sve512/eq.h	??
cpu/cpu/eq.h	??
ppc/vmx/eq.h	??
ppc/vsx/eq.h	??

x86/avx/eq.h	??
x86/avx2/eq.h	??
x86/avx512_knl/eq.h	??
x86/avx512_skylake/eq.h	??
x86/sse2/eq.h	??
x86/sse42/eq.h	??
arm/aarch64/erf_u10.h	??
arm/neon128/erf_u10.h	??
arm/sve/erf_u10.h	??
arm/sve1024/erf_u10.h	??
arm/sve128/erf_u10.h	??
arm/sve2048/erf_u10.h	??
arm/sve256/erf_u10.h	??
arm/sve512/erf_u10.h	??
cpu/cpu/erf_u10.h	??
ppc/vmx/erf_u10.h	??
ppc/vsx/erf_u10.h	??
x86/avx/erf_u10.h	??
x86/avx2/erf_u10.h	??
x86/avx512_knl/erf_u10.h	??
x86/avx512_skylake/erf_u10.h	??
x86/sse2/erf_u10.h	??
x86/sse42/erf_u10.h	??
arm/aarch64/erfc_u15.h	??
arm/neon128/erfc_u15.h	??
arm/sve/erfc_u15.h	??
arm/sve1024/erfc_u15.h	??
arm/sve128/erfc_u15.h	??
arm/sve2048/erfc_u15.h	??
arm/sve256/erfc_u15.h	??
arm/sve512/erfc_u15.h	??
cpu/cpu/erfc_u15.h	??
ppc/vmx/erfc_u15.h	??
ppc/vsx/erfc_u15.h	??
x86/avx/erfc_u15.h	??
x86/avx2/erfc_u15.h	??
x86/avx512_knl/erfc_u15.h	??
x86/avx512_skylake/erfc_u15.h	??
x86/sse2/erfc_u15.h	??
x86/sse42/erfc_u15.h	??
arm/aarch64/exp10_u10.h	??
arm/neon128/exp10_u10.h	??
arm/sve/exp10_u10.h	??
arm/sve1024/exp10_u10.h	??
arm/sve128/exp10_u10.h	??
arm/sve2048/exp10_u10.h	??
arm/sve256/exp10_u10.h	??
arm/sve512/exp10_u10.h	??
cpu/cpu/exp10_u10.h	??
ppc/vmx/exp10_u10.h	??
ppc/vsx/exp10_u10.h	??
x86/avx/exp10_u10.h	??
x86/avx2/exp10_u10.h	??
x86/avx512_knl/exp10_u10.h	??
x86/avx512_skylake/exp10_u10.h	??
x86/sse2/exp10_u10.h	??
x86/sse42/exp10_u10.h	??
arm/aarch64/exp10_u35.h	??

arm/neon128/exp10_u35.h	??
arm/sve/exp10_u35.h	??
arm/sve1024/exp10_u35.h	??
arm/sve128/exp10_u35.h	??
arm/sve2048/exp10_u35.h	??
arm/sve256/exp10_u35.h	??
arm/sve512/exp10_u35.h	??
cpu/cpu/exp10_u35.h	??
ppc/vmx/exp10_u35.h	??
ppc/vsx/exp10_u35.h	??
x86/avx/exp10_u35.h	??
x86/avx2/exp10_u35.h	??
x86/avx512_knl/exp10_u35.h	??
x86/avx512_skylake/exp10_u35.h	??
x86/sse2/exp10_u35.h	??
x86/sse42/exp10_u35.h	??
arm/aarch64/exp2_u10.h	??
arm/neon128/exp2_u10.h	??
arm/sve/exp2_u10.h	??
arm/sve1024/exp2_u10.h	??
arm/sve128/exp2_u10.h	??
arm/sve2048/exp2_u10.h	??
arm/sve256/exp2_u10.h	??
arm/sve512/exp2_u10.h	??
cpu/cpu/exp2_u10.h	??
ppc/vmx/exp2_u10.h	??
ppc/vsx/exp2_u10.h	??
x86/avx/exp2_u10.h	??
x86/avx2/exp2_u10.h	??
x86/avx512_knl/exp2_u10.h	??
x86/avx512_skylake/exp2_u10.h	??
x86/sse2/exp2_u10.h	??
x86/sse42/exp2_u10.h	??
arm/aarch64/exp2_u35.h	??
arm/neon128/exp2_u35.h	??
arm/sve/exp2_u35.h	??
arm/sve1024/exp2_u35.h	??
arm/sve128/exp2_u35.h	??
arm/sve2048/exp2_u35.h	??
arm/sve256/exp2_u35.h	??
arm/sve512/exp2_u35.h	??
cpu/cpu/exp2_u35.h	??
ppc/vmx/exp2_u35.h	??
ppc/vsx/exp2_u35.h	??
x86/avx/exp2_u35.h	??
x86/avx2/exp2_u35.h	??
x86/avx512_knl/exp2_u35.h	??
x86/avx512_skylake/exp2_u35.h	??
x86/sse2/exp2_u35.h	??
x86/sse42/exp2_u35.h	??
arm/aarch64/exp_u10.h	??
arm/neon128/exp_u10.h	??
arm/sve/exp_u10.h	??
arm/sve1024/exp_u10.h	??
arm/sve128/exp_u10.h	??
arm/sve2048/exp_u10.h	??
arm/sve256/exp_u10.h	??
arm/sve512/exp_u10.h	??

cpu/cpu/exp_u10.h	??
ppc/vmx/exp_u10.h	??
ppc/vsx/exp_u10.h	??
x86/avx/exp_u10.h	??
x86/avx2/exp_u10.h	??
x86/avx512_knl/exp_u10.h	??
x86/avx512_skylake/exp_u10.h	??
x86/sse2/exp_u10.h	??
x86/sse42/exp_u10.h	??
arm/aarch64/expm1_u10.h	??
arm/neon128/expm1_u10.h	??
arm/sve/expm1_u10.h	??
arm/sve1024/expm1_u10.h	??
arm/sve128/expm1_u10.h	??
arm/sve2048/expm1_u10.h	??
arm/sve256/expm1_u10.h	??
arm/sve512/expm1_u10.h	??
cpu/cpu/expm1_u10.h	??
ppc/vmx/expm1_u10.h	??
ppc/vsx/expm1_u10.h	??
x86/avx/expm1_u10.h	??
x86/avx2/expm1_u10.h	??
x86/avx512_knl/expm1_u10.h	??
x86/avx512_skylake/expm1_u10.h	??
x86/sse2/expm1_u10.h	??
x86/sse42/expm1_u10.h	??
arm/aarch64/floor.h	??
arm/neon128/floor.h	??
arm/sve/floor.h	??
arm/sve1024/floor.h	??
arm/sve128/floor.h	??
arm/sve2048/floor.h	??
arm/sve256/floor.h	??
arm/sve512/floor.h	??
cpu/cpu/floor.h	??
ppc/vmx/floor.h	??
ppc/vsx/floor.h	??
x86/avx/floor.h	??
x86/avx2/floor.h	??
x86/avx512_knl/floor.h	??
x86/avx512_skylake/floor.h	??
x86/sse2/floor.h	??
x86/sse42/floor.h	??
arm/aarch64/fma.h	??
arm/neon128/fma.h	??
arm/sve/fma.h	??
arm/sve1024/fma.h	??
arm/sve128/fma.h	??
arm/sve2048/fma.h	??
arm/sve256/fma.h	??
arm/sve512/fma.h	??
cpu/cpu/fma.h	??
ppc/vmx/fma.h	??
ppc/vsx/fma.h	??
x86/avx/fma.h	??
x86/avx2/fma.h	??
x86/avx512_knl/fma.h	??
x86/avx512_skylake/fma.h	??

x86/sse2/fma.h	??
x86/sse42/fma.h	??
arm/aarch64/fmod.h	??
arm/neon128/fmod.h	??
arm/sve/fmod.h	??
arm/sve1024/fmod.h	??
arm/sve128/fmod.h	??
arm/sve2048/fmod.h	??
arm/sve256/fmod.h	??
arm/sve512/fmod.h	??
cpu/cpu/fmod.h	??
ppc/vmx/fmod.h	??
ppc/vsx/fmod.h	??
x86/avx/fmod.h	??
x86/avx2/fmod.h	??
x86/avx512_knl/fmod.h	??
x86/avx512_skylake/fmod.h	??
x86/sse2/fmod.h	??
x86/sse42/fmod.h	??
arm/aarch64/fms.h	??
arm/neon128/fms.h	??
arm/sve/fms.h	??
arm/sve1024/fms.h	??
arm/sve128/fms.h	??
arm/sve2048/fms.h	??
arm/sve256/fms.h	??
arm/sve512/fms.h	??
cpu/cpu/fms.h	??
ppc/vmx/fms.h	??
ppc/vsx/fms.h	??
x86/avx/fms.h	??
x86/avx2/fms.h	??
x86/avx512_knl/fms.h	??
x86/avx512_skylake/fms.h	??
x86/sse2/fms.h	??
x86/sse42/fms.h	??
arm/aarch64/fnma.h	??
arm/neon128/fnma.h	??
arm/sve/fnma.h	??
arm/sve1024/fnma.h	??
arm/sve128/fnma.h	??
arm/sve2048/fnma.h	??
arm/sve256/fnma.h	??
arm/sve512/fnma.h	??
cpu/cpu/fnma.h	??
ppc/vmx/fnma.h	??
ppc/vsx/fnma.h	??
x86/avx/fnma.h	??
x86/avx2/fnma.h	??
x86/avx512_knl/fnma.h	??
x86/avx512_skylake/fnma.h	??
x86/sse2/fnma.h	??
x86/sse42/fnma.h	??
arm/aarch64/fnms.h	??
arm/neon128/fnms.h	??
arm/sve/fnms.h	??
arm/sve1024/fnms.h	??
arm/sve128/fnms.h	??

arm/sve2048/fnms.h	??
arm/sve256/fnms.h	??
arm/sve512/fnms.h	??
cpu/cpu/fnms.h	??
ppc/vmx/fnms.h	??
ppc/vsx/fnms.h	??
x86/avx/fnms.h	??
x86/avx2/fnms.h	??
x86/avx512_knl/fnms.h	??
x86/avx512_skylake/fnms.h	??
x86/sse2/fnms.h	??
x86/sse42/fnms.h	??
functions.h	??
arm/aarch64/gather.h	??
arm/neon128/gather.h	??
arm/sve/gather.h	??
arm/sve1024/gather.h	??
arm/sve128/gather.h	??
arm/sve2048/gather.h	??
arm/sve256/gather.h	??
arm/sve512/gather.h	??
cpu/cpu/gather.h	??
ppc/vmx/gather.h	??
ppc/vsx/gather.h	??
x86/avx/gather.h	??
x86/avx2/gather.h	??
x86/avx512_knl/gather.h	??
x86/avx512_skylake/gather.h	??
x86/sse2/gather.h	??
x86/sse42/gather.h	??
arm/aarch64/gather_linear.h	??
arm/neon128/gather_linear.h	??
arm/sve/gather_linear.h	??
arm/sve1024/gather_linear.h	??
arm/sve128/gather_linear.h	??
arm/sve2048/gather_linear.h	??
arm/sve256/gather_linear.h	??
arm/sve512/gather_linear.h	??
cpu/cpu/gather_linear.h	??
ppc/vmx/gather_linear.h	??
ppc/vsx/gather_linear.h	??
x86/avx/gather_linear.h	??
x86/avx2/gather_linear.h	??
x86/avx512_knl/gather_linear.h	??
x86/avx512_skylake/gather_linear.h	??
x86/sse2/gather_linear.h	??
x86/sse42/gather_linear.h	??
arm/aarch64/ge.h	??
arm/neon128/ge.h	??
arm/sve/ge.h	??
arm/sve1024/ge.h	??
arm/sve128/ge.h	??
arm/sve2048/ge.h	??
arm/sve256/ge.h	??
arm/sve512/ge.h	??
cpu/cpu/ge.h	??
ppc/vmx/ge.h	??
ppc/vsx/ge.h	??

x86/avx/ge.h	??
x86/avx2/ge.h	??
x86/avx512_knl/ge.h	??
x86/avx512_skylake/ge.h	??
x86/sse2/ge.h	??
x86/sse42/ge.h	??
arm/aarch64/gt.h	??
arm/neon128/gt.h	??
arm/sve/gt.h	??
arm/sve1024/gt.h	??
arm/sve128/gt.h	??
arm/sve2048/gt.h	??
arm/sve256/gt.h	??
arm/sve512/gt.h	??
cpu/cpu/gt.h	??
ppc/vmx/gt.h	??
ppc/vsx/gt.h	??
x86/avx/gt.h	??
x86/avx2/gt.h	??
x86/avx512_knl/gt.h	??
x86/avx512_skylake/gt.h	??
x86/sse2/gt.h	??
x86/sse42/gt.h	??
arm/aarch64/hypot_u05.h	??
arm/neon128/hypot_u05.h	??
arm/sve/hypot_u05.h	??
arm/sve1024/hypot_u05.h	??
arm/sve128/hypot_u05.h	??
arm/sve2048/hypot_u05.h	??
arm/sve256/hypot_u05.h	??
arm/sve512/hypot_u05.h	??
cpu/cpu/hypot_u05.h	??
ppc/vmx/hypot_u05.h	??
ppc/vsx/hypot_u05.h	??
x86/avx/hypot_u05.h	??
x86/avx2/hypot_u05.h	??
x86/avx512_knl/hypot_u05.h	??
x86/avx512_skylake/hypot_u05.h	??
x86/sse2/hypot_u05.h	??
x86/sse42/hypot_u05.h	??
arm/aarch64/hypot_u35.h	??
arm/neon128/hypot_u35.h	??
arm/sve/hypot_u35.h	??
arm/sve1024/hypot_u35.h	??
arm/sve128/hypot_u35.h	??
arm/sve2048/hypot_u35.h	??
arm/sve256/hypot_u35.h	??
arm/sve512/hypot_u35.h	??
cpu/cpu/hypot_u35.h	??
ppc/vmx/hypot_u35.h	??
ppc/vsx/hypot_u35.h	??
x86/avx/hypot_u35.h	??
x86/avx2/hypot_u35.h	??
x86/avx512_knl/hypot_u35.h	??
x86/avx512_skylake/hypot_u35.h	??
x86/sse2/hypot_u35.h	??
x86/sse42/hypot_u35.h	??
arm/aarch64/if_else1.h	??

arm/neon128/if_else1.h	??
arm/sve/if_else1.h	??
arm/sve1024/if_else1.h	??
arm/sve128/if_else1.h	??
arm/sve2048/if_else1.h	??
arm/sve256/if_else1.h	??
arm/sve512/if_else1.h	??
cpu/cpu/if_else1.h	??
ppc/vmx/if_else1.h	??
ppc/vsx/if_else1.h	??
x86/avx/if_else1.h	??
x86/avx2/if_else1.h	??
x86/avx512_knl/if_else1.h	??
x86/avx512_skylake/if_else1.h	??
x86/sse2/if_else1.h	??
x86/sse42/if_else1.h	??
arm/aarch64/iota.h	??
arm/neon128/iota.h	??
arm/sve/iota.h	??
arm/sve1024/iota.h	??
arm/sve128/iota.h	??
arm/sve2048/iota.h	??
arm/sve256/iota.h	??
arm/sve512/iota.h	??
cpu/cpu/iota.h	??
ppc/vmx/iota.h	??
ppc/vsx/iota.h	??
x86/avx/iota.h	??
x86/avx2/iota.h	??
x86/avx512_knl/iota.h	??
x86/avx512_skylake/iota.h	??
x86/sse2/iota.h	??
x86/sse42/iota.h	??
arm/aarch64/le.h	??
arm/neon128/le.h	??
arm/sve/le.h	??
arm/sve1024/le.h	??
arm/sve128/le.h	??
arm/sve2048/le.h	??
arm/sve256/le.h	??
arm/sve512/le.h	??
cpu/cpu/le.h	??
ppc/vmx/le.h	??
ppc/vsx/le.h	??
x86/avx/le.h	??
x86/avx2/le.h	??
x86/avx512_knl/le.h	??
x86/avx512_skylake/le.h	??
x86/sse2/le.h	??
x86/sse42/le.h	??
arm/aarch64/len.h	??
arm/neon128/len.h	??
arm/sve/len.h	??
arm/sve1024/len.h	??
arm/sve128/len.h	??
arm/sve2048/len.h	??
arm/sve256/len.h	??
arm/sve512/len.h	??

cpu/cpu/len.h	??
ppc/vmx/len.h	??
ppc/vsx/len.h	??
x86/avx/len.h	??
x86/avx2/len.h	??
x86/avx512_knl/len.h	??
x86/avx512_skylake/len.h	??
x86/sse2/len.h	??
x86/sse42/len.h	??
arm/aarch64/lgamma_u10.h	??
arm/neon128/lgamma_u10.h	??
arm/sve/lgamma_u10.h	??
arm/sve1024/lgamma_u10.h	??
arm/sve128/lgamma_u10.h	??
arm/sve2048/lgamma_u10.h	??
arm/sve256/lgamma_u10.h	??
arm/sve512/lgamma_u10.h	??
cpu/cpu/lgamma_u10.h	??
ppc/vmx/lgamma_u10.h	??
ppc/vsx/lgamma_u10.h	??
x86/avx/lgamma_u10.h	??
x86/avx2/lgamma_u10.h	??
x86/avx512_knl/lgamma_u10.h	??
x86/avx512_skylake/lgamma_u10.h	??
x86/sse2/lgamma_u10.h	??
x86/sse42/lgamma_u10.h	??
arm/aarch64/load2a.h	??
arm/neon128/load2a.h	??
arm/sve/load2a.h	??
arm/sve1024/load2a.h	??
arm/sve128/load2a.h	??
arm/sve2048/load2a.h	??
arm/sve256/load2a.h	??
arm/sve512/load2a.h	??
cpu/cpu/load2a.h	??
ppc/vmx/load2a.h	??
ppc/vsx/load2a.h	??
x86/avx/load2a.h	??
x86/avx2/load2a.h	??
x86/avx512_knl/load2a.h	??
x86/avx512_skylake/load2a.h	??
x86/sse2/load2a.h	??
x86/sse42/load2a.h	??
arm/aarch64/load2u.h	??
arm/neon128/load2u.h	??
arm/sve/load2u.h	??
arm/sve1024/load2u.h	??
arm/sve128/load2u.h	??
arm/sve2048/load2u.h	??
arm/sve256/load2u.h	??
arm/sve512/load2u.h	??
cpu/cpu/load2u.h	??
ppc/vmx/load2u.h	??
ppc/vsx/load2u.h	??
x86/avx/load2u.h	??
x86/avx2/load2u.h	??
x86/avx512_knl/load2u.h	??
x86/avx512_skylake/load2u.h	??

x86/sse2/load2u.h	??
x86/sse42/load2u.h	??
arm/aarch64/load3a.h	??
arm/neon128/load3a.h	??
arm/sve/load3a.h	??
arm/sve1024/load3a.h	??
arm/sve128/load3a.h	??
arm/sve2048/load3a.h	??
arm/sve256/load3a.h	??
arm/sve512/load3a.h	??
cpu/cpu/load3a.h	??
ppc/vmx/load3a.h	??
ppc/vsx/load3a.h	??
x86/avx/load3a.h	??
x86/avx2/load3a.h	??
x86/avx512_knl/load3a.h	??
x86/avx512_skylake/load3a.h	??
x86/sse2/load3a.h	??
x86/sse42/load3a.h	??
arm/aarch64/load3u.h	??
arm/neon128/load3u.h	??
arm/sve/load3u.h	??
arm/sve1024/load3u.h	??
arm/sve128/load3u.h	??
arm/sve2048/load3u.h	??
arm/sve256/load3u.h	??
arm/sve512/load3u.h	??
cpu/cpu/load3u.h	??
ppc/vmx/load3u.h	??
ppc/vsx/load3u.h	??
x86/avx/load3u.h	??
x86/avx2/load3u.h	??
x86/avx512_knl/load3u.h	??
x86/avx512_skylake/load3u.h	??
x86/sse2/load3u.h	??
x86/sse42/load3u.h	??
arm/aarch64/load4a.h	??
arm/neon128/load4a.h	??
arm/sve/load4a.h	??
arm/sve1024/load4a.h	??
arm/sve128/load4a.h	??
arm/sve2048/load4a.h	??
arm/sve256/load4a.h	??
arm/sve512/load4a.h	??
cpu/cpu/load4a.h	??
ppc/vmx/load4a.h	??
ppc/vsx/load4a.h	??
x86/avx/load4a.h	??
x86/avx2/load4a.h	??
x86/avx512_knl/load4a.h	??
x86/avx512_skylake/load4a.h	??
x86/sse2/load4a.h	??
x86/sse42/load4a.h	??
arm/aarch64/load4u.h	??
arm/neon128/load4u.h	??
arm/sve/load4u.h	??
arm/sve1024/load4u.h	??
arm/sve128/load4u.h	??

arm/sve2048/load4u.h	??
arm/sve256/load4u.h	??
arm/sve512/load4u.h	??
cpu/cpu/load4u.h	??
ppc/vmx/load4u.h	??
ppc/vsx/load4u.h	??
x86/avx/load4u.h	??
x86/avx2/load4u.h	??
x86/avx512_knl/load4u.h	??
x86/avx512_skylake/load4u.h	??
x86/sse2/load4u.h	??
x86/sse42/load4u.h	??
arm/aarch64/loada.h	??
arm/neon128/loada.h	??
arm/sve/loada.h	??
arm/sve1024/loada.h	??
arm/sve128/loada.h	??
arm/sve2048/loada.h	??
arm/sve256/loada.h	??
arm/sve512/loada.h	??
cpu/cpu/loada.h	??
ppc/vmx/loada.h	??
ppc/vsx/loada.h	??
x86/avx/loada.h	??
x86/avx2/loada.h	??
x86/avx512_knl/loada.h	??
x86/avx512_skylake/loada.h	??
x86/sse2/loada.h	??
x86/sse42/loada.h	??
arm/aarch64/loadla.h	??
arm/neon128/loadla.h	??
arm/sve/loadla.h	??
arm/sve1024/loadla.h	??
arm/sve128/loadla.h	??
arm/sve2048/loadla.h	??
arm/sve256/loadla.h	??
arm/sve512/loadla.h	??
cpu/cpu/loadla.h	??
ppc/vmx/loadla.h	??
ppc/vsx/loadla.h	??
x86/avx/loadla.h	??
x86/avx2/loadla.h	??
x86/avx512_knl/loadla.h	??
x86/avx512_skylake/loadla.h	??
x86/sse2/loadla.h	??
x86/sse42/loadla.h	??
arm/aarch64/loadlu.h	??
arm/neon128/loadlu.h	??
arm/sve/loadlu.h	??
arm/sve1024/loadlu.h	??
arm/sve128/loadlu.h	??
arm/sve2048/loadlu.h	??
arm/sve256/loadlu.h	??
arm/sve512/loadlu.h	??
cpu/cpu/loadlu.h	??
ppc/vmx/loadlu.h	??
ppc/vsx/loadlu.h	??
x86/avx/loadlu.h	??

x86/avx2/loadlu.h	??
x86/avx512_knl/loadlu.h	??
x86/avx512_skylake/loadlu.h	??
x86/sse2/loadlu.h	??
x86/sse42/loadlu.h	??
arm/aarch64/loadu.h	??
arm/neon128/loadu.h	??
arm/sve/loadu.h	??
arm/sve1024/loadu.h	??
arm/sve128/loadu.h	??
arm/sve2048/loadu.h	??
arm/sve256/loadu.h	??
arm/sve512/loadu.h	??
cpu/cpu/loadu.h	??
ppc/vmx/loadu.h	??
ppc/vsx/loadu.h	??
x86/avx/loadu.h	??
x86/avx2/loadu.h	??
x86/avx512_knl/loadu.h	??
x86/avx512_skylake/loadu.h	??
x86/sse2/loadu.h	??
x86/sse42/loadu.h	??
arm/aarch64/log10_u10.h	??
arm/neon128/log10_u10.h	??
arm/sve/log10_u10.h	??
arm/sve1024/log10_u10.h	??
arm/sve128/log10_u10.h	??
arm/sve2048/log10_u10.h	??
arm/sve256/log10_u10.h	??
arm/sve512/log10_u10.h	??
cpu/cpu/log10_u10.h	??
ppc/vmx/log10_u10.h	??
ppc/vsx/log10_u10.h	??
x86/avx/log10_u10.h	??
x86/avx2/log10_u10.h	??
x86/avx512_knl/log10_u10.h	??
x86/avx512_skylake/log10_u10.h	??
x86/sse2/log10_u10.h	??
x86/sse42/log10_u10.h	??
arm/aarch64/log1p_u10.h	??
arm/neon128/log1p_u10.h	??
arm/sve/log1p_u10.h	??
arm/sve1024/log1p_u10.h	??
arm/sve128/log1p_u10.h	??
arm/sve2048/log1p_u10.h	??
arm/sve256/log1p_u10.h	??
arm/sve512/log1p_u10.h	??
cpu/cpu/log1p_u10.h	??
ppc/vmx/log1p_u10.h	??
ppc/vsx/log1p_u10.h	??
x86/avx/log1p_u10.h	??
x86/avx2/log1p_u10.h	??
x86/avx512_knl/log1p_u10.h	??
x86/avx512_skylake/log1p_u10.h	??
x86/sse2/log1p_u10.h	??
x86/sse42/log1p_u10.h	??
arm/aarch64/log2_u10.h	??
arm/neon128/log2_u10.h	??

arm/sve/log2_u10.h	??
arm/sve1024/log2_u10.h	??
arm/sve128/log2_u10.h	??
arm/sve2048/log2_u10.h	??
arm/sve256/log2_u10.h	??
arm/sve512/log2_u10.h	??
cpu/cpu/log2_u10.h	??
ppc/vmx/log2_u10.h	??
ppc/vsx/log2_u10.h	??
x86/avx/log2_u10.h	??
x86/avx2/log2_u10.h	??
x86/avx512_knl/log2_u10.h	??
x86/avx512_skylake/log2_u10.h	??
x86/sse2/log2_u10.h	??
x86/sse42/log2_u10.h	??
arm/aarch64/log2_u35.h	??
arm/neon128/log2_u35.h	??
arm/sve/log2_u35.h	??
arm/sve1024/log2_u35.h	??
arm/sve128/log2_u35.h	??
arm/sve2048/log2_u35.h	??
arm/sve256/log2_u35.h	??
arm/sve512/log2_u35.h	??
cpu/cpu/log2_u35.h	??
ppc/vmx/log2_u35.h	??
ppc/vsx/log2_u35.h	??
x86/avx/log2_u35.h	??
x86/avx2/log2_u35.h	??
x86/avx512_knl/log2_u35.h	??
x86/avx512_skylake/log2_u35.h	??
x86/sse2/log2_u35.h	??
x86/sse42/log2_u35.h	??
arm/aarch64/log_u10.h	??
arm/neon128/log_u10.h	??
arm/sve/log_u10.h	??
arm/sve1024/log_u10.h	??
arm/sve128/log_u10.h	??
arm/sve2048/log_u10.h	??
arm/sve256/log_u10.h	??
arm/sve512/log_u10.h	??
cpu/cpu/log_u10.h	??
ppc/vmx/log_u10.h	??
ppc/vsx/log_u10.h	??
x86/avx/log_u10.h	??
x86/avx2/log_u10.h	??
x86/avx512_knl/log_u10.h	??
x86/avx512_skylake/log_u10.h	??
x86/sse2/log_u10.h	??
x86/sse42/log_u10.h	??
arm/aarch64/log_u35.h	??
arm/neon128/log_u35.h	??
arm/sve/log_u35.h	??
arm/sve1024/log_u35.h	??
arm/sve128/log_u35.h	??
arm/sve2048/log_u35.h	??
arm/sve256/log_u35.h	??
arm/sve512/log_u35.h	??
cpu/cpu/log_u35.h	??

ppc/vmx/log_u35.h	??
ppc/vsx/log_u35.h	??
x86/avx/log_u35.h	??
x86/avx2/log_u35.h	??
x86/avx512_knl/log_u35.h	??
x86/avx512_skylake/log_u35.h	??
x86/sse2/log_u35.h	??
x86/sse42/log_u35.h	??
arm/aarch64/lt.h	??
arm/neon128/lt.h	??
arm/sve/lt.h	??
arm/sve1024/lt.h	??
arm/sve128/lt.h	??
arm/sve2048/lt.h	??
arm/sve256/lt.h	??
arm/sve512/lt.h	??
cpu/cpu/lt.h	??
ppc/vmx/lt.h	??
ppc/vsx/lt.h	??
x86/avx/lt.h	??
x86/avx2/lt.h	??
x86/avx512_knl/lt.h	??
x86/avx512_skylake/lt.h	??
x86/sse2/lt.h	??
x86/sse42/lt.h	??
arm/aarch64/mask_for_loop_tail.h	??
arm/neon128/mask_for_loop_tail.h	??
arm/sve/mask_for_loop_tail.h	??
arm/sve1024/mask_for_loop_tail.h	??
arm/sve128/mask_for_loop_tail.h	??
arm/sve2048/mask_for_loop_tail.h	??
arm/sve256/mask_for_loop_tail.h	??
arm/sve512/mask_for_loop_tail.h	??
cpu/cpu/mask_for_loop_tail.h	??
ppc/vmx/mask_for_loop_tail.h	??
ppc/vsx/mask_for_loop_tail.h	??
x86/avx/mask_for_loop_tail.h	??
x86/avx2/mask_for_loop_tail.h	??
x86/avx512_knl/mask_for_loop_tail.h	??
x86/avx512_skylake/mask_for_loop_tail.h	??
x86/sse2/mask_for_loop_tail.h	??
x86/sse42/mask_for_loop_tail.h	??
arm/aarch64/mask_storea1.h	??
arm/neon128/mask_storea1.h	??
arm/sve/mask_storea1.h	??
arm/sve1024/mask_storea1.h	??
arm/sve128/mask_storea1.h	??
arm/sve2048/mask_storea1.h	??
arm/sve256/mask_storea1.h	??
arm/sve512/mask_storea1.h	??
cpu/cpu/mask_storea1.h	??
ppc/vmx/mask_storea1.h	??
ppc/vsx/mask_storea1.h	??
x86/avx/mask_storea1.h	??
x86/avx2/mask_storea1.h	??
x86/avx512_knl/mask_storea1.h	??
x86/avx512_skylake/mask_storea1.h	??
x86/sse2/mask_storea1.h	??

x86/sse42/mask_storea1.h	??
arm/aarch64/mask_storeu1.h	??
arm/neon128/mask_storeu1.h	??
arm/sve/mask_storeu1.h	??
arm/sve1024/mask_storeu1.h	??
arm/sve128/mask_storeu1.h	??
arm/sve2048/mask_storeu1.h	??
arm/sve256/mask_storeu1.h	??
arm/sve512/mask_storeu1.h	??
cpu/cpu/mask_storeu1.h	??
ppc/vmx/mask_storeu1.h	??
ppc/vsx/mask_storeu1.h	??
x86/avx/mask_storeu1.h	??
x86/avx2/mask_storeu1.h	??
x86/avx512_knl/mask_storeu1.h	??
x86/avx512_skylake/mask_storeu1.h	??
x86/sse2/mask_storeu1.h	??
x86/sse42/mask_storeu1.h	??
arm/aarch64/masko_loada1.h	??
arm/neon128/masko_loada1.h	??
arm/sve/masko_loada1.h	??
arm/sve1024/masko_loada1.h	??
arm/sve128/masko_loada1.h	??
arm/sve2048/masko_loada1.h	??
arm/sve256/masko_loada1.h	??
arm/sve512/masko_loada1.h	??
cpu/cpu/masko_loada1.h	??
ppc/vmx/masko_loada1.h	??
ppc/vsx/masko_loada1.h	??
x86/avx/masko_loada1.h	??
x86/avx2/masko_loada1.h	??
x86/avx512_knl/masko_loada1.h	??
x86/avx512_skylake/masko_loada1.h	??
x86/sse2/masko_loada1.h	??
x86/sse42/masko_loada1.h	??
arm/aarch64/masko_loadu1.h	??
arm/neon128/masko_loadu1.h	??
arm/sve/masko_loadu1.h	??
arm/sve1024/masko_loadu1.h	??
arm/sve128/masko_loadu1.h	??
arm/sve2048/masko_loadu1.h	??
arm/sve256/masko_loadu1.h	??
arm/sve512/masko_loadu1.h	??
cpu/cpu/masko_loadu1.h	??
ppc/vmx/masko_loadu1.h	??
ppc/vsx/masko_loadu1.h	??
x86/avx/masko_loadu1.h	??
x86/avx2/masko_loadu1.h	??
x86/avx512_knl/masko_loadu1.h	??
x86/avx512_skylake/masko_loadu1.h	??
x86/sse2/masko_loadu1.h	??
x86/sse42/masko_loadu1.h	??
arm/aarch64/maskz_loada1.h	??
arm/neon128/maskz_loada1.h	??
arm/sve/maskz_loada1.h	??
arm/sve1024/maskz_loada1.h	??
arm/sve128/maskz_loada1.h	??
arm/sve2048/maskz_loada1.h	??

arm/sve256/maskz_loada1.h	??
arm/sve512/maskz_loada1.h	??
cpu/cpu/maskz_loada1.h	??
ppc/vmx/maskz_loada1.h	??
ppc/vsx/maskz_loada1.h	??
x86/avx/maskz_loada1.h	??
x86/avx2/maskz_loada1.h	??
x86/avx512_knl/maskz_loada1.h	??
x86/avx512_skylake/maskz_loada1.h	??
x86/sse2/maskz_loada1.h	??
x86/sse42/maskz_loada1.h	??
arm/aarch64/maskz_loadu1.h	??
arm/neon128/maskz_loadu1.h	??
arm/sve/maskz_loadu1.h	??
arm/sve1024/maskz_loadu1.h	??
arm/sve128/maskz_loadu1.h	??
arm/sve2048/maskz_loadu1.h	??
arm/sve256/maskz_loadu1.h	??
arm/sve512/maskz_loadu1.h	??
cpu/cpu/maskz_loadu1.h	??
ppc/vmx/maskz_loadu1.h	??
ppc/vsx/maskz_loadu1.h	??
x86/avx/maskz_loadu1.h	??
x86/avx2/maskz_loadu1.h	??
x86/avx512_knl/maskz_loadu1.h	??
x86/avx512_skylake/maskz_loadu1.h	??
x86/sse2/maskz_loadu1.h	??
x86/sse42/maskz_loadu1.h	??
arm/aarch64/max.h	??
arm/neon128/max.h	??
arm/sve/max.h	??
arm/sve1024/max.h	??
arm/sve128/max.h	??
arm/sve2048/max.h	??
arm/sve256/max.h	??
arm/sve512/max.h	??
cpu/cpu/max.h	??
ppc/vmx/max.h	??
ppc/vsx/max.h	??
x86/avx/max.h	??
x86/avx2/max.h	??
x86/avx512_knl/max.h	??
x86/avx512_skylake/max.h	??
x86/sse2/max.h	??
x86/sse42/max.h	??
arm/aarch64/min.h	??
arm/neon128/min.h	??
arm/sve/min.h	??
arm/sve1024/min.h	??
arm/sve128/min.h	??
arm/sve2048/min.h	??
arm/sve256/min.h	??
arm/sve512/min.h	??
cpu/cpu/min.h	??
ppc/vmx/min.h	??
ppc/vsx/min.h	??
x86/avx/min.h	??
x86/avx2/min.h	??

x86/avx512_knl/min.h	??
x86/avx512_skylake/min.h	??
x86/sse2/min.h	??
x86/sse42/min.h	??
arm/aarch64/mul.h	??
arm/neon128/mul.h	??
arm/sve/mul.h	??
arm/sve1024/mul.h	??
arm/sve128/mul.h	??
arm/sve2048/mul.h	??
arm/sve256/mul.h	??
arm/sve512/mul.h	??
cpu/cpu/mul.h	??
ppc/vmx/mul.h	??
ppc/vsx/mul.h	??
x86/avx/mul.h	??
x86/avx2/mul.h	??
x86/avx512_knl/mul.h	??
x86/avx512_skylake/mul.h	??
x86/sse2/mul.h	??
x86/sse42/mul.h	??
arm/aarch64/nbtrue.h	??
arm/neon128/nbtrue.h	??
arm/sve/nbtrue.h	??
arm/sve1024/nbtrue.h	??
arm/sve128/nbtrue.h	??
arm/sve2048/nbtrue.h	??
arm/sve256/nbtrue.h	??
arm/sve512/nbtrue.h	??
cpu/cpu/nbtrue.h	??
ppc/vmx/nbtrue.h	??
ppc/vsx/nbtrue.h	??
x86/avx/nbtrue.h	??
x86/avx2/nbtrue.h	??
x86/avx512_knl/nbtrue.h	??
x86/avx512_skylake/nbtrue.h	??
x86/sse2/nbtrue.h	??
x86/sse42/nbtrue.h	??
arm/aarch64/ne.h	??
arm/neon128/ne.h	??
arm/sve/ne.h	??
arm/sve1024/ne.h	??
arm/sve128/ne.h	??
arm/sve2048/ne.h	??
arm/sve256/ne.h	??
arm/sve512/ne.h	??
cpu/cpu/ne.h	??
ppc/vmx/ne.h	??
ppc/vsx/ne.h	??
x86/avx/ne.h	??
x86/avx2/ne.h	??
x86/avx512_knl/ne.h	??
x86/avx512_skylake/ne.h	??
x86/sse2/ne.h	??
x86/sse42/ne.h	??
arm/aarch64/neg.h	??
arm/neon128/neg.h	??
arm/sve/neg.h	??

arm/sve1024/neg.h	??
arm/sve128/neg.h	??
arm/sve2048/neg.h	??
arm/sve256/neg.h	??
arm/sve512/neg.h	??
cpu/cpu/neg.h	??
ppc/vmx/neg.h	??
ppc/vsx/neg.h	??
x86/avx/neg.h	??
x86/avx2/neg.h	??
x86/avx512_knl/neg.h	??
x86/avx512_skylake/neg.h	??
x86/sse2/neg.h	??
x86/sse42/neg.h	??
arm/aarch64/notb.h	??
arm/neon128/notb.h	??
arm/sve/notb.h	??
arm/sve1024/notb.h	??
arm/sve128/notb.h	??
arm/sve2048/notb.h	??
arm/sve256/notb.h	??
arm/sve512/notb.h	??
cpu/cpu/notb.h	??
ppc/vmx/notb.h	??
ppc/vsx/notb.h	??
x86/avx/notb.h	??
x86/avx2/notb.h	??
x86/avx512_knl/notb.h	??
x86/avx512_skylake/notb.h	??
x86/sse2/notb.h	??
x86/sse42/notb.h	??
arm/aarch64/notl.h	??
arm/neon128/notl.h	??
arm/sve/notl.h	??
arm/sve1024/notl.h	??
arm/sve128/notl.h	??
arm/sve2048/notl.h	??
arm/sve256/notl.h	??
arm/sve512/notl.h	??
cpu/cpu/notl.h	??
ppc/vmx/notl.h	??
ppc/vsx/notl.h	??
x86/avx/notl.h	??
x86/avx2/notl.h	??
x86/avx512_knl/notl.h	??
x86/avx512_skylake/notl.h	??
x86/sse2/notl.h	??
x86/sse42/notl.h	??
nsimd-all.h	??
nsimd.h	??
arm/aarch64/orb.h	??
arm/neon128/orb.h	??
arm/sve/orb.h	??
arm/sve1024/orb.h	??
arm/sve128/orb.h	??
arm/sve2048/orb.h	??
arm/sve256/orb.h	??
arm/sve512/orb.h	??

cpu/cpu/orb.h	??
ppc/vmx/orb.h	??
ppc/vsx/orb.h	??
x86/avx/orb.h	??
x86/avx2/orb.h	??
x86/avx512_knl/orb.h	??
x86/avx512_skylake/orb.h	??
x86/sse2/orb.h	??
x86/sse42/orb.h	??
arm/aarch64/orl.h	??
arm/neon128/orl.h	??
arm/sve/orl.h	??
arm/sve1024/orl.h	??
arm/sve128/orl.h	??
arm/sve2048/orl.h	??
arm/sve256/orl.h	??
arm/sve512/orl.h	??
cpu/cpu/orl.h	??
ppc/vmx/orl.h	??
ppc/vsx/orl.h	??
x86/avx/orl.h	??
x86/avx2/orl.h	??
x86/avx512_knl/orl.h	??
x86/avx512_skylake/orl.h	??
x86/sse2/orl.h	??
x86/sse42/orl.h	??
PFNC.h	??
arm/aarch64/pow_u10.h	??
arm/neon128/pow_u10.h	??
arm/sve/pow_u10.h	??
arm/sve1024/pow_u10.h	??
arm/sve128/pow_u10.h	??
arm/sve2048/pow_u10.h	??
arm/sve256/pow_u10.h	??
arm/sve512/pow_u10.h	??
cpu/cpu/pow_u10.h	??
ppc/vmx/pow_u10.h	??
ppc/vsx/pow_u10.h	??
x86/avx/pow_u10.h	??
x86/avx2/pow_u10.h	??
x86/avx512_knl/pow_u10.h	??
x86/avx512_skylake/pow_u10.h	??
x86/sse2/pow_u10.h	??
x86/sse42/pow_u10.h	??
arm/aarch64/put.h	??
arm/neon128/put.h	??
arm/sve/put.h	??
arm/sve1024/put.h	??
arm/sve128/put.h	??
arm/sve2048/put.h	??
arm/sve256/put.h	??
arm/sve512/put.h	??
cpu/cpu/put.h	??
ppc/vmx/put.h	??
ppc/vsx/put.h	??
x86/avx/put.h	??
x86/avx2/put.h	??
x86/avx512_knl/put.h	??

x86/avx512_skylake/put.h	??
x86/sse2/put.h	??
x86/sse42/put.h	??
arm/aarch64/rec.h	??
arm/neon128/rec.h	??
arm/sve/rec.h	??
arm/sve1024/rec.h	??
arm/sve128/rec.h	??
arm/sve2048/rec.h	??
arm/sve256/rec.h	??
arm/sve512/rec.h	??
cpu/cpu/rec.h	??
ppc/vmx/rec.h	??
ppc/vsx/rec.h	??
x86/avx/rec.h	??
x86/avx2/rec.h	??
x86/avx512_knl/rec.h	??
x86/avx512_skylake/rec.h	??
x86/sse2/rec.h	??
x86/sse42/rec.h	??
arm/aarch64/rec11.h	??
arm/neon128/rec11.h	??
arm/sve/rec11.h	??
arm/sve1024/rec11.h	??
arm/sve128/rec11.h	??
arm/sve2048/rec11.h	??
arm/sve256/rec11.h	??
arm/sve512/rec11.h	??
cpu/cpu/rec11.h	??
ppc/vmx/rec11.h	??
ppc/vsx/rec11.h	??
x86/avx/rec11.h	??
x86/avx2/rec11.h	??
x86/avx512_knl/rec11.h	??
x86/avx512_skylake/rec11.h	??
x86/sse2/rec11.h	??
x86/sse42/rec11.h	??
arm/aarch64/rec8.h	??
arm/neon128/rec8.h	??
arm/sve/rec8.h	??
arm/sve1024/rec8.h	??
arm/sve128/rec8.h	??
arm/sve2048/rec8.h	??
arm/sve256/rec8.h	??
arm/sve512/rec8.h	??
cpu/cpu/rec8.h	??
ppc/vmx/rec8.h	??
ppc/vsx/rec8.h	??
x86/avx/rec8.h	??
x86/avx2/rec8.h	??
x86/avx512_knl/rec8.h	??
x86/avx512_skylake/rec8.h	??
x86/sse2/rec8.h	??
x86/sse42/rec8.h	??
arm/aarch64/reinterpret.h	??
arm/neon128/reinterpret.h	??
arm/sve/reinterpret.h	??
arm/sve1024/reinterpret.h	??

arm/sve128/reinterpret.h	??
arm/sve2048/reinterpret.h	??
arm/sve256/reinterpret.h	??
arm/sve512/reinterpret.h	??
cpu/cpu/reinterpret.h	??
ppc/vmx/reinterpret.h	??
ppc/vsx/reinterpret.h	??
x86/avx/reinterpret.h	??
x86/avx2/reinterpret.h	??
x86/avx512_knl/reinterpret.h	??
x86/avx512_skylake/reinterpret.h	??
x86/sse2/reinterpret.h	??
x86/sse42/reinterpret.h	??
arm/aarch64/reinterpretl.h	??
arm/neon128/reinterpretl.h	??
arm/sve/reinterpretl.h	??
arm/sve1024/reinterpretl.h	??
arm/sve128/reinterpretl.h	??
arm/sve2048/reinterpretl.h	??
arm/sve256/reinterpretl.h	??
arm/sve512/reinterpretl.h	??
cpu/cpu/reinterpretl.h	??
ppc/vmx/reinterpretl.h	??
ppc/vsx/reinterpretl.h	??
x86/avx/reinterpretl.h	??
x86/avx2/reinterpretl.h	??
x86/avx512_knl/reinterpretl.h	??
x86/avx512_skylake/reinterpretl.h	??
x86/sse2/reinterpretl.h	??
x86/sse42/reinterpretl.h	??
arm/aarch64/remainder.h	??
arm/neon128/remainder.h	??
arm/sve/remainder.h	??
arm/sve1024/remainder.h	??
arm/sve128/remainder.h	??
arm/sve2048/remainder.h	??
arm/sve256/remainder.h	??
arm/sve512/remainder.h	??
cpu/cpu/remainder.h	??
ppc/vmx/remainder.h	??
ppc/vsx/remainder.h	??
x86/avx/remainder.h	??
x86/avx2/remainder.h	??
x86/avx512_knl/remainder.h	??
x86/avx512_skylake/remainder.h	??
x86/sse2/remainder.h	??
x86/sse42/remainder.h	??
arm/aarch64/round_to_even.h	??
arm/neon128/round_to_even.h	??
arm/sve/round_to_even.h	??
arm/sve1024/round_to_even.h	??
arm/sve128/round_to_even.h	??
arm/sve2048/round_to_even.h	??
arm/sve256/round_to_even.h	??
arm/sve512/round_to_even.h	??
cpu/cpu/round_to_even.h	??
ppc/vmx/round_to_even.h	??
ppc/vsx/round_to_even.h	??

x86/avx/round_to_even.h	??
x86/avx2/round_to_even.h	??
x86/avx512_knl/round_to_even.h	??
x86/avx512_skylake/round_to_even.h	??
x86/sse2/round_to_even.h	??
x86/sse42/round_to_even.h	??
arm/aarch64/rsqrt11.h	??
arm/neon128/rsqrt11.h	??
arm/sve/rsqrt11.h	??
arm/sve1024/rsqrt11.h	??
arm/sve128/rsqrt11.h	??
arm/sve2048/rsqrt11.h	??
arm/sve256/rsqrt11.h	??
arm/sve512/rsqrt11.h	??
cpu/cpu/rsqrt11.h	??
ppc/vmx/rsqrt11.h	??
ppc/vsx/rsqrt11.h	??
x86/avx/rsqrt11.h	??
x86/avx2/rsqrt11.h	??
x86/avx512_knl/rsqrt11.h	??
x86/avx512_skylake/rsqrt11.h	??
x86/sse2/rsqrt11.h	??
x86/sse42/rsqrt11.h	??
arm/aarch64/rsqrt8.h	??
arm/neon128/rsqrt8.h	??
arm/sve/rsqrt8.h	??
arm/sve1024/rsqrt8.h	??
arm/sve128/rsqrt8.h	??
arm/sve2048/rsqrt8.h	??
arm/sve256/rsqrt8.h	??
arm/sve512/rsqrt8.h	??
cpu/cpu/rsqrt8.h	??
ppc/vmx/rsqrt8.h	??
ppc/vsx/rsqrt8.h	??
x86/avx/rsqrt8.h	??
x86/avx2/rsqrt8.h	??
x86/avx512_knl/rsqrt8.h	??
x86/avx512_skylake/rsqrt8.h	??
x86/sse2/rsqrt8.h	??
x86/sse42/rsqrt8.h	??
scalar_utilities.h	??
arm/aarch64/scatter.h	??
arm/neon128/scatter.h	??
arm/sve/scatter.h	??
arm/sve1024/scatter.h	??
arm/sve128/scatter.h	??
arm/sve2048/scatter.h	??
arm/sve256/scatter.h	??
arm/sve512/scatter.h	??
cpu/cpu/scatter.h	??
ppc/vmx/scatter.h	??
ppc/vsx/scatter.h	??
x86/avx/scatter.h	??
x86/avx2/scatter.h	??
x86/avx512_knl/scatter.h	??
x86/avx512_skylake/scatter.h	??
x86/sse2/scatter.h	??
x86/sse42/scatter.h	??

arm/aarch64/scatter_linear.h	??
arm/neon128/scatter_linear.h	??
arm/sve/scatter_linear.h	??
arm/sve1024/scatter_linear.h	??
arm/sve128/scatter_linear.h	??
arm/sve2048/scatter_linear.h	??
arm/sve256/scatter_linear.h	??
arm/sve512/scatter_linear.h	??
cpu/cpu/scatter_linear.h	??
ppc/vmx/scatter_linear.h	??
ppc/vsx/scatter_linear.h	??
x86/avx/scatter_linear.h	??
x86/avx2/scatter_linear.h	??
x86/avx512_knl/scatter_linear.h	??
x86/avx512_skylake/scatter_linear.h	??
x86/sse2/scatter_linear.h	??
x86/sse42/scatter_linear.h	??
arm/aarch64/set1.h	??
arm/neon128/set1.h	??
arm/sve/set1.h	??
arm/sve1024/set1.h	??
arm/sve128/set1.h	??
arm/sve2048/set1.h	??
arm/sve256/set1.h	??
arm/sve512/set1.h	??
cpu/cpu/set1.h	??
ppc/vmx/set1.h	??
ppc/vsx/set1.h	??
x86/avx/set1.h	??
x86/avx2/set1.h	??
x86/avx512_knl/set1.h	??
x86/avx512_skylake/set1.h	??
x86/sse2/set1.h	??
x86/sse42/set1.h	??
arm/aarch64/set1l.h	??
arm/neon128/set1l.h	??
arm/sve/set1l.h	??
arm/sve1024/set1l.h	??
arm/sve128/set1l.h	??
arm/sve2048/set1l.h	??
arm/sve256/set1l.h	??
arm/sve512/set1l.h	??
cpu/cpu/set1l.h	??
ppc/vmx/set1l.h	??
ppc/vsx/set1l.h	??
x86/avx/set1l.h	??
x86/avx2/set1l.h	??
x86/avx512_knl/set1l.h	??
x86/avx512_skylake/set1l.h	??
x86/sse2/set1l.h	??
x86/sse42/set1l.h	??
arm/aarch64/shl.h	??
arm/neon128/shl.h	??
arm/sve/shl.h	??
arm/sve1024/shl.h	??
arm/sve128/shl.h	??
arm/sve2048/shl.h	??
arm/sve256/shl.h	??

arm/sve512/shl.h	??
cpu/cpu/shl.h	??
ppc/vmx/shl.h	??
ppc/vsx/shl.h	??
x86/avx/shl.h	??
x86/avx2/shl.h	??
x86/avx512_knl/shl.h	??
x86/avx512_skylake/shl.h	??
x86/sse2/shl.h	??
x86/sse42/shl.h	??
arm/aarch64/shr.h	??
arm/neon128/shr.h	??
arm/sve/shr.h	??
arm/sve1024/shr.h	??
arm/sve128/shr.h	??
arm/sve2048/shr.h	??
arm/sve256/shr.h	??
arm/sve512/shr.h	??
cpu/cpu/shr.h	??
ppc/vmx/shr.h	??
ppc/vsx/shr.h	??
x86/avx/shr.h	??
x86/avx2/shr.h	??
x86/avx512_knl/shr.h	??
x86/avx512_skylake/shr.h	??
x86/sse2/shr.h	??
x86/sse42/shr.h	??
arm/aarch64/shra.h	??
arm/neon128/shra.h	??
arm/sve/shra.h	??
arm/sve1024/shra.h	??
arm/sve128/shra.h	??
arm/sve2048/shra.h	??
arm/sve256/shra.h	??
arm/sve512/shra.h	??
cpu/cpu/shra.h	??
ppc/vmx/shra.h	??
ppc/vsx/shra.h	??
x86/avx/shra.h	??
x86/avx2/shra.h	??
x86/avx512_knl/shra.h	??
x86/avx512_skylake/shra.h	??
x86/sse2/shra.h	??
x86/sse42/shra.h	??
arm/aarch64/sin_u10.h	??
arm/neon128/sin_u10.h	??
arm/sve/sin_u10.h	??
arm/sve1024/sin_u10.h	??
arm/sve128/sin_u10.h	??
arm/sve2048/sin_u10.h	??
arm/sve256/sin_u10.h	??
arm/sve512/sin_u10.h	??
cpu/cpu/sin_u10.h	??
ppc/vmx/sin_u10.h	??
ppc/vsx/sin_u10.h	??
x86/avx/sin_u10.h	??
x86/avx2/sin_u10.h	??
x86/avx512_knl/sin_u10.h	??

x86/avx512_skylake/sin_u10.h	??
x86/sse2/sin_u10.h	??
x86/sse42/sin_u10.h	??
arm/aarch64/sin_u35.h	??
arm/neon128/sin_u35.h	??
arm/sve/sin_u35.h	??
arm/sve1024/sin_u35.h	??
arm/sve128/sin_u35.h	??
arm/sve2048/sin_u35.h	??
arm/sve256/sin_u35.h	??
arm/sve512/sin_u35.h	??
cpu/cpu/sin_u35.h	??
ppc/vmx/sin_u35.h	??
ppc/vsx/sin_u35.h	??
x86/avx/sin_u35.h	??
x86/avx2/sin_u35.h	??
x86/avx512_knl/sin_u35.h	??
x86/avx512_skylake/sin_u35.h	??
x86/sse2/sin_u35.h	??
x86/sse42/sin_u35.h	??
arm/aarch64/sinh_u10.h	??
arm/neon128/sinh_u10.h	??
arm/sve/sinh_u10.h	??
arm/sve1024/sinh_u10.h	??
arm/sve128/sinh_u10.h	??
arm/sve2048/sinh_u10.h	??
arm/sve256/sinh_u10.h	??
arm/sve512/sinh_u10.h	??
cpu/cpu/sinh_u10.h	??
ppc/vmx/sinh_u10.h	??
ppc/vsx/sinh_u10.h	??
x86/avx/sinh_u10.h	??
x86/avx2/sinh_u10.h	??
x86/avx512_knl/sinh_u10.h	??
x86/avx512_skylake/sinh_u10.h	??
x86/sse2/sinh_u10.h	??
x86/sse42/sinh_u10.h	??
arm/aarch64/sinh_u35.h	??
arm/neon128/sinh_u35.h	??
arm/sve/sinh_u35.h	??
arm/sve1024/sinh_u35.h	??
arm/sve128/sinh_u35.h	??
arm/sve2048/sinh_u35.h	??
arm/sve256/sinh_u35.h	??
arm/sve512/sinh_u35.h	??
cpu/cpu/sinh_u35.h	??
ppc/vmx/sinh_u35.h	??
ppc/vsx/sinh_u35.h	??
x86/avx/sinh_u35.h	??
x86/avx2/sinh_u35.h	??
x86/avx512_knl/sinh_u35.h	??
x86/avx512_skylake/sinh_u35.h	??
x86/sse2/sinh_u35.h	??
x86/sse42/sinh_u35.h	??
arm/aarch64/sinpi_u05.h	??
arm/neon128/sinpi_u05.h	??
arm/sve/sinpi_u05.h	??
arm/sve1024/sinpi_u05.h	??

arm/sve128/sinpi_u05.h	??
arm/sve2048/sinpi_u05.h	??
arm/sve256/sinpi_u05.h	??
arm/sve512/sinpi_u05.h	??
cpu/cpu/sinpi_u05.h	??
ppc/vmx/sinpi_u05.h	??
ppc/vsx/sinpi_u05.h	??
x86/avx/sinpi_u05.h	??
x86/avx2/sinpi_u05.h	??
x86/avx512_knl/sinpi_u05.h	??
x86/avx512_skylake/sinpi_u05.h	??
x86/sse2/sinpi_u05.h	??
x86/sse42/sinpi_u05.h	??
arm/aarch64/sqrt.h	??
arm/neon128/sqrt.h	??
arm/sve/sqrt.h	??
arm/sve1024/sqrt.h	??
arm/sve128/sqrt.h	??
arm/sve2048/sqrt.h	??
arm/sve256/sqrt.h	??
arm/sve512/sqrt.h	??
cpu/cpu/sqrt.h	??
ppc/vmx/sqrt.h	??
ppc/vsx/sqrt.h	??
x86/avx/sqrt.h	??
x86/avx2/sqrt.h	??
x86/avx512_knl/sqrt.h	??
x86/avx512_skylake/sqrt.h	??
x86/sse2/sqrt.h	??
x86/sse42/sqrt.h	??
arm/aarch64/store2a.h	??
arm/neon128/store2a.h	??
arm/sve/store2a.h	??
arm/sve1024/store2a.h	??
arm/sve128/store2a.h	??
arm/sve2048/store2a.h	??
arm/sve256/store2a.h	??
arm/sve512/store2a.h	??
cpu/cpu/store2a.h	??
ppc/vmx/store2a.h	??
ppc/vsx/store2a.h	??
x86/avx/store2a.h	??
x86/avx2/store2a.h	??
x86/avx512_knl/store2a.h	??
x86/avx512_skylake/store2a.h	??
x86/sse2/store2a.h	??
x86/sse42/store2a.h	??
arm/aarch64/store2u.h	??
arm/neon128/store2u.h	??
arm/sve/store2u.h	??
arm/sve1024/store2u.h	??
arm/sve128/store2u.h	??
arm/sve2048/store2u.h	??
arm/sve256/store2u.h	??
arm/sve512/store2u.h	??
cpu/cpu/store2u.h	??
ppc/vmx/store2u.h	??
ppc/vsx/store2u.h	??

x86/avx/store2u.h	??
x86/avx2/store2u.h	??
x86/avx512_knl/store2u.h	??
x86/avx512_skylake/store2u.h	??
x86/sse2/store2u.h	??
x86/sse42/store2u.h	??
arm/aarch64/store3a.h	??
arm/neon128/store3a.h	??
arm/sve/store3a.h	??
arm/sve1024/store3a.h	??
arm/sve128/store3a.h	??
arm/sve2048/store3a.h	??
arm/sve256/store3a.h	??
arm/sve512/store3a.h	??
cpu/cpu/store3a.h	??
ppc/vmx/store3a.h	??
ppc/vsx/store3a.h	??
x86/avx/store3a.h	??
x86/avx2/store3a.h	??
x86/avx512_knl/store3a.h	??
x86/avx512_skylake/store3a.h	??
x86/sse2/store3a.h	??
x86/sse42/store3a.h	??
arm/aarch64/store3u.h	??
arm/neon128/store3u.h	??
arm/sve/store3u.h	??
arm/sve1024/store3u.h	??
arm/sve128/store3u.h	??
arm/sve2048/store3u.h	??
arm/sve256/store3u.h	??
arm/sve512/store3u.h	??
cpu/cpu/store3u.h	??
ppc/vmx/store3u.h	??
ppc/vsx/store3u.h	??
x86/avx/store3u.h	??
x86/avx2/store3u.h	??
x86/avx512_knl/store3u.h	??
x86/avx512_skylake/store3u.h	??
x86/sse2/store3u.h	??
x86/sse42/store3u.h	??
arm/aarch64/store4a.h	??
arm/neon128/store4a.h	??
arm/sve/store4a.h	??
arm/sve1024/store4a.h	??
arm/sve128/store4a.h	??
arm/sve2048/store4a.h	??
arm/sve256/store4a.h	??
arm/sve512/store4a.h	??
cpu/cpu/store4a.h	??
ppc/vmx/store4a.h	??
ppc/vsx/store4a.h	??
x86/avx/store4a.h	??
x86/avx2/store4a.h	??
x86/avx512_knl/store4a.h	??
x86/avx512_skylake/store4a.h	??
x86/sse2/store4a.h	??
x86/sse42/store4a.h	??
arm/aarch64/store4u.h	??

arm/neon128/store4u.h	??
arm/sve/store4u.h	??
arm/sve1024/store4u.h	??
arm/sve128/store4u.h	??
arm/sve2048/store4u.h	??
arm/sve256/store4u.h	??
arm/sve512/store4u.h	??
cpu/cpu/store4u.h	??
ppc/vmx/store4u.h	??
ppc/vsx/store4u.h	??
x86/avx/store4u.h	??
x86/avx2/store4u.h	??
x86/avx512_knl/store4u.h	??
x86/avx512_skylake/store4u.h	??
x86/sse2/store4u.h	??
x86/sse42/store4u.h	??
arm/aarch64/storea.h	??
arm/neon128/storea.h	??
arm/sve/storea.h	??
arm/sve1024/storea.h	??
arm/sve128/storea.h	??
arm/sve2048/storea.h	??
arm/sve256/storea.h	??
arm/sve512/storea.h	??
cpu/cpu/storea.h	??
ppc/vmx/storea.h	??
ppc/vsx/storea.h	??
x86/avx/storea.h	??
x86/avx2/storea.h	??
x86/avx512_knl/storea.h	??
x86/avx512_skylake/storea.h	??
x86/sse2/storea.h	??
x86/sse42/storea.h	??
arm/aarch64/storela.h	??
arm/neon128/storela.h	??
arm/sve/storela.h	??
arm/sve1024/storela.h	??
arm/sve128/storela.h	??
arm/sve2048/storela.h	??
arm/sve256/storela.h	??
arm/sve512/storela.h	??
cpu/cpu/storela.h	??
ppc/vmx/storela.h	??
ppc/vsx/storela.h	??
x86/avx/storela.h	??
x86/avx2/storela.h	??
x86/avx512_knl/storela.h	??
x86/avx512_skylake/storela.h	??
x86/sse2/storela.h	??
x86/sse42/storela.h	??
arm/aarch64/storelu.h	??
arm/neon128/storelu.h	??
arm/sve/storelu.h	??
arm/sve1024/storelu.h	??
arm/sve128/storelu.h	??
arm/sve2048/storelu.h	??
arm/sve256/storelu.h	??
arm/sve512/storelu.h	??

cpu/cpu/storelu.h	??
ppc/vmx/storelu.h	??
ppc/vsx/storelu.h	??
x86/avx/storelu.h	??
x86/avx2/storelu.h	??
x86/avx512_knl/storelu.h	??
x86/avx512_skylake/storelu.h	??
x86/sse2/storelu.h	??
x86/sse42/storelu.h	??
arm/aarch64/storeu.h	??
arm/neon128/storeu.h	??
arm/sve/storeu.h	??
arm/sve1024/storeu.h	??
arm/sve128/storeu.h	??
arm/sve2048/storeu.h	??
arm/sve256/storeu.h	??
arm/sve512/storeu.h	??
cpu/cpu/storeu.h	??
ppc/vmx/storeu.h	??
ppc/vsx/storeu.h	??
x86/avx/storeu.h	??
x86/avx2/storeu.h	??
x86/avx512_knl/storeu.h	??
x86/avx512_skylake/storeu.h	??
x86/sse2/storeu.h	??
x86/sse42/storeu.h	??
arm/aarch64/sub.h	??
arm/neon128/sub.h	??
arm/sve/sub.h	??
arm/sve1024/sub.h	??
arm/sve128/sub.h	??
arm/sve2048/sub.h	??
arm/sve256/sub.h	??
arm/sve512/sub.h	??
cpu/cpu/sub.h	??
ppc/vmx/sub.h	??
ppc/vsx/sub.h	??
x86/avx/sub.h	??
x86/avx2/sub.h	??
x86/avx512_knl/sub.h	??
x86/avx512_skylake/sub.h	??
x86/sse2/sub.h	??
x86/sse42/sub.h	??
arm/aarch64/subs.h	??
arm/neon128/subs.h	??
arm/sve/subs.h	??
arm/sve1024/subs.h	??
arm/sve128/subs.h	??
arm/sve2048/subs.h	??
arm/sve256/subs.h	??
arm/sve512/subs.h	??
cpu/cpu/subs.h	??
ppc/vmx/subs.h	??
ppc/vsx/subs.h	??
x86/avx/subs.h	??
x86/avx2/subs.h	??
x86/avx512_knl/subs.h	??
x86/avx512_skylake/subs.h	??

x86/sse2/subs.h	??
x86/sse42/subs.h	??
arm/aarch64/tan_u10.h	??
arm/neon128/tan_u10.h	??
arm/sve/tan_u10.h	??
arm/sve1024/tan_u10.h	??
arm/sve128/tan_u10.h	??
arm/sve2048/tan_u10.h	??
arm/sve256/tan_u10.h	??
arm/sve512/tan_u10.h	??
cpu/cpu/tan_u10.h	??
ppc/vmx/tan_u10.h	??
ppc/vsx/tan_u10.h	??
x86/avx/tan_u10.h	??
x86/avx2/tan_u10.h	??
x86/avx512_knl/tan_u10.h	??
x86/avx512_skylake/tan_u10.h	??
x86/sse2/tan_u10.h	??
x86/sse42/tan_u10.h	??
arm/aarch64/tan_u35.h	??
arm/neon128/tan_u35.h	??
arm/sve/tan_u35.h	??
arm/sve1024/tan_u35.h	??
arm/sve128/tan_u35.h	??
arm/sve2048/tan_u35.h	??
arm/sve256/tan_u35.h	??
arm/sve512/tan_u35.h	??
cpu/cpu/tan_u35.h	??
ppc/vmx/tan_u35.h	??
ppc/vsx/tan_u35.h	??
x86/avx/tan_u35.h	??
x86/avx2/tan_u35.h	??
x86/avx512_knl/tan_u35.h	??
x86/avx512_skylake/tan_u35.h	??
x86/sse2/tan_u35.h	??
x86/sse42/tan_u35.h	??
arm/aarch64/tanh_u10.h	??
arm/neon128/tanh_u10.h	??
arm/sve/tanh_u10.h	??
arm/sve1024/tanh_u10.h	??
arm/sve128/tanh_u10.h	??
arm/sve2048/tanh_u10.h	??
arm/sve256/tanh_u10.h	??
arm/sve512/tanh_u10.h	??
cpu/cpu/tanh_u10.h	??
ppc/vmx/tanh_u10.h	??
ppc/vsx/tanh_u10.h	??
x86/avx/tanh_u10.h	??
x86/avx2/tanh_u10.h	??
x86/avx512_knl/tanh_u10.h	??
x86/avx512_skylake/tanh_u10.h	??
x86/sse2/tanh_u10.h	??
x86/sse42/tanh_u10.h	??
arm/aarch64/tanh_u35.h	??
arm/neon128/tanh_u35.h	??
arm/sve/tanh_u35.h	??
arm/sve1024/tanh_u35.h	??
arm/sve128/tanh_u35.h	??

arm/sve2048/tanh_u35.h	??
arm/sve256/tanh_u35.h	??
arm/sve512/tanh_u35.h	??
cpu/cpu/tanh_u35.h	??
ppc/vmx/tanh_u35.h	??
ppc/vsx/tanh_u35.h	??
x86/avx/tanh_u35.h	??
x86/avx2/tanh_u35.h	??
x86/avx512_knl/tanh_u35.h	??
x86/avx512_skylake/tanh_u35.h	??
x86/sse2/tanh_u35.h	??
x86/sse42/tanh_u35.h	??
arm/aarch64/tgamma_u10.h	??
arm/neon128/tgamma_u10.h	??
arm/sve/tgamma_u10.h	??
arm/sve1024/tgamma_u10.h	??
arm/sve128/tgamma_u10.h	??
arm/sve2048/tgamma_u10.h	??
arm/sve256/tgamma_u10.h	??
arm/sve512/tgamma_u10.h	??
cpu/cpu/tgamma_u10.h	??
ppc/vmx/tgamma_u10.h	??
ppc/vsx/tgamma_u10.h	??
x86/avx/tgamma_u10.h	??
x86/avx2/tgamma_u10.h	??
x86/avx512_knl/tgamma_u10.h	??
x86/avx512_skylake/tgamma_u10.h	??
x86/sse2/tgamma_u10.h	??
x86/sse42/tgamma_u10.h	??
arm/aarch64/to_logical.h	??
arm/neon128/to_logical.h	??
arm/sve/to_logical.h	??
arm/sve1024/to_logical.h	??
arm/sve128/to_logical.h	??
arm/sve2048/to_logical.h	??
arm/sve256/to_logical.h	??
arm/sve512/to_logical.h	??
cpu/cpu/to_logical.h	??
ppc/vmx/to_logical.h	??
ppc/vsx/to_logical.h	??
x86/avx/to_logical.h	??
x86/avx2/to_logical.h	??
x86/avx512_knl/to_logical.h	??
x86/avx512_skylake/to_logical.h	??
x86/sse2/to_logical.h	??
x86/sse42/to_logical.h	??
arm/aarch64/to_mask.h	??
arm/neon128/to_mask.h	??
arm/sve/to_mask.h	??
arm/sve1024/to_mask.h	??
arm/sve128/to_mask.h	??
arm/sve2048/to_mask.h	??
arm/sve256/to_mask.h	??
arm/sve512/to_mask.h	??
cpu/cpu/to_mask.h	??
ppc/vmx/to_mask.h	??
ppc/vsx/to_mask.h	??
x86/avx/to_mask.h	??

x86/avx2/to_mask.h	??
x86/avx512_knl/to_mask.h	??
x86/avx512_skylake/to_mask.h	??
x86/sse2/to_mask.h	??
x86/sse42/to_mask.h	??
arm/aarch64/trunc.h	??
arm/neon128/trunc.h	??
arm/sve/trunc.h	??
arm/sve1024/trunc.h	??
arm/sve128/trunc.h	??
arm/sve2048/trunc.h	??
arm/sve256/trunc.h	??
arm/sve512/trunc.h	??
cpu/cpu/trunc.h	??
ppc/vmx/trunc.h	??
ppc/vsx/trunc.h	??
x86/avx/trunc.h	??
x86/avx2/trunc.h	??
x86/avx512_knl/trunc.h	??
x86/avx512_skylake/trunc.h	??
x86/sse2/trunc.h	??
x86/sse42/trunc.h	??
TYApi.h	
TYApi.h includes camera control and data receiving interface, which supports configuration for image resolution, frame rate, exposure time, gain, working mode,etc	99
TYCAPL.h	??
TYCoordinateMapper.h	
Coordinate Conversion API	168
TYDefs.h	
TYDefs.h includes camera control and data receiving data definitions which supports configura- tion for image resolution, frame rate, exposure time, gain, working mode,etc	179
TYFeatureList.h	
Camera feature enumeration list	202
TYImageProc.h	219
TYPParameter.h	??
arm/aarch64/types.h	??
arm/neon128/types.h	??
arm/sve/types.h	??
arm/sve1024/types.h	??
arm/sve128/types.h	??
arm/sve2048/types.h	??
arm/sve256/types.h	??
arm/sve512/types.h	??
cpu/cpu/types.h	??
ppc/vmx/types.h	??
ppc/vsx/types.h	??
x86/avx/types.h	??
x86/avx2/types.h	??
x86/avx512_knl/types.h	??
x86/avx512_skylake/types.h	??
x86/sse2/types.h	??
x86/sse42/types.h	??
TYVer.h	??
arm/aarch64/unzip.h	??
arm/neon128/unzip.h	??
arm/sve/unzip.h	??

arm/sve1024/unzip.h	??
arm/sve128/unzip.h	??
arm/sve2048/unzip.h	??
arm/sve256/unzip.h	??
arm/sve512/unzip.h	??
cpu/cpu/unzip.h	??
ppc/vmx/unzip.h	??
ppc/vsx/unzip.h	??
x86/avx/unzip.h	??
x86/avx2/unzip.h	??
x86/avx512_knl/unzip.h	??
x86/avx512_skylake/unzip.h	??
x86/sse2/unzip.h	??
x86/sse42/unzip.h	??
arm/aarch64/unziphi.h	??
arm/neon128/unziphi.h	??
arm/sve/unziphi.h	??
arm/sve1024/unziphi.h	??
arm/sve128/unziphi.h	??
arm/sve2048/unziphi.h	??
arm/sve256/unziphi.h	??
arm/sve512/unziphi.h	??
cpu/cpu/unziphi.h	??
ppc/vmx/unziphi.h	??
ppc/vsx/unziphi.h	??
x86/avx/unziphi.h	??
x86/avx2/unziphi.h	??
x86/avx512_knl/unziphi.h	??
x86/avx512_skylake/unziphi.h	??
x86/sse2/unziphi.h	??
x86/sse42/unziphi.h	??
arm/aarch64/unziplo.h	??
arm/neon128/unziplo.h	??
arm/sve/unziplo.h	??
arm/sve1024/unziplo.h	??
arm/sve128/unziplo.h	??
arm/sve2048/unziplo.h	??
arm/sve256/unziplo.h	??
arm/sve512/unziplo.h	??
cpu/cpu/unziplo.h	??
ppc/vmx/unziplo.h	??
ppc/vsx/unziplo.h	??
x86/avx/unziplo.h	??
x86/avx2/unziplo.h	??
x86/avx512_knl/unziplo.h	??
x86/avx512_skylake/unziplo.h	??
x86/sse2/unziplo.h	??
x86/sse42/unziplo.h	??
arm/aarch64/upcvt.h	??
arm/neon128/upcvt.h	??
arm/sve/upcvt.h	??
arm/sve1024/upcvt.h	??
arm/sve128/upcvt.h	??
arm/sve2048/upcvt.h	??
arm/sve256/upcvt.h	??
arm/sve512/upcvt.h	??
cpu/cpu/upcvt.h	??
ppc/vmx/upcvt.h	??

ppc/vsx/upcvt.h	??
x86/avx/upcvt.h	??
x86/avx2/upcvt.h	??
x86/avx512_knl/upcvt.h	??
x86/avx512_skylake/upcvt.h	??
x86/sse2/upcvt.h	??
x86/sse42/upcvt.h	??
arm/aarch64/xorb.h	??
arm/neon128/xorb.h	??
arm/sve/xorb.h	??
arm/sve1024/xorb.h	??
arm/sve128/xorb.h	??
arm/sve2048/xorb.h	??
arm/sve256/xorb.h	??
arm/sve512/xorb.h	??
cpu/cpu/xorb.h	??
ppc/vmx/xorb.h	??
ppc/vsx/xorb.h	??
x86/avx/xorb.h	??
x86/avx2/xorb.h	??
x86/avx512_knl/xorb.h	??
x86/avx512_skylake/xorb.h	??
x86/sse2/xorb.h	??
x86/sse42/xorb.h	??
arm/aarch64/xorl.h	??
arm/neon128/xorl.h	??
arm/sve/xorl.h	??
arm/sve1024/xorl.h	??
arm/sve128/xorl.h	??
arm/sve2048/xorl.h	??
arm/sve256/xorl.h	??
arm/sve512/xorl.h	??
cpu/cpu/xorl.h	??
ppc/vmx/xorl.h	??
ppc/vsx/xorl.h	??
x86/avx/xorl.h	??
x86/avx2/xorl.h	??
x86/avx512_knl/xorl.h	??
x86/avx512_skylake/xorl.h	??
x86/sse2/xorl.h	??
x86/sse42/xorl.h	??
zconf.h	??
arm/aarch64/zip.h	??
arm/neon128/zip.h	??
arm/sve/zip.h	??
arm/sve1024/zip.h	??
arm/sve128/zip.h	??
arm/sve2048/zip.h	??
arm/sve256/zip.h	??
arm/sve512/zip.h	??
cpu/cpu/zip.h	??
ppc/vmx/zip.h	??
ppc/vsx/zip.h	??
x86/avx/zip.h	??
x86/avx2/zip.h	??
x86/avx512_knl/zip.h	??
x86/avx512_skylake/zip.h	??
x86/sse2/zip.h	??

x86/sse42/zip.h	??
arm/aarch64/ziphi.h	??
arm/neon128/ziphi.h	??
arm/sve/ziphi.h	??
arm/sve1024/ziphi.h	??
arm/sve128/ziphi.h	??
arm/sve2048/ziphi.h	??
arm/sve256/ziphi.h	??
arm/sve512/ziphi.h	??
cpu/cpu/ziphi.h	??
ppc/vmx/ziphi.h	??
ppc/vsx/ziphi.h	??
x86/avx/ziphi.h	??
x86/avx2/ziphi.h	??
x86/avx512_knl/ziphi.h	??
x86/avx512_skylake/ziphi.h	??
x86/sse2/ziphi.h	??
x86/sse42/ziphi.h	??
arm/aarch64/ziplo.h	??
arm/neon128/ziplo.h	??
arm/sve/ziplo.h	??
arm/sve1024/ziplo.h	??
arm/sve128/ziplo.h	??
arm/sve2048/ziplo.h	??
arm/sve256/ziplo.h	??
arm/sve512/ziplo.h	??
cpu/cpu/ziplo.h	??
ppc/vmx/ziplo.h	??
ppc/vsx/ziplo.h	??
x86/avx/ziplo.h	??
x86/avx2/ziplo.h	??
x86/avx512_knl/ziplo.h	??
x86/avx512_skylake/ziplo.h	??
x86/sse2/ziplo.h	??
x86/sse42/ziplo.h	??
zlib.h	??

Chapter 4

Class Documentation

4.1 AlgVersion Struct Reference

Public Attributes

- int32_t **major**
- int32_t **minor**
- int32_t **patch**
- int32_t **reserved**

4.1.1 Detailed Description

Definition at line 6 of file TYCAPL.h.

The documentation for this struct was generated from the following file:

- TYCAPL.h

4.2 DepthEnhenceParameters Struct Reference

Public Attributes

- float [sigma_s](#)
filter param on space
- float [sigma_r](#)
filter param on range
- int [outlier_win_sz](#)
outlier filter windows ize
- float **outlier_rate**

4.2.1 Detailed Description

Definition at line 96 of file TYImageProc.h.

The documentation for this struct was generated from the following file:

- [TYImageProc.h](#)

4.3 DepthInpainterParameters Struct Reference

Public Attributes

- int **kernel_size**
- int **max_internal_hole**

4.3.1 Detailed Description

Definition at line 77 of file TYImageProc.h.

The documentation for this struct was generated from the following file:

- [TYImageProc.h](#)

4.4 DepthSpeckleFilterParameters Struct Reference

Public Attributes

- int **max_speckle_size**
- int **max_speckle_diff**
- float **max_physical_size**

4.4.1 Detailed Description

Definition at line 54 of file TYImageProc.h.

The documentation for this struct was generated from the following file:

- [TYImageProc.h](#)

4.5 f16 Struct Reference

Public Attributes

- u16 **u**

4.5.1 Detailed Description

Definition at line 986 of file nsimd.h.

The documentation for this struct was generated from the following file:

- nsimd.h

4.6 gz_header_s Struct Reference

Public Attributes

- int **text**
- uLong **time**
- int **xflags**
- int **os**
- Bytef * **extra**
- uInt **extra_len**
- uInt **extra_max**
- Bytef * **name**
- uInt **name_max**
- Bytef * **comment**
- uInt **comm_max**
- int **hcrc**
- int **done**

4.6.1 Detailed Description

Definition at line 114 of file zlib.h.

The documentation for this struct was generated from the following file:

- zlib.h

4.7 gzFile_s Struct Reference

Public Attributes

- unsigned **have**
- unsigned char * **next**
- z_off64_t **pos**

4.7.1 Detailed Description

Definition at line 1837 of file zlib.h.

The documentation for this struct was generated from the following file:

- zlib.h

4.8 nsimd_aarch64_vf16 Struct Reference

Public Attributes

- float32x4_t v0
- float32x4_t v1

4.8.1 Detailed Description

Definition at line 36 of file arm/aarch64/types.h.

The documentation for this struct was generated from the following file:

- arm/aarch64/types.h

4.9 nsimd_aarch64_vlf16 Struct Reference

Public Attributes

- uint32x4_t v0
- uint32x4_t v1

4.9.1 Detailed Description

Definition at line 55 of file arm/aarch64/types.h.

The documentation for this struct was generated from the following file:

- arm/aarch64/types.h

4.10 nsimd_avx2_vf16 Struct Reference

Public Attributes

- __m256 v0
- __m256 v1

4.10.1 Detailed Description

Definition at line 32 of file x86/avx2/types.h.

The documentation for this struct was generated from the following file:

- x86/avx2/types.h

4.11 nsimd_avx2_vlf16 Struct Reference

Public Attributes

- `__m256 v0`
- `__m256 v1`

4.11.1 Detailed Description

Definition at line 44 of file `x86/avx2/types.h`.

The documentation for this struct was generated from the following file:

- `x86/avx2/types.h`

4.12 nsimd_avx512_knl_vf16 Struct Reference

Public Attributes

- `__m512 v0`
- `__m512 v1`

4.12.1 Detailed Description

Definition at line 32 of file `x86/avx512_knl/types.h`.

The documentation for this struct was generated from the following file:

- `x86/avx512_knl/types.h`

4.13 nsimd_avx512_knl_vlf16 Struct Reference

Public Attributes

- `__mmask16 v0`
- `__mmask16 v1`

4.13.1 Detailed Description

Definition at line 44 of file `x86/avx512_knl/types.h`.

The documentation for this struct was generated from the following file:

- `x86/avx512_knl/types.h`

4.14 nsimd_avx512_skylake_vf16 Struct Reference

Public Attributes

- `__m512 v0`
- `__m512 v1`

4.14.1 Detailed Description

Definition at line 32 of file `x86/avx512_skylake/types.h`.

The documentation for this struct was generated from the following file:

- `x86/avx512_skylake/types.h`

4.15 nsimd_avx512_skylake_vlf16 Struct Reference

Public Attributes

- `__mmask16 v0`
- `__mmask16 v1`

4.15.1 Detailed Description

Definition at line 44 of file `x86/avx512_skylake/types.h`.

The documentation for this struct was generated from the following file:

- `x86/avx512_skylake/types.h`

4.16 nsimd_avx_vf16 Struct Reference

Public Attributes

- `__m256 v0`
- `__m256 v1`

4.16.1 Detailed Description

Definition at line 32 of file `x86/avx/types.h`.

The documentation for this struct was generated from the following file:

- `x86/avx/types.h`

4.17 nsimd_avx_vlf16 Struct Reference

Public Attributes

- `__m256 v0`
- `__m256 v1`

4.17.1 Detailed Description

Definition at line 44 of file `x86/avx/types.h`.

The documentation for this struct was generated from the following file:

- `x86/avx/types.h`

4.18 nsimd_cpu_vf16 Struct Reference

Public Attributes

- `f32 v0`
- `f32 v1`
- `f32 v2`
- `f32 v3`
- `f32 v4`
- `f32 v5`
- `f32 v6`
- `f32 v7`

4.18.1 Detailed Description

Definition at line 36 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.19 nsimd_cpu_vf32 Struct Reference

Public Attributes

- `f32 v0`
- `f32 v1`
- `f32 v2`
- `f32 v3`

4.19.1 Detailed Description

Definition at line 32 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.20 nsimd_cpu_vf64 Struct Reference

Public Attributes

- `f64 v0`
- `f64 v1`

4.20.1 Detailed Description

Definition at line 30 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.21 nsimd_cpu_vi16 Struct Reference

Public Attributes

- `i16 v0`
- `i16 v1`
- `i16 v2`
- `i16 v3`
- `i16 v4`
- `i16 v5`
- `i16 v6`
- `i16 v7`

4.21.1 Detailed Description

Definition at line 50 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.22 nsimd_cpu_vi32 Struct Reference

Public Attributes

- i32 v0
- i32 v1
- i32 v2
- i32 v3

4.22.1 Detailed Description

Definition at line 46 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.23 nsimd_cpu_vi64 Struct Reference

Public Attributes

- i64 v0
- i64 v1

4.23.1 Detailed Description

Definition at line 44 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.24 nsimd_cpu_vi8 Struct Reference

Public Attributes

- i8 v0
- i8 v1
- i8 v2
- i8 v3
- i8 v4
- i8 v5
- i8 v6
- i8 v7
- i8 v8
- i8 v9
- i8 v10
- i8 v11
- i8 v12
- i8 v13
- i8 v14
- i8 v15

4.24.1 Detailed Description

Definition at line 58 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.25 `nsimd_cpu_vlf16` Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**
- unsigned int **v4**
- unsigned int **v5**
- unsigned int **v6**
- unsigned int **v7**

4.25.1 Detailed Description

Definition at line 111 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.26 `nsimd_cpu_vlf32` Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**

4.26.1 Detailed Description

Definition at line 107 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.27 nsimd_cpu_vlf64 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**

4.27.1 Detailed Description

Definition at line 105 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.28 nsimd_cpu_vli16 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**
- unsigned int **v4**
- unsigned int **v5**
- unsigned int **v6**
- unsigned int **v7**

4.28.1 Detailed Description

Definition at line 125 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.29 nsimd_cpu_vli32 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**

4.29.1 Detailed Description

Definition at line 121 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.30 nsimd_cpu_vli64 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**

4.30.1 Detailed Description

Definition at line 119 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.31 nsimd_cpu_vli8 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**
- unsigned int **v4**
- unsigned int **v5**
- unsigned int **v6**
- unsigned int **v7**
- unsigned int **v8**
- unsigned int **v9**
- unsigned int **v10**
- unsigned int **v11**
- unsigned int **v12**
- unsigned int **v13**
- unsigned int **v14**
- unsigned int **v15**

4.31.1 Detailed Description

Definition at line 133 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.32 nsimd_cpu_vlu16 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**
- unsigned int **v4**
- unsigned int **v5**
- unsigned int **v6**
- unsigned int **v7**

4.32.1 Detailed Description

Definition at line 155 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.33 nsimd_cpu_vlu32 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**

4.33.1 Detailed Description

Definition at line 151 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.34 nsimd_cpu_vlu64 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**

4.34.1 Detailed Description

Definition at line 149 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.35 nsimd_cpu_vlu8 Struct Reference

Public Attributes

- unsigned int **v0**
- unsigned int **v1**
- unsigned int **v2**
- unsigned int **v3**
- unsigned int **v4**
- unsigned int **v5**
- unsigned int **v6**
- unsigned int **v7**
- unsigned int **v8**
- unsigned int **v9**
- unsigned int **v10**
- unsigned int **v11**
- unsigned int **v12**
- unsigned int **v13**
- unsigned int **v14**
- unsigned int **v15**

4.35.1 Detailed Description

Definition at line 163 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.36 nsimd_cpu_vu16 Struct Reference

Public Attributes

- u16 v0
- u16 v1
- u16 v2
- u16 v3
- u16 v4
- u16 v5
- u16 v6
- u16 v7

4.36.1 Detailed Description

Definition at line 80 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.37 nsimd_cpu_vu32 Struct Reference

Public Attributes

- u32 v0
- u32 v1
- u32 v2
- u32 v3

4.37.1 Detailed Description

Definition at line 76 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.38 nsimd_cpu_vu64 Struct Reference

Public Attributes

- u64 v0
- u64 v1

4.38.1 Detailed Description

Definition at line 74 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.39 nsimd_cpu_vu8 Struct Reference

Public Attributes

- `u8 v0`
- `u8 v1`
- `u8 v2`
- `u8 v3`
- `u8 v4`
- `u8 v5`
- `u8 v6`
- `u8 v7`
- `u8 v8`
- `u8 v9`
- `u8 v10`
- `u8 v11`
- `u8 v12`
- `u8 v13`
- `u8 v14`
- `u8 v15`

4.39.1 Detailed Description

Definition at line 88 of file `cpu/cpu/types.h`.

The documentation for this struct was generated from the following file:

- `cpu/cpu/types.h`

4.40 nsimd_neon128_vf16 Struct Reference

Public Attributes

- `float32x4_t v0`
- `float32x4_t v1`

4.40.1 Detailed Description

Definition at line 36 of file arm/neon128/types.h.

The documentation for this struct was generated from the following file:

- arm/neon128/types.h

4.41 nsimd_neon128_vf64 Struct Reference

Public Attributes

- double **v0**
- double **v1**

4.41.1 Detailed Description

Definition at line 30 of file arm/neon128/types.h.

The documentation for this struct was generated from the following file:

- arm/neon128/types.h

4.42 nsimd_neon128_vlf16 Struct Reference

Public Attributes

- uint32x4_t **v0**
- uint32x4_t **v1**

4.42.1 Detailed Description

Definition at line 55 of file arm/neon128/types.h.

The documentation for this struct was generated from the following file:

- arm/neon128/types.h

4.43 nsimd_neon128_vlf64 Struct Reference

Public Attributes

- u64 **v0**
- u64 **v1**

4.43.1 Detailed Description

Definition at line 49 of file arm/neon128/types.h.

The documentation for this struct was generated from the following file:

- arm/neon128/types.h

4.44 nsimd_sse2_vf16 Struct Reference

Public Attributes

- __m128 v0
- __m128 v1

4.44.1 Detailed Description

Definition at line 32 of file x86/sse2/types.h.

The documentation for this struct was generated from the following file:

- x86/sse2/types.h

4.45 nsimd_sse2_vlf16 Struct Reference

Public Attributes

- __m128 v0
- __m128 v1

4.45.1 Detailed Description

Definition at line 44 of file x86/sse2/types.h.

The documentation for this struct was generated from the following file:

- x86/sse2/types.h

4.46 nsimd_sse42_vf16 Struct Reference

Public Attributes

- __m128 v0
- __m128 v1

4.46.1 Detailed Description

Definition at line 32 of file x86/sse42/types.h.

The documentation for this struct was generated from the following file:

- x86/sse42/types.h

4.47 nsimd_sse42_vlf16 Struct Reference

Public Attributes

- __m128 v0
- __m128 v1

4.47.1 Detailed Description

Definition at line 44 of file x86/sse42/types.h.

The documentation for this struct was generated from the following file:

- x86/sse42/types.h

4.48 nsimd_vmx_vf16 Struct Reference

Public Attributes

- __vector float v0
- __vector float v1

4.48.1 Detailed Description

Definition at line 32 of file ppc/vmx/types.h.

The documentation for this struct was generated from the following file:

- ppc/vmx/types.h

4.49 nsimd_vmx_vf64 Struct Reference

Public Attributes

- f64 v0
- f64 v1

4.49.1 Detailed Description

Definition at line 30 of file `ppc/vmx/types.h`.

The documentation for this struct was generated from the following file:

- `ppc/vmx/types.h`

4.50 nsimd_vmx_vi64 Struct Reference

Public Attributes

- `i64 v0`
- `i64 v1`

4.50.1 Detailed Description

Definition at line 33 of file `ppc/vmx/types.h`.

The documentation for this struct was generated from the following file:

- `ppc/vmx/types.h`

4.51 nsimd_vmx_vlf16 Struct Reference

Public Attributes

- `__vector __bool int v0`
- `__vector __bool int v1`

4.51.1 Detailed Description

Definition at line 44 of file `ppc/vmx/types.h`.

The documentation for this struct was generated from the following file:

- `ppc/vmx/types.h`

4.52 nsimd_vmx_vlf64 Struct Reference

Public Attributes

- `u64 v0`
- `u64 v1`

4.52.1 Detailed Description

Definition at line 42 of file ppc/vmx/types.h.

The documentation for this struct was generated from the following file:

- ppc/vmx/types.h

4.53 nsimd_vmx_vli64 Struct Reference

Public Attributes

- u64 v0
- u64 v1

4.53.1 Detailed Description

Definition at line 45 of file ppc/vmx/types.h.

The documentation for this struct was generated from the following file:

- ppc/vmx/types.h

4.54 nsimd_vmx_vlu64 Struct Reference

Public Attributes

- u64 v0
- u64 v1

4.54.1 Detailed Description

Definition at line 49 of file ppc/vmx/types.h.

The documentation for this struct was generated from the following file:

- ppc/vmx/types.h

4.55 nsimd_vmx_vu64 Struct Reference

Public Attributes

- u64 v0
- u64 v1

4.55.1 Detailed Description

Definition at line 37 of file ppc/vmx/types.h.

The documentation for this struct was generated from the following file:

- ppc/vmx/types.h

4.56 nsimd_vsx_vf16 Struct Reference

Public Attributes

- `__vector float v0`
- `__vector float v1`

4.56.1 Detailed Description

Definition at line 32 of file ppc/vsx/types.h.

The documentation for this struct was generated from the following file:

- ppc/vsx/types.h

4.57 nsimd_vsx_vlf16 Struct Reference

Public Attributes

- `__vector __bool int v0`
- `__vector __bool int v1`

4.57.1 Detailed Description

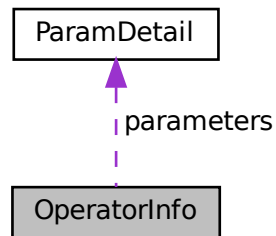
Definition at line 44 of file ppc/vsx/types.h.

The documentation for this struct was generated from the following file:

- ppc/vsx/types.h

4.58 OperatorInfo Struct Reference

Collaboration diagram for OperatorInfo:



Public Attributes

- char **name** [64]
- char **description** [128]
- bool **enabled**
- [ParamDetail](#) * **parameters**
- uint32_t **param_count**

4.58.1 Detailed Description

Definition at line 38 of file TYCAPL.h.

The documentation for this struct was generated from the following file:

- TYCAPL.h

4.59 ParamDetail Struct Reference

Public Attributes

- char **name** [64]
- AlgParamType **type**
- char **description** [128]
-
- union {
 bool **bool_value**
 int32_t **int_value**
 float **float_value**
 };
- int32_t **min_int**
- int32_t **max_int**
- int32_t **step_int**
- float **min_float**
- float **max_float**
- float **step_float**

4.59.1 Detailed Description

Definition at line 19 of file TYCAPL.h.

The documentation for this struct was generated from the following file:

- TYCAPL.h

4.60 pattern_bin_param Struct Reference

Public Attributes

- uint32_t **offset**
- uint8_t **data** [512]

4.60.1 Detailed Description

Definition at line 1081 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.61 pattern_gray_param Struct Reference

Public Attributes

- uint32_t **phase_num**
- uint32_t **param1**
- uint32_t **param2**
- uint32_t **param3**

4.61.1 Detailed Description

Definition at line 1073 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.62 pattern_sine_param Struct Reference

Public Attributes

- uint32_t **phase_num**
- float **period**

4.62.1 Detailed Description

Definition at line 1067 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.63 TY_PHC_GROUP_ATTR::phc_group_attr Struct Reference

Public Attributes

- `uint8_t type`
- `uint8_t amp_thresh`
- `uint16_t ch`
- `uint8_t chn_type`
- `uint8_t rsvd [27]`

4.63.1 Detailed Description

Definition at line 1051 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.64 TY_ACC_BIAS Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- `float data [3]`

4.64.1 Detailed Description

a 3x3 matrix

.	.	.
BIASx	BIASy	BIASz

Definition at line 1131 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.65 TY_ACC_MISALIGNMENT Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- float **data** [3 *3]

4.65.1 Detailed Description

a 3x3 matrix

|.|.|

.	.	.
1	-GAMAy _z	GAMAz _y
GAMAx _z	1	-GAMAz _x
-GAMAx _y	GAMAy _x	1

Definition at line 1143 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.66 TY_ACC_SCALE Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- float **data** [3 *3]

4.66.1 Detailed Description

a 3x3 matrix

.	.	.
SCALE _x	0	0
0	SCALE _y	0
0	0	SCALE _z

Definition at line 1154 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.67 TY_AEC_ROI_PARAM Struct Reference

Public Attributes

- uint32_t **x**
- uint32_t **y**
- uint32_t **w**
- uint32_t **h**

4.67.1 Detailed Description

Definition at line 1031 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.68 TY_BYTEARRAY_ATTR Struct Reference

byte array data structure

```
#include <TYDefs.h>
```

Public Attributes

- int32_t [size](#)
Bytes array size in bytes.
- int32_t [unit_size](#)
- int32_t [valid_size](#)

4.68.1 Detailed Description

byte array data structure

See also

[TYGetByteArray](#)

Definition at line 899 of file TYDefs.h.

4.68.2 Member Data Documentation

4.68.2.1 unit_size

```
int32_t TY_BYTEARRAY_ATTR::unit_size
```

unit size in bytes for special parse

Definition at line 902 of file TYDefs.h.

4.68.2.2 valid_size

```
int32_t TY_BYTEARRAY_ATTR::valid_size
```

valid size in bytes in case has reserved member, Must be multiple of unit_size, mem_length = valid_size/unit_size

Definition at line 905 of file TYDefs.h.

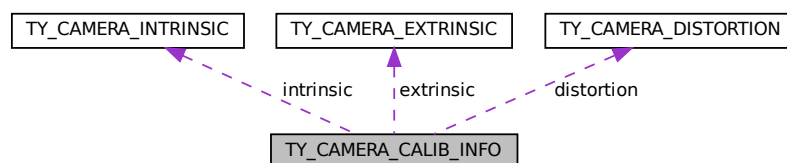
The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.69 TY_CAMERA_CALIB_INFO Struct Reference

```
#include <TYDefs.h>
```

Collaboration diagram for TY_CAMERA_CALIB_INFO:



Public Attributes

- `int32_t intrinsicWidth`
- `int32_t intrinsicHeight`
- `TY_CAMERA_INTRINSIC` `intrinsic`
- `TY_CAMERA_EXTRINSIC` `extrinsic`
- `TY_CAMERA_DISTORTION` `distortion`

4.69.1 Detailed Description

camera 's cailbration data

See also

[TYGetStruct](#)

Definition at line 974 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.70 TY_CAMERA_DISTORTION Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- `float data` [12]
Definition is compatible with opencv3.0+ :k1,k2,p1,p2,k3,k4,k5,k6,s1,s2,s3,s4.

4.70.1 Detailed Description

camera distortion parameters

See also

[TYGetStruct](#) Usage:

```
TY_CAMERA_DISTORTION distortion;
TYGetStruct(hDevice, some_compoent, TY_STRUCT_CAM_DISTORTION, &distortion, sizeof(distortion));
```

Definition at line 966 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.71 TY_CAMERA_EXTRINSIC Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- `float data` [4 *4]

4.71.1 Detailed Description

a 4x4 matrix

.	.	.	.
r11	r12	r13	t1
r21	r22	r23	t2
r31	r32	r33	t3
0	0	0	1

See also

[TYGetStruct](#) Usage:

```
TY_CAMERA_EXTRINSIC extrinsic;
TYGetStruct(hDevice, some_compoent, TY_STRUCT_EXTRINSIC, &extrinsic, sizeof(extrinsic));
```

Definition at line 954 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.72 TY_CAMERA_INTRINSIC Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- float **data** [3 *3]

4.72.1 Detailed Description

a 3x3 matrix

.	.	.
fx	0	cx
0	fy	cy
0	0	1

See also

[TYGetStruct](#) Usage:

```
TY_CAMERA_INTRINSIC intrinsic;
TYGetStruct(hDevice, some_compoent, TY_STRUCT_CAM_INTRINSIC, &intrinsic, sizeof(intrinsic));
```

Definition at line 936 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.73 TY_CAMERA_ROTATION Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- float **data** [3 *3]

4.73.1 Detailed Description

a 3x3 matrix

.	.	.
r00	r01	r02
r10	r11	r12
r20	r21	r22

See also

[TYGetStruct](#) Usage:

```
TY_CAMERA_ROTATION rotation;
TYGetStruct(hDevice, some_compoent, TY_STRUCT_CAM_ROTATION, &rotation, sizeof(rotation));
```

Definition at line 1312 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.74 TY_CAMERA_STATISTICS Struct Reference

Public Attributes

- uint64_t **packetReceived**
- uint64_t **packetLost**
- uint64_t **imageOutputed**
- uint64_t **imageDropped**
- uint8_t **rsvd** [1024]

4.74.1 Detailed Description

Definition at line 1105 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.75 TY_CAMERA_TO_IMU Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- float **data** [4 *4]

4.75.1 Detailed Description

a 4x4 matrix

.	.	.	.
r11	r12	r13	t1
r21	r22	r23	t2
r31	r32	r33	t3
0	0	0	1

Definition at line 1197 of file TYDefs.h.

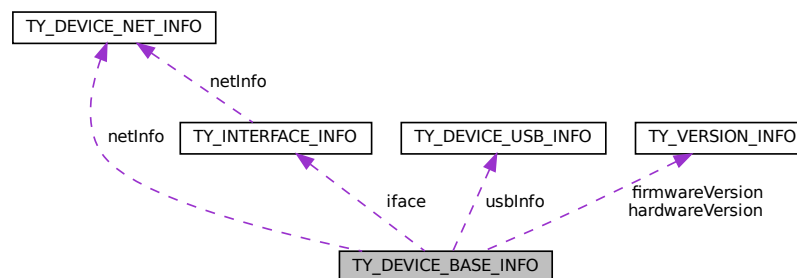
The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.76 TY_DEVICE_BASE_INFO Struct Reference

```
#include <TYDefs.h>
```

Collaboration diagram for TY_DEVICE_BASE_INFO:



Public Attributes

- [TY_INTERFACE_INFO](#) **iface**
- char **id** [32]
device serial number
- char **vendorName** [32]
- char **userDefinedName** [32]
- char **modelName** [32]
device model name
- [TY_VERSION_INFO](#) **hardwareVersion**
deprecated
- [TY_VERSION_INFO](#) **firmwareVersion**
deprecated
- union {
 [TY_DEVICE_NET_INFO](#) **netInfo**
 [TY_DEVICE_USB_INFO](#) **usbInfo**
};
- char **buildHash** [256]
- char **configVersion** [256]
- char **reserved** [256]

4.76.1 Detailed Description

See also

[TYGetDeviceList](#)

Definition at line 837 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.77 TY_DEVICE_NET_INFO Struct Reference

device network information

```
#include <TYDefs.h>
```

Public Attributes

- char **mac** [32]
- char **ip** [32]
- char **netmask** [32]
- char **gateway** [32]
- char **broadcast** [32]
- char **tlversion** [32]
- char **reserved** [64]

4.77.1 Detailed Description

device network information

Definition at line 808 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.78 TY_DEVICE_USB_INFO Struct Reference

Public Attributes

- int **bus**
- int **addr**
- char **reserved** [248]

4.78.1 Detailed Description

Definition at line 819 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.79 TY_DI_WORKMODE Struct Reference

Public Attributes

- TY_E_DI_MODE **mode**
- TY_E_DI_INT_ACTION **int_act**
- uint32_t **mode_supported**
- uint32_t **int_act_supported**
- uint32_t **status**
- uint32_t **reserved** [3]

4.79.1 Detailed Description

Definition at line 1279 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.80 TY_DO_WORKMODE Struct Reference

Public Attributes

- TY_E_DO_MODE **mode**
- TY_E_VOLT_T **volt**
- uint32_t **freq**
- uint32_t **duty**
- uint32_t **mode_supported**
- uint32_t **volt_supported**
- uint32_t **reserved** [3]

4.80.1 Detailed Description

Definition at line 1256 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.81 TY_ENUM_ENTRY Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- char **description** [64]
- uint32_t **value**
- uint32_t **reserved** [3]

4.81.1 Detailed Description

enum feature entry information

See also

[TYGetEnumEntryInfo](#)

Definition at line 910 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.82 TY_EVENT_INFO Struct Reference

Public Attributes

- TY_EVENT **eventId**
- char **message** [124]

4.82.1 Detailed Description

Definition at line 1250 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.83 TY_FEATURE_INFO Struct Reference

Public Attributes

- bool **isValid**
true if feature exists, false otherwise
- TY_ACCESS_MODE **accessMode**
feature access privilege
- bool **writableAtRun**
feature can be written while capturing
- char **reserved0** [1]
- TY_COMPONENT_ID **componentID**
owner of this feature
- TY_FEATURE_ID **featureID**
feature unique id
- char **name** [32]
describe string
- TY_COMPONENT_ID **bindComponentID**
component ID current feature bind to
- TY_FEATURE_ID **bindFeatureID**
feature ID current feature bind to
- TY_VISIBILITY_TYPE **visibility**
- char **reserved** [248]

4.83.1 Detailed Description

Definition at line 863 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.84 TY_FLOAT_RANGE Struct Reference

float range data structure

```
#include <TYDefs.h>
```

Public Attributes

- float **min**
- float **max**
- float **inc**
increasing step
- float **reserved** [1]

4.84.1 Detailed Description

float range data structure

See also

[TYGetFloatRange](#)

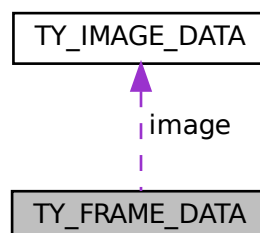
Definition at line 889 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.85 TY_FRAME_DATA Struct Reference

Collaboration diagram for TY_FRAME_DATA:



Public Attributes

- void * [userBuffer](#)
Pointer to user enqueued buffer, user should enqueue this buffer in the end of callback.
- int32_t [bufferSize](#)
Size of userBuffer.
- int32_t [validCount](#)
Number of valid data.
- int32_t [reserved](#) [6]
Reserved: reserved[0],laser_val;.
- [TY_IMAGE_DATA image](#) [10]
Buffer data, max to 10 images per frame, each buffer data could be an image or something else.

4.85.1 Detailed Description

Definition at line 1240 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.86 TY_GYRO_BIAS Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- float **data** [3]

4.86.1 Detailed Description

a 3x3 matrix

.	.	.
BIASx	BIASy	BIASz

Definition at line 1163 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.87 TY_GYRO_MISALIGNMENT Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- float **data** [3 *3]

4.87.1 Detailed Description

a 3x3 matrix

.	.	.
1	-ALPHAy _z	ALPHA _z y
0	1	-ALPHA _z x
0	0	1

Definition at line 1174 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.88 TY_GYRO_SCALE Struct Reference

```
#include <TYDefs.h>
```

Public Attributes

- float **data** [3 *3]

4.88.1 Detailed Description

a 3x3 matrix

.	.	.
SCALE _x	0	0
0	SCALE _y	0
0	0	SCALE _z

Definition at line 1185 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.89 TY_IMAGE_DATA Struct Reference

Public Attributes

- uint64_t [timestamp](#)
Timestamp in microseconds.
- int32_t [imageIndex](#)
image index used in trigger mode
- int32_t [status](#)
Status of this buffer.
- TY_COMPONENT_ID [componentID](#)
Where current data come from.
- int32_t [size](#)
Buffer size.
- void * [buffer](#)
Pointer to data buffer.
- int32_t [width](#)
Image width in pixels.
- int32_t [height](#)
Image height in pixels.
- TYPixFmt [pixelFormat](#)
Pixel format, see TY_PIXEL_FORMAT_LIST.
- int32_t [reserved](#) [9]
Reserved.

4.89.1 Detailed Description

Definition at line 1225 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.90 TY_IMU_DATA Struct Reference

Public Attributes

- uint64_t **timestamp**
- float **acc_x**
- float **acc_y**
- float **acc_z**
- float **gyro_x**
- float **gyro_y**
- float **gyro_z**
- float **temperature**
- float **reserved** [1]

4.90.1 Detailed Description

Definition at line 1114 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.91 TY_INT_RANGE Struct Reference

Public Attributes

- int32_t **min**
- int32_t **max**
- int32_t **inc**
increasing step
- int32_t **reserved** [1]

4.91.1 Detailed Description

Definition at line 879 of file TYDefs.h.

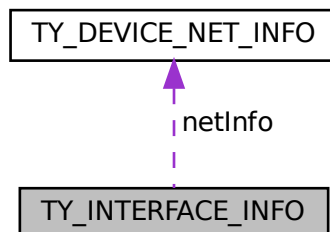
The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.92 TY_INTERFACE_INFO Struct Reference

```
#include <TYDefs.h>
```

Collaboration diagram for TY_INTERFACE_INFO:



Public Attributes

- char **name** [32]
- char **id** [32]
- TY_INTERFACE_TYPE **type**
- char **reserved** [4]
- [TY_DEVICE_NET_INFO](#) **netInfo**

4.92.1 Detailed Description

See also

[TYGetInterfaceList](#)

Definition at line 827 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.93 TY_LASER_PARAM Struct Reference

Public Attributes

- uint32_t **idx**
- uint32_t **en**
- uint32_t **power**

4.93.1 Detailed Description

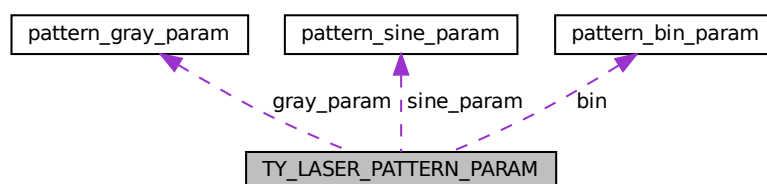
Definition at line 1215 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

4.94 TY_LASER_PATTERN_PARAM Struct Reference

Collaboration diagram for TY_LASER_PATTERN_PARAM:



Public Attributes

- uint32_t **img_index**
 - uint32_t **type**
 -
- ```

union {
 uint8_t payload [512+16]
 pattern_sine_param sine_param
 pattern_gray_param gray_param
 pattern_bin_param bin
};

```

### 4.94.1 Detailed Description

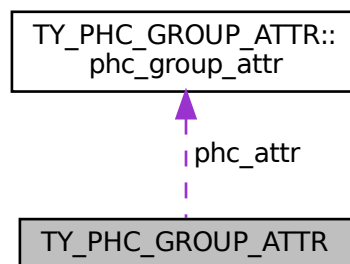
Definition at line 1087 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 4.95 TY\_PHC\_GROUP\_ATTR Struct Reference

Collaboration diagram for TY\_PHC\_GROUP\_ATTR:



## Classes

- struct [phc\\_group\\_attr](#)

## Public Attributes

- uint32\_t **offset**
- uint32\_t **size**
- struct [TY\\_PHC\\_GROUP\\_ATTR::phc\\_group\\_attr](#) **phc\_attr** [16]

### 4.95.1 Detailed Description

Definition at line 1047 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 4.96 TY\_PIXEL\_COLOR\_DESC Struct Reference

### Public Attributes

- `int16_t x`
- `int16_t y`
- `uint8_t bgr_ch1`
- `uint8_t bgr_ch2`
- `uint8_t bgr_ch3`
- `uint8_t rsvd`

### 4.96.1 Detailed Description

Definition at line 20 of file TYCoordinateMapper.h.

The documentation for this struct was generated from the following file:

- [TYCoordinateMapper.h](#)

## 4.97 TY\_PIXEL\_DESC Struct Reference

### Public Attributes

- `int16_t x`
- `int16_t y`
- `uint16_t depth`
- `uint16_t rsvd`

### 4.97.1 Detailed Description

Definition at line 12 of file TYCoordinateMapper.h.

The documentation for this struct was generated from the following file:

- [TYCoordinateMapper.h](#)

## 4.98 TY\_TEMP\_DATA Struct Reference

### Public Attributes

- uint32\_t **id**
- char **name** [16]
- char **temp** [16]
- char **desc** [16]

#### 4.98.1 Detailed Description

Definition at line 1293 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 4.99 TY\_TOF\_FREQ Struct Reference

### Public Attributes

- uint32\_t **freq1**
- uint32\_t **freq2**

#### 4.99.1 Detailed Description

Definition at line 1202 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 4.100 TY\_TRIGGER\_PARAM Struct Reference

### Public Attributes

- TY\_TRIGGER\_MODE **mode**
- int8\_t **fps**
- int8\_t **rsvd**

#### 4.100.1 Detailed Description

Definition at line 985 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 4.101 TY\_TRIGGER\_PARAM\_EX Struct Reference

### Public Attributes

- TY\_TRIGGER\_MODE **mode**
- 

```
union {
 struct {
 int8_t fps
 int8_t duty
 int32_t laser_stream
 int32_t led_stream
 int32_t led_expo
 int32_t led_gain
 }
 struct {
 int32_t ir_gain [2]
 }
 int32_t rsvd [32]
};
```

### 4.101.1 Detailed Description

Definition at line 993 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 4.102 TY\_TRIGGER\_TIMER\_LIST Struct Reference

### Public Attributes

- uint64\_t **start\_time\_us**
- uint32\_t **offset\_us\_count**
- uint32\_t **offset\_us\_list** [50]

### 4.102.1 Detailed Description

Definition at line 1016 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 4.103 TY\_TRIGGER\_TIMER\_PERIOD Struct Reference

### Public Attributes

- uint64\_t **start\_time\_us**
- uint32\_t **trigger\_count**
- uint32\_t **period\_us**

### 4.103.1 Detailed Description

Definition at line 1024 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 4.104 TY\_VECT\_3F Struct Reference

### Public Attributes

- float **x**
- float **y**
- float **z**

### 4.104.1 Detailed Description

Definition at line 917 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 4.105 TY\_VERSION\_INFO Struct Reference

### Public Attributes

- int32\_t **major**
- int32\_t **minor**
- int32\_t **patch**
- int32\_t **reserved**

### 4.105.1 Detailed Description

Definition at line 799 of file TYDefs.h.

The documentation for this struct was generated from the following file:

- [TYDefs.h](#)

## 4.106 TYEnumEntry Struct Reference

### Public Attributes

- `int32_t` **value**
- `char` **name** [64]
- `char` **tooltip** [512]
- `char` **description** [512]
- `char` **displayName** [512]

### 4.106.1 Detailed Description

Definition at line 16 of file TYParameter.h.

The documentation for this struct was generated from the following file:

- [TYParameter.h](#)

## 4.107 z\_stream\_s Struct Reference

### Public Attributes

- `z_const Bytef *` **next\_in**
- `uInt` **avail\_in**
- `uLong` **total\_in**
- `Bytef *` **next\_out**
- `uInt` **avail\_out**
- `uLong` **total\_out**
- `z_const char *` **msg**
- `struct internal_state FAR *` **state**
- `alloc_func` **zalloc**
- `free_func` **zfree**
- `voidpf` **opaque**
- `int` **data\_type**
- `uLong` **adler**
- `uLong` **reserved**

### 4.107.1 Detailed Description

Definition at line 86 of file zlib.h.

The documentation for this struct was generated from the following file:

- [zlib.h](#)

## Chapter 5

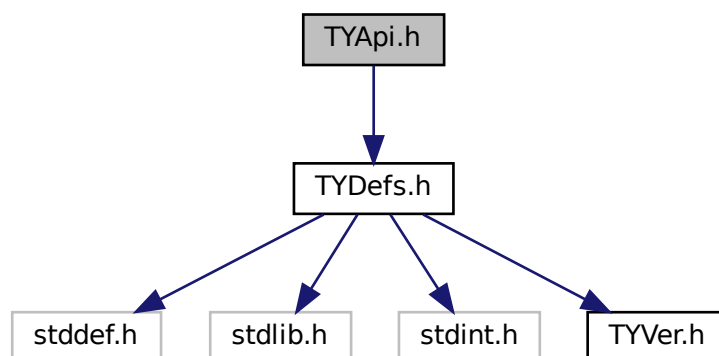
# File Documentation

### 5.1 TYApi.h File Reference

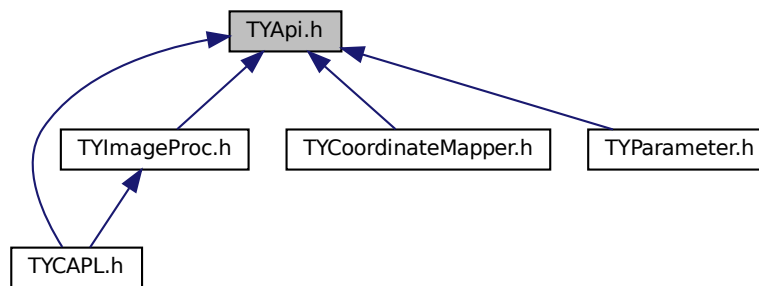
[TYApi.h](#) includes camera control and data receiving interface, which supports configuration for image resolution, frame rate, exposure time, gain, working mode, etc.

```
#include "TYDefs.h"
```

Include dependency graph for TYApi.h:



This graph shows which files directly or indirectly include this file:



## Typedefs

- typedef void(\* **TY\_EVENT\_CALLBACK**) (**TY\_EVENT\_INFO** \*, void \*userdata)
- typedef void(\* **TY\_IMU\_CALLBACK**) (**TY\_IMU\_DATA** \*, void \*userdata)

## Functions

- TY\_CAPI **TYInitLib** (void)
- TY\_CAPI **TYLibVersion** (**TY\_VERSION\_INFO** \*version)  
*Get current library version.*
- TY\_EXTC const TY\_EXPORT char \*TY\_STDC **TYErrorString** (TY\_STATUS errorID)  
*Get error information.*
- TY\_CAPI **TYDeinitLib** (void)  
*Deinit this library.*
- TY\_CAPI **TYSetLogLevel** (TY\_LOG\_TYPE type, TY\_LOG\_LEVEL lvl)  
*Set log level.*
- TY\_CAPI **TYSetLogPrefix** (const char \*prefix)  
*set log prefix*
- TY\_CAPI **TYEnableLog** (TY\_LOG\_TYPE type)  
*Enable logger by type.*
- TY\_CAPI **TYDisableLog** (TY\_LOG\_TYPE type)  
*Disable logger by type.*
- TY\_CAPI **TYCfgLogFile** (const char \*filePath, uint32\_t max\_size=10 \*1024 \*1024, uint32\_t max\_files=10)  
*Append log to specified file(s), max\_size must not be 0. If max\_files Not 0, it will use rotation files. Rotate files as: log.txt -> log.1.txt log.1.txt -> log.2.txt log.2.txt -> log.3.txt log.3.txt -> delete.*
- TY\_CAPI **TYCfgLogServer** (TY\_SERVER\_TYPE prot\_type, char \*ip, uint16\_t port)  
*Append log to Tcp/Udp server.*
- TY\_CAPI **TYUpdateInterfaceList** (void)  
*Update current interfaces. call before TYGetInterfaceList.*
- TY\_CAPI **TYGetInterfaceNumber** (uint32\_t \*pNumIfaces)  
*Get number of current interfaces.*
- TY\_CAPI **TYGetInterfaceList** (**TY\_INTERFACE\_INFO** \*pIfaceInfos, uint32\_t bufferCount, uint32\_t \*filledCount)  
*Get interface info list.*



- TY\_CAPI [TYHasInterface](#) (const char \*ifaceID, bool \*value)  
*Check if has interface.*
- TY\_CAPI [TYOpenInterface](#) (const char \*ifaceID, [TY\\_INTERFACE\\_HANDLE](#) \*outHandle)  
*Open specified interface.*
- TY\_CAPI [TYCloseInterface](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle)  
*Close interface.*
- TY\_CAPI [TYUpdateDeviceList](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle)  
*Update current connected devices.*
- TY\_CAPI [TYUpdateAllDeviceList](#) (void)  
*Update current connected devices.*
- TY\_CAPI [TYGetDeviceNumber](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle, uint32\_t \*deviceNumber)  
*Get number of current connected devices.*
- TY\_CAPI [TYGetDeviceList](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle, [TY\\_DEVICE\\_BASE\\_INFO](#) \*deviceInfos, uint32\_t bufferCount, uint32\_t \*filledDeviceCount)  
*Get device info list.*
- TY\_CAPI [TYHasDevice](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle, const char \*deviceID, bool \*value)  
*Check whether the interface has the specified device.*
- TY\_CAPI [TYOpenDevice](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle, const char \*deviceID, [TY\\_DEV\\_HANDLE](#) \*outDeviceHandle, [TY\\_FW\\_ERRORCODE](#) \*outFwErrorcode=NULL)  
*Open device by device ID.*
- TY\_CAPI [TYOpenDeviceWithIP](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle, const char \*IP, [TY\\_DEV\\_HANDLE](#) \*deviceHandle)  
*Open device by device IP, useful when a device is not listed.*
- TY\_CAPI [TYGetDeviceInterface](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_INTERFACE\\_HANDLE](#) \*piface)  
*Get interface handle by device handle.*
- TY\_CAPI [TYForceDeviceIP](#) ([TY\\_INTERFACE\\_HANDLE](#) ifaceHandle, const char \*MAC, const char \*newIP, const char \*newNetMask, const char \*newGateway)  
*Force a ethernet device to use new IP address, useful when device use persistent IP and cannot be found.*
- TY\_CAPI [TYCloseDevice](#) ([TY\\_DEV\\_HANDLE](#) hDevice, bool reboot=false)  
*Close device by device handle.*
- TY\_CAPI [TYGetDeviceInfo](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_DEVICE\\_BASE\\_INFO](#) \*info)  
*Get base info of the open device.*
- TY\_CAPI [TYGetComponentIDs](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) \*componentIDs)  
*Get all components IDs.*
- TY\_CAPI [TYGetEnabledComponents](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) \*componentIDs)  
*Get all enabled components IDs.*
- TY\_CAPI [TYEnableComponents](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentIDs)  
*Enable components.*
- TY\_CAPI [TYDisableComponents](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentIDs)  
*Disable components.*
- TY\_CAPI [TYGetFrameBufferSize](#) ([TY\\_DEV\\_HANDLE](#) hDevice, uint32\_t \*bufferSize)  
*Get total buffer size of one frame in current configuration.*
- TY\_CAPI [TYEnqueueBuffer](#) ([TY\\_DEV\\_HANDLE](#) hDevice, void \*buffer, uint32\_t bufferSize)  
*Enqueue a user allocated buffer.*
- TY\_CAPI [TYClearBufferQueue](#) ([TY\\_DEV\\_HANDLE](#) hDevice)  
*Clear the internal buffer queue, so that user can release all the buffer.*
- TY\_CAPI [TYStartCapture](#) ([TY\\_DEV\\_HANDLE](#) hDevice)  
*Start capture.*
- TY\_CAPI [TYStopCapture](#) ([TY\\_DEV\\_HANDLE](#) hDevice)  
*Stop capture.*
- TY\_CAPI [TYSendSoftTrigger](#) ([TY\\_DEV\\_HANDLE](#) hDevice)

*Send a software trigger to capture a frame when device works in trigger mode.*

- TY\_CAPI [TYRegisterEventCallback](#) (TY\_DEV\_HANDLE hDevice, TY\_EVENT\_CALLBACK callback, void \*userdata)

*Register device status callback. Register NULL to clean callback.*

- TY\_CAPI [TYRegisterImuCallback](#) (TY\_DEV\_HANDLE hDevice, TY\_IMU\_CALLBACK callback, void \*userdata)

*Register imu callback. Register NULL to clean callback.*

- TY\_CAPI [TYFetchFrame](#) (TY\_DEV\_HANDLE hDevice, TY\_FRAME\_DATA \*frame, int32\_t timeout)

*Fetch one frame.*

- TY\_CAPI [TYHasFeature](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, bool \*value)

*Check whether a component has a specific feature.*

- TY\_CAPI [TYGetFeatureInfo](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, TY\_FEATURE\_INFO \*featureInfo)

*Get feature info.*

- TY\_CAPI [TYGetIntRange](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, TY\_INT\_RANGE \*intRange)

*Get value range of integer feature.*

- TY\_CAPI [TYGetInt](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, int32\_t \*value)

*Get value of integer feature.*

- TY\_CAPI [TYSetInt](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, int32\_t value)

*Set value of integer feature.*

- TY\_CAPI [TYGetFloatRange](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, TY\_FLOAT\_RANGE \*floatRange)

*Get value range of float feature.*

- TY\_CAPI [TYGetFloat](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, float \*value)

*Get value of float feature.*

- TY\_CAPI [TYSetFloat](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, float value)

*Set value of float feature.*

- TY\_CAPI [TYGetEnumEntryCount](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, uint32\_t \*entryCount)

*Get number of enum entries.*

- TY\_CAPI [TYGetEnumEntryInfo](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, TY\_ENUM\_ENTRY \*entries, uint32\_t entryCount, uint32\_t \*filledEntryCount)

*Get list of enum entries.*

- TY\_CAPI [TYGetEnum](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, uint32\_t \*value)

*Get current value of enum feature.*

- TY\_CAPI [TYSetEnum](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, uint32\_t value)

*Set value of enum feature.*

- TY\_CAPI [TYGetBool](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, bool \*value)

*Get value of bool feature.*

- TY\_CAPI [TYSetBool](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, bool value)

*Set value of bool feature.*

- TY\_CAPI [TYGetStringLength](#) (TY\_DEV\_HANDLE hDevice, TY\_COMPONENT\_ID componentID, TY\_FEATURE\_ID featureID, uint32\_t \*size)

*Get internal buffer size of string feature.*

- TY\_CAPI [TYGetString](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, char \*buffer, uint32\_t bufferSize)

*Get value of string feature.*

- TY\_CAPI [TYSetString](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, const char \*buffer)

*Set value of string feature.*

- TY\_CAPI [TYGetStruct](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, void \*pStruct, uint32\_t structSize)

*Get value of struct.*

- TY\_CAPI [TYSetStruct](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, void \*pStruct, uint32\_t structSize)

*Set value of struct.*

- TY\_CAPI [TYGetByteArraySize](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, uint32\_t \*pSize)

*Get the size of specified byte array zone.*

- TY\_CAPI [TYGetByteArray](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, uint8\_t \*pBuffer, uint32\_t bufferSize)

*Read byte array from device.*

- TY\_CAPI [TYSetByteArray](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, const uint8\_t \*pBuffer, uint32\_t bufferSize)

*Write byte array to device.*

- TY\_CAPI [TYGetByteArrayAttr](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_ID](#) featureID, [TY\\_BYTEARRAY\\_ATTR](#) \*pAttr)

*Write byte array to device.*

- TY\_CAPI [TYGetDeviceFeatureNumber](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, uint32\_t \*size)

*Get the size of device features.*

- TY\_CAPI [TYGetDeviceFeatureInfo](#) ([TY\\_DEV\\_HANDLE](#) hDevice, [TY\\_COMPONENT\\_ID](#) componentID, [TY\\_FEATURE\\_INFO](#) \*featureInfo, uint32\_t entryCount, uint32\_t \*filledEntryCount)

*Get the all features by comp id.*

- TY\_CAPI [TYGetDeviceXMLSize](#) ([TY\\_DEV\\_HANDLE](#) hDevice, uint32\_t \*size)

*Get the Device xml size.*

- TY\_CAPI [TYGetDeviceXML](#) ([TY\\_DEV\\_HANDLE](#) hDevice, char \*xml, const uint32\_t in\_size, uint32\_t \*out\_size)

*Get the Device xml string.*

### 5.1.1 Detailed Description

[TYApi.h](#) includes camera control and data receiving interface, which supports configuration for image resolution, frame rate, exposure time, gain, working mode, etc.

### 5.1.2 Function Documentation

### 5.1.2.1 TYCfgLogFile()

```
TY_CAPI TYCfgLogFile (
 const char * filePath,
 uint32_t max_size = 10 * 1024 * 1024,
 uint32_t max_files = 10)
```

Append log to specified file(s), max\_size must not be 0. If max\_files Not 0, it will use rotation files. Rotate files as: log.txt -> log.1.txt log.1.txt -> log.2.txt log.2.txt -> log.3.txt log.3.txt -> delete.

#### Parameters

|    |                  |                                                    |
|----|------------------|----------------------------------------------------|
| in | <i>filePath</i>  | Path to the log file.                              |
| in | <i>max_size</i>  | max single files size, can't be 0, default is 10M. |
| in | <i>max_files</i> | Max files to keep, default is 10.                  |

#### Return values

|                        |                                                                                                                           |
|------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>    | Succeed.                                                                                                                  |
| <i>TY_STATUS_ERROR</i> | Failed to add file<br><br>Suggestions:<br>Please check if the file path is correct and if you have permission to write to |

### 5.1.2.2 TYCfgLogServer()

```
TY_CAPI TYCfgLogServer (
 TY_SERVER_TYPE prot_type,
 char * ip,
 uint16_t port)
```

Append log to Tcp/Udp server.

#### Parameters

|    |                  |                                         |
|----|------------------|-----------------------------------------|
| in | <i>prot_type</i> | Protocol of the server, "tcp" or "udp". |
| in | <i>ip</i>        | IP address of the server.               |
| in | <i>port</i>      | Port of the server.                     |

#### Return values

|                        |                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>    | Succeed.                                                                                |
| <i>TY_STATUS_ERROR</i> | Failed to add server<br><br>Suggestions:<br>Please check if the ip and port are correct |

## Return values

|                                          |                                                                                         |
|------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>TY_STATUS_INVALID_PARAMETER</code> | Unsupported protocol<br><br>Suggestions:<br>Unsupported protocol, please use tcp or udp |
|------------------------------------------|-----------------------------------------------------------------------------------------|

## 5.1.2.3 TYClearBufferQueue()

```
TY_CAPI TYClearBufferQueue (
 TY_DEV_HANDLE hDevice)
```

Clear the internal buffer queue, so that user can release all the buffer.

## Parameters

|    |                |                |
|----|----------------|----------------|
| in | <i>hDevice</i> | Device handle. |
|----|----------------|----------------|

## Return values

|                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>TY_STATUS_OK</code>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <code>TY_STATUS_INVALID_HANDLE</code> | TYClearBufferQueue called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYClearBufferQueue(hDevice);</pre> ^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdated |
| <code>TY_STATUS_BUSY</code>           | Device is capturing.                                                                                                                                                                                                                                                                                                                                                                                                                               |

## 5.1.2.4 TYCloseDevice()

```
TY_CAPI TYCloseDevice (
 TY_DEV_HANDLE hDevice,
 bool reboot = false)
```

Close device by device handle.

## Parameters

|    |                |                            |
|----|----------------|----------------------------|
| in | <i>hDevice</i> | Device handle.             |
| in | <i>reboot</i>  | Reboot device after close. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYCloseDevice called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYCloseDevice(hDevice, reboot);</code><br/>                           ^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:<br/>           1.TYOpenDevice failed to open device and get correct handle<br/>           2.Memory in stack to store handle data is corrupted<br/>           3.After getting handle, you updated device list by calling TYUpdatedDeviceList</p> |
| <i>TY_STATUS_TIMEOUT</i>        | <p>Failed to close device</p> <p>Suggestions:<br/>Possible reasons:<br/>           1.Network communication is abnormal, please check whether the network is normal</p>                                                                                                                                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_DEVICE_ERROR</i>   | <p>Failed to close device</p> <p>Suggestions:<br/>Possible reasons:<br/>           1.Camera device is abnormal and cannot be closed normally.</p>                                                                                                                                                                                                                                                                                                                                                                                                  |
| <i>TY_STATUS_IDLE</i>           | Device has been closed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

## 5.1.2.5 TYCloseInterface()

```
TY_CAPI TYCloseInterface (
 TY_INTERFACE_HANDLE ifaceHandle)
```

Close interface.

## Parameters

|    |                    |                         |
|----|--------------------|-------------------------|
| in | <i>ifaceHandle</i> | Interface to be closed. |
|----|--------------------|-------------------------|

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <i>TY_STATUS_NOT_INITED</i>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <i>TY_STATUS_INVALID_INTERFACE</i> | <p>TYCloseInterface called with invalid interface handle</p> <p>Suggestions:<br/>Please check interface handle<br/>Like this:<br/> <code>TYCloseInterface(ifaceHandle);</code><br/>                           ^ is invalid</p> <p>The ifaceHandle parameter you input is not recorded</p> <p>Possible reasons:<br/>           1.TYOpenInterface failed to open interface and get correct handle<br/>           2.Memory in stack to store handle data is corrupted<br/>           3.After getting handle, you updated interface list by calling TYUpdateInterfaceList</p> |

### 5.1.2.6 TYDeinitLib()

```
TY_CAPI TYDeinitLib (
 void)
```

Deinit this library.

**Return values**

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 5.1.2.7 TYDisableComponents()

```
TY_CAPI TYDisableComponents (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentIDs)
```

Disable components.

**Parameters**

|    |                     |                            |
|----|---------------------|----------------------------|
| in | <i>hDevice</i>      | Device handle.             |
| in | <i>componentIDs</i> | Components to be disabled. |

**Return values**

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYDisableComponents called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/>    TYDisableComponents(hDevice, componentIDs);<br/>                                ^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"><li>1.TYOpenDevice failed to open device and get correct handle</li><li>2.Memory in stack to store handle data is corrupted</li><li>3.After getting handle, you updated device list by calling TYUpo</li></ol> |
| <i>TY_STATUS_INVALID_PARAMETER</i> | <p>Invalid component IDs</p> <p>Suggestions:<br/>Please check componentIDs parameter<br/>Like this:<br/>    TYDisableComponents(hDevice, componentIDs);<br/>                                ^ is invalid</p> <p>componentIDs should be the value returned by TYGetComponentIDs<br/>You can also view the components of the camera by obtaining the x</p>                                                                                                                                                                                                             |
| <i>TY_STATUS_INVALID_COMPONENT</i> | Some components specified by componentIDs are invalid.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

## Return values

|                       |                                                                                                                                                                                                                                                   |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_BUSY</i> | Camera device is capturing<br><br>Suggestions:<br>Please call <code>TYEnableComponents</code> when the camera device is stopped<br>Like this:<br><code>TYStopCapture(hDevice);</code><br><code>TYDisableComponents(hDevice, componentIDs);</code> |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## See also

[TY\\_DEVICE\\_COMPONENT\\_LIST](#)

### 5.1.2.8 TYDisableLog()

```
TY_CAPI TYDisableLog (
 TY_LOG_TYPE type)
```

Disable logger by type.

## Parameters

|    |             |              |
|----|-------------|--------------|
| in | <i>type</i> | Logger type. |
|----|-------------|--------------|

## Return values

|                        |                                                                                                              |
|------------------------|--------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>    | Succeed.                                                                                                     |
| <i>TY_STATUS_ERROR</i> | Failed to disable the selected logger<br><br>Suggestions:<br>Please check if the selected logger is disabled |

### 5.1.2.9 TYEnableComponents()

```
TY_CAPI TYEnableComponents (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentIDs)
```

Enable components.

## Parameters

|    |                     |                           |
|----|---------------------|---------------------------|
| in | <i>hDevice</i>      | Device handle.            |
| in | <i>componentIDs</i> | Components to be enabled. |



## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYEnableComponents called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYEnableComponents(hDevice, componentIDs);</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUp</p> |
| <i>TY_STATUS_INVALID_PARAMETER</i> | <p>Invalid component IDs</p> <p>Suggestions:<br/>Please check componentIDs parameter<br/>Like this:<br/> <pre>TYEnableComponents(hDevice, componentIDs);</pre> ^ is invalid<br/> componentIDs should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                       |
| <i>TY_STATUS_INVALID_COMPONENT</i> | Some components specified by componentIDs are invalid.                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>TY_STATUS_BUSY</i>              | <p>Camera device is capturing</p> <p>Suggestions:<br/>Please call TYEnableComponents when the camera device is stopped<br/>Like this:<br/> <pre>TYStopCapture(hDevice);</pre> <pre>TYEnableComponents(hDevice, componentIDs);</pre></p>                                                                                                                                                                                                                                          |

## See also

[TY\\_DEVICE\\_COMPONENT\\_LIST](#)

## 5.1.2.10 TYEnableLog()

```
TY_CAPI TYEnableLog (
 TY_LOG_TYPE type)
```

Enable logger by type.

## Parameters

|    |             |              |
|----|-------------|--------------|
| in | <i>type</i> | Logger type. |
|----|-------------|--------------|

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                        |                                                                                                                                            |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_ERROR</i> | Failed to enable the selected logger<br><br>Suggestions:<br>Please call TYCfgLogFile or TYCfgLogServer before enable log to file or log to |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|

## 5.1.2.11 TYEnqueueBuffer()

```

TY_CAPI TYEnqueueBuffer (
 TY_DEV_HANDLE hDevice,
 void * buffer,
 uint32_t bufferSize)

```

Enqueue a user allocated buffer.

## Parameters

|    |                   |                           |
|----|-------------------|---------------------------|
| in | <i>hDevice</i>    | Device handle.            |
| in | <i>buffer</i>     | Buffer to be enqueued.    |
| in | <i>bufferSize</i> | Size of the input buffer. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <i>TY_STATUS_INVALID_HANDLE</i> | TYEnqueueBuffer called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br>TYEnqueueBuffer(hDevice, buffer, bufferSize);<br>^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdated |
| <i>TY_STATUS_NULL_POINTER</i>   | TYEnqueueBuffer called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br>TYEnqueueBuffer(hDevice, buffer, bufferSize);<br>^ is NULL                                                                                                                                                                                                                                                                                       |
| <i>TY_STATUS_WRONG_SIZE</i>     | TYEnqueueBuffer called with wrong size<br><br>Suggestions:<br>Please check your code<br>Like this:<br>TYEnqueueBuffer(hDevice, buffer, bufferSize);<br>^ is 0 or negative value                                                                                                                                                                                                                                                                          |

## Return values

|                               |                                                                                                                                                  |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_TIMEOUT</i>      | Failed to enqueue frame buffer<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the network |
| <i>TY_STATUS_DEVICE_ERROR</i> | Failed to enqueue frame buffer<br><br>Suggestions:<br>Possible reasons:<br>1.Camera device is abnormal and cannot get the frame buffer size.     |

## 5.1.2.12 TYErrorString()

```
TY_EXTC const TY_EXPORT char* TY_STDC TYErrorString (
 TY_STATUS errorID)
```

Get error information.

## Parameters

|    |                |           |
|----|----------------|-----------|
| in | <i>errorID</i> | Error id. |
|----|----------------|-----------|

## Return values

|              |         |
|--------------|---------|
| <i>Error</i> | string. |
|--------------|---------|

## 5.1.2.13 TYFetchFrame()

```
TY_CAPI TYFetchFrame (
 TY_DEV_HANDLE hDevice,
 TY_FRAME_DATA * frame,
 int32_t timeout)
```

Fetch one frame.

## Parameters

|     |                |                                           |
|-----|----------------|-------------------------------------------|
| in  | <i>hDevice</i> | Device handle.                            |
| out | <i>frame</i>   | Frame data to be filled.                  |
| in  | <i>timeout</i> | Timeout in milliseconds. <0 for infinite. |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYFetchFrame called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYFetchFrame(hDevice, pFrame, timeout); ^ is invalid</pre> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdated</li> </ol> |
| <i>TY_STATUS_NULL_POINTER</i>   | <p>TYFetchFrame called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYFetchFrame(hDevice, pFrame, timeout); ^ is NULL</pre>                                                                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_IDLE</i>           | <p>Camera device is not started</p> <p>Suggestions:</p> <p>Please start the camera device first</p> <p>Like this:</p> <pre>TYStartCapture(hDevice); TYFetchFrame(hDevice, pFrame, timeout);</pre>                                                                                                                                                                                                                                                                                                                                      |
| <i>TY_STATUS_WRONG_MODE</i>     | <p>Callback has been registered, this function is disabled.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <i>TY_STATUS_TIMEOUT</i>        | <p>Failed to get frame</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.Camera device is abnormal and cannot get frame.</li> <li>2.Network communication is abnormal, please check whether the network</li> <li>3.Timeout, frame acquisition timeout</li> </ol>                                                                                                                                                                                                                               |

## 5.1.2.14 TYForceDeviceIP()

```
TY_CAPI TYForceDeviceIP (
 TY_INTERFACE_HANDLE ifaceHandle,
 const char * MAC,
 const char * newIP,
 const char * newNetMask,
 const char * newGateway)
```

Force a ethernet device to use new IP address, useful when device use persistent IP and cannot be found.

## Parameters

|    |                    |                                            |
|----|--------------------|--------------------------------------------|
| in | <i>ifaceHandle</i> | Interface handle.                          |
| in | <i>MAC</i>         | Device MAC, should be "xx:xx:xx:xx:xx:xx". |
| in | <i>newIP</i>       | New IP.                                    |
| in | <i>newNetMask</i>  | New subnet mask.                           |
| in | <i>newGateway</i>  | New gateway.                               |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TY_STATUS_OK</b>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>TY_STATUS_NOT_INITED</b>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>TY_STATUS_INVALID_INTERFACE</b> | <p>TYForceDeviceIP called with invalid interface handle</p> <p>Suggestions:<br/>Please check interface handle<br/>Like this:<br/> <code>TYForceDeviceIP(ifaceHandle, MAC, newIP, newNetMask, newGateway)</code><br/> ^ is invalid<br/> The ifaceHandle parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenInterface failed to open interface and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated interface list by calling TY</p>                                                                                                                           |
| <b>TY_STATUS_WRONG_TYPE</b>        | <p>TYForceDeviceIP called with invalid interface type</p> <p>Suggestions:<br/>Please check interface type<br/>Usually you can get interface information by calling TYGetInterfaceList<br/>You can use TYIsNetworkInterface to check the interface type<br/>Only network interfaces can call TYForceDeviceIP<br/>Like this:<br/> <code>TY_INTERFACE_INFO info; uint32_t num;<br/>TYGetInterfaceList(&amp;info, 1, &amp;num);<br/>if (TYIsNetworkInterface(info[0].type)) {<br/>    TY_INTERFACE_HANDLE hIface;<br/>    TYOpenInterface(info[0].id, &amp;hIface);<br/>    TYForceDeviceIP(hIface, MAC, newIP, newNetMask, newGateway);<br/>}</code></p> |
| <b>TY_STATUS_NULL_POINTER</b>      | <p>TYForceDeviceIP called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYForceDeviceIP(ifaceHandle, MAC, newIP, newNetMask, newGateway)</code><br/> ^ or ^ or ^ or ^ is NULL</p>                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>TY_STATUS_INVALID_PARAMETER</b> | <p>Invalid MAC address:</p> <p>Suggestions:<br/>Please check MAC parameter<br/>Like this:<br/> <code>TYForceDeviceIP(ifaceHandle, MAC, newIP, newNetMask, newGateway)</code><br/> ^ is invalid<br/> MAC address should be six bytes of hexadecimal separated by colons<br/> For example: 00:11:22:aa:bb:cc</p>                                                                                                                                                                                                                                                                                                                                        |
| <b>TY_STATUS_TIMEOUT</b>           | <p>Failed to force set IP</p> <p>Suggestions:<br/>Possible reasons:<br/> 1.Network communication is abnormal, please check whether the network is normal<br/> 2.There is no camera with a matching target MAC address in the network</p>                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>TY_STATUS_DEVICE_ERROR</b>      | <p>Failed to force set IP</p> <p>Suggestions:<br/>Possible reasons:<br/> 1.New IP, NetMask, Gateway are incorrect, camera device refuses to set IP</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

### 5.1.2.15 TYGetBool()

```
TY_CAPI TYGetBool (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 bool * value)
```

Get value of bool feature.

#### Parameters

|     |                    |                |
|-----|--------------------|----------------|
| in  | <i>hDevice</i>     | Device handle. |
| in  | <i>componentID</i> | Component ID.  |
| in  | <i>featureID</i>   | Feature ID.    |
| out | <i>value</i>       | Bool value.    |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetBool called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYGetBool(hDevice, componentID, featureID, pValue);</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYGetBool(hDevice, componentID, featureID, pValue);</pre> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                  |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYGetBool(hDevice, componentID, featureID, pValue);</pre> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGet<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                 |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <pre>TYGetBool(hDevice, componentID, featureID, pValue);</pre> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeatur</p>                                                                                                                                                                                                                      |



## Return values

|                                          |                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>TY_STATUS_INVALID_COMPONENT</code> | <p>Invalid component ID</p> <p>Suggestions:</p> <p>Please check componentID parameter</p> <p>Like this:</p> <pre>TYGetByteArray(hDevice, componentID, featureID, buffer, bufferSize);</pre> <p>^ is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs</p> <p>You can also view the components of the camera by obtaining the x</p>                                                    |
| <code>TY_STATUS_INVALID_FEATURE</code>   | <p>Invalid feature ID</p> <p>Suggestions:</p> <p>Please check featureID parameter</p> <p>Like this:</p> <pre>TYGetByteArray(hDevice, componentID, featureID, buffer, bufferSize);</pre> <p>^ is invalid</p> <p>You entered an invalid featureID parameter</p> <p>You can get a list of features of the camera device through TYGet</p> <p>You can also view the features of the camera device by obtaining t</p> |
| <code>TY_STATUS_WRONG_TYPE</code>        | <p>Feature type mismatch</p> <p>Suggestions:</p> <p>Please check the feature type</p> <p>Like this:</p> <pre>TYGetByteArray(hDevice, componentID, featureID, buffer, bufferSize);</pre> <p>^ type mismatch</p> <p>The feature type you entered does not match. You can use TYFeature</p>                                                                                                                         |
| <code>TY_STATUS_NULL_POINTER</code>      | <p>TYGetByteArray called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYGetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize);</pre> <p>^ is NULL</p>                                                                                                                                                                                             |
| <code>TY_STATUS_WRONG_SIZE</code>        | <p>Array size mismatch</p> <p>Suggestions:</p> <p>Please check the array size</p> <p>Like this:</p> <pre>TYGetByteArray(hDevice, componentID, featureID, buffer, bufferSize);</pre> <p>^ is inv</p> <p>The array size you entered does not match</p>                                                                                                                                                             |
| <code>TY_STATUS_TIMEOUT</code>           | <p>Failed to get byte array feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.Network communication is abnormal, please check whether the ne</li> </ol>                                                                                                                                                                                                          |
| <code>TY_STATUS_DEVICE_ERROR</code>      | <p>Failed to get byte array feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.The feature of the camera device is not available or not imple</li> <li>2.Camera device is abnormal and cannot get byte array feature</li> </ol>                                                                                                                                   |

## 5.1.2.17 TYGetByteArrayAttr()

```
TY_CAPI TYGetByteArrayAttr (
 TY_DEV_HANDLE hDevice,
```



```

TY_COMPONENT_ID componentID,
TY_FEATURE_ID featureID,
TY_BYTEARRAY_ATTR * pAttr)

```

Write byte array to device.

#### Parameters

|     |                    |                                    |
|-----|--------------------|------------------------------------|
| in  | <i>hDevice</i>     | Device handle.                     |
| in  | <i>componentID</i> | Component ID.                      |
| in  | <i>featureID</i>   | Feature ID.                        |
| out | <i>pAttr</i>       | Byte array attribute to be filled. |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetByteArrayAttr called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYGetByteArrayAttr(hDevice, componentID, featureID, pAttr);</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYGetByteArrayAttr(hDevice, componentID, featureID, pAttr);</pre> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                           |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYGetByteArrayAttr(hDevice, componentID, featureID, pAttr);</pre> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetL<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                         |
| <i>TY_STATUS_NOT_PERMITTED</i>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <pre>TYGetByteArrayAttr(hDevice, componentID, featureID, pAttr);</pre> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeature</p>                                                                                                                                                                                                                              |

## Return values

|                               |                                                                                                                                                                                             |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_NULL_POINTER</i> | TYGetByteArrayAttr called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetByteArrayAttr(hDevice, componentID, featureID, pAttr); ^ is NULL</pre> |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 5.1.2.18 TYGetByteArraySize()

```
TY_CAPI TYGetByteArraySize (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 uint32_t * pSize)
```

Get the size of specified byte array zone.

## Parameters

|     |                    |                                    |
|-----|--------------------|------------------------------------|
| in  | <i>hDevice</i>     | Device handle.                     |
| in  | <i>componentID</i> | Component ID.                      |
| in  | <i>featureID</i>   | Feature ID.                        |
| out | <i>pSize</i>       | Size of specified byte array zone. |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <i>TY_STATUS_INVALID_HANDLE</i>    | TYGetByteArraySize called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYGetByteArraySize(hDevice, componentID, featureID, pSize); ^ is invalid</pre> The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUp |
| <i>TY_STATUS_INVALID_COMPONENT</i> | Invalid component ID<br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br><pre>TYGetByteArraySize(hDevice, componentID, featureID, pSize); ^ is invalid</pre> componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the x                                                                                                                                                    |

### Return values

|                                         |                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_INVALID_FEATURE</i></b> | <p><b>Invalid feature ID</b></p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYGetByteArraySize(hDevice, componentID, featureID, pSize);</code><br/> ^ is invalid</p> <p>You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetFeatures<br/> You can also view the features of the camera device by obtaining the featureID</p> |
| <b><i>TY_STATUS_WRONG_TYPE</i></b>      | <p><b>Feature type mismatch</b></p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <code>TYGetByteArraySize(hDevice, componentID, featureID, pSize);</code><br/> ^ type mismatch</p> <p>The feature type you entered does not match. You can use TYFeatureType to get the feature type</p>                                                                                                             |
| <b><i>TY_STATUS_NULL_POINTER</i></b>    | <p><b>TYGetByteArraySize called with NULL pointer</b></p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetByteArraySize(hDevice, componentID, featureID, pSize);</code><br/> ^ is NULL</p>                                                                                                                                                                                                          |

### 5.1.2.19 TYGetComponentIDs()

```
TY_CAPI TYGetComponentIDs (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID * componentIds)
```

Get all components IDs.

### Parameters

|     |                     |                                                |
|-----|---------------------|------------------------------------------------|
| in  | <i>hDevice</i>      | Device handle.                                 |
| out | <i>componentIDs</i> | All component IDs this device has. (bit flag). |

### Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetComponentIDs called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYGetComponentIDs(hDevice, outComponentIDs);</pre> <p style="margin-left: 150px;">^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdatedDeviceList</li> </ol> |

## Return values

|                               |                                                                                                                                                                             |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_NULL_POINTER</i> | TYGetComponentIDs called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetComponentIDs(hDevice, outComponentIDs); ^ is NULL</pre> |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## See also

[TY\\_DEVICE\\_COMPONENT\\_LIST](#)

## 5.1.2.20 TYGetDeviceFeatureInfo()

```
TY_CAPI TYGetDeviceFeatureInfo (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_INFO * featureInfo,
 uint32_t entryCount,
 uint32_t * filledEntryCount)
```

Get the all features by comp id.

## Parameters

|     |                         |                                              |
|-----|-------------------------|----------------------------------------------|
| in  | <i>hDevice</i>          | Device handle.                               |
| in  | <i>componentID</i>      | Component ID.                                |
| out | <i>featureInfo</i>      | Output feature info.                         |
| in  | <i>entryCount</i>       | Array size of input parameter "featureInfo". |
| out | <i>filledEntryCount</i> | Number of filled featureInfo.                |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <i>TY_STATUS_INVALID_HANDLE</i> | TYGetDeviceFeatureInfo called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYGetDeviceFeatureInfo(hDevice, componentID, featureInfo, entryCount, filledEntryCount); ^ is invalid</pre> The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdateDeviceList |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_COMPONENT</i> | Invalid component ID<br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br><pre>TYGetDeviceFeatureInfo(hDevice, componentID, featureInfo, entryID, &amp;featureInfo, entryID)</pre> ^ is invalid<br>componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the x |
| <i>TY_STATUS_NULL_POINTER</i>      | TYGetDeviceFeatureInfo called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetDeviceFeatureInfo(hDevice, componentID, featureInfo, entryID, &amp;featureInfo, entryID)</pre> ^ or                                                                                                                                |
| <i>TY_STATUS_TIMEOUT</i>           | Failed to get feature info<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the n                                                                                                                                                                                                                      |
| <i>TY_STATUS_DEVICE_ERROR</i>      | Failed to get feature info<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not impl<br>2.Camera device is abnormal and cannot get feature info                                                                                                                                                           |

## 5.1.2.21 TYGetDeviceFeatureNumber()

```

TY_CAPI TYGetDeviceFeatureNumber (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 uint32_t * size)

```

Get the size of device features.

## Parameters

|     |                    |                          |
|-----|--------------------|--------------------------|
| in  | <i>hDevice</i>     | Device handle.           |
| in  | <i>componentID</i> | Component ID.            |
| out | <i>size</i>        | Size of all feature cnt. |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_INVALID_HANDLE</i></b>    | <b>TYGetDeviceFeatureNumber called with invalid device handle</b><br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYGetDeviceFeatureNumber(hDevice, componentID, size);                         ^ is invalid</pre> The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUp |
| <b><i>TY_STATUS_INVALID_COMPONENT</i></b> | <b>Invalid component ID</b><br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br><pre>TYGetDeviceFeatureNumber(hDevice, componentID, size);                         ^ is invalid</pre> componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the xn                                                                                                                                                         |
| <b><i>TY_STATUS_NULL_POINTER</i></b>      | <b>TYGetDeviceFeatureNumber called with NULL pointer</b><br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetDeviceFeatureNumber(hDevice, componentID, size);                         ^ is NULL</pre>                                                                                                                                                                                                                                                                               |
| <b><i>TY_STATUS_TIMEOUT</i></b>           | <b>Failed to get feature number</b><br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the n                                                                                                                                                                                                                                                                                                                                                          |
| <b><i>TY_STATUS_DEVICE_ERROR</i></b>      | <b>Failed to get feature number</b><br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not impl<br>2.Camera device is abnormal and cannot get feature number                                                                                                                                                                                                                                                                                             |

## 5.1.2.22 TYGetDeviceInfo()

```
TY_CAPI TYGetDeviceInfo (
 TY_DEV_HANDLE hDevice,
 TY_DEVICE_BASE_INFO * info)
```

Get base info of the open device.

## Parameters

|     |                |                |
|-----|----------------|----------------|
| in  | <i>hDevice</i> | Device handle. |
| out | <i>info</i>    | Base info out. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetDeviceInfo called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetDeviceInfo(hDevice, info);</code><br/>                                           ^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:<br/>           1.TYOpenDevice failed to open device and get correct handle<br/>           2.Memory in stack to store handle data is corrupted<br/>           3.After getting handle, you updated device list by calling TYUpdatedDeviceList</p> |
| <i>TY_STATUS_NULL_POINTER</i>   | <p>TYGetDeviceInfo called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetDeviceInfo(hDevice, info);</code><br/>                                           ^ is NULL</p>                                                                                                                                                                                                                                                                                                                                             |

## 5.1.2.23 TYGetDeviceInterface()

```

TY_CAPI TYGetDeviceInterface (
 TY_DEV_HANDLE hDevice,
 TY_INTERFACE_HANDLE * pIface)

```

Get interface handle by device handle.

## Parameters

|     |                |                   |
|-----|----------------|-------------------|
| in  | <i>hDevice</i> | Device handle.    |
| out | <i>pIface</i>  | Interface handle. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetDeviceInterface called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetDeviceInterface(hDevice, pIface);</code><br/>                                           ^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:<br/>           1.TYOpenDevice failed to open device and get correct handle<br/>           2.Memory in stack to store handle data is corrupted<br/>           3.After getting handle, you updated device list by calling TYUpdatedDeviceList</p> |

## Return values

|                                      |                                                                                                                                                                                                  |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_NULL_POINTER</i></b> | TYGetDeviceInterface called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetDeviceInterface(hDevice, pIface);                         ^ is NULL</pre> |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 5.1.2.24 TYGetDeviceList()

```
TY_CAPI TYGetDeviceList (
 TY_INTERFACE_HANDLE ifaceHandle,
 TY_DEVICE_BASE_INFO * deviceInfos,
 uint32_t bufferSize,
 uint32_t * filledDeviceCount)
```

Get device info list.

## Parameters

|     |                          |                                                        |
|-----|--------------------------|--------------------------------------------------------|
| in  | <i>ifaceHandle</i>       | Interface handle.                                      |
| out | <i>deviceInfos</i>       | Device info array to be filled.                        |
| in  | <i>bufferCount</i>       | Array size of deviceInfos.                             |
| out | <i>filledDeviceCount</i> | Number of filled <a href="#">TY_DEVICE_BASE_INFO</a> . |

## Return values

|                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_OK</i></b>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b><i>TY_STATUS_NOT_INITED</i></b>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b><i>TY_STATUS_INVALID_INTERFACE</i></b> | TYGetDeviceList called with invalid interface handle<br><br>Suggestions:<br>Please check interface handle<br>Like this:<br><pre>TYGetDeviceList(ifaceHandle, pDeviceInfos, bufferSize, pFilledDe                         ^ is invalid</pre> The ifaceHandle parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenInterface failed to open interface and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated interface list by calling TYU |
| <b><i>TY_STATUS_NULL_POINTER</i></b>      | TYGetDeviceList called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetDeviceList(ifaceHandle, pDeviceInfos, bufferSize, pFilledCo                         ^ is NULL or ^ is 0 or ^ is NULL</pre>                                                                                                                                                                                                                                                                          |



### 5.1.2.25 TYGetDeviceNumber()

```
TY_CAPI TYGetDeviceNumber (
 TY_INTERFACE_HANDLE ifaceHandle,
 uint32_t * deviceNumber)
```

Get number of current connected devices.

#### Parameters

|     |                     |                              |
|-----|---------------------|------------------------------|
| in  | <i>ifaceHandle</i>  | Interface handle.            |
| out | <i>deviceNumber</i> | Number of connected devices. |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <i>TY_STATUS_NOT_INITED</i>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>TY_STATUS_INVALID_INTERFACE</i> | <p>TYGetDeviceNumber called with invalid interface handle</p> <p>Suggestions:</p> <p>Please check interface handle</p> <p>Like this:</p> <pre>TYGetDeviceNumber(ifaceHandle, pDeviceNumber);                 ^ is invalid</pre> <p>The ifaceHandle parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenInterface failed to open interface and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated interface list by calling TYU</li> </ol> |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYGetDeviceNumber called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYGetDeviceNumber(ifaceHandle, deviceNumber);                 ^ is NULL</pre>                                                                                                                                                                                                                                                                                                                                                                        |

### 5.1.2.26 TYGetDeviceXML()

```
TY_CAPI TYGetDeviceXML (
 TY_DEV_HANDLE hDevice,
 char * xml,
 const uint32_t in_size,
 uint32_t * out_size)
```

Get the Device xml string.

#### Parameters

|     |                 |                                 |
|-----|-----------------|---------------------------------|
| in  | <i>hDevice</i>  | Device handle.                  |
| in  | <i>xml</i>      | The buffer to store xml         |
| in  | <i>in_size</i>  | The size buffer                 |
| out | <i>out_size</i> | The actual size write in buffer |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <i>TY_STATUS_NOT_INITED</i>     | Not call TYInitLib                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <i>TY_STATUS_WRONG_SIZE</i>     | <p>XML buffer size is not enough</p> <p>Suggestions:<br/>XML buffer size is not enough<br/>Like this:<br/> <code>TYGetDeviceXML(hDevice, xml, in_size, out_size);</code><br/> ^ is invalid<br/> XML buffer size is not enough, please use TYGetDeviceXMLSize to get the size</p>                                                                                                                                                                                                                        |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetDeviceXML called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetDeviceXML(hDevice, xml, in_size, out_size);</code><br/> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpdateDeviceList</p> |
| <i>TY_STATUS_NULL_POINTER</i>   | <p>TYGetDeviceXML called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetDeviceXML(hDevice, xml, in_size, out_size);</code><br/> ^ or ^ is NULL</p>                                                                                                                                                                                                                                                                                                     |

## 5.1.2.27 TYGetDeviceXMLSize()

```

TY_CAPI TYGetDeviceXMLSize (
 TY_DEV_HANDLE hDevice,
 uint32_t * size)

```

Get the Device xml size.

## Parameters

|     |                |                               |
|-----|----------------|-------------------------------|
| in  | <i>hDevice</i> | Device handle.                |
| out | <i>size</i>    | The size of device xml string |

## Return values

|                             |                    |
|-----------------------------|--------------------|
| <i>TY_STATUS_OK</i>         | Succeed.           |
| <i>TY_STATUS_NOT_INITED</i> | Not call TYInitLib |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetDeviceXMLSize called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYGetDeviceXMLSize(hDevice, size);                     ^ is invalid</pre> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdated</li> </ol> |
| <i>TY_STATUS_NULL_POINTER</i>   | <p>TYGetDeviceXMLSize called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYGetDeviceXMLSize(hDevice, size);                     ^ is NULL</pre>                                                                                                                                                                                                                                                                                                                                                          |

## 5.1.2.28 TYGetEnabledComponents()

```
TY_CAPI TYGetEnabledComponents (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID * componentIDs)
```

Get all enabled components IDs.

## Parameters

|     |                     |                                  |
|-----|---------------------|----------------------------------|
| in  | <i>hDevice</i>      | Device handle.                   |
| out | <i>componentIDs</i> | Enabled component IDs.(bit flag) |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetEnabledComponents called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYGetEnabledComponents(hDevice, componentIDs);                         ^ is invalid</pre> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdated</li> </ol> |
| <i>TY_STATUS_NULL_POINTER</i>   | componentIDs is NULL.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

See also

[TY\\_DEVICE\\_COMPONENT\\_LIST](#)

### 5.1.2.29 TYGetEnum()

```
TY_CAPI TYGetEnum (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 uint32_t * value)
```

Get current value of enum feature.

#### Parameters

|     |                    |                |
|-----|--------------------|----------------|
| in  | <i>hDevice</i>     | Device handle. |
| in  | <i>componentID</i> | Component ID.  |
| in  | <i>featureID</i>   | Feature ID.    |
| out | <i>value</i>       | Enum value.    |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetEnum called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYGetEnum(hDevice, componentID, featureID, pValue);</pre> <p>^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpd</li> </ol> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID: d</p> <p>Suggestions:</p> <p>Please check componentID parameter</p> <p>Like this:</p> <pre>TYGetEnum(hDevice, componentID, featureID, pValue);</pre> <p>^ is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs</p> <p>You can also view the components of the camera by obtaining the x</p>                                                                                                                                                                                                    |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID: d</p> <p>Suggestions:</p> <p>Please check featureID parameter</p> <p>Like this:</p> <pre>TYGetEnum(hDevice, componentID, featureID, pValue);</pre> <p>^ is invalid</p> <p>You entered an invalid featureID parameter</p> <p>You can get a list of features of the camera device through TYGetI</p> <p>You can also view the features of the camera device by obtaining t</p>                                                                                                                                                |

### Return values

|                               |                                                                                                                                                                                                                                                                                                    |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_WRONG_TYPE</i>   | <p>Feature type mismatch</p> <p>Suggestions:</p> <p>Please check the feature type</p> <p>Like this:</p> <pre>TYGetEnum(hDevice, componentID, featureID, pValue);</pre> <p style="text-align: right;">^ type mismatch</p> <p>The feature type you entered does not match. You can use TYFeature</p> |
| <i>TY_STATUS_NULL_POINTER</i> | <p>TYGetEnum called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYGetEnum(hDevice, componentID, featureID, pValue);</pre> <p style="text-align: right;">^ is NULL</p>                                                                           |
| <i>TY_STATUS_TIMEOUT</i>      | <p>Failed to get enum feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.Network communication is abnormal, please check whether the net</li> </ol>                                                                                                 |
| <i>TY_STATUS_DEVICE_ERROR</i> | <p>Failed to get enum feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.The feature of the camera device is not available or not impl</li> <li>2.Camera device is abnormal and cannot get enum feature</li> </ol>                                  |

### 5.1.2.30 TYGetEnumEntryCount()

```

TY_CAPI TYGetEnumEntryCount (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 uint32_t * entryCount)

```

Get number of enum entries.

## Parameters

|     |                    |                |
|-----|--------------------|----------------|
| in  | <i>hDevice</i>     | Device handle. |
| in  | <i>componentID</i> | Component ID.  |
| in  | <i>featureID</i>   | Feature ID.    |
| out | <i>entryCount</i>  | Entry count.   |

### Return values

|              |          |
|--------------|----------|
| TY STATUS OK | Succeed. |
|--------------|----------|

## Return values

|                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_INVALID_HANDLE</i></b>    | <b>TYGetEnumEntryCount called with invalid device handle</b><br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYGetEnumEntryCount(hDevice, componentID, featureID, pEntryCount, &amp;filledEntryCount);</pre> ^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdateDeviceList |
| <b><i>TY_STATUS_INVALID_COMPONENT</i></b> | <b>Invalid component ID</b><br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br><pre>TYGetEnumEntryCount(hDevice, componentID, featureID, pEntryCount, &amp;filledEntryCount);</pre> ^ is invalid<br>componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the xrt::device                                                                                                                                                         |
| <b><i>TY_STATUS_INVALID_FEATURE</i></b>   | <b>Invalid feature ID</b><br><br>Suggestions:<br>Please check featureID parameter<br>Like this:<br><pre>TYGetEnumEntryCount(hDevice, componentID, featureID, pEntryCount, &amp;filledEntryCount);</pre> ^ is invalid<br>You entered an invalid featureID parameter<br>You can get a list of features of the camera device through TYGetFeatureIDs<br>You can also view the features of the camera device by obtaining the xrt::device                                                                                            |
| <b><i>TY_STATUS_WRONG_TYPE</i></b>        | <b>Feature type mismatch</b><br><br>Suggestions:<br>Please check the feature type<br>Like this:<br><pre>TYGetEnumEntryCount(hDevice, componentID, featureID, pEntryCount, &amp;filledEntryCount);</pre> ^ type mismatch<br>The feature type you entered does not match. You can use TYFeatureType                                                                                                                                                                                                                                |
| <b><i>TY_STATUS_NULL_POINTER</i></b>      | <b>TYGetEnumEntryCount called with NULL pointer</b><br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetEnumEntryCount(hDevice, componentID, featureID, pEntryCount, &amp;filledEntryCount);</pre> ^ is NULL                                                                                                                                                                                                                                                                                                |

## 5.1.2.31 TYGetEnumEntryInfo()

```

TY_CAPI TYGetEnumEntryInfo (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 TY_ENUM_ENTRY * entries,
 uint32_t entryCount,
 uint32_t * filledEntryCount)

```

Get list of enum entries.

## Parameters

|     |                         |                                          |
|-----|-------------------------|------------------------------------------|
| in  | <i>hDevice</i>          | Device handle.                           |
| in  | <i>componentID</i>      | Component ID.                            |
| in  | <i>featureID</i>        | Feature ID.                              |
| out | <i>entries</i>          | Output entries.                          |
| in  | <i>entryCount</i>       | Array size of input parameter "entries". |
| out | <i>filledEntryCount</i> | Number of filled entries.                |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetEnumEntryInfo called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetEnumEntryInfo(hDevice, componentID, featureID, entries, entryCount)</code><br/> <code>^</code> is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpdateDeviceList</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID: d</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYGetEnumEntryInfo(hDevice, componentID, featureID, entries, entryCount)</code><br/> <code>^</code> is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs</p> <p>You can also view the components of the camera by obtaining the xrt::device object</p>                                                                                                                                                    |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID: d</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYGetEnumEntryInfo(hDevice, componentID, featureID, entries, entryCount)</code><br/> <code>^</code> is invalid</p> <p>You entered an invalid featureID parameter</p> <p>You can get a list of features of the camera device through TYGetFeatureIDs</p> <p>You can also view the features of the camera device by obtaining the xrt::device object</p>                                                                                   |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <code>TYGetEnumEntryInfo(hDevice, componentID, featureID, entries, entryCount)</code><br/> <code>^</code> type mismatch</p> <p>The feature type you entered does not match. You can use TYFeatureType to get the feature type</p>                                                                                                                                                                                                                 |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYGetEnumEntryInfo called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetEnumEntryInfo(hDevice, componentID, featureID, pEnumDescriptions, entryCount)</code><br/> <code>^</code></p>                                                                                                                                                                                                                                                                                                            |

### 5.1.2.32 TYGetFeatureInfo()

```

TY_CAPI TYGetFeatureInfo (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 TY_FEATURE_INFO * featureInfo)

```

Get feature info.

#### Parameters

|     |                    |                |
|-----|--------------------|----------------|
| in  | <i>hDevice</i>     | Device handle. |
| in  | <i>componentID</i> | Component ID.  |
| in  | <i>featureID</i>   | Feature ID.    |
| out | <i>featureInfo</i> | Feature info.  |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetFeatureInfo called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetFeatureInfo(hDevice, componentID, featureID, pFeatureInfo),</code><br/> <code>^</code> is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYGetFeatureInfo(hDevice, componentID, featureID, pFeatureInfo),</code><br/> <code>^</code> is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                         |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYGetFeatureInfo(hDevice, componentID, featureID, pFeatureInfo),</code><br/> <code>^</code> is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetF<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                       |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYGetFeatureInfo called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetFeatureInfo(hDevice, componentID, featureID, pFeatureInfo),</code><br/> <code>^</code> is NULL</p>                                                                                                                                                                                                                                                                                             |



## 5.1.2.33 TYGetFloat()

```

TY_CAPI TYGetFloat (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 float * value)

```

Get value of float feature.

## Parameters

|     |                    |                |
|-----|--------------------|----------------|
| in  | <i>hDevice</i>     | Device handle. |
| in  | <i>componentID</i> | Component ID.  |
| in  | <i>featureID</i>   | Feature ID.    |
| out | <i>value</i>       | Float value.   |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetFloat called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetFloat(hDevice, componentID, featureID, pValue);</code><br/> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYGetFloat(hDevice, componentID, featureID, pValue);</code><br/> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                   |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYGetFloat(hDevice, componentID, featureID, pValue);</code><br/> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetF<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                 |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <code>TYGetFloat(hDevice, componentID, featureID, pValue);</code><br/> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeatur</p>                                                                                                                                                                                                                       |

## Return values

|                               |                                                                                                                                                                                                            |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_NULL_POINTER</i> | TYGetFloat called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetFloat(hDevice, componentID, featureID, pValue);                 ^ is NULL</pre>               |
| <i>TY_STATUS_TIMEOUT</i>      | Failed to get float feature<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the network is normal                                                    |
| <i>TY_STATUS_DEVICE_ERROR</i> | Failed to get float feature<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not implemented<br>2.Camera device is abnormal and cannot get float feature |

## 5.1.2.34 TYGetFloatRange()

```

TY_CAPI TYGetFloatRange (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 TY_FLOAT_RANGE * floatRange)

```

Get value range of float feature.

## Parameters

|     |                    |                           |
|-----|--------------------|---------------------------|
| in  | <i>hDevice</i>     | Device handle.            |
| in  | <i>componentID</i> | Component ID.             |
| in  | <i>featureID</i>   | Feature ID.               |
| out | <i>floatRange</i>  | Float range to be filled. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <i>TY_STATUS_INVALID_HANDLE</i> | TYGetFloatRange called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYGetFloatRange(hDevice, componentID, featureID, pFloatRange);                 ^ is invalid</pre> The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdateDeviceList |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_COMPONENT</i> | Invalid component ID<br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br><pre>TYGetFloatRange(hDevice, componentID, featureID, pFloatRange);</pre> ^ is invalid<br><br>componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the x                                                 |
| <i>TY_STATUS_INVALID_FEATURE</i>   | Invalid feature ID<br><br>Suggestions:<br>Please check featureID parameter<br>Like this:<br><pre>TYGetFloatRange(hDevice, componentID, featureID, pFloatRange);</pre> ^ is invalid<br><br>You entered an invalid featureID parameter<br>You can get a list of features of the camera device through TYGetF<br>You can also view the features of the camera device by obtaining t |
| <i>TY_STATUS_WRONG_TYPE</i>        | Feature type mismatch<br><br>Suggestions:<br>Please check the feature type<br>Like this:<br><pre>TYGetFloatRange(hDevice, componentID, featureID, pFloatRange);</pre> ^ type mismatch<br><br>The feature type you entered does not match. You can use TYFeatur                                                                                                                   |
| <i>TY_STATUS_NULL_POINTER</i>      | TYGetFloatRange called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetFloatRange(hDevice, componentID, featureID, pFloatRange);</pre> ^ is NULL                                                                                                                                                                                      |

## 5.1.2.35 TYGetFrameBufferSize()

```
TY_CAPI TYGetFrameBufferSize (
 TY_DEV_HANDLE hDevice,
 uint32_t * bufferSize)
```

Get total buffer size of one frame in current configuration.

## Parameters

|     |                   |                        |
|-----|-------------------|------------------------|
| in  | <i>hDevice</i>    | Device handle.         |
| out | <i>bufferSize</i> | Buffer size per frame. |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetFrameBufferSize called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetFrameBufferSize(hDevice, outSize);</code><br/>                                           ^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:<br/>           1.TYOpenDevice failed to open device and get correct handle<br/>           2.Memory in stack to store handle data is corrupted<br/>           3.After getting handle, you updated device list by calling TYUpdated</p> |
| <i>TY_STATUS_NULL_POINTER</i>   | <p>TYGetFrameBufferSize called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetFrameBufferSize(hDevice, outSize);</code><br/>                                           ^ is NULL</p>                                                                                                                                                                                                                                                                                                                                   |
| <i>TY_STATUS_TIMEOUT</i>        | <p>Failed to get frame buffer size</p> <p>Suggestions:<br/>Possible reasons:<br/>           1.Network communication is abnormal, please check whether the network</p>                                                                                                                                                                                                                                                                                                                                                                                                   |
| <i>TY_STATUS_DEVICE_ERROR</i>   | <p>Failed to get frame buffer size</p> <p>Suggestions:<br/>Possible reasons:<br/>           1.Camera device is abnormal and cannot get the frame buffer size.</p>                                                                                                                                                                                                                                                                                                                                                                                                       |

## 5.1.2.36 TYGetInt()

```

TY_CAPI TYGetInt (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 int32_t * value)

```

Get value of integer feature.

## Parameters

|     |                    |                |
|-----|--------------------|----------------|
| in  | <i>hDevice</i>     | Device handle. |
| in  | <i>componentID</i> | Component ID.  |
| in  | <i>featureID</i>   | Feature ID.    |
| out | <i>value</i>       | Integer value. |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TY_STATUS_INVALID_HANDLE</b>    | <p>TYGetInt called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYGetInt(hDevice, componentID, featureID, pValue);</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <b>TY_STATUS_INVALID_COMPONENT</b> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYGetInt(hDevice, componentID, featureID, pValue);</pre> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                 |
| <b>TY_STATUS_INVALID_FEATURE</b>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYGetInt(hDevice, componentID, featureID, pValue);</pre> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetI<br/> You can also view the features of the camera device by obtaining t</p>                                                                                               |
| <b>TY_STATUS_WRONG_TYPE</b>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <pre>TYGetInt(hDevice, componentID, featureID, pValue);</pre> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeature</p>                                                                                                                                                                                                                    |
| <b>TY_STATUS_NULL_POINTER</b>      | <p>TYGetInt called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <pre>TYGetInt(hDevice, componentID, featureID, pValue);</pre> ^ is NULL</p>                                                                                                                                                                                                                                                                                             |
| <b>TY_STATUS_TIMEOUT</b>           | <p>Failed to get int feature</p> <p>Suggestions:<br/>Possible reasons:<br/>1.Network communication is abnormal, please check whether the n</p>                                                                                                                                                                                                                                                                                                                                  |
| <b>TY_STATUS_DEVICE_ERROR</b>      | <p>Failed to get int feature</p> <p>Suggestions:<br/>Possible reasons:<br/>1.The feature of the camera device is not available or not imple<br/>2.Camera device is abnormal and cannot get int feature</p>                                                                                                                                                                                                                                                                      |

### 5.1.2.37 TYGetInterfaceList()

```
TY_CAPI TYGetInterfaceList (
 TY_INTERFACE_INFO * pIfaceInfos,
 uint32_t bufferSize,
 uint32_t * filledCount)
```

Get interface info list.

#### Parameters

|     |                    |                                                      |
|-----|--------------------|------------------------------------------------------|
| out | <i>pIfaceInfos</i> | Array of interface infos to be filled.               |
| in  | <i>bufferCount</i> | Array size of interface infos.                       |
| out | <i>filledCount</i> | Number of filled <a href="#">TY_INTERFACE_INFO</a> . |

#### Return values

|                               |                                                                                                                                                                                                                               |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>           | Succeed.                                                                                                                                                                                                                      |
| <i>TY_STATUS_NOT_INITED</i>   | TYInitLib not called.                                                                                                                                                                                                         |
| <i>TY_STATUS_NULL_POINTER</i> | TYGetInterfaceList called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetInterfaceList (pIfaceInfos, bufferSize, filledCount);                     ^           or ^ is NULL</pre> |

### 5.1.2.38 TYGetInterfaceNumber()

```
TY_CAPI TYGetInterfaceNumber (
 uint32_t * pNumIfaces)
```

Get number of current interfaces.

#### Parameters

|     |                   |                       |
|-----|-------------------|-----------------------|
| out | <i>pNumIfaces</i> | Number of interfaces. |
|-----|-------------------|-----------------------|

#### Return values

|                               |                                                                                                                                                                                          |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>           | Succeed.                                                                                                                                                                                 |
| <i>TY_STATUS_NOT_INITED</i>   | TYInitLib not called.                                                                                                                                                                    |
| <i>TY_STATUS_NULL_POINTER</i> | TYGetInterfaceNumber called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYGetInterfaceNumber (pNumIfaces);                     ^ is NULL</pre> |

## 5.1.2.39 TYGetIntRange()

```

TY_CAPI TYGetIntRange (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 TY_INT_RANGE * intRange)

```

Get value range of integer feature.

## Parameters

|     |                    |                             |
|-----|--------------------|-----------------------------|
| in  | <i>hDevice</i>     | Device handle.              |
| in  | <i>componentID</i> | Component ID.               |
| in  | <i>featureID</i>   | Feature ID.                 |
| out | <i>intRange</i>    | Integer range to be filled. |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetIntRange called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetIntRange(hDevice, componentID, featureID, pIntRange);</code><br/> <code>^</code> is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYGetIntRange(hDevice, componentID, featureID, pIntRange);</code><br/> <code>^</code> is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                      |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYGetIntRange(hDevice, componentID, featureID, pIntRange);</code><br/> <code>^</code> is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGet<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                     |

## Return values

|                               |                                                                                                                                                                                                                                                                                |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_WRONG_TYPE</i>   | <p>Feature type mismatch</p> <p>Suggestions:</p> <p>Please check the feature type</p> <p>Like this:</p> <pre>TYGetIntRange(hDevice, componentID, featureID, pintRange);</pre> <p>^ type mismatch</p> <p>The feature type you entered does not match. You can use TYFeature</p> |
| <i>TY_STATUS_NULL_POINTER</i> | <p>TYGetIntRange called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYGetIntRange(hDevice, componentID, featureID, pintRange);</pre> <p>^ is NULL</p>                                                                       |

## 5.1.2.40 TYGetString()

```

TY_CAPI TYGetString (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 char * buffer,
 uint32_t bufferSize)

```

Get value of string feature.

## Parameters

|     |                    |                 |
|-----|--------------------|-----------------|
| in  | <i>hDevice</i>     | Device handle.  |
| in  | <i>componentID</i> | Component ID.   |
| in  | <i>featureID</i>   | Feature ID.     |
| out | <i>buffer</i>      | String buffer.  |
| in  | <i>bufferSize</i>  | Size of buffer. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYGetString called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYGetString(hDevice, componentID, featureID, buffer, bufferSize);</pre> <p>^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUp</li> </ol> |



## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:</p> <p>Please check componentID parameter</p> <p>Like this:</p> <pre>TYGetString(hDevice, componentID, featureID, buffer, bufferSize)</pre> <p>^ is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs</p> <p>You can also view the components of the camera by obtaining the x</p>                                                    |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:</p> <p>Please check featureID parameter</p> <p>Like this:</p> <pre>TYGetString(hDevice, componentID, featureID, buffer, bufferSize)</pre> <p>^ is invalid</p> <p>You entered an invalid featureID parameter</p> <p>You can get a list of features of the camera device through TYGet</p> <p>You can also view the features of the camera device by obtaining t</p> |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:</p> <p>Please check the feature type</p> <p>Like this:</p> <pre>TYGetString(hDevice, componentID, featureID, buffer, bufferSize)</pre> <p>^ type mismatch</p> <p>The feature type you entered does not match. You can use TYFeature</p>                                                                                                                         |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYGetString called with NULL pointer</p> <p>Suggestions:</p> <p>Please check your code</p> <p>Like this:</p> <pre>TYGetString(hDevice, componentID, featureID, pBuffer, bufferSize)</pre> <p>^ is NULL</p>                                                                                                                                                                                                |
| <i>TY_STATUS_TIMEOUT</i>           | <p>Failed to get string feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.Network communication is abnormal, please check whether the n</li> </ol>                                                                                                                                                                                                           |
| <i>TY_STATUS_DEVICE_ERROR</i>      | <p>Failed to get string feature</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.The feature of the camera device is not available or not impl</li> <li>2.Camera device is abnormal and cannot get string feature</li> </ol>                                                                                                                                        |

## See also

[TYGetStringLength](#)

## 5.1.2.41 TYGetStringLength()

```
TY_CAPI TYGetStringLength (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
```

```
TY_FEATURE_ID featureID,
uint32_t * size)
```

Get internal buffer size of string feature.

## Parameters

|     |                    |                               |
|-----|--------------------|-------------------------------|
| in  | <i>hDevice</i>     | Device handle.                |
| in  | <i>componentID</i> | Component ID.                 |
| in  | <i>featureID</i>   | Feature ID.                   |
| out | <i>size</i>        | String length including '\0'. |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetString called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYGetString(hDevice, componentID, featureID, buffer, bufferSize);</code><br/> <code>^</code> is invalid</p> <p>The hDevice parameter you input is not recorded<br/>Possible reasons:<br/> 1. TYOpenDevice failed to open device and get correct handle<br/> 2. Memory in stack to store handle data is corrupted<br/> 3. After getting handle, you updated device list by calling TYUpdateDeviceList</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYGetString(hDevice, componentID, featureID, buffer, bufferSize);</code><br/> <code>^</code> is invalid</p> <p>componentID should be the value returned by TYGetComponentIDs<br/>You can also view the components of the camera by obtaining the xrt::device object</p>                                                                                                                                                   |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYGetString(hDevice, componentID, featureID, buffer, bufferSize);</code><br/> <code>^</code> is invalid</p> <p>You entered an invalid featureID parameter<br/>You can get a list of features of the camera device through TYGetFeatureIDs<br/>You can also view the features of the camera device by obtaining the xrt::device object</p>                                                                                     |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <code>TYGetString(hDevice, componentID, featureID, buffer, bufferSize);</code><br/> <code>^</code> type mismatch</p> <p>The feature type you entered does not match. You can use TYFeatureType to get the feature type</p>                                                                                                                                                                                                          |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYGetStringLength called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYGetStringLength(hDevice, componentID, featureID, pLength);</code><br/> <code>^</code> is NULL</p>                                                                                                                                                                                                                                                                                                             |

## See also

[TYGetString](#)

### 5.1.2.42 TYGetStruct()

```

TY_CAPI TYGetStruct (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 void * pStruct,
 uint32_t structSize)

```

Get value of struct.

#### Parameters

|     |                    |                               |
|-----|--------------------|-------------------------------|
| in  | <i>hDevice</i>     | Device handle.                |
| in  | <i>componentID</i> | Component ID.                 |
| in  | <i>featureID</i>   | Feature ID.                   |
| out | <i>pStruct</i>     | Pointer of struct.            |
| in  | <i>structSize</i>  | Size of input buffer pStruct. |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYGetStruct called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYGetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYGetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the xn</p>                                                                                                                                                   |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYGetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetI<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                  |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <pre>TYGetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeature</p>                                                                                                                                                                                                                       |

## Return values

|                               |                                                                                                                                                                                                                          |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_NULL_POINTER</i> | TYGetStruct called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br>TYGetStruct(hDevice, componentID, featureID, pStruct, structSize, ^ is NULL                                          |
| <i>TY_STATUS_WRONG_SIZE</i>   | Struct size mismatch<br><br>Suggestions:<br>Please check the struct size<br>Like this:<br>TYGetStruct(hDevice, componentID, featureID, pStruct, structSize, ^ is inval<br><br>The struct size you entered does not match |
| <i>TY_STATUS_TIMEOUT</i>      | Failed to get struct feature<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the n                                                                                 |
| <i>TY_STATUS_DEVICE_ERROR</i> | Failed to get struct feature<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not imple<br>2.Camera device is abnormal and cannot get struct feature                   |

## 5.1.2.43 TYHasDevice()

```

TY_CAPI TYHasDevice (
 TY_INTERFACE_HANDLE ifaceHandle,
 const char * deviceID,
 bool * value)

```

Check whether the interface has the specified device.

## Parameters

|     |                    |                                                                         |
|-----|--------------------|-------------------------------------------------------------------------|
| in  | <i>ifaceHandle</i> | Interface handle.                                                       |
| in  | <i>deviceID</i>    | Device ID string, can be get from <a href="#">TY_DEVICE_BASE_INFO</a> . |
| out | <i>value</i>       | True if the device exists.                                              |

## Return values

|                             |                       |
|-----------------------------|-----------------------|
| <i>TY_STATUS_OK</i>         | Succeed.              |
| <i>TY_STATUS_NOT_INITED</i> | TYInitLib not called. |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_INTERFACE</i> | <p>TYHasDevice called with invalid interface handle</p> <p>Suggestions:<br/>Please check interface handle<br/>Like this:<br/> <code>TYHasDevice(ifaceHandle, deviceID, value);</code><br/>                           ^ is invalid</p> <p>The ifaceHandle parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenInterface failed to open interface and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated interface list by calling TYU</li> </ol> |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYHasDevice called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYHasDevice(ifaceHandle, deviceID, value);</code><br/>                                                   ^      or      ^ is NULL</p>                                                                                                                                                                                                                                                                                                                                |

## 5.1.2.44 TYHasFeature()

```

TY_CAPI TYHasFeature (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 bool * value)

```

Check whether a component has a specific feature.

## Parameters

|     |                    |                      |
|-----|--------------------|----------------------|
| in  | <i>hDevice</i>     | Device handle.       |
| in  | <i>componentID</i> | Component ID.        |
| in  | <i>featureID</i>   | Feature ID.          |
| out | <i>value</i>       | Whether has feature. |

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYHasFeature called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYHasFeature(hDevice, componentID, featureID, value);</code><br/>                                                   ^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUp</li> </ol> |



### 5.1.2.46 TYLibVersion()

```
TY_CAPI TYLibVersion (
 TY_VERSION_INFO * version)
```

Get current library version.

#### Parameters

|     |                |                                   |
|-----|----------------|-----------------------------------|
| out | <i>version</i> | Version information to be filled. |
|-----|----------------|-----------------------------------|

#### Return values

|                               |                                                                                                                                      |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>           | Succeed.                                                                                                                             |
| <i>TY_STATUS_NULL_POINTER</i> | TYLibVersion called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br>TYLibVersion(ver);<br>^ is NULL |

### 5.1.2.47 TYOpenDevice()

```
TY_CAPI TYOpenDevice (
 TY_INTERFACE_HANDLE ifaceHandle,
 const char * deviceID,
 TY_DEV_HANDLE * outDeviceHandle,
 TY_FW_ERRORCODE * outFwErrorcode = NULL)
```

Open device by device ID.

#### Parameters

|     |                        |                                                                                  |
|-----|------------------------|----------------------------------------------------------------------------------|
| in  | <i>ifaceHandle</i>     | Interface handle.                                                                |
| in  | <i>deviceID</i>        | Device ID string, can be get from <a href="#">TY_DEVICE_BASE_INFO</a> .          |
| out | <i>outDeviceHandle</i> | Handle of opened device. Valid only if TY_STATUS_OK or TY_FW_ERRORCODE returned. |
| out | <i>outFwErrorcode</i>  | Firmware errorcode. Valid only if TY_FW_ERRORCODE returned.                      |

#### Return values

|                             |                       |
|-----------------------------|-----------------------|
| <i>TY_STATUS_OK</i>         | Succeed.              |
| <i>TY_STATUS_NOT_INITED</i> | TYInitLib not called. |



## Return values

|                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_INVALID_INTERFACE</i></b> | <b>TYOpenDevice called with invalid interface handle</b> <p>Suggestions:<br/>Please check interface handle<br/>Like this:<br/> <code>TYOpenDevice(ifaceHandle, deviceID, outDeviceHandle, outFwErrorcode, ^);</code><br/> ^ is invalid<br/> The ifaceHandle parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenInterface failed to open interface and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated interface list by calling TYUpdateDeviceList</p>                                                                                                                   |
| <b><i>TY_STATUS_NULL_POINTER</i></b>      | <b>TYOpenDevice called with NULL pointer</b> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYOpenDevice(ifaceHandle, deviceID, outDeviceHandle, outFwErrorcode, ^);</code><br/> ^ or ^ is NULL</p>                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b><i>TY_STATUS_INVALID_PARAMETER</i></b> | <b>TYOpenDevice called with invalid device ID: s</b> <p>Suggestions:<br/>Please check deviceID parameter<br/>Like this:<br/> <code>TYOpenDevice(ifaceHandle, deviceID, outDeviceHandle, outFwErrorcode, ^);</code><br/> ^ is invalid<br/> Usually you get device information by calling TYUpdateDeviceList, and then open device by calling TYOpenDevice (When your device online status changes); you may need to update device list again</p>                                                                                                                                                                                                                    |
| <b><i>TY_STATUS_BUSY</i></b>              | <b>Failed to open device</b> <p>Suggestions:<br/>Possible reasons:<br/> 1.Camera is occupied, please check if other processes on this machine or other host machines are occupying the camera. If the camera is occupied, please wait for a while.<br/> 2.A third-party program is written into the camera, please contact the camera manufacturer.</p>                                                                                                                                                                                                                                                                                                            |
| <b><i>TY_STATUS_FIRMWARE_ERROR</i></b>    | <b>Device opened successfully, but firmware error code is not 0</b> <p>Suggestions:<br/>Some functions of the device may have exceptions, please check the firmware error code.<br/> <code>TY_FW_ERRORCODE outFwErrorcode;</code><br/> <code>TYOpenDevice(ifaceHandle, deviceID, outDeviceHandle, &amp;outFwErrorcode, ^);</code><br/> if(outFwErrorcode != 0) {<br/>     parse_firmware_errcode(outFwErrorcode);<br/> }</p>                                                                                                                                                                                                                                       |
| <b><i>TY_STATUS_DEVICE_ERROR</i></b>      | <b>Failed to open device</b> <p>Suggestions:<br/>Possible reasons:<br/> 1.A third-party program is written into the camera, please contact the camera manufacturer.<br/> 2.The camera IP address is not in the same network segment as the host.<br/> If the camera IP address is not in the same network segment as the host, you need to set up a routing connection between your host and the camera. Otherwise, you can modify the camera IP address or the host IP address.<br/> If you need to modify the camera IP address, please refer to the camera manual.<br/> 3.Network communication is abnormal, please check whether the network is connected.</p> |

### 5.1.2.48 TYOpenDeviceWithIP()

```
TY_CAPI TYOpenDeviceWithIP (
 TY_INTERFACE_HANDLE ifaceHandle,
 const char * IP,
 TY_DEV_HANDLE * deviceHandle)
```

Open device by device IP, useful when a device is not listed.

#### Parameters

|     |                     |                          |
|-----|---------------------|--------------------------|
| in  | <i>ifaceHandle</i>  | Interface handle.        |
| in  | <i>IP</i>           | Device IP.               |
| out | <i>deviceHandle</i> | Handle of opened device. |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_NOT_INITED</i>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <i>TY_STATUS_INVALID_INTERFACE</i> | <p>TYOpenDeviceWithIP called with invalid interface handle</p> <p>Suggestions:<br/>Please check interface handle<br/>Like this:<br/> <code>TYOpenDeviceWithIP(ifaceHandle, IP, outDeviceHandle);</code><br/> <code>^</code> is invalid</p> <p>The ifaceHandle parameter you input is not recorded</p> <p>Possible reasons:<br/> 1.TYOpenInterface failed to open interface and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated interface list by calling TYUpdateDeviceList</p> |
| <i>TY_STATUS_NULL_POINTER</i>      | <p>TYOpenDeviceWithIP called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYOpenDeviceWithIP(ifaceHandle, IP, outDeviceHandle);</code><br/> <code>^</code> or <code>^</code> is NULL</p>                                                                                                                                                                                                                                                                                                               |
| <i>TY_STATUS_INVALID_PARAMETER</i> | <p>TYOpenDeviceWithIP called with invalid IP address</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYOpenDeviceWithIP(ifaceHandle, IP, outDeviceHandle);</code><br/> <code>^</code> is invalid</p> <p>A valid IP address should be like:<br/> 192.168.31.1</p> <p>Usually you get device information by calling TYUpdateDeviceList,<br/>and then open device by calling TYOpenDevice</p> <p>When your device online status changes,<br/>you may need to update device list again</p>                                      |
| <i>TY_STATUS_BUSY</i>              | <p>Failed to open device</p> <p>Suggestions:<br/>Possible reasons:<br/> 1.Camera is occupied, please check if other processes on this machine or other host machines are occupying the camera. If the camera is occupied, please wait for a while and then try again.<br/> 2.A third-party program is written into the camera, please contact the third-party program developer.</p>                                                                                                                                                                 |

## Return values

|                                      |                                                                                                                                                                                                                                                                                                                           |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_DEVICE_ERROR</i></b> | Failed to open device<br><br>Suggestions:<br>Possible reasons:<br>1.A third-party program is written into the camera, please contact the camera manufacturer.<br>2.The camera IP address cannot communicate with the host IP address.<br>3.Network communication is abnormal, please check whether the network is normal. |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 5.1.2.49 TYOpenInterface()

```
TY_CAPI TYOpenInterface (
 const char * ifaceID,
 TY_INTERFACE_HANDLE * outHandle)
```

Open specified interface.

## Parameters

|     |                  |                                                                          |
|-----|------------------|--------------------------------------------------------------------------|
| in  | <i>ifaceID</i>   | Interface ID string, can be get from <a href="#">TY_INTERFACE_INFO</a> . |
| out | <i>outHandle</i> | Handle of opened interface.                                              |

## Return values

|                                           |                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_OK</i></b>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b><i>TY_STATUS_NOT_INITED</i></b>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b><i>TY_STATUS_NULL_POINTER</i></b>      | TYOpenInterface called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><pre>TYOpenInterface(ifaceID, outHandle);</pre><br>^ or ^ is NULL                                                                                                                                                                                                                                                 |
| <b><i>TY_STATUS_INVALID_INTERFACE</i></b> | TYOpenInterface called with invalid interface ID<br><br>Suggestions:<br>Please check ifaceID parameter<br>Like this:<br><pre>TYOpenInterface(ifaceID, outHandle);</pre><br>^ is invalid<br>Usually you get interface information by calling TYUpdateInterfaceList and then open interface by calling TYOpenInterface<br>When your host interface (network or USB) changes;<br>you may need to update interface list again |

## See also

[TYGetInterfaceList](#)

### 5.1.2.50 TYRegisterEventCallback()

```
TY_CAPI TYRegisterEventCallback (
 TY_DEV_HANDLE hDevice,
 TY_EVENT_CALLBACK callback,
 void * userdata)
```

Register device status callback. Register NULL to clean callback.

#### Parameters

|    |                 |                    |
|----|-----------------|--------------------|
| in | <i>hDevice</i>  | Device handle.     |
| in | <i>callback</i> | Callback function. |
| in | <i>userdata</i> | User private data. |

#### Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>             | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYRegisterEventCallback called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYRegisterEventCallback(hDevice, callback, userdata);</pre> <p>^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdated</li> </ol> |
| <i>TY_STATUS_BUSY</i>           | Device is capturing.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

### 5.1.2.51 TYRegisterImuCallback()

```
TY_CAPI TYRegisterImuCallback (
 TY_DEV_HANDLE hDevice,
 TY_IMU_CALLBACK callback,
 void * userdata)
```

Register imu callback. Register NULL to clean callback.

#### Parameters

|    |                 |                    |
|----|-----------------|--------------------|
| in | <i>hDevice</i>  | Device handle.     |
| in | <i>callback</i> | Callback function. |
| in | <i>userdata</i> | User private data. |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_HANDLE</i> | TYRegisterImuCallback called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYRegisterImuCallback(hDevice, callback, userdata);</pre> ^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdated |
| <i>TY_STATUS_BUSY</i>           | Device is capturing.                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

## 5.1.2.52 TYSendSoftTrigger()

```
TY_CAPI TYSendSoftTrigger (
 TY_DEV_HANDLE hDevice)
```

Send a software trigger to capture a frame when device works in trigger mode.

## Parameters

|    |                |                |
|----|----------------|----------------|
| in | <i>hDevice</i> | Device handle. |
|----|----------------|----------------|

## Return values

|                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>              | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <i>TY_STATUS_INVALID_HANDLE</i>  | TYSendSoftTrigger called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br><pre>TYSendSoftTrigger(hDevice);</pre> ^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdated |
| <i>TY_STATUS_INVALID_FEATURE</i> | Not support soft trigger.                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <i>TY_STATUS_IDLE</i>            | Camera device is not started<br><br>Suggestions:<br>Please start the camera device first<br>Like this:<br><pre>TYStartCapture(hDevice);</pre> <pre>TYSendSoftTrigger(hDevice);</pre>                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_WRONG_MODE</i>      | Not in trigger mode.                                                                                                                                                                                                                                                                                                                                                                                                                             |

## Return values

|                               |                                                                                                                                                                                                                       |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_DEVICE_ERROR</i> | Failed to send soft trigger<br><br>Suggestions:<br>Possible reasons:<br>1.Camera device is abnormal and cannot send soft trigger.<br>2.Network communication is abnormal, please check whether the network is normal. |
| <i>TY_STATUS_BUSY</i>         | Failed to send soft trigger<br><br>Suggestions:<br>Possible reasons:<br>1.Camera is busy, the last soft trigger is not completed, please try again later.                                                             |

## 5.1.2.53 TYSetBool()

```

TY_CAPI TYSetBool (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 bool value)

```

Set value of bool feature.

## Parameters

|    |                    |                |
|----|--------------------|----------------|
| in | <i>hDevice</i>     | Device handle. |
| in | <i>componentID</i> | Component ID.  |
| in | <i>featureID</i>   | Feature ID.    |
| in | <i>value</i>       | Bool value.    |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_INVALID_HANDLE</i>    | TYSetBool called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br>TYSetBool(hDevice, componentID, featureID, value);<br>^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdateDeviceList. |
| <i>TY_STATUS_INVALID_COMPONENT</i> | Invalid component ID<br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br>TYSetBool(hDevice, componentID, featureID, value);<br>^ is invalid<br>componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the x                                                                                                                                                          |

## Return values

|                                  |                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_FEATURE</i> | Invalid feature ID<br><br>Suggestions:<br>Please check featureID parameter<br>Like this:<br><pre>TYSetBool(hDevice, componentID, featureID, value);</pre> ^ is invalid<br>You entered an invalid featureID parameter<br>You can get a list of features of the camera device through TYGetFeatures<br>You can also view the features of the camera device by obtaining the feature list |
| <i>TY_STATUS_NOT_PERMITTED</i>   | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                           |
| <i>TY_STATUS_WRONG_TYPE</i>      | Feature type mismatch<br><br>Suggestions:<br>Please check the feature type<br>Like this:<br><pre>TYSetBool(hDevice, componentID, featureID, value);</pre> ^ type mismatch<br>The feature type you entered does not match. You can use TYFeatureType to get the feature type                                                                                                            |
| <i>TY_STATUS_BUSY</i>            | Device is capturing, the feature is locked.                                                                                                                                                                                                                                                                                                                                            |
| <i>TY_STATUS_TIMEOUT</i>         | Failed to set bool feature<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the network is connected                                                                                                                                                                                                                              |
| <i>TY_STATUS_DEVICE_ERROR</i>    | Failed to set bool feature<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not implemented<br>2.Camera device is abnormal and cannot set bool feature                                                                                                                                                                               |

## 5.1.2.54 TYSetByteArray()

```

TY_CAPI TYSetByteArray (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 const uint8_t * pBuffer,
 uint32_t bufferSize)

```

Write byte array to device.

## Parameters

|     |                    |                 |
|-----|--------------------|-----------------|
| in  | <i>hDevice</i>     | Device handle.  |
| in  | <i>componentID</i> | Component ID.   |
| in  | <i>featureID</i>   | Feature ID.     |
| out | <i>pBuffer</i>     | Byte buffer.    |
| in  | <i>bufferSize</i>  | Size of buffer. |

## Return values

|                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>TY_STATUS_OK</i></b>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b><i>TY_STATUS_INVALID_HANDLE</i></b>    | <p>TYSetByteArray called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYSetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize)</code><br/> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpdateDeviceList</p> |
| <b><i>TY_STATUS_INVALID_COMPONENT</i></b> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYSetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize)</code><br/> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the xrtDeviceList</p>                                                                                                                                                        |
| <b><i>TY_STATUS_INVALID_FEATURE</i></b>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYSetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize)</code><br/> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetFeatureIDs<br/> You can also view the features of the camera device by obtaining the xrtDeviceList</p>                                                                                         |
| <b><i>TY_STATUS_NOT_PERMITTED</i></b>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b><i>TY_STATUS_WRONG_TYPE</i></b>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <code>TYSetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize)</code><br/> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeatureType to get the feature type</p>                                                                                                                                                                                                           |
| <b><i>TY_STATUS_NULL_POINTER</i></b>      | <p>TYSetByteArray called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <code>TYSetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize)</code><br/> ^ is NULL</p>                                                                                                                                                                                                                                                                                                          |
| <b><i>TY_STATUS_WRONG_SIZE</i></b>        | <p>Array size mismatch</p> <p>Suggestions:<br/>Please check the array size<br/>Like this:<br/> <code>TYSetByteArray(hDevice, componentID, featureID, pBuffer, bufferSize)</code><br/> ^ is incorrect<br/> The array size you entered does not match</p>                                                                                                                                                                                                                                                                     |
| <b><i>TY_STATUS_BUSY</i></b>              | Device is capturing, the feature is locked.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b><i>TY_STATUS_TIMEOUT</i></b>           | <p>Failed to set byte array feature</p> <p>Suggestions:<br/>Possible reasons:<br/> 1.Network communication is abnormal, please check whether the network is connected</p>                                                                                                                                                                                                                                                                                                                                                   |



## Return values

|                               |                                                                                                                                                                                                                      |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_DEVICE_ERROR</i> | Failed to set byte array feature<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not implemented<br>2.Camera device is abnormal and cannot set byte array feature |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 5.1.2.55 TYSetEnum()

```
TY_CAPI TYSetEnum (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 uint32_t value)
```

Set value of enum feature.

## Parameters

|    |                    |                |
|----|--------------------|----------------|
| in | <i>hDevice</i>     | Device handle. |
| in | <i>componentID</i> | Component ID.  |
| in | <i>featureID</i>   | Feature ID.    |
| in | <i>value</i>       | Enum value.    |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <i>TY_STATUS_INVALID_HANDLE</i>    | TYSetEnum called with invalid device handle<br><br>Suggestions:<br>Please check device handle<br>Like this:<br>TYSetEnum(hDevice, componentID, featureID, value);<br>^ is invalid<br>The hDevice parameter you input is not recorded<br>Possible reasons:<br>1.TYOpenDevice failed to open device and get correct handle<br>2.Memory in stack to store handle data is corrupted<br>3.After getting handle, you updated device list by calling TYUpdateDeviceList |
| <i>TY_STATUS_INVALID_COMPONENT</i> | Invalid component ID<br><br>Suggestions:<br>Please check componentID parameter<br>Like this:<br>TYSetEnum(hDevice, componentID, featureID, value);<br>^ is invalid<br>componentID should be the value returned by TYGetComponentIDs<br>You can also view the components of the camera by obtaining the xrt::device::get_components()                                                                                                                             |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_FEATURE</i>   | Invalid feature ID<br><br>Suggestions:<br>Please check featureID parameter<br>Like this:<br><pre>TYSetEnum(hDevice, componentID, featureID, value);</pre> ^ is invalid<br><br>You entered an invalid featureID parameter<br>You can get a list of features of the camera device through TYGetEnumEntryInfo<br>You can also view the features of the camera device by obtaining the feature list |
| <i>TY_STATUS_NOT_PERMITTED</i>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                    |
| <i>TY_STATUS_WRONG_TYPE</i>        | Feature type mismatch<br><br>Suggestions:<br>Please check the feature type<br>Like this:<br><pre>TYSetEnum(hDevice, componentID, featureID, value);</pre> ^ type mismatch<br><br>The feature type you entered does not match. You can use TYFeatureType to get the feature type                                                                                                                 |
| <i>TY_STATUS_INVALID_PARAMETER</i> | Out of range<br><br>Suggestions:<br>Please check the value<br>Like this:<br><pre>TYSetEnum(hDevice, componentID, featureID, value);</pre> ^ is out of range<br><br>The value is out of range, please use TYGetEnumEntryInfo to get the feature value range                                                                                                                                      |
| <i>TY_STATUS_BUSY</i>              | Device is capturing, the feature is locked.                                                                                                                                                                                                                                                                                                                                                     |
| <i>TY_STATUS_TIMEOUT</i>           | Failed to set enum feature<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the network is connected                                                                                                                                                                                                                                       |
| <i>TY_STATUS_DEVICE_ERROR</i>      | Failed to set enum feature<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not implemented<br>2.Camera device is abnormal and cannot set enum feature                                                                                                                                                                                        |

## 5.1.2.56 TYSetFloat()

```

TY_CAPI TYSetFloat (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 float value)

```

Set value of float feature.

## Parameters

|    |                    |                |
|----|--------------------|----------------|
| in | <i>hDevice</i>     | Device handle. |
| in | <i>componentID</i> | Component ID.  |
| in | <i>featureID</i>   | Feature ID.    |
| in | <i>value</i>       | Float value.   |

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYSetFloat called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <code>TYSetFloat(hDevice, componentID, featureID, value);</code><br/> <code>^</code> is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUp</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <code>TYSetFloat(hDevice, componentID, featureID, value);</code><br/> <code>^</code> is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the xn</p>                                                                                                                                                 |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <code>TYSetFloat(hDevice, componentID, featureID, value);</code><br/> <code>^</code> is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetf<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                |
| <i>TY_STATUS_NOT_PERMITTED</i>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_WRONG_TYPE</i>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <code>TYSetFloat(hDevice, componentID, featureID, value);</code><br/> <code>^</code> type mismatch<br/> The feature type you entered does not match. You can use TYFeatur</p>                                                                                                                                                                                                                      |
| <i>TY_STATUS_OUT_OF_RANGE</i>      | <p>Out of range</p> <p>Suggestions:<br/>Please check the value<br/>Like this:<br/> <code>TYSetFloat(hDevice, componentID, featureID, value);</code><br/> <code>^</code> is out of range<br/> The value is out of range, please use TYGetFloatRange to get the r</p>                                                                                                                                                                                                                                   |
| <i>TY_STATUS_BUSY</i>              | Device is capturing, the feature is locked.                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>TY_STATUS_TIMEOUT</i>           | <p>Failed to set float feature</p> <p>Suggestions:<br/>Possible reasons:<br/>1.Network communication is abnormal, please check whether the ne</p>                                                                                                                                                                                                                                                                                                                                                     |
| <i>TY_STATUS_DEVICE_ERROR</i>      | <p>Failed to set float feature</p> <p>Suggestions:<br/>Possible reasons:<br/>1.The feature of the camera device is not available or not imple<br/>2.Camera device is abnormal and cannot set float feature</p>                                                                                                                                                                                                                                                                                        |

### 5.1.2.57 TYSetInt()

```

TY_CAPI TYSetInt (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 int32_t value)

```

Set value of integer feature.

#### Parameters

|    |                    |                |
|----|--------------------|----------------|
| in | <i>hDevice</i>     | Device handle. |
| in | <i>componentID</i> | Component ID.  |
| in | <i>featureID</i>   | Feature ID.    |
| in | <i>value</i>       | Integer value. |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYSetInt called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYSetInt(hDevice, componentID, featureID, value);</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYSetInt(hDevice, componentID, featureID, value);</pre> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                 |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYSetInt(hDevice, componentID, featureID, value);</pre> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetf<br/> You can also view the features of the camera device by obtaining t</p>                                                                                               |
| <i>TY_STATUS_NOT_PERMITTED</i>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

## Return values

|                               |                                                                                                                                                                                                                                                                                                               |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_WRONG_TYPE</i>   | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <code>TYSetInt(hDevice, componentID, featureID, value);</code><br/> <span style="margin-left: 150px;">^ type mismatch</span></p> <p>The feature type you entered does not match. You can use TYFeature</p> |
| <i>TY_STATUS_OUT_OF_RANGE</i> | <p>Out of range</p> <p>Suggestions:<br/>Please check the value<br/>Like this:<br/> <code>TYSetInt(hDevice, componentID, featureID, value);</code><br/> <span style="margin-left: 150px;">^ is out of range</span></p> <p>The value is out of range, please use TYGetIntRange to get the ran</p>               |
| <i>TY_STATUS_BUSY</i>         | Device is capturing, the feature is locked.                                                                                                                                                                                                                                                                   |
| <i>TY_STATUS_TIMEOUT</i>      | <p>Failed to set int feature</p> <p>Suggestions:<br/>Possible reasons:<br/>1.Network communication is abnormal, please check whether the ne</p>                                                                                                                                                               |
| <i>TY_STATUS_DEVICE_ERROR</i> | <p>Failed to set int feature</p> <p>Suggestions:<br/>Possible reasons:<br/>1.The feature of the camera device is not available or not impl<br/>2.Camera device is abnormal and cannot set int feature</p>                                                                                                     |

#### 5.1.2.58 TYSetLogLevel()

```
TY_CAPI TYSetLogLevel (
 TY_LOG_TYPE type,
 TY_LOG_LEVEL lvl)
```

Set log level.

### Parameters

|    |             |              |
|----|-------------|--------------|
| in | <i>type</i> | Logger type. |
| in | <i>lvl</i>  | Log level.   |

### Return values

|              |          |
|--------------|----------|
| TY STATUS OK | Succeed. |
|--------------|----------|

### 5.1.2.59 TYSetLogPrefix()

```
TY_CAPI TYSetLogPrefix (
 const char * prefix)
```

set log prefix

#### Parameters

|    |               |                |
|----|---------------|----------------|
| in | <i>prefix</i> | Prefix string. |
|----|---------------|----------------|

#### Return values

|                                    |                                                                                                                                                                                                       |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                              |
| <i>TY_STATUS_INVALID_PARAMETER</i> | Prefix is empty or prefix is too long<br><br>Suggestions:<br>Prefix is empty or prefix is too long, cannot be set<br>Like this:<br>TYSetLogPrefix(prefix);<br>^ prefix is empty or prefix is too long |

### 5.1.2.60 TYSetString()

```
TY_CAPI TYSetString (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 const char * buffer)
```

Set value of string feature.

#### Parameters

|    |                    |                |
|----|--------------------|----------------|
| in | <i>hDevice</i>     | Device handle. |
| in | <i>componentID</i> | Component ID.  |
| in | <i>featureID</i>   | Feature ID.    |
| in | <i>buffer</i>      | String buffer. |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TY_STATUS_INVALID_HANDLE</b>    | <p>TYSetString called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYSetString(hDevice, componentID, featureID, pBuffer);</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <b>TY_STATUS_INVALID_COMPONENT</b> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYSetString(hDevice, componentID, featureID, pBuffer);</pre> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                    |
| <b>TY_STATUS_INVALID_FEATURE</b>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYSetString(hDevice, componentID, featureID, pBuffer);</pre> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGetF<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                  |
| <b>TY_STATUS_NOT_PERMITTED</b>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>TY_STATUS_WRONG_TYPE</b>        | <p>Feature type mismatch</p> <p>Suggestions:<br/>Please check the feature type<br/>Like this:<br/> <pre>TYSetString(hDevice, componentID, featureID, pBuffer);</pre> ^ type mismatch<br/> The feature type you entered does not match. You can use TYFeature</p>                                                                                                                                                                                                                       |
| <b>TY_STATUS_NULL_POINTER</b>      | <p>TYSetString called with NULL pointer</p> <p>Suggestions:<br/>Please check your code<br/>Like this:<br/> <pre>TYSetString(hDevice, componentID, featureID, pBuffer);</pre> ^ is NULL</p>                                                                                                                                                                                                                                                                                             |
| <b>TY_STATUS_OUT_OF_RANGE</b>      | Input string is too long.                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>TY_STATUS_BUSY</b>              | Device is capturing, the feature is locked.                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>TY_STATUS_TIMEOUT</b>           | <p>Failed to set string feature</p> <p>Suggestions:<br/>Possible reasons:<br/>1.Network communication is abnormal, please check whether the n</p>                                                                                                                                                                                                                                                                                                                                      |
| <b>TY_STATUS_DEVICE_ERROR</b>      | <p>Failed to set string feature</p> <p>Suggestions:<br/>Possible reasons:<br/>1.The feature of the camera device is not available or not impl<br/>2.Camera device is abnormal and cannot set string feature</p>                                                                                                                                                                                                                                                                        |

### 5.1.2.61 TYSetStruct()

```

TY_CAPI TYSetStruct (
 TY_DEV_HANDLE hDevice,
 TY_COMPONENT_ID componentID,
 TY_FEATURE_ID featureID,
 void * pStruct,
 uint32_t structSize)

```

Set value of struct.

#### Parameters

|    |                    |                    |
|----|--------------------|--------------------|
| in | <i>hDevice</i>     | Device handle.     |
| in | <i>componentID</i> | Component ID.      |
| in | <i>featureID</i>   | Feature ID.        |
| in | <i>pStruct</i>     | Pointer of struct. |
| in | <i>structSize</i>  | Size of struct.    |

#### Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYSetStruct called with invalid device handle</p> <p>Suggestions:<br/>Please check device handle<br/>Like this:<br/> <pre>TYSetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ is invalid<br/> The hDevice parameter you input is not recorded<br/> Possible reasons:<br/> 1.TYOpenDevice failed to open device and get correct handle<br/> 2.Memory in stack to store handle data is corrupted<br/> 3.After getting handle, you updated device list by calling TYUpd</p> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>Invalid component ID</p> <p>Suggestions:<br/>Please check componentID parameter<br/>Like this:<br/> <pre>TYSetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ is invalid<br/> componentID should be the value returned by TYGetComponentIDs<br/> You can also view the components of the camera by obtaining the x</p>                                                                                                                                                    |
| <i>TY_STATUS_INVALID_FEATURE</i>   | <p>Invalid feature ID</p> <p>Suggestions:<br/>Please check featureID parameter<br/>Like this:<br/> <pre>TYSetStruct(hDevice, componentID, featureID, pStruct, structSize)</pre> ^ is invalid<br/> You entered an invalid featureID parameter<br/> You can get a list of features of the camera device through TYGet<br/> You can also view the features of the camera device by obtaining t</p>                                                                                                   |
| <i>TY_STATUS_NOT_PERMITTED</i>     | The feature is not writable.                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |



## Return values

|                               |                                                                                                                                                                                                                                                                   |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_WRONG_TYPE</i>   | Feature type mismatch<br><br>Suggestions:<br>Please check the feature type<br>Like this:<br><code>TYSetStruct(hDevice, componentID, featureID, pStruct, structSize, ^ type mismatch)</code><br>The feature type you entered does not match. You can use TYFeature |
| <i>TY_STATUS_NULL_POINTER</i> | TYSetStruct called with NULL pointer<br><br>Suggestions:<br>Please check your code<br>Like this:<br><code>TYSetStruct(hDevice, componentID, featureID, pStruct, structSize, ^ is NULL)</code>                                                                     |
| <i>TY_STATUS_WRONG_SIZE</i>   | Struct size mismatch<br><br>Suggestions:<br>Please check the struct size<br>Like this:<br><code>TYSetStruct(hDevice, componentID, featureID, pStruct, structSize, ^ is inva</code><br>The struct size you entered does not match                                  |
| <i>TY_STATUS_BUSY</i>         | Device is capturing, the feature is locked.                                                                                                                                                                                                                       |
| <i>TY_STATUS_TIMEOUT</i>      | Failed to set struct feature<br><br>Suggestions:<br>Possible reasons:<br>1.Network communication is abnormal, please check whether the ne                                                                                                                         |
| <i>TY_STATUS_DEVICE_ERROR</i> | Failed to set struct feature<br><br>Suggestions:<br>Possible reasons:<br>1.The feature of the camera device is not available or not impl<br>2.Camera device is abnormal and cannot set struct feature                                                             |

## 5.1.2.62 TYStartCapture()

```
TY_CAPI TYStartCapture (
 TY_DEV_HANDLE hDevice)
```

Start capture.

## Parameters

|    |                |                |
|----|----------------|----------------|
| in | <i>hDevice</i> | Device handle. |
|----|----------------|----------------|

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_HANDLE</i>    | <p>TYStartCapture called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYStartCapture(hDevice);</pre> <p>^ is invalid</p> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUp</li> </ol> |
| <i>TY_STATUS_INVALID_COMPONENT</i> | <p>No components are enabled</p> <p>Suggestions:</p> <p>Please enable the components of the camera device first</p> <p>Like this:</p> <pre>TYEnableComponents(hDevice, componentIDs);</pre> <pre>TYStartCapture(hDevice);</pre>                                                                                                                                                                                                                                                                                             |
| <i>TY_STATUS_BUSY</i>              | <p>Camera device has been started.</p> <p>Suggestions:</p> <p>Please stop the camera device first</p> <p>Like this:</p> <pre>TYStopCapture(hDevice);</pre> <pre>TYStartCapture(hDevice);</pre>                                                                                                                                                                                                                                                                                                                              |
| <i>TY_STATUS_DEVICE_ERROR</i>      | <p>Failed to start camera</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.Camera device is abnormal and cannot start the camera.</li> <li>2.Network communication is abnormal, please check whether the n</li> <li>3.Camera is busy, please try again</li> </ol>                                                                                                                                                                                                                  |

## 5.1.2.63 TYStopCapture()

```
TY_CAPI TYStopCapture (
 TY_DEV_HANDLE hDevice)
```

Stop capture.

## Parameters

|    |                |                |
|----|----------------|----------------|
| in | <i>hDevice</i> | Device handle. |
|----|----------------|----------------|

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## Return values

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_INVALID_HANDLE</i> | <p>TYStopCapture called with invalid device handle</p> <p>Suggestions:</p> <p>Please check device handle</p> <p>Like this:</p> <pre>TYStopCapture(hDevice);     ^     is invalid</pre> <p>The hDevice parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenDevice failed to open device and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated device list by calling TYUpdated</li> </ol> |
| <i>TY_STATUS_IDLE</i>           | <p>Camera device has been stopped</p> <p>Suggestions:</p> <p>The camera device has stopped, usually after starting</p> <p>Like this:</p> <pre>TYStartCapture(hDevice); TYStopCapture(hDevice);</pre>                                                                                                                                                                                                                                                                                                                            |
| <i>TY_STATUS_DEVICE_ERROR</i>   | <p>Failed to stop camera</p> <p>Suggestions:</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.Camera device is abnormal and cannot stop the camera.</li> </ol>                                                                                                                                                                                                                                                                                                                                             |

## 5.1.2.64 TYUpdateAllDeviceList()

```
TY_CAPI TYUpdateAllDeviceList (
 void)
```

Update current connected devices.

## Return values

|                             |                       |
|-----------------------------|-----------------------|
| <i>TY_STATUS_OK</i>         | Succeed.              |
| <i>TY_STATUS_NOT_INITED</i> | TYInitLib not called. |

## 5.1.2.65 TYUpdateDeviceList()

```
TY_CAPI TYUpdateDeviceList (
 TY_INTERFACE_HANDLE ifaceHandle)
```

Update current connected devices.

## Parameters

|    |                    |                   |
|----|--------------------|-------------------|
| in | <i>ifaceHandle</i> | Interface handle. |
|----|--------------------|-------------------|

## Return values

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <i>TY_STATUS_NOT_INITED</i>        | TYInitLib not called.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <i>TY_STATUS_INVALID_INTERFACE</i> | <p>TYUpdateDeviceList called with invalid interface handle</p> <p>Suggestions:</p> <p>Please check interface handle</p> <p>Like this:</p> <pre>TYUpdateDeviceList(ifaceHandle);                     ^ is invalid</pre> <p>The ifaceHandle parameter you input is not recorded</p> <p>Possible reasons:</p> <ol style="list-style-type: none"> <li>1.TYOpenInterface failed to open interface and get correct handle</li> <li>2.Memory in stack to store handle data is corrupted</li> <li>3.After getting handle, you updated interface list by calling TYU</li> </ol> |

## 5.1.2.66 TYUpdateInterfaceList()

```
TY_CAPI TYUpdateInterfaceList (
 void)
```

Update current interfaces. call before TYGetInterfaceList.

## Return values

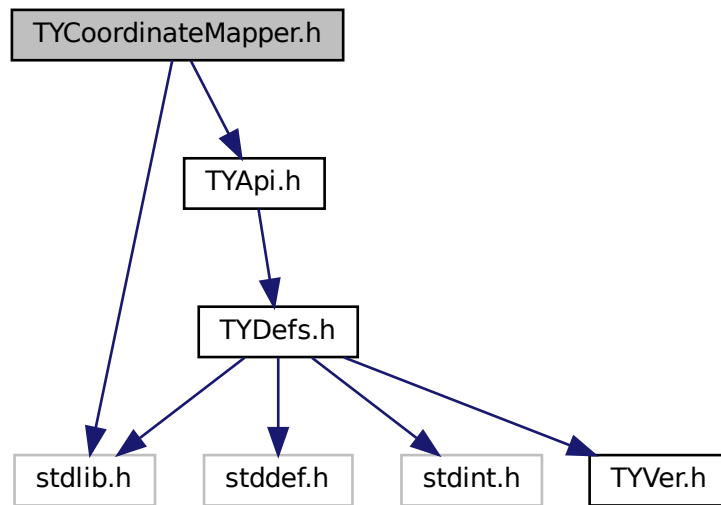
|                             |                       |
|-----------------------------|-----------------------|
| <i>TY_STATUS_OK</i>         | Succeed.              |
| <i>TY_STATUS_NOT_INITED</i> | TYInitLib not called. |

## 5.2 TYCoordinateMapper.h File Reference

Coordinate Conversion API.

```
#include <stdlib.h>
#include "TYApi.h"
```

Include dependency graph for TYCoordinateMapper.h:



## Classes

- struct [TY\\_PIXEL\\_DESC](#)
- struct [TY\\_PIXEL\\_COLOR\\_DESC](#)

## Typedefs

- typedef struct [TY\\_PIXEL\\_DESC](#) [TY\\_PIXEL\\_DESC](#)
- typedef struct [TY\\_PIXEL\\_COLOR\\_DESC](#) [TY\\_PIXEL\\_COLOR\\_DESC](#)

## Functions

- TY\_CAPI [TYInvertExtrinsic](#) (const [TY\\_CAMERA\\_EXTRINSIC](#) \*orgExtrinsic, [TY\\_CAMERA\\_EXTRINSIC](#) \*invExtrinsic)  
*Calculate 4x4 extrinsic matrix's inverse matrix.*
- TY\_CAPI [TYMapDepthToPoint3d](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*src\_calib, uint32\_t depthW, uint32\_t depthH, const [TY\\_PIXEL\\_DESC](#) \*depthPixels, uint32\_t count, [TY\\_VECT\\_3F](#) \*point3d, float f\_scale\_unit=1.0f)  
*Map pixels on depth image to 3D points.*
- TY\_CAPI [TYMapPoint3dToDepth](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*dst\_calib, const [TY\\_VECT\\_3F](#) \*point3d, uint32\_t count, uint32\_t depthW, uint32\_t depthH, [TY\\_PIXEL\\_DESC](#) \*depth, float f\_scale\_unit=1.0f)  
*Map 3D points to pixels on depth image. Reverse operation of TYMapDepthToPoint3d.*
- TY\_CAPI [TYMapDepthImageToPoint3d](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*src\_calib, int32\_t imageW, int32\_t imageH, const uint16\_t \*depth, [TY\\_VECT\\_3F](#) \*point3d, float f\_scale\_unit=1.0f)  
*Map depth image to 3D points. 0 depth pixels maps to (NaN, NaN, NaN).*

- TY\_CAPI [TYMapPoint3dToDepthImage](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*dst\_calib, const [TY\\_VECT\\_3F](#) \*point3d, uint32\_t count, uint32\_t depthW, uint32\_t depthH, uint16\_t \*depth, float f\_target\_scale=1.0f)  
*Map 3D points to depth image. (NAN, NAN, NAN) will be skipped.*
- TY\_CAPI [TYMapPoint3dToPoint3d](#) (const [TY\\_CAMERA\\_EXTRINSIC](#) \*extrinsic, const [TY\\_VECT\\_3F](#) \*point3dFrom, int32\_t count, [TY\\_VECT\\_3F](#) \*point3dTo)  
*Map 3D points to another coordinate.*
- TY\_CAPI [TYMapDepthToColorCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const [TY\\_PIXEL\\_DESC](#) \*depth, uint32\_t count, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t mappedW, uint32\_t mappedH, [TY\\_PIXEL\\_DESC](#) \*mappedDepth, float f\_scale\_unit=1.0f)  
*Map depth pixels to color coordinate pixels.*
- TY\_CAPI [TYMapDepthImageToColorCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const uint16\_t \*depth, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t mappedW, uint32\_t mappedH, uint16\_t \*mappedDepth, float f\_scale\_unit=1.0f)  
*Map original depth image to color coordinate depth image.*
- TY\_CAPI [TYMapRGBPixelsToDepthCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const uint16\_t \*depth, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t rgbW, uint32\_t rgbH, [TY\\_PIXEL\\_COLOR\\_DESC](#) \*src, uint32\_t cnt, uint32\_t min\_distance, uint32\_t max\_distance, [TY\\_PIXEL\\_COLOR\\_DESC](#) \*dst, float f\_scale\_unit=1.0f)  
*Map original RGB pixels to depth coordinate.*
- TY\_CAPI [TYMapRGBImageToDepthCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const uint16\_t \*depth, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t rgbW, uint32\_t rgbH, const uint8\_t \*inRgb, uint8\_t \*mappedRgb, float f\_scale\_unit=1.0f)  
*Map original RGB image to depth coordinate RGB image.*
- TY\_CAPI [TYMapRGB48ImageToDepthCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const uint16\_t \*depth, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t rgbW, uint32\_t rgbH, const uint16\_t \*inRgb, uint16\_t \*mappedRgb, float f\_scale\_unit=1.0f)  
*Map original RGB48 image to depth coordinate RGB image.*
- TY\_CAPI [TYMapMono16ImageToDepthCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const uint16\_t \*depth, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t rgbW, uint32\_t rgbH, const uint16\_t \*gray, uint16\_t \*mappedGray, float f\_scale\_unit=1.0f)  
*Map original MONO16 image to depth coordinate MONO16 image.*
- TY\_CAPI [TYMapMono8ImageToDepthCoordinate](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*depth\_calib, uint32\_t depthW, uint32\_t depthH, const uint16\_t \*depth, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*color\_calib, uint32\_t monoW, uint32\_t monoH, const uint8\_t \*inMono, uint8\_t \*mappedMono, float f\_scale\_unit=1.0f)  
*Map original MONO8 image to depth coordinate MONO8 image.*

## 5.2.1 Detailed Description

Coordinate Conversion API.

### Note

Considering performance, we leave the responsibility of parameters check to users.

### Copyright

Copyright(C)2016-2018 Percipio All Rights Reserved

## 5.2.2 Function Documentation

### 5.2.2.1 TYInvertExtrinsic()

```
TY_CAPI TYInvertExtrinsic (
 const TY_CAMERA_EXTRINSIC * orgExtrinsic,
 TY_CAMERA_EXTRINSIC * invExtrinsic)
```

Calculate 4x4 extrinsic matrix's inverse matrix.

#### Parameters

|     |                     |                         |
|-----|---------------------|-------------------------|
| in  | <i>orgExtrinsic</i> | Input extrinsic matrix. |
| out | <i>invExtrinsic</i> | Inverse matrix.         |

#### Return values

|                        |                     |
|------------------------|---------------------|
| <i>TY_STATUS_OK</i>    | Succeed.            |
| <i>TY_STATUS_ERROR</i> | Calculation failed. |

### 5.2.2.2 TYMapDepthImageToColorCoordinate()

```
TY_CAPI TYMapDepthImageToColorCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
 uint32_t depthW,
 uint32_t depthH,
 const uint16_t * depth,
 const TY_CAMERA_CALIB_INFO * color_calib,
 uint32_t mappedW,
 uint32_t mappedH,
 uint16_t * mappedDepth,
 float f_scale_unit = 1.0f)
```

Map original depth image to color coordinate depth image.

#### Parameters

|     |                    |                                 |
|-----|--------------------|---------------------------------|
| in  | <i>depth_calib</i> | Depth image's calibration data. |
| in  | <i>depthW</i>      | Width of current depth image.   |
| in  | <i>depthH</i>      | Height of current depth image.  |
| in  | <i>depth</i>       | Depth image.                    |
| in  | <i>color_calib</i> | Color image's calibration data. |
| in  | <i>mappedW</i>     | Width of target depth image.    |
| in  | <i>mappedH</i>     | Height of target depth image.   |
| out | <i>mappedDepth</i> | Output pixels.                  |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 5.2.2.3 TYMapDepthImageToPoint3d()

```
TY_CAPI TYMapDepthImageToPoint3d (
 const TY_CAMERA_CALIB_INFO * src_calib,
 int32_t imageW,
 int32_t imageH,
 const uint16_t * depth,
 TY_VECT_3F * point3d,
 float f_scale_unit = 1.0f)
```

Map depth image to 3D points. 0 depth pixels maps to (NaN, NaN, NaN).

#### Parameters

|     |                  |                                 |
|-----|------------------|---------------------------------|
| in  | <i>src_calib</i> | Depth image's calibration data. |
| in  | <i>depthW</i>    | Width of depth image.           |
| in  | <i>depthH</i>    | Height of depth image.          |
| in  | <i>depth</i>     | Depth image.                    |
| out | <i>point3d</i>   | Output point3D image.           |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 5.2.2.4 TYMapDepthToColorCoordinate()

```
TY_CAPI TYMapDepthToColorCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
 uint32_t depthW,
 uint32_t depthH,
 const TY_PIXEL_DESC * depth,
 uint32_t count,
 const TY_CAMERA_CALIB_INFO * color_calib,
 uint32_t mappedW,
 uint32_t mappedH,
 TY_PIXEL_DESC * mappedDepth,
 float f_scale_unit = 1.0f)
```

Map depth pixels to color coordinate pixels.

#### Parameters

|    |                    |                                 |
|----|--------------------|---------------------------------|
| in | <i>depth_calib</i> | Depth image's calibration data. |
| in | <i>depthW</i>      | Width of current depth image.   |
| in | <i>depthH</i>      | Height of current depth image.  |
| in | <i>depth</i>       | Depth image pixels.             |



## Parameters

|     |                    |                                 |
|-----|--------------------|---------------------------------|
| in  | <i>count</i>       | Number of depth image pixels.   |
| in  | <i>color_calib</i> | Color image's calibration data. |
| in  | <i>mappedW</i>     | Width of target depth image.    |
| in  | <i>mappedH</i>     | Height of target depth image.   |
| out | <i>mappedDepth</i> | Output pixels.                  |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## 5.2.2.5 TYMapDepthToPoint3d()

```

TY_CAPI TYMapDepthToPoint3d (
 const TY_CAMERA_CALIB_INFO * src_calib,
 uint32_t depthW,
 uint32_t depthH,
 const TY_PIXEL_DESC * depthPixels,
 uint32_t count,
 TY_VECT_3F * point3d,
 float f_scale_unit = 1.0f)

```

Map pixels on depth image to 3D points.

## Parameters

|     |                    |                                 |
|-----|--------------------|---------------------------------|
| in  | <i>src_calib</i>   | Depth image's calibration data. |
| in  | <i>depthW</i>      | Width of depth image.           |
| in  | <i>depthH</i>      | Height of depth image.          |
| in  | <i>depthPixels</i> | Pixels on depth image.          |
| in  | <i>count</i>       | Number of depth pixels.         |
| out | <i>point3d</i>     | Output point3D.                 |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## 5.2.2.6 TYMapMono16ImageToDepthCoordinate()

```

TY_CAPI TYMapMono16ImageToDepthCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
 uint32_t depthW,

```

```

uint32_t depthH,
const uint16_t * depth,
const TY_CAMERA_CALIB_INFO * color_calib,
uint32_t rgbW,
uint32_t rgbH,
const uint16_t * gray,
uint16_t * mappedGray,
float f_scale_unit = 1.0f)

```

Map original MONO16 image to depth coordinate MONO16 image.

#### Parameters

|     |                    |                                 |
|-----|--------------------|---------------------------------|
| in  | <i>depth_calib</i> | Depth image's calibration data. |
| in  | <i>depthW</i>      | Width of current depth image.   |
| in  | <i>depthH</i>      | Height of current depth image.  |
| in  | <i>depth</i>       | Current depth image.            |
| in  | <i>color_calib</i> | Color image's calibration data. |
| in  | <i>rgbW</i>        | Width of MONO16 image.          |
| in  | <i>rgbH</i>        | Height of MONO16 image.         |
| in  | <i>gray</i>        | Current MONO16 image.           |
| out | <i>mappedGray</i>  | Output MONO16 image.            |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 5.2.2.7 TYMapMono8ImageToDepthCoordinate()

```

TY_CAPI TYMapMono8ImageToDepthCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
 uint32_t depthW,
 uint32_t depthH,
 const uint16_t * depth,
 const TY_CAMERA_CALIB_INFO * color_calib,
 uint32_t monoW,
 uint32_t monoH,
 const uint8_t * inMono,
 uint8_t * mappedMono,
 float f_scale_unit = 1.0f)

```

Map original MONO8 image to depth coordinate MONO8 image.

#### Parameters

|    |                    |                                 |
|----|--------------------|---------------------------------|
| in | <i>depth_calib</i> | Depth image's calibration data. |
| in | <i>depthW</i>      | Width of current depth image.   |
| in | <i>depthH</i>      | Height of current depth image.  |
| in | <i>depth</i>       | Current depth image.            |
| in | <i>color_calib</i> | Color image's calibration data. |

## Parameters

|     |                   |                        |
|-----|-------------------|------------------------|
| in  | <i>monoW</i>      | Width of MONO8 image.  |
| in  | <i>monoH</i>      | Height of MONO8 image. |
| in  | <i>inMono</i>     | Current MONO8 image.   |
| out | <i>mappedMono</i> | Output MONO8 image.    |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## 5.2.2.8 TYMapPoint3dToDepth()

```
TY_CAPI TYMapPoint3dToDepth (
 const TY_CAMERA_CALIB_INFO * dst_calib,
 const TY_VECT_3F * point3d,
 uint32_t count,
 uint32_t depthW,
 uint32_t depthH,
 TY_PIXEL_DESC * depth,
 float f_scale_unit = 1.0f)
```

Map 3D points to pixels on depth image. Reverse operation of TYMapDepthToPoint3d.

## Parameters

|     |                  |                                        |
|-----|------------------|----------------------------------------|
| in  | <i>dst_calib</i> | Target depth image's calibration data. |
| in  | <i>point3d</i>   | Input 3D points.                       |
| in  | <i>count</i>     | Number of points.                      |
| in  | <i>depthW</i>    | Width of target depth image.           |
| in  | <i>depthH</i>    | Height of target depth image.          |
| out | <i>depth</i>     | Output depth pixels.                   |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## 5.2.2.9 TYMapPoint3dToDepthImage()

```
TY_CAPI TYMapPoint3dToDepthImage (
 const TY_CAMERA_CALIB_INFO * dst_calib,
 const TY_VECT_3F * point3d,
 uint32_t count,
 uint32_t depthW,
```

```
uint32_t depthH,
uint16_t * depth,
float f_target_scale = 1.0f)
```

Map 3D points to depth image. (NAN, NAN, NAN) will be skipped.

#### Parameters

|         |                  |                                        |
|---------|------------------|----------------------------------------|
| in      | <i>dst_calib</i> | Target depth image's calibration data. |
| in      | <i>point3d</i>   | Input 3D points.                       |
| in      | <i>count</i>     | Number of points.                      |
| in      | <i>depthW</i>    | Width of target depth image.           |
| in      | <i>depthH</i>    | Height of target depth image.          |
| in, out | <i>depth</i>     | Depth image buffer.                    |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 5.2.2.10 TYMapPoint3dToPoint3d()

```
TY_CAPI TYMapPoint3dToPoint3d (
 const TY_CAMERA_EXTRINSIC * extrinsic,
 const TY_VECT_3F * point3dFrom,
 int32_t count,
 TY_VECT_3F * point3dTo)
```

Map 3D points to another coordinate.

#### Parameters

|     |                    |                             |
|-----|--------------------|-----------------------------|
| in  | <i>extrinsic</i>   | Extrinsic matrix.           |
| in  | <i>point3dFrom</i> | Source 3D points.           |
| in  | <i>count</i>       | Number of source 3D points. |
| out | <i>point3dTo</i>   | Target 3D points.           |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

### 5.2.2.11 TYMapRGB48ImageToDepthCoordinate()

```
TY_CAPI TYMapRGB48ImageToDepthCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
```

```

uint32_t depthW,
uint32_t depthH,
const uint16_t * depth,
const TY_CAMERA_CALIB_INFO * color_calib,
uint32_t rgbW,
uint32_t rgbH,
const uint16_t * inRgb,
uint16_t * mappedRgb,
float f_scale_unit = 1.0f)

```

Map original RGB48 image to depth coordinate RGB image.

#### Parameters

|     |                    |                                 |
|-----|--------------------|---------------------------------|
| in  | <i>depth_calib</i> | Depth image's calibration data. |
| in  | <i>depthW</i>      | Width of current depth image.   |
| in  | <i>depthH</i>      | Height of current depth image.  |
| in  | <i>depth</i>       | Current depth image.            |
| in  | <i>color_calib</i> | Color image's calibration data. |
| in  | <i>rgbW</i>        | Width of RGB48 image.           |
| in  | <i>rgbH</i>        | Height of RGB48 image.          |
| in  | <i>inRgb</i>       | Current RGB48 image.            |
| out | <i>mappedRgb</i>   | Output RGB48 image.             |

#### Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

#### 5.2.2.12 TYMapRGBImageToDepthCoordinate()

```

TY_CAPI TYMapRGBImageToDepthCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
 uint32_t depthW,
 uint32_t depthH,
 const uint16_t * depth,
 const TY_CAMERA_CALIB_INFO * color_calib,
 uint32_t rgbW,
 uint32_t rgbH,
 const uint8_t * inRgb,
 uint8_t * mappedRgb,
 float f_scale_unit = 1.0f)

```

Map original RGB image to depth coordinate RGB image.

#### Parameters

|    |                    |                                 |
|----|--------------------|---------------------------------|
| in | <i>depth_calib</i> | Depth image's calibration data. |
| in | <i>depthW</i>      | Width of current depth image.   |
| in | <i>depthH</i>      | Height of current depth image.  |
| in | <i>depth</i>       | Current depth image.            |

## Parameters

|     |                    |                                 |
|-----|--------------------|---------------------------------|
| in  | <i>color_calib</i> | Color image's calibration data. |
| in  | <i>rgbW</i>        | Width of RGB image.             |
| in  | <i>rgbH</i>        | Height of RGB image.            |
| in  | <i>inRgb</i>       | Current RGB image.              |
| out | <i>mappedRgb</i>   | Output RGB image.               |

## Return values

|                     |          |
|---------------------|----------|
| <i>TY_STATUS_OK</i> | Succeed. |
|---------------------|----------|

## 5.2.2.13 TYMapRGBPixelsToDepthCoordinate()

```

TY_CAPI TYMapRGBPixelsToDepthCoordinate (
 const TY_CAMERA_CALIB_INFO * depth_calib,
 uint32_t depthW,
 uint32_t depthH,
 const uint16_t * depth,
 const TY_CAMERA_CALIB_INFO * color_calib,
 uint32_t rgbW,
 uint32_t rgbH,
 TY_PIXEL_COLOR_DESC * src,
 uint32_t cnt,
 uint32_t min_distance,
 uint32_t max_distance,
 TY_PIXEL_COLOR_DESC * dst,
 float f_scale_unit = 1.0f)

```

Map original RGB pixels to depth coordinate.

## Parameters

|     |                     |                                                                                                          |
|-----|---------------------|----------------------------------------------------------------------------------------------------------|
| in  | <i>depth_calib</i>  | Depth image's calibration data.                                                                          |
| in  | <i>depthW</i>       | Width of current depth image.                                                                            |
| in  | <i>depthH</i>       | Height of current depth image.                                                                           |
| in  | <i>depth</i>        | Current depth image.                                                                                     |
| in  | <i>color_calib</i>  | Color image's calibration data.                                                                          |
| in  | <i>rgbW</i>         | Width of RGB image.                                                                                      |
| in  | <i>rgbH</i>         | Height of RGB image.                                                                                     |
| in  | <i>src</i>          | Input RGB pixels info.                                                                                   |
| in  | <i>cnt</i>          | Input src RGB pixels cnt                                                                                 |
| in  | <i>min_distance</i> | The min distance(mm), which is generally set to the minimum measured distance of the current camera      |
| in  | <i>max_distance</i> | The longest distance(mm), which is generally set to the longest measuring distance of the current camera |
| out | <i>dst</i>          | Output RGB pixels info.                                                                                  |

## Return values

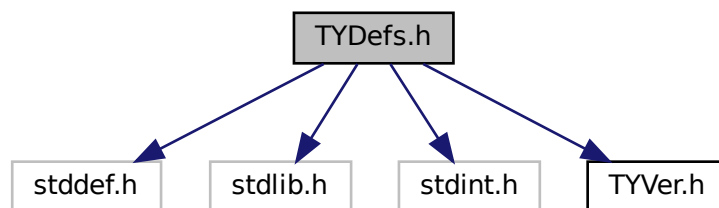
|                           |          |
|---------------------------|----------|
| <code>TY_STATUS_OK</code> | Succeed. |
|---------------------------|----------|

## 5.3 TYDefs.h File Reference

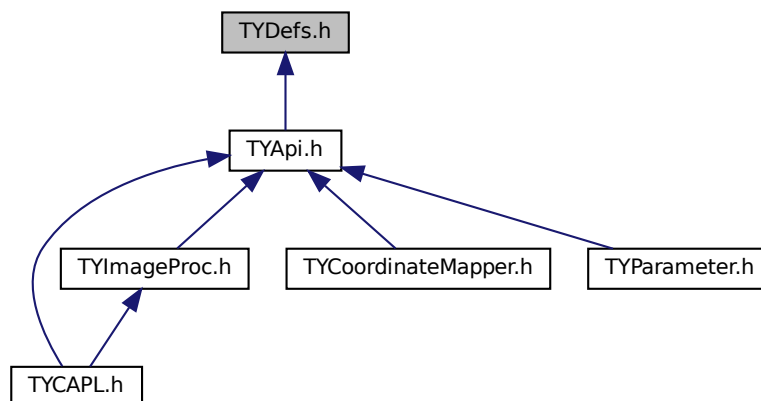
[TYDefs.h](#) includes camera control and data receiving data definitions which supports configuration for image resolution, frame rate, exposure time, gain, working mode, etc.

```
#include <stddef.h>
#include <stdlib.h>
#include <stdint.h>
#include "TYVer.h"
```

Include dependency graph for TYDefs.h:



This graph shows which files directly or indirectly include this file:



## Classes

- struct [TY\\_VERSION\\_INFO](#)
- struct [TY\\_DEVICE\\_NET\\_INFO](#)
  - device network information*
- struct [TY\\_DEVICE\\_USB\\_INFO](#)
- struct [TY\\_INTERFACE\\_INFO](#)
- struct [TY\\_DEVICE\\_BASE\\_INFO](#)
- struct [TY\\_FEATURE\\_INFO](#)
- struct [TY\\_INT\\_RANGE](#)
- struct [TY\\_FLOAT\\_RANGE](#)
  - float range data structure*
- struct [TY\\_BYTEARRAY\\_ATTR](#)
  - byte array data structure*
- struct [TY\\_ENUM\\_ENTRY](#)
- struct [TY\\_VECT\\_3F](#)
- struct [TY\\_CAMERA\\_INTRINSIC](#)
- struct [TY\\_CAMERA\\_EXTRINSIC](#)
- struct [TY\\_CAMERA\\_DISTORTION](#)
- struct [TY\\_CAMERA\\_CALIB\\_INFO](#)
- struct [TY\\_TRIGGER\\_PARAM](#)
- struct [TY\\_TRIGGER\\_PARAM\\_EX](#)
- struct [TY\\_TRIGGER\\_TIMER\\_LIST](#)
- struct [TY\\_TRIGGER\\_TIMER\\_PERIOD](#)
- struct [TY\\_AEC\\_ROI\\_PARAM](#)
- struct [TY\\_PHC\\_GROUP\\_ATTR](#)
- struct [TY\\_PHC\\_GROUP\\_ATTR::phc\\_group\\_attr](#)
- struct [pattern\\_sine\\_param](#)
- struct [pattern\\_gray\\_param](#)
- struct [pattern\\_bin\\_param](#)
- struct [TY\\_LASER\\_PATTERN\\_PARAM](#)
- struct [TY\\_CAMERA\\_STATISTICS](#)
- struct [TY\\_IMU\\_DATA](#)
- struct [TY\\_ACC\\_BIAS](#)
- struct [TY\\_ACC\\_MISALIGNMENT](#)
- struct [TY\\_ACC\\_SCALE](#)
- struct [TY\\_GYRO\\_BIAS](#)
- struct [TY\\_GYRO\\_MISALIGNMENT](#)
- struct [TY\\_GYRO\\_SCALE](#)
- struct [TY\\_CAMERA\\_TO\\_IMU](#)
- struct [TY\\_TOF\\_FREQ](#)
- struct [TY\\_LASER\\_PARAM](#)
- struct [TY\\_IMAGE\\_DATA](#)
- struct [TY\\_FRAME\\_DATA](#)
- struct [TY\\_EVENT\\_INFO](#)
- struct [TY\\_DO\\_WORKMODE](#)
- struct [TY\\_DI\\_WORKMODE](#)
- struct [TY\\_TEMP\\_DATA](#)
- struct [TY\\_CAMERA\\_ROTATION](#)



## Macros

- `#define _STDBOOL_H`
- `#define __bool_true_false_are_defined 1`
- `#define bool _Bool`
- `#define true 1`
- `#define false 0`
- `#define TY_DLLIMPORT __attribute__((visibility("default")))`
- `#define TY_DLLEXPORT __attribute__((visibility("default")))`
- `#define TY_STDC`
- `#define TY_CDEC`
- `#define TY_EXPORT TY_DLLIMPORT`
- `#define TY_EXTC`
- `#define TY_CAPI TY_EXTC TY_EXPORT TY_STATUS TY_STDC`
- `#define TY_INT_SGBM_COST_PARAM TY_INT_SGBM_UNIQUE_MAX_COST`
- `#define TY_BOOL_FLASHLIGHT TY_BOOL_IR_FLASHLIGHT`
- `#define TY_INT_FLASHLIGHT_INTENSITY TY_INT_IR_FLASHLIGHT_INTENSITY`
- `#define TY_INT_AE_TARGET_V TY_INT_AE_TARGET_Y`
- `#define TY_DECLARE_IMAGE_MODE0(pix, res) TY_IMAGE_MODE_##pix##_##res = TY_PIXEL_FOR↵  
MAT_##pix | TY_RESOLUTION_MODE_##res`
- `#define TY_DECLARE_IMAGE_MODE1(pix)`

## Typedefs

- typedef enum [TY\\_STATUS\\_LIST](#) [TY\\_STATUS\\_LIST](#)  
*API call return status.*
- typedef int32\_t [TY\\_STATUS](#)
- typedef enum [TY\\_FW\\_ERRORCODE\\_LIST](#) [TY\\_FW\\_ERRORCODE\\_LIST](#)
- typedef uint32\_t [TY\\_FW\\_ERRORCODE](#)
- typedef enum [TY\\_EVENT\\_LIST](#) [TY\\_ENENT\\_LIST](#)
- typedef int32\_t [TY\\_EVENT](#)
- typedef void \* [TY\\_INTERFACE\\_HANDLE](#)  
*Interface handle.*
- typedef void \* [TY\\_DEV\\_HANDLE](#)  
*Device Handle.*
- typedef enum [TY\\_DEVICE\\_COMPONENT\\_LIST](#) [TY\\_DEVICE\\_COMPONENT\\_LIST](#)
- typedef uint32\_t [TY\\_COMPONENT\\_ID](#)  
*component unique id*
- typedef enum [TY\\_FEATURE\\_TYPE\\_LIST](#) [TY\\_FEATURE\\_TYPE\\_LIST](#)  
*Feature Format Type definitions.*
- typedef uint32\_t [TY\\_FEATURE\\_TYPE](#)
- typedef enum [TY\\_FEATURE\\_ID\\_LIST](#) [TY\\_FEATURE\\_ID\\_LIST](#)  
*feature for component definitions*
- typedef uint32\_t [TY\\_FEATURE\\_ID](#)  
*feature unique id*
- typedef enum [TY\\_CONFIG\\_MODE\\_LIST](#) [TY\\_CONFIG\\_MODE\\_LIST](#)
- typedef uint32\_t [TY\\_CONFIG\\_MODE](#)
- typedef enum [TY\\_DEPTH\\_QUALITY\\_LIST](#) [TY\\_DEPTH\\_QUALITY\\_LIST](#)
- typedef uint32\_t [TY\\_DEPTH\\_QUALITY](#)
- typedef enum [TY\\_TRIGGER\\_POL\\_LIST](#) [TY\\_TRIGGER\\_POL\\_LIST](#)  
*set external trigger signal edge*
- typedef uint32\_t [TY\\_TRIGGER\\_POL](#)
- typedef enum [TY\\_INTERFACE\\_TYPE\\_LIST](#) [TY\\_INTERFACE\\_TYPE\\_LIST](#)

- typedef uint32\_t **TY\_INTERFACE\_TYPE**
- typedef enum [TY\\_ACCESS\\_MODE\\_LIST](#) **TY\_ACCESS\_MODE\_LIST**
- typedef uint8\_t **TY\_ACCESS\_MODE**
- typedef enum [TY\\_STREAM\\_ASYNC\\_MODE\\_LIST](#) **TY\_STREAM\_ASYNC\_MODE\_LIST**
  - stream async mode*
- typedef uint8\_t **TY\_STREAM\_ASYNC\_MODE**
- typedef enum TYLensOpticalType **TYLensOpticalType**
- typedef enum [TYPixFmtList](#) **TYPixFmtList**
  - pixel format definitions*
- typedef uint32\_t **TYPixFmt**
- typedef enum [TY\\_PIXEL\\_BITS\\_LIST](#) **TY\_PIXEL\_BITS\_LIST**
- typedef uint32\_t **TY\_PIXEL\_BITS**
- typedef enum [TY\\_PIXEL\\_FORMAT\\_LIST](#) **TY\_PIXEL\_FORMAT\_LIST**
  - pixel format definitions*
- typedef uint32\_t **TY\_PIXEL\_FORMAT**
- typedef enum [TY\\_RESOLUTION\\_MODE\\_LIST](#) **TY\_RESOLUTION\_MODE\_LIST**
  - predefined resolution list*
- typedef int32\_t **TY\_RESOLUTION\_MODE**
- typedef enum [TY\\_IMAGE\\_MODE\\_LIST](#) **TY\_IMAGE\_MODE\_LIST**
  - Predefined Image Mode List image mode controls image resolution & format predefined image modes named like TY\_IMAGE\_MODE\_MONO\_160x120, TY\_IMAGE\_MODE\_RGB\_1280x960.*
- typedef uint32\_t **TY\_IMAGE\_MODE**
- typedef enum [TY\\_TRIGGER\\_MODE\\_LIST](#) **TY\_TRIGGER\_MODE\_LIST**
- typedef int16\_t **TY\_TRIGGER\_MODE**
- typedef enum [TY\\_TIME\\_SYNC\\_TYPE\\_LIST](#) **TY\_TIME\_SYNC\_TYPE\_LIST**
  - type of time sync*
- typedef uint32\_t **TY\_TIME\_SYNC\_TYPE**
- typedef uint32\_t **TY\_E\_VOLT\_T**
- typedef uint32\_t **TY\_E\_DO\_MODE**
- typedef uint32\_t **TY\_E\_DI\_MODE**
- typedef uint32\_t **TY\_E\_DI\_INT\_ACTION**
- typedef uint32\_t **TY\_TEMPERATURE\_ID**
- typedef enum [TY\\_LOG\\_LEVEL\\_LIST](#) **TY\_LOG\_LEVEL\_LIST**
- typedef int32\_t **TY\_LOG\_LEVEL**
- typedef enum [TY\\_LOG\\_TYPE\\_LIST](#) **TY\_LOG\_TYPE\_LIST**
- typedef int32\_t **TY\_LOG\_TYPE**
- typedef enum [TY\\_SERVER\\_TYPE\\_LIST](#) **TY\_SERVER\_TYPE\_LIST**
- typedef int32\_t **TY\_SERVER\_TYPE**
- typedef struct [TY\\_VERSION\\_INFO](#) **TY\_VERSION\_INFO**
- typedef struct [TY\\_DEVICE\\_NET\\_INFO](#) **TY\_DEVICE\_NET\_INFO**
  - device network information*
- typedef struct [TY\\_DEVICE\\_USB\\_INFO](#) **TY\_DEVICE\_USB\_INFO**
- typedef struct [TY\\_INTERFACE\\_INFO](#) **TY\_INTERFACE\_INFO**
- typedef struct [TY\\_DEVICE\\_BASE\\_INFO](#) **TY\_DEVICE\_BASE\_INFO**
- typedef enum [TY\\_VISIBILITY\\_TYPE](#) **TY\_VISIBILITY\_TYPE**
- typedef struct [TY\\_FEATURE\\_INFO](#) **TY\_FEATURE\_INFO**
- typedef struct [TY\\_INT\\_RANGE](#) **TY\_INT\_RANGE**
- typedef struct [TY\\_FLOAT\\_RANGE](#) **TY\_FLOAT\_RANGE**
  - float range data structure*
- typedef struct [TY\\_BYTEARRAY\\_ATTR](#) **TY\_BYTEARRAY\_ATTR**
  - byte array data structure*
- typedef struct [TY\\_ENUM\\_ENTRY](#) **TY\_ENUM\_ENTRY**
- typedef struct [TY\\_VECT\\_3F](#) **TY\_VECT\_3F**

- typedef struct [TY\\_CAMERA\\_INTRINSIC](#) [TY\\_CAMERA\\_INTRINSIC](#)
- typedef struct [TY\\_CAMERA\\_EXTRINSIC](#) [TY\\_CAMERA\\_EXTRINSIC](#)
- typedef struct [TY\\_CAMERA\\_DISTORTION](#) [TY\\_CAMERA\\_DISTORTION](#)
- typedef struct [TY\\_CAMERA\\_CALIB\\_INFO](#) [TY\\_CAMERA\\_CALIB\\_INFO](#)
- typedef struct [TY\\_TRIGGER\\_PARAM](#) [TY\\_TRIGGER\\_PARAM](#)
- typedef struct [TY\\_TRIGGER\\_PARAM\\_EX](#) [TY\\_TRIGGER\\_PARAM\\_EX](#)
- typedef struct [TY\\_TRIGGER\\_TIMER\\_LIST](#) [TY\\_TRIGGER\\_TIMER\\_LIST](#)
- typedef struct [TY\\_TRIGGER\\_TIMER\\_PERIOD](#) [TY\\_TRIGGER\\_TIMER\\_PERIOD](#)
- typedef struct [TY\\_AEC\\_ROI\\_PARAM](#) [TY\\_AEC\\_ROI\\_PARAM](#)
- typedef struct [TY\\_PHC\\_GROUP\\_ATTR](#) [TY\\_PHC\\_GROUP\\_ATTR](#)
- typedef struct [TY\\_LASER\\_PATTERN\\_PARAM](#) [TY\\_LASER\\_PATTERN\\_PARAM](#)
- typedef struct [TY\\_CAMERA\\_STATISTICS](#) [TY\\_CAMERA\\_STATISTICS](#)
- typedef struct [TY\\_IMU\\_DATA](#) [TY\\_IMU\\_DATA](#)
- typedef struct [TY\\_ACC\\_BIAS](#) [TY\\_ACC\\_BIAS](#)
- typedef struct [TY\\_ACC\\_MISALIGNMENT](#) [TY\\_ACC\\_MISALIGNMENT](#)
- typedef struct [TY\\_ACC\\_SCALE](#) [TY\\_ACC\\_SCALE](#)
- typedef struct [TY\\_GYRO\\_BIAS](#) [TY\\_GYRO\\_BIAS](#)
- typedef struct [TY\\_GYRO\\_MISALIGNMENT](#) [TY\\_GYRO\\_MISALIGNMENT](#)
- typedef struct [TY\\_GYRO\\_SCALE](#) [TY\\_GYRO\\_SCALE](#)
- typedef struct [TY\\_CAMERA\\_TO\\_IMU](#) [TY\\_CAMERA\\_TO\\_IMU](#)
- typedef struct [TY\\_TOF\\_FREQ](#) [TY\\_TOF\\_FREQ](#)
- typedef enum [TY\\_IMU\\_FPS\\_LIST](#) [TY\\_IMU\\_FPS\\_LIST](#)
- typedef struct [TY\\_LASER\\_PARAM](#) [TY\\_LASER\\_PARAM](#)
- typedef struct [TY\\_IMAGE\\_DATA](#) [TY\\_IMAGE\\_DATA](#)
- typedef struct [TY\\_FRAME\\_DATA](#) [TY\\_FRAME\\_DATA](#)
- typedef struct [TY\\_EVENT\\_INFO](#) [TY\\_EVENT\\_INFO](#)
- typedef struct [TY\\_DO\\_WORKMODE](#) [TY\\_DO\\_WORKMODE](#)
- typedef struct [TY\\_DI\\_WORKMODE](#) [TY\\_DI\\_WORKMODE](#)
- typedef struct [TY\\_TEMP\\_DATA](#) [TY\\_TEMP\\_DATA](#)
- typedef struct [TY\\_CAMERA\\_ROTATION](#) [TY\\_CAMERA\\_ROTATION](#)

## Enumerations

- enum [TY\\_STATUS\\_LIST](#) : int32\_t {  
[TY\\_STATUS\\_OK](#) = 0, [TY\\_STATUS\\_ERROR](#) = -1001, [TY\\_STATUS\\_NOT\\_INITED](#) = -1002, [TY\\_STATUS\\_↵](#)  
[\\_NOT\\_IMPLEMENTED](#) = -1003,  
[TY\\_STATUS\\_NOT\\_PERMITTED](#) = -1004, [TY\\_STATUS\\_DEVICE\\_ERROR](#) = -1005, [TY\\_STATUS\\_INVA↵](#)  
[LID\\_PARAMETER](#) = -1006, [TY\\_STATUS\\_INVALID\\_HANDLE](#) = -1007,  
[TY\\_STATUS\\_INVALID\\_COMPONENT](#) = -1008, [TY\\_STATUS\\_INVALID\\_FEATURE](#) = -1009, [TY\\_STATU↵](#)  
[S\\_WRONG\\_TYPE](#) = -1010, [TY\\_STATUS\\_WRONG\\_SIZE](#) = -1011,  
[TY\\_STATUS\\_OUT\\_OF\\_MEMORY](#) = -1012, [TY\\_STATUS\\_OUT\\_OF\\_RANGE](#) = -1013, [TY\\_STATUS\\_TIM↵](#)  
[EOUT](#) = -1014, [TY\\_STATUS\\_WRONG\\_MODE](#) = -1015,  
[TY\\_STATUS\\_BUSY](#) = -1016, [TY\\_STATUS\\_IDLE](#) = -1017, [TY\\_STATUS\\_NO\\_DATA](#) = -1018, [TY\\_STATU↵](#)  
[S\\_NO\\_BUFFER](#) = -1019,  
[TY\\_STATUS\\_NULL\\_POINTER](#) = -1020, [TY\\_STATUS\\_READONLY\\_FEATURE](#) = -1021, [TY\\_STATUS\\_I↵](#)  
[NVALID\\_DESCRIPTOR](#) = -1022, [TY\\_STATUS\\_INVALID\\_INTERFACE](#) = -1023,  
[TY\\_STATUS\\_FIRMWARE\\_ERROR](#) = -1024, [TY\\_STATUS\\_ERROR\\_XML](#) = -1025, [TY\\_STATUS\\_DEV\\_E↵](#)  
[PERM](#) = -1, [TY\\_STATUS\\_DEV\\_EIO](#) = -5,  
[TY\\_STATUS\\_DEV\\_ENOMEM](#) = -12, [TY\\_STATUS\\_DEV\\_EBUSY](#) = -16, [TY\\_STATUS\\_DEV\\_EINVAL](#) = -22  
 }

*API call return status.*

- enum **TY\_FW\_ERRORCODE\_LIST** : uint32\_t {  
**TY\_FW\_ERRORCODE\_CAM0\_NOT\_DETECTED** = 0x00000001, **TY\_FW\_ERRORCODE\_CAM1\_NOT\_DETECTED** = 0x00000002, **TY\_FW\_ERRORCODE\_CAM2\_NOT\_DETECTED** = 0x00000004, **TY\_FW\_ERRORCODE\_POE\_NOT\_INIT** = 0x00000008,  
**TY\_FW\_ERRORCODE\_RECMAP\_NOT\_CORRECT** = 0x00000010, **TY\_FW\_ERRORCODE\_LOOKUPTABLE\_NOT\_CORRECT** = 0x00000020, **TY\_FW\_ERRORCODE\_DRV8899\_NOT\_INIT** = 0x00000040, **TY\_FW\_ERRORCODE\_FOC\_START\_ERR** = 0x00000080,  
**TY\_FW\_ERRORCODE\_PHASE\_CALIB\_ERR** = 0x00000100, **TY\_FW\_ERRORCODE\_CONFIG\_NOT\_FOUND** = 0x00010000, **TY\_FW\_ERRORCODE\_CONFIG\_NOT\_CORRECT** = 0x00020000, **TY\_FW\_ERRORCODE\_XML\_NOT\_FOUND** = 0x00040000,  
**TY\_FW\_ERRORCODE\_XML\_NOT\_CORRECT** = 0x00080000, **TY\_FW\_ERRORCODE\_XML\_OVERRIDE\_FAILED** = 0x00100000, **TY\_FW\_ERRORCODE\_CAM\_INIT\_FAILED** = 0x00200000, **TY\_FW\_ERRORCODE\_LASER\_INIT\_FAILED** = 0x00400000 }
- enum **TY\_EVENT\_LIST** : int32\_t { **TY\_EVENT\_DEVICE\_OFFLINE** = -2001, **TY\_EVENT\_LICENSE\_ERROR** = -2002, **TY\_EVENT\_FW\_INIT\_ERROR** = -2003 }
- enum **TY\_DEVICE\_COMPONENT\_LIST** : uint32\_t {  
**TY\_COMPONENT\_DEVICE** = 0x80000000, **TY\_COMPONENT\_DEPTH\_CAM** = 0x00010000, **TY\_COMPONENT\_IR\_CAM\_LEFT** = 0x00040000, **TY\_COMPONENT\_IR\_CAM\_RIGHT** = 0x00080000,  
**TY\_COMPONENT\_RGB\_CAM\_LEFT** = 0x00100000, **TY\_COMPONENT\_RGB\_CAM\_RIGHT** = 0x00200000, **TY\_COMPONENT\_LASER** = 0x00400000, **TY\_COMPONENT\_IMU** = 0x00800000,  
**TY\_COMPONENT\_BRIGHT\_HISTO** = 0x01000000, **TY\_COMPONENT\_STORAGE** = 0x02000000,  
**TY\_COMPONENT\_RGB\_CAM** = **TY\_COMPONENT\_RGB\_CAM\_LEFT** }
- enum **TY\_FEATURE\_TYPE\_LIST** : uint32\_t {  
**TY\_FEATURE\_INT** = 0x1000, **TY\_FEATURE\_FLOAT** = 0x2000, **TY\_FEATURE\_ENUM** = 0x3000, **TY\_FEATURE\_BOOL** = 0x4000,  
**TY\_FEATURE\_STRING** = 0x5000, **TY\_FEATURE\_BYTEARRAY** = 0x6000, **TY\_FEATURE\_STRUCT** = 0x7000 }

*Feature Format Type definitions.*

- enum **TY\_FEATURE\_ID\_LIST** : uint32\_t {  
**TY\_STRUCT\_CAM\_INTRINSIC** = 0x0000 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_EXTRINSIC\_TO\_DEPTH** = 0x0001 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_EXTRINSIC\_TO\_IR\_LEFT** = 0x0002 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_CAM\_RECTIFIED\_ROTATION** = 0x0003 | **TY\_FEATURE\_STRUCT**,  
**TY\_STRUCT\_CAM\_DISTORTION** = 0x0006 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_CAM\_CALIB\_DATA** = 0x0007 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_CAM\_RECTIFIED\_INTRI** = 0x0008 | **TY\_FEATURE\_STRUCT**, **TY\_BYTEARRAY\_CUSTOM\_BLOCK** = 0x000A | **TY\_FEATURE\_BYTEARRAY**,  
**TY\_BYTEARRAY\_ISP\_BLOCK** = 0x000B | **TY\_FEATURE\_BYTEARRAY**, **TY\_INT\_PERSISTENT\_IP** = 0x0010 | **TY\_FEATURE\_INT**, **TY\_INT\_PERSISTENT\_SUBMASK** = 0x0011 | **TY\_FEATURE\_INT**, **TY\_INT\_PERSISTENT\_GATEWAY** = 0x0012 | **TY\_FEATURE\_INT**,  
**TY\_BOOL\_GVSP\_RESEND** = 0x0013 | **TY\_FEATURE\_BOOL**, **TY\_INT\_PACKET\_DELAY** = 0x0014 | **TY\_FEATURE\_INT**, **TY\_INT\_ACCEPTABLE\_PERCENT** = 0x0015 | **TY\_FEATURE\_INT**, **TY\_INT\_NTP\_SERVER\_IP** = 0x0016 | **TY\_FEATURE\_INT**,  
**TY\_INT\_PACKET\_SIZE** = 0x0017 | **TY\_FEATURE\_INT**, **TY\_INT\_LINK\_CMD\_TIMEOUT** = 0x0018 | **TY\_FEATURE\_INT**, **TY\_STRUCT\_CAM\_STATISTICS** = 0x00ff | **TY\_FEATURE\_STRUCT**, **TY\_INT\_WIDTH\_MAX** = 0x0100 | **TY\_FEATURE\_INT**,  
**TY\_INT\_HEIGHT\_MAX** = 0x0101 | **TY\_FEATURE\_INT**, **TY\_INT\_OFFSET\_X** = 0x0102 | **TY\_FEATURE\_INT**, **TY\_INT\_OFFSET\_Y** = 0x0103 | **TY\_FEATURE\_INT**, **TY\_INT\_WIDTH** = 0x0104 | **TY\_FEATURE\_INT**,  
**TY\_INT\_HEIGHT** = 0x0105 | **TY\_FEATURE\_INT**, **TY\_ENUM\_IMAGE\_MODE** = 0x0109 | **TY\_FEATURE\_ENUM**, **TY\_FLOAT\_SCALE\_UNIT** = 0x010a | **TY\_FEATURE\_FLOAT**, **TY\_ENUM\_TRIGGER\_POL** = 0x0201 | **TY\_FEATURE\_ENUM**,  
**TY\_INT\_FRAME\_PER\_TRIGGER** = 0x0202 | **TY\_FEATURE\_INT**, **TY\_STRUCT\_TRIGGER\_PARAM** = 0x0523 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_TRIGGER\_PARAM\_EX** = 0x0525 | **TY\_FEATURE\_STRUCT**, **TY\_STRUCT\_TRIGGER\_TIMER\_LIST** = 0x0526 | **TY\_FEATURE\_STRUCT**,  
**TY\_STRUCT\_TRIGGER\_TIMER\_PERIOD** = 0x0527 | **TY\_FEATURE\_STRUCT**, **TY\_BOOL\_KEEP\_ALIVE\_ONOFF** = 0x0203 | **TY\_FEATURE\_BOOL**, **TY\_INT\_KEEP\_ALIVE\_TIMEOUT** = 0x0204 | **TY\_FEATURE\_INT**,  
**TY\_BOOL\_CMOS\_SYNC** = 0x0205 | **TY\_FEATURE\_BOOL**, **TY\_INT\_TRIGGER\_DELAY\_US** = 0x0206 | **TY\_FEATURE\_INT**, **TY\_BOOL\_TRIGGER\_OUT\_IO** = 0x0207 | **TY\_FEATURE\_BOOL**, **TY\_INT\_TRIGGER\_DURATION\_US** = 0x0208 | **TY\_FEATURE\_INT**,  
**TY\_ENUM\_STREAM\_ASYNC** = 0x0209 | **TY\_FEATURE\_ENUM**,

```

TY_INT_CAPTURE_TIME_US = 0x0210 | TY_FEATURE_INT, TY_ENUM_TIME_SYNC_TYPE =
0x0211 | TY_FEATURE_ENUM, TY_BOOL_TIME_SYNC_READY = 0x0212 | TY_FEATURE_BOOL,
TY_BOOL_IR_FLASHLIGHT = 0x0213 | TY_FEATURE_BOOL,
TY_INT_IR_FLASHLIGHT_INTENSITY = 0x0214 | TY_FEATURE_INT, TY_STRUCT_DO0_WORKMODE
= 0x0215 | TY_FEATURE_STRUCT, TY_STRUCT_DI0_WORKMODE = 0x0216 | TY_FEATURE_STRUCT,
TY_STRUCT_DO1_WORKMODE = 0x0217 | TY_FEATURE_STRUCT,
TY_STRUCT_DI1_WORKMODE = 0x0218 | TY_FEATURE_STRUCT, TY_STRUCT_DO2_WORKMODE =
0x0219 | TY_FEATURE_STRUCT, TY_STRUCT_DI2_WORKMODE = 0x0220 | TY_FEATURE_STRUCT,
TY_BOOL_RGB_FLASHLIGHT = 0x0221 | TY_FEATURE_BOOL,
TY_INT_RGB_FLASHLIGHT_INTENSITY = 0x0222 | TY_FEATURE_INT, TY_ENUM_CONFIG_MODE =
0x0221 | TY_FEATURE_ENUM, TY_ENUM_TEMPERATURE_ID = 0x0223 | TY_FEATURE_ENUM, TY_↵
STRUCT_TEMPERATURE = 0x0224 | TY_FEATURE_STRUCT,
TY_BOOL_AUTO_EXPOSURE = 0x0300 | TY_FEATURE_BOOL, TY_INT_EXPOSURE_TIME = 0x0301 |
TY_FEATURE_INT, TY_BOOL_AUTO_GAIN = 0x0302 | TY_FEATURE_BOOL, TY_INT_GAIN = 0x0303 |
TY_FEATURE_INT,
TY_BOOL_AUTO_AWB = 0x0304 | TY_FEATURE_BOOL, TY_STRUCT_AEC_ROI = 0x0305 | TY_FEAT↵
URE_STRUCT, TY_INT_TOF_HDR_RATIO = 0x0306 | TY_FEATURE_INT, TY_INT_TOF_JITTER_THRESHOLD
= 0x0307 | TY_FEATURE_INT,
TY_FLOAT_EXPOSURE_TIME_US = 0x0308 | TY_FEATURE_FLOAT, TY_INT_LASER_POWER = 0x0500
| TY_FEATURE_INT, TY_BOOL_LASER_AUTO_CTRL = 0x0501 | TY_FEATURE_BOOL, TY_STRUCT_↵
LASER_PATTERN = 0x0502 | TY_FEATURE_STRUCT,
TY_INT_LASER_CAM_TRIG_POS = 0x0503 | TY_FEATURE_INT, TY_INT_LASER_CAM_TRIG_LEN =
0x0504 | TY_FEATURE_INT, TY_INT_LASER_LUT_TRIG_POS = 0x0505 | TY_FEATURE_INT, TY_INT↵
_LASER_LUT_NUM = 0x0506 | TY_FEATURE_INT,
TY_INT_LASER_PATTERN_OFFSET = 0x0507 | TY_FEATURE_INT, TY_INT_LASER_MIRROR_NUM =
0x0508 | TY_FEATURE_INT, TY_INT_LASER_MIRROR_SEL = 0x0509 | TY_FEATURE_INT, TY_INT_L↵
ASER_LUT_IDX = 0x050a | TY_FEATURE_INT,
TY_INT_LASER_FACET_IDX = 0x050b | TY_FEATURE_INT, TY_INT_LASER_FACET_POS = 0x050c |
TY_FEATURE_INT, TY_INT_LASER_MODE = 0x050d | TY_FEATURE_INT, TY_INT_CONST_DRV_DUTY
= 0x050e | TY_FEATURE_INT,
TY_STRUCT_LASER_ENABLE_BY_IDX = 0x0530 | TY_FEATURE_STRUCT, TY_STRUCT_LASER_POWER_BY_IDX
= 0x0531 | TY_FEATURE_STRUCT, TY_STRUCT_FLOOD_ENABLE_BY_IDX = 0x0532 | TY_FEATURE↵
_STRUCT, TY_STRUCT_FLOOD_POWER_BY_IDX = 0x0533 | TY_FEATURE_STRUCT,
TY_BOOL_UNDISTORTION = 0x0510 | TY_FEATURE_BOOL, TY_BOOL_BRIGHTNESS_HISTOGRAM
= 0x0511 | TY_FEATURE_BOOL, TY_BOOL_DEPTH_POSTPROC = 0x0512 | TY_FEATURE_BOOL,
TY_INT_R_GAIN = 0x0520 | TY_FEATURE_INT,
TY_INT_G_GAIN = 0x0521 | TY_FEATURE_INT, TY_INT_B_GAIN = 0x0522 | TY_FEATURE_INT,
TY_INT_ANALOG_GAIN = 0x0524 | TY_FEATURE_INT, TY_BOOL_HDR = 0x0525 | TY_FEATURE↵
_BOOL,
TY_BYTEARRAY_HDR_PARAMETER = 0x0526 | TY_FEATURE_BYTEARRAY, TY_INT_AE_TARGET_↵
T_Y = 0x0527 | TY_FEATURE_INT, TY_BOOL_IMU_DATA_ONOFF = 0x0600 | TY_FEATURE_BOOL,
TY_STRUCT_IMU_ACC_BIAS = 0x0601 | TY_FEATURE_STRUCT,
TY_STRUCT_IMU_ACC_MISALIGNMENT = 0x0602 | TY_FEATURE_STRUCT, TY_STRUCT_IMU_ACC_SCALE
= 0x0603 | TY_FEATURE_STRUCT, TY_STRUCT_IMU_GYRO_BIAS = 0x0604 | TY_FEATURE_STRUCT,
TY_STRUCT_IMU_GYRO_MISALIGNMENT = 0x0605 | TY_FEATURE_STRUCT,
TY_STRUCT_IMU_GYRO_SCALE = 0x0606 | TY_FEATURE_STRUCT, TY_STRUCT_IMU_CAM_TO_IMU
= 0x0607 | TY_FEATURE_STRUCT, TY_ENUM_IMU_FPS = 0x0608 | TY_FEATURE_ENUM, TY_INT_SGBM_IMAGE_NUM
= 0x0610 | TY_FEATURE_INT,
TY_INT_SGBM_DISPARITY_NUM = 0x0611 | TY_FEATURE_INT, TY_INT_SGBM_DISPARITY_OFFSET
= 0x0612 | TY_FEATURE_INT, TY_INT_SGBM_MATCH_WIN_HEIGHT = 0x0613 | TY_FEATURE_INT,
TY_INT_SGBM_SEMI_PARAM_P1 = 0x0614 | TY_FEATURE_INT,
TY_INT_SGBM_SEMI_PARAM_P2 = 0x0615 | TY_FEATURE_INT, TY_INT_SGBM_UNIQUE_FACTOR
= 0x0616 | TY_FEATURE_INT, TY_INT_SGBM_UNIQUE_ABSDIFF = 0x0617 | TY_FEATURE_INT,
TY_INT_SGBM_UNIQUE_MAX_COST = 0x0618 | TY_FEATURE_INT,
TY_BOOL_SGBM_HFILTER_HALF_WIN = 0x0619 | TY_FEATURE_BOOL, TY_INT_SGBM_MATCH_WIN_WIDTH
= 0x061A | TY_FEATURE_INT, TY_BOOL_SGBM_MEDFILTER = 0x061B | TY_FEATURE_BOOL,
TY_BOOL_SGBM_LRC = 0x061C | TY_FEATURE_BOOL,
TY_INT_SGBM_LRC_DIFF = 0x061D | TY_FEATURE_INT, TY_INT_SGBM_MEDFILTER_THRESH =

```

```

0x061E | TY_FEATURE_INT, TY_INT_SGBM_SEMI_PARAM_P1_SCALE = 0x061F | TY_FEATURE_INT,
TY_INT_SGPM_PHASE_NUM = 0x0620 | TY_FEATURE_INT,
TY_INT_SGPM_NORMAL_PHASE_SCALE = 0x0621 | TY_FEATURE_INT, TY_INT_SGPM_NORMAL_PHASE_OFFSET
= 0x0622 | TY_FEATURE_INT, TY_INT_SGPM_REF_PHASE_SCALE = 0x0623 | TY_FEATURE_INT,
TY_INT_SGPM_REF_PHASE_OFFSET = 0x0624 | TY_FEATURE_INT,
TY_FLOAT_SGPM_EPI_HS = 0x0625 | TY_FEATURE_FLOAT, TY_INT_SGPM_EPI_HF = 0x0626 | TY_F↵
EATURE_INT, TY_BOOL_SGPM_EPI_EN = 0x0627 | TY_FEATURE_BOOL, TY_INT_SGPM_EPI_CH0 =
0x0628 | TY_FEATURE_INT,
TY_INT_SGPM_EPI_CH1 = 0x0629 | TY_FEATURE_INT, TY_INT_SGPM_EPI_THRESH = 0x062A
| TY_FEATURE_INT, TY_BOOL_SGPM_ORDER_FILTER_EN = 0x062B | TY_FEATURE_BOOL,
TY_INT_SGPM_ORDER_FILTER_CHN = 0x062C | TY_FEATURE_INT,
TY_INT_DEPTH_MIN_MM = 0x062D | TY_FEATURE_INT, TY_INT_DEPTH_MAX_MM = 0x062E | TY_FE↵
ATURE_INT, TY_INT_SGBM_TEXTURE_OFFSET = 0x062F | TY_FEATURE_INT, TY_INT_SGBM_TEXTURE_THRESH
= 0x0630 | TY_FEATURE_INT,
TY_STRUCT_PHC_GROUP_ATTR = 0x0710 | TY_FEATURE_STRUCT, TY_ENUM_DEPTH_QUALITY
= 0x0900 | TY_FEATURE_ENUM, TY_INT_FILTER_THRESHOLD = 0x0901 | TY_FEATURE_INT,
TY_INT_TOF_CHANNEL = 0x0902 | TY_FEATURE_INT,
TY_INT_TOF_MODULATION_THRESHOLD = 0x0903 | TY_FEATURE_INT, TY_STRUCT_TOF_FREQ =
0x0904 | TY_FEATURE_STRUCT, TY_BOOL_TOF_ANTI_INTERFERENCE = 0x0905 | TY_FEATURE_↵
BOOL, TY_INT_TOF_ANTI_SUNLIGHT_INDEX = 0x0906 | TY_FEATURE_INT,
TY_INT_MAX_SPECKLE_SIZE = 0x0907 | TY_FEATURE_INT, TY_INT_MAX_SPECKLE_DIFF = 0x0908 |
TY_FEATURE_INT, TY_ENUM_LENS_OPTICAL_TYPE = 0x0909 | TY_FEATURE_ENUM }

```

*feature for component definitions*

- enum **TY\_CONFIG\_MODE\_LIST** : uint32\_t {  
**TY\_CONFIG\_MODE\_PRESET0** = 0, **TY\_CONFIG\_MODE\_PRESET1**, **TY\_CONFIG\_MODE\_PRESET2**,  
**TY\_CONFIG\_MODE\_USERSET0** = (1 << 16),  
**TY\_CONFIG\_MODE\_USERSET1**, **TY\_CONFIG\_MODE\_USERSET2** }
- enum **TY\_DEPTH\_QUALITY\_LIST** : uint32\_t { **TY\_DEPTH\_QUALITY\_BASIC** = 1, **TY\_DEPTH\_QUALIT↵**  
**Y\_MEDIUM** = 2, **TY\_DEPTH\_QUALITY\_HIGH** = 4 }
- enum **TY\_TRIGGER\_POL\_LIST** : uint32\_t { **TY\_TRIGGER\_POL\_FALLINGEDGE** = 0, **TY\_TRIGGER\_P↵**  
**OL\_RISINGEDGE** = 1 }

*set external trigger signal edge*

- enum **TY\_INTERFACE\_TYPE\_LIST** : uint32\_t {  
**TY\_INTERFACE\_UNKNOWN** = 0, **TY\_INTERFACE\_RAW** = 1, **TY\_INTERFACE\_USB** = 2, **TY\_INTERF↵**  
**ACE\_ETHERNET** = 4,  
**TY\_INTERFACE\_IEEE80211** = 8, **TY\_INTERFACE\_ALL** = 0xffff }
- enum **TY\_ACCESS\_MODE\_LIST** : uint32\_t { **TY\_ACCESS\_READABLE** = 0x1, **TY\_ACCESS\_WRITABLE**  
= 0x2 }
- enum **TY\_STREAM\_ASYNC\_MODE\_LIST** : uint32\_t {  
**TY\_STREAM\_ASYNC\_OFF** = 0, **TY\_STREAM\_ASYNC\_DEPTH** = 1, **TY\_STREAM\_ASYNC\_RGB** = 2, **T↵**  
**Y\_STREAM\_ASYNC\_DEPTH\_RGB** = 3,  
**TY\_STREAM\_ASYNC\_ALL** = 0xff }

*stream async mode*

- enum **TYLensOpticalType** : uint32\_t { **TY\_LENS\_PINHOLE** = 0, **TY\_LENS\_FISHEYE** = 1 }
- enum **TYPixFmtList** : uint32\_t {  
**TYPixelFormatMono8** = 0x01080001, **TYPixelFormatBayerGBRG8** = 0x0108000A, **TYPixelFormat↵**  
**BayerBGGR8** = 0x0108000B, **TYPixelFormatBayerGRBG8** = 0x01080008,  
**TYPixelFormatBayerRGGB8** = 0x01080009, **TYPixelFormatPacketMono10** = 0x010A0046, **TYPixel↵**  
**FormatPacketBayerGBRG10** = 0x010A0054, **TYPixelFormatPacketBayerBGGR10** = 0x010A0052,  
**TYPixelFormatPacketBayerGRBG10** = 0x010A0056, **TYPixelFormatPacketBayerRGGB10** = 0x010↵  
A0058, **TYPixelFormatPacketMono12** = 0x010C0047, **TYPixelFormatPacketBayerGBRG12** = 0x010↵  
C0055,  
**TYPixelFormatPacketBayerBGGR12** = 0x010C0053, **TYPixelFormatPacketBayerGRBG12** = 0x010↵  
C0057, **TYPixelFormatPacketBayerRGGB12** = 0x010C0059, **TYPixelFormatMono10** = 0x810A0046,  
**TYPixelFormatBayerGBRG10** = 0x810A0054, **TYPixelFormatBayerBGGR10** = 0x810A0052, **TYPixel↵**  
**FormatBayerGRBG10** = 0x810A0056, **TYPixelFormatBayerRGGB10** = 0x810A0058,



```

TYPixelFormatMono12 = 0x810C0047, TYPixelFormatBayerGBRG12 = 0x810C0055, TYPixelFormat↵
BayerBGGR12 = 0x810C0053, TYPixelFormatBayerGRBG12 = 0x810C0057,
TYPixelFormatBayerRGGB12 = 0x810C0059, TYPixelFormatMono14 = 0x810E0104, TYPixelFormat↵
BayerGBRG14 = 0x810E0107, TYPixelFormatBayerBGGR14 = 0x810E0108,
TYPixelFormatBayerGRBG14 = 0x810E0105, TYPixelFormatBayerRGGB14 = 0x810E0106, TYPixel↵
FormatMono16 = 0x01100007, TYPixelFormatBayerGBRG16 = 0x01100030,
TYPixelFormatBayerBGGR16 = 0x01100031, TYPixelFormatBayerGRBG16 = 0x0110002E, TYPixel↵
FormatBayerRGGB16 = 0x0110002F, TYPixelFormatRGB8 = 0x02180014,
TYPixelFormatBGR8 = 0x02180015, TYPixelFormatYUV422_8 = 0x02100032, TYPixelFormatYUV422↵
_8_UYVY = 0x0210001F, TYPixelFormatYCbCr420_8_YY_CbCr_Planar = 0x020C0113,
TYPixelFormatYCbCr420_8_YY_CrCb_Planar = 0x020C0115, TYPixelFormatYCbCr420_8_YY_CbCr↵
_Semiplanar = 0x020C0112, TYPixelFormatYCbCr420_8_YY_CrCb_Semiplanar = 0x020C0114, TY↵
PixelFormatCoord3D_C16 = 0x011000B8,
TYPixelFormatCoord3D_ABC16 = 0x023000B9, TYPixelFormatCoord3D_ABC32f = 0x026000C0, TY↵
PixelFormatJPEG = 0x82180015, TYPixelFormatToFIRFourGroupMono16 = 0x81400016,
TYPixelFormatInvalid = 0xFFFFFFFF }

```

*pixel format definitions*

- enum **TY\_PIXEL\_BITS\_LIST** : uint32\_t {  
**TY\_PIXEL\_8BIT** = 0x1 << 28, **TY\_PIXEL\_16BIT** = 0x2 << 28, **TY\_PIXEL\_24BIT** = 0x3 << 28, **TY\_PIX**↵  
**EL\_32BIT** = 0x4 << 28,  
**TY\_PIXEL\_10BIT** = 0x5 << 28, **TY\_PIXEL\_12BIT** = 0x6 << 28, **TY\_PIXEL\_14BIT** = 0x7 << 28, **TY\_PIX**↵  
**EL\_48BIT** = (uint32\_t)0x8 << 28,  
**TY\_PIXEL\_64BIT** = (uint32\_t)0xa << 28 }
- enum **TY\_PIXEL\_FORMAT\_LIST** : uint32\_t {  
**TY\_PIXEL\_FORMAT\_UNDEFINED** = 0, **TY\_PIXEL\_FORMAT\_MONO** = (TY\_PIXEL\_8BIT | (0x0 << 24)),  
**TY\_PIXEL\_FORMAT\_BAYER8GB** = (TY\_PIXEL\_8BIT | (0x1 << 24)), **TY\_PIXEL\_FORMAT\_BAYER8BG** =  
(TY\_PIXEL\_8BIT | (0x2 << 24)),  
**TY\_PIXEL\_FORMAT\_BAYER8GR** = (TY\_PIXEL\_8BIT | (0x3 << 24)), **TY\_PIXEL\_FORMAT\_BAYER8RG**  
= (TY\_PIXEL\_8BIT | (0x4 << 24)), **TY\_PIXEL\_FORMAT\_BAYER8GRBG** = TY\_PIXEL\_FORMAT\_BAY↵  
**ER8GB**, **TY\_PIXEL\_FORMAT\_BAYER8RGGB** = TY\_PIXEL\_FORMAT\_BAYER8BG,  
**TY\_PIXEL\_FORMAT\_BAYER8GBRG** = TY\_PIXEL\_FORMAT\_BAYER8GR, **TY\_PIXEL\_FORMAT\_BAY**↵  
**ER8BGR** = TY\_PIXEL\_FORMAT\_BAYER8RG, **TY\_PIXEL\_FORMAT\_CSI\_MONO10** = (TY\_PIXEL\_10BIT  
| (0x0 << 24)), **TY\_PIXEL\_FORMAT\_CSI\_BAYER10GRBG** = (TY\_PIXEL\_10BIT | (0x1 << 24)),  
**TY\_PIXEL\_FORMAT\_CSI\_BAYER10RGGB** = (TY\_PIXEL\_10BIT | (0x2 << 24)), **TY\_PIXEL\_FORMAT\_CSI\_BAYER10GBRG**  
= (TY\_PIXEL\_10BIT | (0x3 << 24)), **TY\_PIXEL\_FORMAT\_CSI\_BAYER10BGGR** = (TY\_PIXEL\_10BIT | (0x4  
<< 24)), **TY\_PIXEL\_FORMAT\_CSI\_MONO12** = (TY\_PIXEL\_12BIT | (0x0 << 24)),  
**TY\_PIXEL\_FORMAT\_CSI\_BAYER12GRBG** = (TY\_PIXEL\_12BIT | (0x1 << 24)), **TY\_PIXEL\_FORMAT\_CSI\_BAYER12RGGB**  
= (TY\_PIXEL\_12BIT | (0x2 << 24)), **TY\_PIXEL\_FORMAT\_CSI\_BAYER12GBRG** = (TY\_PIXEL\_12BIT | (0x3  
<< 24)), **TY\_PIXEL\_FORMAT\_CSI\_BAYER12BGGR** = (TY\_PIXEL\_12BIT | (0x4 << 24)),  
**TY\_PIXEL\_FORMAT\_DEPTH16** = (TY\_PIXEL\_16BIT | (0x0 << 24)), **TY\_PIXEL\_FORMAT\_YVYU** =  
(TY\_PIXEL\_16BIT | (0x1 << 24)), **TY\_PIXEL\_FORMAT\_YUYV** = (TY\_PIXEL\_16BIT | (0x2 << 24)),  
**TY\_PIXEL\_FORMAT\_MONO16** = (TY\_PIXEL\_16BIT | (0x3 << 24)),  
**TY\_PIXEL\_FORMAT\_TOF\_IR\_MONO16** = (TY\_PIXEL\_64BIT | (0x4 << 24)), **TY\_PIXEL\_FORMAT\_RGB**  
= (TY\_PIXEL\_24BIT | (0x0 << 24)), **TY\_PIXEL\_FORMAT\_BGR** = (TY\_PIXEL\_24BIT | (0x1 << 24)),  
**TY\_PIXEL\_FORMAT\_JPEG** = (TY\_PIXEL\_24BIT | (0x2 << 24)),  
**TY\_PIXEL\_FORMAT\_MJPG** = (TY\_PIXEL\_24BIT | (0x3 << 24)), **TY\_PIXEL\_FORMAT\_RGB48** = (T↵  
**Y\_PIXEL\_48BIT** | (0x0 << 24)), **TY\_PIXEL\_FORMAT\_BGR48** = (TY\_PIXEL\_48BIT | (0x1 << 24)),  
**TY\_PIXEL\_FORMAT\_XYZ48** = (TY\_PIXEL\_48BIT | (0x2 << 24)) }

*pixel format definitions*

- enum **TY\_RESOLUTION\_MODE\_LIST** : uint32\_t {  
**TY\_RESOLUTION\_MODE\_160x100** = (160<<12)+100, **TY\_RESOLUTION\_MODE\_160x120** = (160<<12)+120,  
**TY\_RESOLUTION\_MODE\_240x320** = (240<<12)+320, **TY\_RESOLUTION\_MODE\_320x180** = (320<<12)+180,  
**TY\_RESOLUTION\_MODE\_320x200** = (320<<12)+200, **TY\_RESOLUTION\_MODE\_320x240** = (320<<12)+240,  
**TY\_RESOLUTION\_MODE\_480x640** = (480<<12)+640, **TY\_RESOLUTION\_MODE\_640x360** = (640<<12)+360,  
**TY\_RESOLUTION\_MODE\_640x400** = (640<<12)+400, **TY\_RESOLUTION\_MODE\_640x480** = (640<<12)+480,  
**TY\_RESOLUTION\_MODE\_960x1280** = (960<<12)+1280, **TY\_RESOLUTION\_MODE\_1280x720** =  
(1280<<12)+720,  
**TY\_RESOLUTION\_MODE\_1280x800** = (1280<<12)+800, **TY\_RESOLUTION\_MODE\_1280x960** =

```
(1280<<12)+960, TY_RESOLUTION_MODE_1600x1200 = (1600<<12)+1200, TY_RESOLUTION_MODE_800x600
= (800<<12)+600,
TY_RESOLUTION_MODE_1920x1080 = (1920<<12)+1080, TY_RESOLUTION_MODE_2560x1920 =
(2560<<12)+1920, TY_RESOLUTION_MODE_2592x1944 = (2592<<12)+1944, TY_RESOLUTION_MODE_1920x1440
= (1920<<12)+1440,
TY_RESOLUTION_MODE_240x96 = (240<<12)+96, TY_RESOLUTION_MODE_2048x1536 = (2048<<12)+1536
}
```

*predefined resolution list*

- enum `TY_IMAGE_MODE_LIST` : uint32\_t {  
`TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_`↵  
`IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO),  
`TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_`↵  
`IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO),  
`TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_`↵  
`IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO),  
`TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_`↵  
`IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO),  
`TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_`↵  
`IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO),  
`TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_`↵  
`IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO),  
`TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_`↵  
`IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO),  
`TY_DECLARE_IMAGE_MODE1`=(MONO), `TY_DECLARE_IMAGE_MODE1`=(MONO) }

*Predefined Image Mode List image mode controls image resolution & format predefined image modes named like  
TY\_IMAGE\_MODE\_MONO\_160x120, TY\_IMAGE\_MODE\_RGB\_1280x960.*

- enum `TY_TRIGGER_MODE_LIST` : uint32\_t {  
`TY_TRIGGER_MODE_OFF` = 0, `TY_TRIGGER_MODE_SLAVE` = 1, `TY_TRIGGER_MODE_M_SIG` = 2,  
`TY_TRIGGER_MODE_M_PER` = 3,  
`TY_TRIGGER_MODE_SIG_PASS` = 18, `TY_TRIGGER_MODE_PER_PASS` = 19, `TY_TRIGGER_MODE_`↵  
`TIMER_LIST` = 20, `TY_TRIGGER_MODE_TIMER_PERIOD` = 21,  
`TY_TRIGGER_MODE28` = 28, `TY_TRIGGER_MODE29` = 29, `TY_TRIGGER_MODE_PER_PASS2` = 30,  
`TY_TRIGGER_WORK_MODE31` = 31,  
`TY_TRIGGER_MODE_SIG_LASER` = 34 }
- enum `TY_TIME_SYNC_TYPE_LIST` : uint32\_t {  
`TY_TIME_SYNC_TYPE_NONE` = 0, `TY_TIME_SYNC_TYPE_HOST` = 1, `TY_TIME_SYNC_TYPE_NTP` = 2,  
`TY_TIME_SYNC_TYPE_PTP` = 3,  
`TY_TIME_SYNC_TYPE_CAN` = 4, `TY_TIME_SYNC_TYPE_PTP_MASTER` = 5 }

*type of time sync*

- enum `TY_LOG_LEVEL_LIST` {  
`TY_LOG_LEVEL_VERBOSE` = 1, `TY_LOG_LEVEL_DEBUG` = 2, `TY_LOG_LEVEL_INFO` = 3, `TY_LOG_`↵  
`LEVEL_WARNING` = 4,  
`TY_LOG_LEVEL_ERROR` = 5, `TY_LOG_LEVEL_NEVER` = 9 }
- enum `TY_LOG_TYPE_LIST` { `TY_LOG_TYPE_STD`, `TY_LOG_TYPE_FILE`, `TY_LOG_TYPE_SERVER` }
- enum `TY_SERVER_TYPE_LIST` { `TY_SERVER_TYPE_UDP`, `TY_SERVER_TYPE_TCP` }
- enum `TY_VISIBILITY_TYPE` { `BEGINNER` = 0, `EXPERT` = 1, `GURU` = 2, `INVLISIBLE` = 3 }
- enum { `TY_PATTERN_SINE_TYPE` = 0, `TY_PATTERN_GRAY_TYPE`, `TY_PATTERN_BIN_TYPE`, `TY_`↵  
`PATTERN_EMPTY_TYPE` = 0xffffffff }
- enum { `TY_NORMAL_PHASE_TYPE` = 0, `TY_REFER_PHASE_TYPE` }
- enum `TY_IMU_FPS_LIST` { `TY_IMU_FPS_100HZ` = 0, `TY_IMU_FPS_200HZ`, `TY_IMU_FPS_400HZ` }

## Variables

- typedef enum
- typedef `TY_DO_5V` = 1



- typedef **TY\_DO\_12V** = 2
- typedef **TY\_E\_VOLT\_T\_LIST**
- typedef **TY\_DO\_HIGH** = 1
- typedef **TY\_DO\_PWM** = 2
- typedef **TY\_DO\_CAM\_TRIG** = 3
- typedef **TY\_E\_DO\_MODE\_LIST**
- typedef **TY\_DI\_NE\_INT** = 1
- typedef **TY\_DI\_PE\_INT** = 2
- typedef **TY\_E\_DI\_MODE\_LIST**
- typedef **TY\_DI\_INT\_TRIG\_CAP** = 1
- typedef **TY\_DI\_INT\_EVENT** = 2
- typedef **TY\_E\_DI\_INT\_ACTION\_LIST**
- typedef **TY\_TEMPERATURE\_RIGHT** = 1
- typedef **TY\_TEMPERATURE\_COLOR** = 2
- typedef **TY\_TEMPERATURE\_CPU** = 3
- typedef **TY\_TEMPERATURE\_MAIN\_BOARD** = 4
- typedef **TY\_TEMPERATURE\_ID\_LIST**

### 5.3.1 Detailed Description

[TYDefs.h](#) includes camera control and data receiving data definitions which supports configuration for image resolution, frame rate, exposure time, gain, working mode, etc.

### 5.3.2 Macro Definition Documentation

#### 5.3.2.1 TY\_DECLARE\_IMAGE\_MODE1

```
#define TY_DECLARE_IMAGE_MODE1(
 pix)
```

**Value:**

```
TY_DECLARE_IMAGE_MODE0(pix, 160x100), \
TY_DECLARE_IMAGE_MODE0(pix, 160x120), \
TY_DECLARE_IMAGE_MODE0(pix, 320x180), \
TY_DECLARE_IMAGE_MODE0(pix, 320x200), \
TY_DECLARE_IMAGE_MODE0(pix, 320x240), \
TY_DECLARE_IMAGE_MODE0(pix, 480x640), \
TY_DECLARE_IMAGE_MODE0(pix, 640x360), \
TY_DECLARE_IMAGE_MODE0(pix, 640x400), \
TY_DECLARE_IMAGE_MODE0(pix, 640x480), \
TY_DECLARE_IMAGE_MODE0(pix, 960x1280), \
TY_DECLARE_IMAGE_MODE0(pix, 1280x720), \
TY_DECLARE_IMAGE_MODE0(pix, 1280x960), \
TY_DECLARE_IMAGE_MODE0(pix, 1280x800), \
TY_DECLARE_IMAGE_MODE0(pix, 1600x1200), \
TY_DECLARE_IMAGE_MODE0(pix, 800x600), \
TY_DECLARE_IMAGE_MODE0(pix, 1920x1080), \
TY_DECLARE_IMAGE_MODE0(pix, 2560x1920), \
TY_DECLARE_IMAGE_MODE0(pix, 2592x1944), \
TY_DECLARE_IMAGE_MODE0(pix, 1920x1440), \
TY_DECLARE_IMAGE_MODE0(pix, 2048x1536), \
TY_DECLARE_IMAGE_MODE0(pix, 240x96)
```

Definition at line 621 of file TYDefs.h.

### 5.3.3 Typedef Documentation

#### 5.3.3.1 TY\_ACC\_BIAS

```
typedef struct TY_ACC_BIAS TY_ACC_BIAS
```

a 3x3 matrix

| .     | .     | .     |
|-------|-------|-------|
| BIASx | BIASy | BIASz |

#### 5.3.3.2 TY\_ACC\_MISALIGNMENT

```
typedef struct TY_ACC_MISALIGNMENT TY_ACC_MISALIGNMENT
```

a 3x3 matrix

|.|.|

| .      | .      | .       |
|--------|--------|---------|
| 1      | -GAMAz | GAMAz   |
| GAMAxz | 1      | -GAMAzx |
| -GAMAx | GAMAy  | 1       |

#### 5.3.3.3 TY\_ACC\_SCALE

```
typedef struct TY_ACC_SCALE TY_ACC_SCALE
```

a 3x3 matrix

| .      | .      | .      |
|--------|--------|--------|
| SCALEx | 0      | 0      |
| 0      | SCALEy | 0      |
| 0      | 0      | SCALEz |

#### 5.3.3.4 TY\_ACCESS\_MODE\_LIST

```
typedef enum TY_ACCESS_MODE_LIST TY_ACCESS_MODE_LIST
```

Indicate a feature is readable or writable

See also

[TYGetFeatureInfo](#)

#### 5.3.3.5 TY\_BYTEARRAY\_ATTR

```
typedef struct TY_BYTEARRAY_ATTR TY_BYTEARRAY_ATTR
```

byte array data structure

See also

[TYGetByteArray](#)

#### 5.3.3.6 TY\_CAMERA\_CALIB\_INFO

```
typedef struct TY_CAMERA_CALIB_INFO TY_CAMERA_CALIB_INFO
```

camera 's caillbration data

See also

[TYGetStruct](#)

#### 5.3.3.7 TY\_CAMERA\_DISTORTION

```
typedef struct TY_CAMERA_DISTORTION TY_CAMERA_DISTORTION
```

camera distortion parameters

See also

[TYGetStruct](#) Usage:

```
TY_CAMERA_DISTORTION distortion;
TYGetStruct(hDevice, some_component, TY_STRUCT_CAM_DISTORTION, &distortion, sizeof(distortion));
```

#### 5.3.3.8 TY\_CAMERA\_EXTRINSIC

```
typedef struct TY_CAMERA_EXTRINSIC TY_CAMERA_EXTRINSIC
```

a 4x4 matrix

| .   | .   | .   | .  |
|-----|-----|-----|----|
| r11 | r12 | r13 | t1 |
| r21 | r22 | r23 | t2 |
| r31 | r32 | r33 | t3 |
| 0   | 0   | 0   | 1  |

See also

[TYGetStruct](#) Usage:

```
TY_CAMERA_EXTRINSIC extrinsic;
TYGetStruct(hDevice, some_compoent, TY_STRUCT_EXTRINSIC, &extrinsic, sizeof(extrinsic));
```

### 5.3.3.9 TY\_CAMERA\_INTRINSIC

```
typedef struct TY_CAMERA_INTRINSIC TY_CAMERA_INTRINSIC
```

a 3x3 matrix

| .  | .  | .  |
|----|----|----|
| fx | 0  | cx |
| 0  | fy | cy |
| 0  | 0  | 1  |

See also

[TYGetStruct](#) Usage:

```
TY_CAMERA_INTRINSIC intrinsic;
TYGetStruct(hDevice, some_compoent, TY_STRUCT_CAM_INTRINSIC, &intrinsic, sizeof(intrinsic));
```

### 5.3.3.10 TY\_CAMERA\_ROTATION

```
typedef struct TY_CAMERA_ROTATION TY_CAMERA_ROTATION
```

a 3x3 matrix

| .   | .   | .   |
|-----|-----|-----|
| r00 | r01 | r02 |
| r10 | r11 | r12 |
| r20 | r21 | r22 |

See also

[TYGetStruct](#) Usage:

```
TY_CAMERA_ROTATION rotation;
TYGetStruct(hDevice, some_compoent, TY_STRUCT_CAM_ROTATION, &rotation, sizeof(rotation));
```

#### 5.3.3.11 TY\_CAMERA\_TO\_IMU

```
typedef struct TY_CAMERA_TO_IMU TY_CAMERA_TO_IMU
```

a 4x4 matrix

| .   | .   | .   | .  |
|-----|-----|-----|----|
| r11 | r12 | r13 | t1 |
| r21 | r22 | r23 | t2 |
| r31 | r32 | r33 | t3 |
| 0   | 0   | 0   | 1  |

#### 5.3.3.12 TY\_COMPONENT\_ID

```
typedef uint32_t TY_COMPONENT_ID
```

component unique id

See also

[TY\\_DEVICE\\_COMPONENT\\_LIST](#)

Definition at line 192 of file TYDefs.h.

#### 5.3.3.13 TY\_DEVICE\_BASE\_INFO

```
typedef struct TY_DEVICE_BASE_INFO TY_DEVICE_BASE_INFO
```

See also

[TYGetDeviceList](#)

#### 5.3.3.14 TY\_DEVICE\_COMPONENT\_LIST

```
typedef enum TY_DEVICE_COMPONENT_LIST TY_DEVICE_COMPONENT_LIST
```

@brief Device Component list A device contains several component. Each component can be controlled by its own features, such as image width, exposure time, etc..

See also

To Know how to get feature information please refer to sample code DumpAllFeatures

### 5.3.3.15 TY\_ENUM\_ENTRY

```
typedef struct TY_ENUM_ENTRY TY_ENUM_ENTRY
```

enum feature entry information

See also

[TYGetEnumEntryInfo](#)

### 5.3.3.16 TY\_FEATURE\_ID

```
typedef uint32_t TY_FEATURE_ID
```

feature unique id

See also

[TY\\_FEATURE\\_ID\\_LIST](#)

Definition at line 378 of file TYDefs.h.

### 5.3.3.17 TY\_FLOAT\_RANGE

```
typedef struct TY_FLOAT_RANGE TY_FLOAT_RANGE
```

float range data structure

See also

[TYGetFloatRange](#)

### 5.3.3.18 TY\_GYRO\_BIAS

```
typedef struct TY_GYRO_BIAS TY_GYRO_BIAS
```

a 3x3 matrix

|       |       |       |
|-------|-------|-------|
| .     | .     | .     |
| BIASx | BIASy | BIASz |

### 5.3.3.19 TY\_GYRO\_MISALIGNMENT

```
typedef struct TY_GYRO_MISALIGNMENT TY_GYRO_MISALIGNMENT
```

a 3x3 matrix

| . | .        | .        |
|---|----------|----------|
| 1 | -ALPHAyz | ALPHAzy  |
| 0 | 1        | -ALPHAzx |
| 0 | 0        | 1        |

### 5.3.3.20 TY\_GYRO\_SCALE

```
typedef struct TY_GYRO_SCALE TY_GYRO_SCALE
```

a 3x3 matrix

| .      | .      | .      |
|--------|--------|--------|
| SCALEx | 0      | 0      |
| 0      | SCALEy | 0      |
| 0      | 0      | SCALEz |

### 5.3.3.21 TY\_INTERFACE\_INFO

```
typedef struct TY_INTERFACE_INFO TY_INTERFACE_INFO
```

See also

[TYGetInterfaceList](#)

### 5.3.3.22 TY\_INTERFACE\_TYPE\_LIST

```
typedef enum TY_INTERFACE_TYPE_LIST TY_INTERFACE_TYPE_LIST
```

Interface type definition

See also

[TYGetInterfaceList](#)

#### 5.3.3.23 TY\_PIXEL\_BITS\_LIST

```
typedef enum TY_PIXEL_BITS_LIST TY_PIXEL_BITS_LIST
```

Pixel size type definitions to define the pixel size in bits

See also

[TY\\_PIXEL\\_FORMAT\\_LIST](#)

#### 5.3.3.24 TY\_TRIGGER\_MODE\_LIST

```
typedef enum TY_TRIGGER_MODE_LIST TY_TRIGGER_MODE_LIST
```

See also

refer to sample SimpleView\_TriggerMode for detail usage

### 5.3.4 Enumeration Type Documentation

#### 5.3.4.1 TY\_ACCESS\_MODE\_LIST

```
enum TY_ACCESS_MODE_LIST : uint32_t
```

Indicate a feature is readable or writable

See also

[TYGetFeatureInfo](#)

Definition at line 432 of file TYDefs.h.

#### 5.3.4.2 TY\_DEVICE\_COMPONENT\_LIST

```
enum TY_DEVICE_COMPONENT_LIST : uint32_t
```

@brief Device Component list A device contains several component. Each component can be controlled by its own features, such as image width, exposure time, etc..

See also

To Know how to get feature information please refer to sample code DumpAllFeatures



## Enumerator

|                            |                                                             |
|----------------------------|-------------------------------------------------------------|
| TY_COMPONENT_DEVICE        | Abstract component stands for whole device, always enabled. |
| TY_COMPONENT_DEPTH_CAM     | Depth camera.                                               |
| TY_COMPONENT_IR_CAM_LEFT   | Left IR camera.                                             |
| TY_COMPONENT_IR_CAM_RIGHT  | Right IR camera.                                            |
| TY_COMPONENT_RGB_CAM_LEFT  | Left RGB camera.                                            |
| TY_COMPONENT_RGB_CAM_RIGHT | Right RGB camera.                                           |
| TY_COMPONENT_LASER         | Laser.                                                      |
| TY_COMPONENT_IMU           | Inertial Measurement Unit.                                  |
| TY_COMPONENT_BRIGHT_HISTO  | virtual component for brightness histogram of ir            |
| TY_COMPONENT_STORAGE       | virtual component for device storage                        |
| TY_COMPONENT_RGB_CAM       | Some device has only one RGB camera, map it to left.        |

Definition at line 177 of file TYDefs.h.

## 5.3.4.3 TY\_FEATURE\_ID\_LIST

```
enum TY_FEATURE_ID_LIST : uint32_t
```

feature for component definitions

## Enumerator

|                                  |                                                                                                                                                                                   |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TY_STRUCT_CAM_INTRINSIC          | see <a href="#">TY_CAMERA_INTRINSIC</a>                                                                                                                                           |
| TY_STRUCT_EXTRINSIC_TO_DEPTH     | extrinsic between depth cam and current component , see <a href="#">TY_CAMERA_EXTRINSIC</a>                                                                                       |
| TY_STRUCT_EXTRINSIC_TO_IR_LEFT   | extrinsic between left IR and current compoent, see <a href="#">TY_CAMERA_EXTRINSIC</a>                                                                                           |
| TY_STRUCT_CAM_RECTIFIED_ROTATION | see <a href="#">TY_CAMERA_ROTATION</a>                                                                                                                                            |
| TY_STRUCT_CAM_DISTORTION         | see <a href="#">TY_CAMERA_DISTORTION</a>                                                                                                                                          |
| TY_STRUCT_CAM_CALIB_DATA         | see <a href="#">TY_CAMERA_CALIB_INFO</a>                                                                                                                                          |
| TY_STRUCT_CAM_RECTIFIED_INTRI    | the rectified intrinsic. see <a href="#">TY_CAMERA_INTRINSIC</a>                                                                                                                  |
| TY_BYTEARRAY_CUSTOM_BLOCK        | used for reading/writing custom block                                                                                                                                             |
| TY_BYTEARRAY_ISP_BLOCK           | used for reading/writing fpn block                                                                                                                                                |
| TY_INT_PACKET_DELAY              | microseconds                                                                                                                                                                      |
| TY_INT_NTP_SERVER_IP             | Ntp server IP.                                                                                                                                                                    |
| TY_INT_LINK_CMD_TIMEOUT          | milliseconds                                                                                                                                                                      |
| TY_STRUCT_CAM_STATISTICS         | statistical information, see <a href="#">TY_CAMERA_STATISTICS</a>                                                                                                                 |
| TY_INT_WIDTH                     | Image width.                                                                                                                                                                      |
| TY_INT_HEIGHT                    | Image height.                                                                                                                                                                     |
| TY_ENUM_IMAGE_MODE               | Resolution-PixelFormat mode, see <a href="#">TY_IMAGE_MODE_LIST</a> .                                                                                                             |
| TY_FLOAT_SCALE_UNIT              | scale unit depth image is uint16 pixel format with default millimeter unit ,for some device can output Sub-millimeter accuracy data the acutal depth (mm)= PixelValue * ScaleUnit |
| TY_ENUM_TRIGGER_POL              | Trigger POL, see <a href="#">TY_TRIGGER_POL_LIST</a> .                                                                                                                            |
| TY_INT_FRAME_PER_TRIGGER         | Number of frames captured per trigger.                                                                                                                                            |

## Enumerator

|                                 |                                                                       |
|---------------------------------|-----------------------------------------------------------------------|
| TY_STRUCT_TRIGGER_PARAM         | param of trigger, see <a href="#">TY_TRIGGER_PARAM</a>                |
| TY_STRUCT_TRIGGER_PARAM_EX      | param of trigger, see <a href="#">TY_TRIGGER_PARAM_EX</a>             |
| TY_STRUCT_TRIGGER_TIMER_LIST    | param of trigger mode 20, see <a href="#">TY_TRIGGER_TIMER_LIST</a>   |
| TY_STRUCT_TRIGGER_TIMER_PERIOD  | param of trigger mode 21, see <a href="#">TY_TRIGGER_TIMER_PERIOD</a> |
| TY_BOOL_KEEP_ALIVE_ONOFF        | Keep Alive switch.                                                    |
| TY_INT_KEEP_ALIVE_TIMEOUT       | Keep Alive timeout.                                                   |
| TY_BOOL_CMOS_SYNC               | Cmos sync switch.                                                     |
| TY_INT_TRIGGER_DELAY_US         | Trigger delay time, in microseconds.                                  |
| TY_BOOL_TRIGGER_OUT_IO          | Trigger out IO.                                                       |
| TY_INT_TRIGGER_DURATION_US      | Trigger duration time, in microseconds.                               |
| TY_ENUM_STREAM_ASYNC            | stream async switch, see <a href="#">TY_STREAM_ASYNC_MODE</a>         |
| TY_INT_CAPTURE_TIME_US          | capture time in multi-ir                                              |
| TY_ENUM_TIME_SYNC_TYPE          | see <a href="#">TY_TIME_SYNC_TYPE</a>                                 |
| TY_BOOL_TIME_SYNC_READY         | time sync done status                                                 |
| TY_BOOL_IR_FLASHLIGHT           | Enable switch for floodlight used in ir component.                    |
| TY_INT_IR_FLASHLIGHT_INTENSITY  | ir component flashlight intensity level                               |
| TY_STRUCT_DO0_WORKMODE          | DO_0 workmode, see <a href="#">TY_DO_WORKMODE</a> .                   |
| TY_STRUCT_DI0_WORKMODE          | DI_0 workmode, see <a href="#">TY_DI_WORKMODE</a> .                   |
| TY_STRUCT_DO1_WORKMODE          | DO_1 workmode, see <a href="#">TY_DO_WORKMODE</a> .                   |
| TY_STRUCT_DI1_WORKMODE          | DI_1 workmode, see <a href="#">TY_DI_WORKMODE</a> .                   |
| TY_STRUCT_DO2_WORKMODE          | DO_2 workmode, see <a href="#">TY_DO_WORKMODE</a> .                   |
| TY_STRUCT_DI2_WORKMODE          | DI_2 workmode, see <a href="#">TY_DI_WORKMODE</a> .                   |
| TY_BOOL_RGB_FLASHLIGHT          | Enable switch for floodlight used in rgb component.                   |
| TY_INT_RGB_FLASHLIGHT_INTENSITY | rgb component flashlight intensity level                              |
| TY_BOOL_AUTO_EXPOSURE           | Auto exposure switch.                                                 |
| TY_INT_EXPOSURE_TIME            | Exposure time.                                                        |
| TY_BOOL_AUTO_GAIN               | Auto gain switch.                                                     |
| TY_INT_GAIN                     | Sensor Gain.                                                          |
| TY_BOOL_AUTO_AWB                | Auto white balance.                                                   |
| TY_STRUCT_AEC_ROI               | region of aec statistics, see <a href="#">TY_AEC_ROI_PARAM</a>        |
| TY_INT_TOF_HDR_RATIO            | tof sensor hdr ratio for depth                                        |
| TY_INT_TOF_JITTER_THRESHOLD     | tof jitter threshold for depth                                        |
| TY_FLOAT_EXPOSURE_TIME_US       | the exposure time, unit: us                                           |
| TY_INT_LASER_POWER              | Laser power level.                                                    |
| TY_BOOL_LASER_AUTO_CTRL         | Laser auto ctrl.                                                      |
| TY_STRUCT_LASER_ENABLE_BY_IDX   | Laser enable by device index.                                         |
| TY_STRUCT_LASER_POWER_BY_IDX    | Laser power by device index.                                          |
| TY_STRUCT_FLOOD_ENABLE_BY_IDX   | Flood enable by device index.                                         |
| TY_STRUCT_FLOOD_POWER_BY_IDX    | Flood power by device index.                                          |
| TY_BOOL_UNDISTORTION            | Output undistorted image.                                             |
| TY_BOOL_BRIGHTNESS_HISTOGRAM    | Output bright histogram.                                              |
| TY_BOOL_DEPTH_POSTPROC          | Do depth image postproc.                                              |
| TY_INT_R_GAIN                   | Gain of R channel.                                                    |
| TY_INT_G_GAIN                   | Gain of G channel.                                                    |
| TY_INT_B_GAIN                   | Gain of B channel.                                                    |
| TY_INT_ANALOG_GAIN              | Analog gain.                                                          |

## Enumerator

|                                 |                                                                          |
|---------------------------------|--------------------------------------------------------------------------|
| TY_BOOL_HDR                     | HDR func enable/disable.                                                 |
| TY_BYTEARRAY_HDR_PARAMETER      | HDR parameters.                                                          |
| TY_BOOL_IMU_DATA_ONOFF          | AE target y. IMU Data Onoff                                              |
| TY_STRUCT_IMU_ACC_BIAS          | IMU acc bias matrix, see <a href="#">TY_ACC_BIAS</a> .                   |
| TY_STRUCT_IMU_ACC_MISALIGNMENT  | IMU acc misalignment matrix, see <a href="#">TY_ACC_MISALIGNMENT</a> .   |
| TY_STRUCT_IMU_ACC_SCALE         | IMU acc scale matrix, see <a href="#">TY_ACC_SCALE</a> .                 |
| TY_STRUCT_IMU_GYRO_BIAS         | IMU gyro bias matrix, see <a href="#">TY_GYRO_BIAS</a> .                 |
| TY_STRUCT_IMU_GYRO_MISALIGNMENT | IMU gyro misalignment matrix, see <a href="#">TY_GYRO_MISALIGNMENT</a> . |
| TY_STRUCT_IMU_GYRO_SCALE        | IMU gyro scale matrix, see <a href="#">TY_GYRO_SCALE</a> .               |
| TY_STRUCT_IMU_CAM_TO_IMU        | IMU camera to imu matrix, see <a href="#">TY_CAMERA_TO_IMU</a> .         |
| TY_ENUM_IMU_FPS                 | IMU fps, see <a href="#">TY_IMU_FPS_LIST</a> .                           |
| TY_INT_SGBM_IMAGE_NUM           | SGBM image channel num.                                                  |
| TY_INT_SGBM_DISPARITY_NUM       | SGBM disparity num.                                                      |
| TY_INT_SGBM_DISPARITY_OFFSET    | SGBM disparity offset.                                                   |
| TY_INT_SGBM_MATCH_WIN_HEIGHT    | SGBM match window height.                                                |
| TY_INT_SGBM_SEMI_PARAM_P1       | SGBM semi global param p1.                                               |
| TY_INT_SGBM_SEMI_PARAM_P2       | SGBM semi global param p2.                                               |
| TY_INT_SGBM_UNIQUE_FACTOR       | SGBM uniqueness factor param.                                            |
| TY_INT_SGBM_UNIQUE_ABSDIFF      | SGBM uniqueness min absolute diff.                                       |
| TY_INT_SGBM_UNIQUE_MAX_COST     | SGBM uniqueness max cost param.                                          |
| TY_BOOL_SGBM_HFILTER_HALF_WIN   | SGBM enable half window size.                                            |
| TY_INT_SGBM_MATCH_WIN_WIDTH     | SGBM match window width.                                                 |
| TY_BOOL_SGBM_MEDFILTER          | SGBM enable median filter.                                               |
| TY_BOOL_SGBM_LRC                | SGBM enable left right consist check.                                    |
| TY_INT_SGBM_LRC_DIFF            | SGBM max diff.                                                           |
| TY_INT_SGBM_MEDFILTER_THRESH    | SGBM median filter thresh.                                               |
| TY_INT_SGBM_SEMI_PARAM_P1_SCALE | SGBM semi global param p1 scale.                                         |
| TY_INT_SGPM_PHASE_NUM           | Phase num to calc a depth.                                               |
| TY_INT_SGPM_NORMAL_PHASE_SCALE  | phase scale when calc a depth                                            |
| TY_INT_SGPM_NORMAL_PHASE_OFFSET | Phase offset when calc a depth.                                          |
| TY_INT_SGPM_REF_PHASE_SCALE     | Reference Phase scale when calc a depth.                                 |
| TY_INT_SGPM_REF_PHASE_OFFSET    | Reference Phase offset when calc a depth.                                |
| TY_FLOAT_SGPM_EPI_HS            | Epipolar Constraint pattern scale.                                       |
| TY_INT_SGPM_EPI_HF              | Epipolar Constraint pattern offset.                                      |
| TY_BOOL_SGPM_EPI_EN             | Epipolar Constraint enable.                                              |
| TY_INT_SGPM_EPI_CH0             | Epipolar Constraint channel0.                                            |
| TY_INT_SGPM_EPI_CH1             | Epipolar Constraint channel1.                                            |
| TY_INT_SGPM_EPI_THRESH          | Epipolar Constraint thresh.                                              |
| TY_BOOL_SGPM_ORDER_FILTER_EN    | Phase order filter enable.                                               |
| TY_INT_SGPM_ORDER_FILTER_CHN    | Phase order filter channel.                                              |
| TY_INT_DEPTH_MIN_MM             | min depth in mm output                                                   |
| TY_INT_DEPTH_MAX_MM             | max depth in mm ouput                                                    |
| TY_INT_SGBM_TEXTURE_OFFSET      | texture filter value offset                                              |
| TY_INT_SGBM_TEXTURE_THRESH      | texture filter threshold                                                 |
| TY_STRUCT_PHC_GROUP_ATTR        | Phase compute group attribute.                                           |

## Enumerator

|                                 |                                                       |
|---------------------------------|-------------------------------------------------------|
| TY_ENUM_DEPTH_QUALITY           | the quality of generated depth, see TY_DEPTH_QUALITY  |
| TY_INT_FILTER_THRESHOLD         | the threshold of the noise filter, 0 for disabled     |
| TY_INT_TOF_CHANNEL              | the frequency channel of tof                          |
| TY_INT_TOF_MODULATION_THRESHOLD | the threshold of the tof modulation                   |
| TY_STRUCT_TOF_FREQ              | the frequency of tof, see <a href="#">TY_TOF_FREQ</a> |
| TY_BOOL_TOF_ANTI_INTERFERENCE   | cooperation if multi-device used                      |
| TY_INT_TOF_ANTI_SUNLIGHT_INDEX  | the index of anti-sunlight                            |
| TY_INT_MAX_SPECKLE_SIZE         | the max size of speckle                               |
| TY_INT_MAX_SPECKLE_DIFF         | the max diff of speckle                               |

Definition at line 211 of file TYDefs.h.

#### 5.3.4.4 TY\_INTERFACE\_TYPE\_LIST

```
enum TY_INTERFACE_TYPE_LIST : uint32_t
```

Interface type definition

See also

[TYGetInterfaceList](#)

Definition at line 419 of file TYDefs.h.

#### 5.3.4.5 TY\_PIXEL\_BITS\_LIST

```
enum TY_PIXEL_BITS_LIST : uint32_t
```

Pixel size type definitions to define the pixel size in bits

See also

[TY\\_PIXEL\\_FORMAT\\_LIST](#)

Definition at line 529 of file TYDefs.h.

#### 5.3.4.6 TY\_PIXEL\_FORMAT\_LIST

```
enum TY_PIXEL_FORMAT_LIST : uint32_t
```

pixel format definitions

## Enumerator

|                                 |                     |
|---------------------------------|---------------------|
| TY_PIXEL_FORMAT_MONO            | 0x10000000          |
| TY_PIXEL_FORMAT_BAYER8GB        | 0x11000000          |
| TY_PIXEL_FORMAT_BAYER8BG        | 0x12000000          |
| TY_PIXEL_FORMAT_BAYER8GR        | 0x13000000          |
| TY_PIXEL_FORMAT_BAYER8RG        | 0x14000000          |
| TY_PIXEL_FORMAT_CSI_MONO10      | 0x50000000          |
| TY_PIXEL_FORMAT_CSI_BAYER10GRBG | 0x51000000          |
| TY_PIXEL_FORMAT_CSI_BAYER10RGGB | 0x52000000          |
| TY_PIXEL_FORMAT_CSI_BAYER10GBRG | 0x53000000          |
| TY_PIXEL_FORMAT_CSI_BAYER10BGGR | 0x54000000          |
| TY_PIXEL_FORMAT_CSI_MONO12      | 0x60000000          |
| TY_PIXEL_FORMAT_CSI_BAYER12GRBG | 0x61000000          |
| TY_PIXEL_FORMAT_CSI_BAYER12RGGB | 0x62000000          |
| TY_PIXEL_FORMAT_CSI_BAYER12GBRG | 0x63000000          |
| TY_PIXEL_FORMAT_CSI_BAYER12BGGR | 0x64000000          |
| TY_PIXEL_FORMAT_DEPTH16         | 0x20000000          |
| TY_PIXEL_FORMAT_YVYU            | 0x21000000, yvyu422 |
| TY_PIXEL_FORMAT_YUYV            | 0x22000000, yuyv422 |
| TY_PIXEL_FORMAT_MONO16          | 0x23000000,         |
| TY_PIXEL_FORMAT_TOF_IR_MONO16   | 0xa4000000,         |
| TY_PIXEL_FORMAT_RGB             | 0x30000000          |
| TY_PIXEL_FORMAT_BGR             | 0x31000000          |
| TY_PIXEL_FORMAT_JPEG            | 0x32000000          |
| TY_PIXEL_FORMAT_MJPEG           | 0x33000000          |
| TY_PIXEL_FORMAT_RGB48           | 0x80000000          |
| TY_PIXEL_FORMAT_BGR48           | 0x81000000          |
| TY_PIXEL_FORMAT_XYZ48           | 0x82000000          |

Definition at line 547 of file TYDefs.h.

### 5.3.4.7 TY\_RESOLUTION\_MODE\_LIST

```
enum TY_RESOLUTION_MODE_LIST : uint32_t
```

predefined resolution list

## Enumerator

|                            |            |
|----------------------------|------------|
| TY_RESOLUTION_MODE_160x100 | 0x000a0078 |
| TY_RESOLUTION_MODE_160x120 | 0x000a0078 |
| TY_RESOLUTION_MODE_240x320 | 0x000f0140 |
| TY_RESOLUTION_MODE_320x180 | 0x001400b4 |
| TY_RESOLUTION_MODE_320x200 | 0x001400c8 |
| TY_RESOLUTION_MODE_320x240 | 0x001400f0 |
| TY_RESOLUTION_MODE_480x640 | 0x001e0280 |
| TY_RESOLUTION_MODE_640x360 | 0x00280168 |

## Enumerator

|                              |            |
|------------------------------|------------|
| TY_RESOLUTION_MODE_640x400   | 0x00280190 |
| TY_RESOLUTION_MODE_640x480   | 0x002801e0 |
| TY_RESOLUTION_MODE_960x1280  | 0x003c0500 |
| TY_RESOLUTION_MODE_1280x720  | 0x005002d0 |
| TY_RESOLUTION_MODE_1280x800  | 0x00500320 |
| TY_RESOLUTION_MODE_1280x960  | 0x005003c0 |
| TY_RESOLUTION_MODE_1600x1200 | 0x006404b0 |
| TY_RESOLUTION_MODE_800x600   | 0x00320258 |
| TY_RESOLUTION_MODE_1920x1080 | 0x00780438 |
| TY_RESOLUTION_MODE_2560x1920 | 0x00a00780 |
| TY_RESOLUTION_MODE_2592x1944 | 0x00a20798 |
| TY_RESOLUTION_MODE_1920x1440 | 0x007805a0 |
| TY_RESOLUTION_MODE_240x96    | 0x000f0060 |
| TY_RESOLUTION_MODE_2048x1536 | 0x00800600 |

Definition at line 591 of file TYDefs.h.

#### 5.3.4.8 TY\_TRIGGER\_MODE\_LIST

```
enum TY_TRIGGER_MODE_LIST : uint32_t
```

#### See also

refer to sample SimpleView\_TriggerMode for detail usage

## Enumerator

|                           |                                                                                |
|---------------------------|--------------------------------------------------------------------------------|
| TY_TRIGGER_MODE_OFF       | not trigger mode, continuous mode                                              |
| TY_TRIGGER_MODE_SLAVE     | slave mode, receive soft/hardware triggers                                     |
| TY_TRIGGER_MODE_M_SIG     | master mode 1, sending one trigger signal once received a soft trigger         |
| TY_TRIGGER_MODE_M_PER     | master mode 2, periodic sending one trigger signals, 'fps' param should be set |
| TY_TRIGGER_MODE_SIG_PASS  | discard, using TY_TRIGGER_MODE28                                               |
| TY_TRIGGER_MODE_PER_PASS  | discard, using TY_TRIGGER_MODE29                                               |
| TY_TRIGGER_MODE_PER_PASS2 | trigger mode 30, Alternate output depth image/ir image                         |

Definition at line 692 of file TYDefs.h.

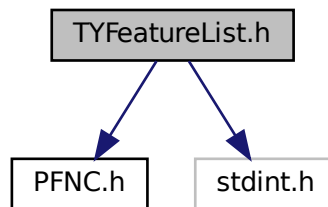
## 5.4 TYFeatureList.h File Reference

Camera feature enumeration list.

```
#include "PFNC.h"
```

```
#include <stdint.h>
```

Include dependency graph for TYFeatureList.h:



## Macros

- `#define TY_DEV_TL_VER_MAJ` "DeviceTLVersionMajor"  
*Integer. Major version of the Transport Layer of the device.*
- `#define TY_DEV_TL_VER_MIN` "DeviceTLVersionMinor"  
*Integer. Minor version of the Transport Layer of the device.*
- `#define TY_DEV_TYPE` "DeviceType"  
*Enumeration. Returns the device type.*
- `#define TY_DEV_VENDOR` "DeviceVendorName"  
*String. Name of the device vendor.*
- `#define TY_DEV_MODEL` "DeviceModelName"  
*String. Model of the device.*
- `#define TY_DEV_MANU_INFO` "DeviceManufacturerInfo"  
*String. Manufacturer information about the device.*
- `#define TY_DEV_VER` "DeviceVersion"  
*String. Version of the device.*
- `#define TY_DEV_HW_MODEL` "DeviceHardWareModel"  
*String. Hardware model of the device.*
- `#define TY_DEV_HW_VER` "DeviceHardwareVersion"  
*String. Version of the Board Hardware.*
- `#define TY_DEV_FW_VER` "DeviceFirmwareVersion"  
*String. Version of the firmware in the device.*
- `#define TY_DEV_SN` "DeviceSerialNumber"  
*String. Serial number of the device.*
- `#define TY_DEV_BUILD_HASH` "DeviceBuildHash"  
*String. Build hash of the device.*
- `#define TY_DEV_CFG_VER` "DeviceConfigVersion"  
*String. Config version of the device.*
- `#define TY_DEV_TECH_MODEL` "DeviceTechModel"  
*String. Technical model of the device.*
- `#define TY_DEV_GEN_TIME` "DeviceGeneratedTime"  
*String. Generated time of the device.*
- `#define TY_DEV_CAL_TIME` "DeviceCalibrationTime"

- String. Calibration time of the device.*

  - `#define TY_DEV_CONN_SEL "DeviceConnectionSelector"`

*Integer. Selects which Connection of the device to control.*
- `#define TY_DEV_LINK_SEL "DeviceLinkSelector"`

*Integer. Selects which Link of the device to control.*
- `#define TY_DEV_LINK_SPEED "DeviceLinkSpeed"`

*Integer. Speed of the selected link.*
- `#define TY_DEV_LINK_HB_MODE "DeviceLinkHeartbeatMode"`

*Enumeration. Activate or deactivate the Link's heartbeat.*
- `#define TY_DEV_LINK_HB_TIMEOUT "DeviceLinkHeartbeatTimeout"`

*Integer. Speed of the selected link.*
- `#define TY_DEV_STREAM_CH_COUNT "DeviceStreamChannelCount"`

*Integer. Indicates the number of streaming channels supported by the device.*
- `#define TY_DEV_STREAM_CH_SEL "DeviceStreamChannelSelector"`

*Integer. Selects the stream channel to configure.*
- `#define TY_DEV_STREAM_CH_TYPE "DeviceStreamChannelType"`

*Enumeration. Type of the selected stream channel.*
- `#define TY_DEV_STREAM_CH_LINK "DeviceStreamChannelLink"`

*Integer. Link associated with the selected stream channel.*
- `#define TY_DEV_STREAM_CH_PKT_SIZE "DeviceStreamChannelPacketSize"`

*Integer. Packet size used on the selected stream channel.*
- `#define TY_DEV_CHARSET "DeviceCharacterSet"`

*Enumeration. Character set used by the strings of the device's bootstrap registers.*
- `#define TY_DEV_RESET "DeviceReset"`

*Command. Reset the device to the state it had after power-up.*
- `#define TY_DEV_ERR_CODE "DeviceErrCode"`

*Integer. Error code of the device.*
- `#define TY_DEV_FRAME_TIMEOUT "DeviceFrameRecvTimeOut"`

*Integer. The waiting timeout for the device to send a single frame of data (unit: ms)*
- `#define TY_DEV_TEMP_SEL "DeviceTemperatureSelector"`

*Enumeration. Selects the temperature sensor to read.*
- `#define TY_DEV_TEMP "DeviceTemperature"`

*Float. Temperature of the selected by DeviceTemperatureSelector.*
- `#define TY_DEV_TEMP_STR "DeviceTemperatureString"`

*String. Temperature of the selected by DeviceTemperatureSelector.*
- `#define TY_DEV_STREAM_ASYNC "DeviceStreamAsyncMode"`

*Enumeration. Controls stream asynchronous mode.*
- `#define TY_DEV_TIME_SYNC "DeviceTimeSyncMode"`

*Enumeration. Selects the time synchronization mode.*
- `#define TY_NTP_SRV_IP "NTPServerIP"`

*Integer. NTP server IP address.*
- `#define TY_DEV_COMPRESS "DeviceDataCompressType"`

*Enumeration. Selects the data compression type.*
- `#define TY_ACQ_MODE "AcquisitionMode"`

*Enumeration. Sets the acquisition mode of the device.*
- `#define TY_ACQ_START "AcquisitionStart"`

*Command. Starts the acquisition of images.*
- `#define TY_ACQ_STOP "AcquisitionStop"`

*Command. Stops the acquisition of images.*
- `#define TY_ACQ_FPS "AcquisitionFrameRate"`

*Float. Controls the acquisition frame rate.*



- `#define TY_ACQ_FPS_EN` "AcquisitionFrameRateEnable"  
*Boolean. Enables the acquisition frame rate control.*
- `#define TY_EXP_AUTO` "ExposureAuto"  
*Boolean. Sets the automatic exposure mode.*
- `#define TY_EXP_AUTO_MIN` "ExposureAutoLowerLimit"  
*Float. Lower limit of the auto exposure time.*
- `#define TY_EXP_AUTO_MAX` "ExposureAutoUpperLimit"  
*Float. Upper limit of the auto exposure time.*
- `#define TY_EXP_TARGET` "ExposureTargetBrightness"  
*Float. Target brightness for auto exposure.*
- `#define TY_EXP_TIME` "ExposureTime"  
*Float. Sets the exposure time.*
- `#define TY_HDR_EN` "HDREnable"  
*Boolean. Enables HDR mode.*
- `#define TY_HDR_CTRL` "HDRControl"  
*Bytearray. Controls HDR parameters.*
- `#define TY_TRIG_SEL` "TriggerSelector"  
*Enumeration. Selects the trigger type to configure.*
- `#define TY_TRIG_MODE` "TriggerMode"  
*Enumeration. Controls whether the selected trigger is active.*
- `#define TY_TRIG_SW` "TriggerSoftware"  
*Command. Generates a software trigger signal.*
- `#define TY_TRIG_SRC` "TriggerSource"  
*Enumeration. Specifies the internal signal or physical input line to use as the trigger source.*
- `#define TY_TRIG_ACT` "TriggerActivation"  
*Enumeration. Specifies the activation mode of the trigger.*
- `#define TY_TRIG_DELAY` "TriggerDelay"  
*Float. Specifies the delay in microseconds to apply after the trigger reception before activating it.*
- `#define TY_TRIG_OUT` "TriggerOutIO"  
*Boolean. Controls trigger output IO.*
- `#define TY_TRIG_DUR` "TriggerDurationUs"  
*Integer. Specifies the duration in microseconds of the trigger signal.*
- `#define TY_CMOS_SYNC` "CmosSync"  
*Boolean. Controls CMOS synchronization.*
- `#define TY_TIME_SYNC_ACK` "TimeSyncAck"  
*Boolean. Time synchronization acknowledge.*
- `#define TY_GAIN_SEL` "GainSelector"  
*Enumeration. Selects which gain to control.*
- `#define TY_GAIN` "Gain"  
*Float. Controls the selected gain as a factor.*
- `#define TY_WB_AUTO` "BalanceWhiteAuto"  
*Boolean. Balance White Auto is the 'automatic' counterpart of the manual white balance feature.*
- `#define TY_AF_AOI_X` "AutoFunctionAOIOffsetX"  
*Integer. Horizontal offset of the auto function AOI.*
- `#define TY_AF_AOI_Y` "AutoFunctionAOIOffsetY"  
*Integer. Vertical offset of the auto function AOI.*
- `#define TY_AF_AOI_W` "AutoFunctionAOIWidth"  
*Integer. Width of the auto function AOI.*
- `#define TY_AF_AOI_H` "AutoFunctionAOIHeight"  
*Integer. Height of the auto function AOI.*
- `#define TY_SENSOR_W` "SensorWidth"

- Integer. Width of the camera sensor in pixels.*

  - `#define TY_SENSOR_H "SensorHeight"`
- Integer. Height of the camera sensor in pixels.*

  - `#define TY_PIX_FMT "PixelFormat"`
- Enumeration. Format of the pixels provided by the device.*

  - `#define TY_BIN_H "BinningHorizontal"`
- Enumeration. Number of horizontal pixels to combine together.*

  - `#define TY_BIN_V "BinningVertical"`
- Enumeration. Number of vertical pixels to combine together.*

  - `#define TY_COMP_EN "ComponentEnable"`
- Boolean. Enables the component.*

  - `#define TY_DEPTH_SCALE "DepthScaleUnit"`
- Float. Depth scale unit.*

  - `#define TY_DEPTH_SGBM_IMG_NUM "DepthSgbmImageNumber"`
- Integer. SGBM image number.*

  - `#define TY_DEPTH_SGBM_DISP_NUM "DepthSgbmDisparityNumber"`
- Integer. SGBM disparity number.*

  - `#define TY_DEPTH_SGBM_DISP_OFF "DepthSgbmDisparityOffset"`
- Integer. SGBM disparity offset.*

  - `#define TY_DEPTH_SGBM_WIN_H "DepthSgbmMatchWinHeight"`
- Integer. SGBM match window height.*

  - `#define TY_DEPTH_SGBM_P1 "DepthSgbmSemiParamP1"`
- Integer. SGBM semi global param p1.*

  - `#define TY_DEPTH_SGBM_P2 "DepthSgbmSemiParamP2"`
- Integer. SGBM semi global param p2.*

  - `#define TY_DEPTH_SGBM_UNIQUE "DepthSgbmUniqueFactor"`
- Integer. SGBM unique factor.*

  - `#define TY_DEPTH_SGBM_UNIQUE_DIFF "DepthSgbmUniqueAbsDiff"`
- Integer. SGBM unique absolute difference.*

  - `#define TY_DEPTH_SGBM_UNIQUE_COST "DepthSgbmUniqueMaxCost"`
- Integer. SGBM unique max cost.*

  - `#define TY_DEPTH_SGBM_H_WIN "DepthSgbmHFilterHalfWin"`
- Boolean. SGBM horizontal filter half window.*

  - `#define TY_DEPTH_SGBM_WIN_W "DepthSgbmMatchWinWidth"`
- Integer. SGBM match window width.*

  - `#define TY_DEPTH_SGBM_MED "DepthSgbmMedFilter"`
- Boolean. SGBM median filter.*

  - `#define TY_DEPTH_SGBM_LRC "DepthSgbmLRC"`
- Boolean. SGBM LRC.*

  - `#define TY_DEPTH_SGBM_LRC_DIFF "DepthSgbmLRCDiff"`
- Integer. SGBM LRC difference.*

  - `#define TY_DEPTH_SGBM_MED_THRES "DepthSgbmMedFilterThresh"`
- Integer. SGBM median filter threshold.*

  - `#define TY_DEPTH_SGBM_P1_SCALE "DepthSgbmSemiParamP1Scale"`
- Integer. SGBM semi param P1 scale.*

  - `#define TY_DEPTH_MIN "DepthRangeMin"`
- Integer. Minimum depth range.*

  - `#define TY_DEPTH_MAX "DepthRangeMax"`
- Integer. Maximum depth range.*

  - `#define TY_DEPTH_SGBM_TEX_OFF "DepthSgbmTextureFilterValueOffset"`
- Integer. Texture filter value offset.*

- `#define TY_DEPTH_SGBM_TEX_THRES` "DepthSgbmTextureFilterThreshold"  
*Integer. Texture filter threshold.*
- `#define TY_DEPTH_SGBM_TEX_EN` "DepthSgbmTextureFilterEnable"  
*Boolean. Texture filter enable.*
- `#define TY_DEPTH_SGBM_TEX_WIN` "DepthSgbmTextureFilterWindowSize"  
*Integer. Texture filter window size.*
- `#define TY_DEPTH_SGBM_TEX_MAX` "DepthSgbmTextureFilterMaxDistance"  
*Boolean. Texture filter maximum distance.*
- `#define TY_DEPTH_SGBM_SAT_EN` "DepthSgbmSaturateFilterEnable"  
*Boolean. Saturate filter enable.*
- `#define TY_DEPTH_SGBM_SAT_VAL` "DepthSgbmSaturateFilterVal"  
*Integer. Texture filter threshold.*
- `#define TY_DEPTH_SGBM_SAT_BLUR` "DepthSgbmSaturateFilterBlurSize"  
*Integer. Saturate filter blur size.*
- `#define TY_DEPTH_SGBM_SAT_DILATE` "DepthSgbmSaturateFilterDilateSize"  
*Integer. Saturate filter dilate size.*
- `#define TY_DEPTH_TOF_THRES` "DepthStreamTofFilterThreshold"  
*Integer. ToF filter threshold.*
- `#define TY_DEPTH_TOF_CH` "DepthStreamTofChannel"  
*Integer. ToF channel.*
- `#define TY_DEPTH_TOF_MOD_THRES` "DepthStreamTofModulationThreshold"  
*Integer. ToF modulation threshold.*
- `#define TY_DEPTH_TOF_QUALITY` "DepthStreamTofDepthQuality"  
*Enumeration. ToF depth quality.*
- `#define TY_DEPTH_TOF_HDR` "DepthStreamTofHdrRatio"  
*Integer. ToF HDR ratio.*
- `#define TY_DEPTH_TOF_JITTER` "DepthStreamTofJitterThreshold"  
*Integer. ToF jitter threshold.*
- `#define TY_TOF_MOD` "ToFModParam"  
*Integer. ToF modulation parameter.*
- `#define TY_TOF_FMOD0` "ToFFmodParam0"  
*Integer. ToF FMOD parameter 0.*
- `#define TY_TOF_FMOD1` "ToFFmodParam1"  
*Integer. ToF FMOD parameter 1.*
- `#define TY_TOF_DELAY0` "ToFLightDelayParam0"  
*Integer. ToF light delay parameter 0.*
- `#define TY_TOF_DELAY1` "ToFLightDelayParam1"  
*Integer. ToF light delay parameter 1.*
- `#define TY_TOF_DELAY2` "ToFLightDelayParam2"  
*Integer. ToF light delay parameter 2.*
- `#define TY_INTRIN_W` "IntrinsicWidth"  
*Integer. Intrinsic width.*
- `#define TY_INTRIN_H` "IntrinsicHeight"  
*Integer. Intrinsic height.*
- `#define TY_INTRIN` "Intrinsic"  
*Bytearray. Intrinsic parameters.*
- `#define TY_REC_INTRIN` "Intrinsic2"  
*Bytearray. Rectified Intrinsic parameters.*
- `#define TY_REC_ROT` "Rotation"  
*Bytearray. Rectified Rotation matrix.*
- `#define TY_DISTORT` "Distortion"

- Bytearray. Distortion parameters.*

  - `#define TY_EXTRIN "Extrinsic"`
- Bytearray. Extrinsic parameters to Depth image.*

  - `#define TY_PAYLOAD "PayloadSize"`
- Integer. Unit size of data payload in bytes.*

  - `#define TY_GEV_IF_SEL "GevInterfaceSelector"`
- Integer. Selects the network interface to control.*

  - `#define TY_GEV_MAC "GevMACAddress"`
- Integer. MAC address of the network interface.*

  - `#define TY_GEV_IP_CFG_LLA "GevCurrentIPConfigurationLLA"`
- Boolean. Controls whether the Link Local Address IP configuration scheme is activated.*

  - `#define TY_GEV_IP_CFG_DHCP "GevCurrentIPConfigurationDHCP"`
- Boolean. Controls whether the DHCP IP configuration scheme is activated.*

  - `#define TY_GEV_IP_CFG_PERSIST "GevCurrentIPConfigurationPersistentIP"`
- Boolean. Controls whether the Persistent IP configuration scheme is activated.*

  - `#define TY_GEV_IP "GevCurrentIPAddress"`
- Integer. Indicates the current IP address.*

  - `#define TY_GEV_SUBNET "GevCurrentSubnetMask"`
- Integer. Indicates the current subnet mask.*

  - `#define TY_GEV_GATEWAY "GevCurrentDefaultGateway"`
- Integer. Indicates the current default gateway.*

  - `#define TY_GEV_URL1 "GevFirstURL"`
- String. The first choice of URL for the XML device description file.*

  - `#define TY_GEV_URL2 "GevSecondURL"`
- String. The second choice of URL for the XML device description file.*

  - `#define TY_GEV_PERSIST_IP "GevPersistentIPAddress"`
- Integer. Indicates the persistent IP address.*

  - `#define TY_GEV_PERSIST_SUBNET "GevPersistentSubnetMask"`
- Integer. Indicates the persistent subnet mask.*

  - `#define TY_GEV_PERSIST_GATEWAY "GevPersistentDefaultGateway"`
- Integer. Indicates the persistent default gateway.*

  - `#define TY_GEV_GVCP_ACK "GevGVCPPendingAck"`
- Boolean. If enabled the device will issue a PENDING ACK in response to all GVCP commands.*

  - `#define TY_GEV_CCP "GevCCP"`
- Enumeration. Controls the device access privilege of an application.*

  - `#define TY_GEV_STREAM_CH_SEL "GevStreamChannelSelector"`
- Integer. Selects the stream channel to configure.*

  - `#define TY_GEV_SCP_IF_IDX "GevSCPInterfaceIndex"`
- Integer. Index of network interface to use.*

  - `#define TY_GEV_SCP_PORT "GevSCPHostPort"`
- Integer. Port to which the device must send data stream images.*

  - `#define TY_GEV_SCP_DIR "GevSCPDirection"`
- Integer. Indicates the direction of the data transfer.*

  - `#define TY_GEV_SCP_PKT_SIZE "GevSCPSPacketSize"`
- Integer. Indicates the packet size in bytes.*

  - `#define TY_GEV_SCPD "GevSCPD"`
- Integer. Indicates the delay in ticks inserted between each packet.*

  - `#define TY_GEV_SCPD "GevSCPD"`
- Integer. Indicates the destination IP address.*

  - `#define TY_GEV_SCPD "GevSCPD"`
- Integer. Indicates the destination port.*

  - `#define TY_GEV_SCSP "GevSCSP"`

- `#define TY_USER_SET_CUR "UserSetCurrent"`  
*Integer. Indicates the user set that is currently in use.*
- `#define TY_USER_SET_SEL "UserSetSelector"`  
*Enumeration. Selects the feature User Set to load, save or configure.*
- `#define TY_USER_SET_DESC "UserSetDescription"`  
*String. User set description.*
- `#define TY_USER_SET_LOAD "UserSetLoad"`  
*Command. Loads the User Set specified by UserSetSelector to the device.*
- `#define TY_USER_SET_SAVE "UserSetSave"`  
*Command. Save the User Set specified by UserSetSelector to the non-volatile memory of the device.*
- `#define TY_USER_SET_DEF "UserSetDefault"`  
*Enumeration. Indicates which User Set is used as default startup set.*
- `#define TY_SRC_COUNT "SourceCount"`  
*Integer. Controls or returns the number of sources supported by the device.*
- `#define TY_SRC_SEL "SourceSelector"`  
*Enumeration. Selects the source to control.*
- `#define TY_SRC_ID "SourceIDValue"`  
*Integer. Source ID value.*
- `#define TY_CAP_TIME_STAT "CaptureTimeStatistic"`  
*Integer. Capture time statistic.*
- `#define TY_IR_UNDIST "IRUndistortion"`  
*Boolean. IR undistortion.*
- `#define TY_POST_PROC0 "PostProcessParams0"`  
*Bytearray. Post process parameters 0.*
- `#define TY_POST_PROC1 "PostProcessParams1"`  
*Bytearray. Post process parameters 1.*
- `#define TY_LIGHT_SEL "LightControllerSelector"`  
*Enumeration. Selects the light to control.*
- `#define TY_LIGHT_EN "LightEnable"`  
*Boolean. Enable/Disable the Light.*
- `#define TY_LIGHT_BRIGHTNESS "LightBrightness"`  
*Integer. Config the Light Brightness.*

## Typedefs

- `typedef enum TY_DEV_TYPE_LIST TY_DEV_TYPE_LIST`
- `typedef enum TY_DEV_LINK_HB_MODE_LIST TY_DEV_LINK_HB_MODE_LIST`
- `typedef enum TY_DEV_STREAM_CH_TYPE_LIST TY_DEV_STREAM_CH_TYPE_LIST`
- `typedef enum TY_DEV_CHARSET_LIST TY_DEV_CHARSET_LIST`
- `typedef enum TY_DEV_TEMP_SEL_LIST TY_DEV_TEMP_SEL_LIST`
- `typedef enum TY_DEV_STREAM_ASYNC_LIST TY_DEV_STREAM_ASYNC_LIST`
- `typedef enum TY_DEV_TIME_SYNC_LIST TY_DEV_TIME_SYNC_LIST`
- `typedef enum TY_DEV_COMPRESS_LIST TY_DEV_COMPRESS_LIST`
- `typedef enum TY_ACQ_MODE_LIST TY_ACQ_MODE_LIST`
- `typedef enum TY_TRIG_SEL_LIST TY_TRIG_SEL_LIST`
- `typedef enum TY_TRIG_MODE_LIST TY_TRIG_MODE_LIST`
- `typedef enum TY_TRIG_SRC_LIST TY_TRIG_SRC_LIST`
- `typedef enum TY_TRIG_ACT_LIST TY_TRIG_ACT_LIST`
- `typedef enum TY_GAIN_SEL_LIST TY_GAIN_SEL_LIST`
- `typedef enum TY_PIX_FMT_LIST TY_PIX_FMT_LIST`
- `typedef enum TY_BIN_H_LIST TY_BIN_H_LIST`

- typedef enum [TY\\_BIN\\_V\\_LIST](#) [TY\\_BIN\\_V\\_LIST](#)
- typedef enum [TY\\_DEPTH\\_TOF\\_QUALITY\\_LIST](#) [TY\\_DEPTH\\_TOF\\_QUALITY\\_LIST](#)
- typedef enum [TY\\_GEV\\_CCP\\_LIST](#) [TY\\_GEV\\_CCP\\_LIST](#)
- typedef enum [TY\\_USER\\_SET\\_SEL\\_LIST](#) [TY\\_USER\\_SET\\_SEL\\_LIST](#)
- typedef enum [TY\\_SRC\\_SEL\\_LIST](#) [TY\\_SRC\\_SEL\\_LIST](#)
- typedef enum [TY\\_LIGHT\\_SEL\\_LIST](#) [TY\\_LIGTH\\_SEL\\_LIST](#)

## Enumerations

- enum [TY\\_DEV\\_TYPE\\_LIST](#) : uint32\_t { [DEV\\_TYPE\\_TRANSMITTER](#), [DEV\\_TYPE\\_RECEIVER](#), [DEV\\_TYPE\\_TRANSCEIVER](#), [DEV\\_TYPE\\_PERIPHERAL](#) }
- enum [TY\\_DEV\\_LINK\\_HB\\_MODE\\_LIST](#) : uint32\_t { [DEV\\_LINK\\_HB\\_ON](#), [DEV\\_LINK\\_HB\\_OFF](#) }
- enum [TY\\_DEV\\_STREAM\\_CH\\_TYPE\\_LIST](#) : uint32\_t { [DEV\\_STREAM\\_CH\\_TRANSMITTER](#), [DEV\\_STREAM\\_CH\\_RECEIVER](#) }
- enum [TY\\_DEV\\_CHARSET\\_LIST](#) : uint32\_t { [DEV\\_CHARSET\\_UTF8](#) = 1, [DEV\\_CHARSET\\_ASCII](#) }
- enum [TY\\_DEV\\_TEMP\\_SEL\\_LIST](#) : uint32\_t { [DEV\\_TEMP\\_SEL\\_MAINBOARD](#), [DEV\\_TEMP\\_SEL\\_LEFT](#), [DEV\\_TEMP\\_SEL\\_RIGHT](#), [DEV\\_TEMP\\_SEL\\_TEXTURE](#), [DEV\\_TEMP\\_SEL\\_LASER](#), [DEV\\_TEMP\\_SEL\\_IRFLASH](#), [DEV\\_TEMP\\_SEL\\_TEXTURE\\_FLASH](#), [DEV\\_TEMP\\_SEL\\_CPUCORE](#) = 8 }
- enum [TY\\_DEV\\_STREAM\\_ASYNC\\_LIST](#) : uint32\_t { [DEV\\_STREAM\\_ASYNC\\_OFF](#), [DEV\\_STREAM\\_ASYNC\\_DEPTH](#), [DEV\\_STREAM\\_ASYNC\\_RGB](#), [DEV\\_STREAM\\_ASYNC\\_DEPTH](#), [DEV\\_STREAM\\_ASYNC\\_ALL](#) = 255 }
- enum [TY\\_DEV\\_TIME\\_SYNC\\_LIST](#) : uint32\_t { [DEV\\_TIME\\_SYNC\\_NONE](#), [DEV\\_TIME\\_SYNC\\_HOST](#), [DEV\\_TIME\\_SYNC\\_NTP](#), [DEV\\_TIME\\_SYNC\\_PTP\\_SLAVE](#), [DEV\\_TIME\\_SYNC\\_CAN](#), [DEV\\_TIME\\_SYNC\\_PTP\\_MASTER](#) }
- enum [TY\\_DEV\\_COMPRESS\\_LIST](#) : uint32\_t { [DEV\\_COMPRESS\\_NONE](#) = 1, [DEV\\_COMPRESS\\_HW](#) }
- enum [TY\\_ACQ\\_MODE\\_LIST](#) : uint32\_t { [ACQ\\_MODE\\_SINGLE\\_FRAME](#), [ACQ\\_MODE\\_MULTI\\_FRAME](#), [ACQ\\_MODE\\_CONTINUOUS](#) }
- enum [TY\\_TRIG\\_SEL\\_LIST](#) : uint32\_t { [TRIG\\_SEL\\_FRAME\\_ACQUISITIONSTART](#), [TRIG\\_SEL\\_FRAME\\_ACQUISITIONEND](#), [TRIG\\_SEL\\_FRAME\\_ACQUISITIONAC](#), [TRIG\\_SEL\\_FRAME\\_FRAMESTART](#), [TRIG\\_SEL\\_FRAME\\_FRAMEEND](#), [TRIG\\_SEL\\_FRAME\\_FRAMEACTIVE](#), [TRIG\\_SEL\\_FRAME\\_FRAMEBURSTSTART](#), [TRIG\\_SEL\\_FRAME\\_FRAMEBURSTEND](#), [TRIG\\_SEL\\_FRAME\\_FRAMEBURSTACTIVE](#), [TRIG\\_SEL\\_FRAME\\_LINESTART](#), [TRIG\\_SEL\\_FRAME\\_EXPOSURESTART](#), [TRIG\\_SEL\\_FRAME\\_EXPOSUREEND](#), [TRIG\\_SEL\\_FRAME\\_EXPOSUREACTIVE](#) }
- enum [TY\\_TRIG\\_MODE\\_LIST](#) : uint32\_t { [TRIG\\_MODE\\_OFF](#), [TRIG\\_MODE\\_ON](#) }
- enum [TY\\_TRIG\\_SRC\\_LIST](#) : uint32\_t { [TRIG\\_SRC\\_LINE0](#), [TRIG\\_SRC\\_LINE1](#), [TRIG\\_SRC\\_LINE2](#), [TRIG\\_SRC\\_LINE3](#), [TRIG\\_SRC\\_LINE4](#), [TRIG\\_SRC\\_LINE5](#), [TRIG\\_SRC\\_LINE6](#), [TRIG\\_SRC\\_LINE7](#), [TRIG\\_SRC\\_SOFTWARE](#) }
- enum [TY\\_TRIG\\_ACT\\_LIST](#) : uint32\_t { [TRIG\\_ACT\\_RISING\\_EDGE](#), [TRIG\\_ACT\\_FALLING\\_EDGE](#), [TRIG\\_ACT\\_ANY\\_EDGE](#), [TRIG\\_ACT\\_LEVEL\\_HIGH](#), [TRIG\\_ACT\\_LEVEL\\_LOW](#) }
- enum [TY\\_GAIN\\_SEL\\_LIST](#) : uint32\_t { [GAIN\\_SEL\\_ANALOG\\_ALL](#), [GAIN\\_SEL\\_DIGITAL\\_ALL](#), [GAIN\\_SEL\\_DIGITALRED](#), [GAIN\\_SEL\\_DIGITALGREEN](#), [GAIN\\_SEL\\_DIGITALBLUE](#) }
- enum [TY\\_PIX\\_FMT\\_LIST](#) : uint32\_t { [CSIMono10P](#) = 0x810A0046, [CSIBayerGB10P](#) = 0x810A0054, [CSIBayerBG10P](#) = 0x810A0052, [CSIBayerGR10P](#) = 0x810A0056, [CSIBayerRG10P](#) = 0x810A0058, [CSIMono12P](#) = 0x810C0047, [CSIBayerGB12P](#) = 0x810C0055, [CSIBayerBG12P](#) = 0x810C0053, [CSIBayerGR12P](#) = 0x810C0057, [CSIBayerRG12P](#) = 0x810C0059, [CSIMono14P](#) = 0x810E0104, [CSIBayerGB14P](#) = 0x810E0107, [CSIBayerBG14P](#) = 0x810E0108, [CSIBayerGR14P](#) = 0x810E0105, [CSIBayerRG14P](#) = 0x810E0106, [JPEG](#) = 0x82180015, [TofIR\\_FourGroup\\_Mono16](#) = 0x81400016 }

- enum `TY_BIN_H_LIST` : `uint32_t` { `BIN_H1` = 1, `BIN_H2`, `BIN_H3`, `BIN_H4` }
- enum `TY_BIN_V_LIST` : `uint32_t` { `BIN_V1` = 1, `BIN_V2`, `BIN_V3`, `BIN_V4` }
- enum `TY_DEPTH_TOF_QUALITY_LIST` : `uint32_t` { `DEPTH_TOF_QUALITY_LOW` = 1, `DEPTH_TOF_QUALITY_MEDIUM` = 2, `DEPTH_TOF_QUALITY_HIGH` = 4 }
- enum `TY_GEV_CCP_LIST` : `uint32_t` { `GEV_CCP_OPEN`, `GEV_CCP_EXCLUSIVE`, `GEV_CCP_CONTROL` }
- enum `TY_USER_SET_SEL_LIST` : `uint32_t` { `USER_SET_SEL_DEFAULT0`, `USER_SET_SEL_DEFAULT1`, `USER_SET_SEL_DEFAULT2`, `USER_SET_SEL_DEFAULT3`, `USER_SET_SEL_DEFAULT4`, `USER_SET_SEL_DEFAULT5`, `USER_SET_SEL_DEFAULT6`, `USER_SET_SEL_DEFAULT7`, `USER_SET_SEL_USER_SET0`, `USER_SET_SEL_USER_SET1`, `USER_SET_SEL_USER_SET2`, `USER_SET_SEL_USER_SET3`, `USER_SET_SEL_USER_SET4`, `USER_SET_SEL_USER_SET5`, `USER_SET_SEL_USER_SET6`, `USER_SET_SEL_USER_SET7` }
- enum `TY_SRC_SEL_LIST` : `uint32_t` { `SRC_SEL_DEPTH`, `SRC_SEL_TEXTURE`, `SRC_SEL_LEFT`, `SRC_SEL_RIGHT` }
- enum `TY_LIGHT_SEL_LIST` : `uint32_t` { `LIGHT_SEL_LASER_0`, `LIGHT_SEL_FLOOD_0`, `LIGHT_SEL_TEXTURE_FLOOD` }

### 5.4.1 Detailed Description

Camera feature enumeration list.

This file contains all feature names and their detailed descriptions extracted from PercipioStdGenICam.xml Integrates XML file content from both PMD and SWIFT versions

### 5.4.2 Enumeration Type Documentation

#### 5.4.2.1 TY\_ACQ\_MODE\_LIST

```
enum TY_ACQ_MODE_LIST : uint32_t
```

Enumerator

|                                    |                                |
|------------------------------------|--------------------------------|
| <code>ACQ_MODE_SINGLE_FRAME</code> | Single frame acquisition mode. |
| <code>ACQ_MODE_MULTI_FRAME</code>  | Multi frame acquisition mode.  |
| <code>ACQ_MODE_CONTINUOUS</code>   | Continuous acquisition mode.   |

Definition at line 261 of file TYFeatureList.h.

#### 5.4.2.2 TY\_BIN\_H\_LIST

```
enum TY_BIN_H_LIST : uint32_t
```

**Enumerator**

|        |                              |
|--------|------------------------------|
| BIN_H1 | Horizontal binning factor 1. |
| BIN_H2 | Horizontal binning factor 2. |
| BIN_H3 | Horizontal binning factor 3. |
| BIN_H4 | Horizontal binning factor 4. |

Definition at line 544 of file TYFeatureList.h.

**5.4.2.3 TY\_BIN\_V\_LIST**

```
enum TY_BIN_V_LIST : uint32_t
```

**Enumerator**

|        |                            |
|--------|----------------------------|
| BIN_V1 | Vertical binning factor 1. |
| BIN_V2 | Vertical binning factor 2. |
| BIN_V3 | Vertical binning factor 3. |
| BIN_V4 | Vertical binning factor 4. |

Definition at line 567 of file TYFeatureList.h.

**5.4.2.4 TY\_DEPTH\_TOF\_QUALITY\_LIST**

```
enum TY_DEPTH_TOF_QUALITY_LIST : uint32_t
```

**Enumerator**

|                          |                       |
|--------------------------|-----------------------|
| DEPTH_TOF_QUALITY_LOW    | Low depth quality.    |
| DEPTH_TOF_QUALITY_MEDIUM | Medium depth quality. |
| DEPTH_TOF_QUALITY_HIGH   | High depth quality.   |

Definition at line 625 of file TYFeatureList.h.

**5.4.2.5 TY\_DEV\_CHARSET\_LIST**

```
enum TY_DEV_CHARSET_LIST : uint32_t
```

**Enumerator**

|                   |                                 |
|-------------------|---------------------------------|
| DEV_CHARSET_UTF8  | Device use UTF8 character set.  |
| DEV_CHARSET_ASCII | Device use ASCII character set. |
| CII               |                                 |



Definition at line 130 of file TYFeatureList.h.

#### 5.4.2.6 TY\_DEV\_COMPRESS\_LIST

```
enum TY_DEV_COMPRESS_LIST : uint32_t
```

##### Enumerator

|                   |                           |
|-------------------|---------------------------|
| DEV_COMPRESS_NONE | No compression.           |
| DEV_COMPRESS_HW   | LZ4 hardware compression. |

Definition at line 241 of file TYFeatureList.h.

#### 5.4.2.7 TY\_DEV\_LINK\_HB\_MODE\_LIST

```
enum TY_DEV_LINK_HB_MODE_LIST : uint32_t
```

##### Enumerator

|                 |                             |
|-----------------|-----------------------------|
| DEV_LINK_HB_ON  | Speed of the selected link. |
| DEV_LINK_HB_OFF | Speed of the selected link. |

Definition at line 85 of file TYFeatureList.h.

#### 5.4.2.8 TY\_DEV\_STREAM\_ASYNC\_LIST

```
enum TY_DEV_STREAM_ASYNC_LIST : uint32_t
```

##### Enumerator

|                              |                                                                             |
|------------------------------|-----------------------------------------------------------------------------|
| DEV_STREAM_ASYNC_OFF         | None of the data streams are output asynchronously.                         |
| DEV_STREAM_ASYNC_DEPTH       | The acquisition of range (distance) data is output asynchronously.          |
| DEV_STREAM_ASYNC_RGB         | The acquisition of intensity data is output asynchronously.                 |
| DEV_STREAM_ASYNC_DEPTHANDRGB | Intensity and range (distance) data are acquired and output asynchronously. |
| DEV_STREAM_ASYNC_ALL         | All of the data streams are output asynchronously.                          |

Definition at line 187 of file TYFeatureList.h.

#### 5.4.2.9 TY\_DEV\_STREAM\_CH\_TYPE\_LIST

```
enum TY_DEV_STREAM_CH_TYPE_LIST : uint32_t
```

##### Enumerator

|                           |                                  |
|---------------------------|----------------------------------|
| DEV_STREAM_CH_TRANSMITTER | Data stream transmitter channel. |
| DEV_STREAM_CH_RECEIVER    | Data stream receiver channel.    |

Definition at line 108 of file TYFeatureList.h.

#### 5.4.2.10 TY\_DEV\_TEMP\_SEL\_LIST

```
enum TY_DEV_TEMP_SEL_LIST : uint32_t
```

##### Enumerator

|                            |                                            |
|----------------------------|--------------------------------------------|
| DEV_TEMP_SEL_MAINBOARD     | Temperature of the mainboard.              |
| DEV_TEMP_SEL_LEFT          | Temperature of the left binocular module.  |
| DEV_TEMP_SEL_RIGHT         | Temperature of the right binocular module. |
| DEV_TEMP_SEL_TEXTURE       | Temperature of the texture module.         |
| DEV_TEMP_SEL_LASER         | Temperature of the laser.                  |
| DEV_TEMP_SEL_IRFLASH       | Temperature of the ir flash.               |
| DEV_TEMP_SEL_TEXTURE_FLASH | Temperature of the Texture flash.          |
| DEV_TEMP_SEL_CPUCORE       | Temperature of the CPU core.               |

Definition at line 153 of file TYFeatureList.h.

#### 5.4.2.11 TY\_DEV\_TIME\_SYNC\_LIST

```
enum TY_DEV_TIME_SYNC_LIST : uint32_t
```

##### Enumerator

|                          |                             |
|--------------------------|-----------------------------|
| DEV_TIME_SYNC_NONE       | No synchronization.         |
| DEV_TIME_SYNC_HOST       | Host synchronization.       |
| DEV_TIME_SYNC_NTP        | NTP synchronization.        |
| DEV_TIME_SYNC_PTP_SLAVE  | PTP slave synchronization.  |
| DEV_TIME_SYNC_CAN        | CAN synchronization.        |
| DEV_TIME_SYNC_PTP_MASTER | PTP master synchronization. |

Definition at line 212 of file TYFeatureList.h.

#### 5.4.2.12 TY\_DEV\_TYPE\_LIST

```
enum TY_DEV_TYPE_LIST : uint32_t
```

##### Enumerator

|                      |                                                   |
|----------------------|---------------------------------------------------|
| DEV_TYPE_TRANSMITTER | Data stream transmitter device.                   |
| DEV_TYPE_RECEIVER    | Data stream receiver device.                      |
| DEV_TYPE_TRANSCEIVER | Data stream receiver and transmitter device.      |
| DEV_TYPE_PERIPHERAL  | Controlable device (with no data stream handling) |

Definition at line 45 of file TYFeatureList.h.

#### 5.4.2.13 TY\_GAIN\_SEL\_LIST

```
enum TY_GAIN_SEL_LIST : uint32_t
```

##### Enumerator

|                       |                     |
|-----------------------|---------------------|
| GAIN_SEL_ANALOG_ALL   | All analog gains.   |
| GAIN_SEL_DIGITAL_ALL  | All digital gains.  |
| GAIN_SEL_DIGITALRED   | Digital red gain.   |
| GAIN_SEL_DIGITALGREEN | Digital green gain. |
| GAIN_SEL_DIGITALBLUE  | Digital blue gain.  |

Definition at line 423 of file TYFeatureList.h.

#### 5.4.2.14 TY\_GEV\_CCP\_LIST

```
enum TY_GEV_CCP_LIST : uint32_t
```

##### Enumerator

|                   |                   |
|-------------------|-------------------|
| GEV_CCP_OPEN      | Open Access.      |
| GEV_CCP_EXCLUSIVE | Exclusive Access. |
| GEV_CCP_CONTROL   | Control Access.   |

Definition at line 685 of file TYFeatureList.h.

#### 5.4.2.15 TY\_LIGHT\_SEL\_LIST

```
enum TY_LIGHT_SEL_LIST : uint32_t
```

## Enumerator

|                         |                                                             |
|-------------------------|-------------------------------------------------------------|
| LIGHT_SEL_LASER_0       | Laser controller will effects depth.                        |
| LIGHT_SEL_FLOOD_0       | Flood controller always effects Left/Right cam when needed. |
| LIGHT_SEL_TEXTURE_FLOOD | Flood controller effects Texture cam.                       |

Definition at line 813 of file TYFeatureList.h.

## 5.4.2.16 TY\_PIX\_FMT\_LIST

```
enum TY_PIX_FMT_LIST : uint32_t
```

## Enumerator

|                        |                                           |
|------------------------|-------------------------------------------|
| CSIMono10P             | Camera Serial Interface Packed Mono10.    |
| CSIBayerGB10P          | Camera Serial Interface Packed BayerGB10. |
| CSIBayerBG10P          | Camera Serial Interface Packed BayerBG10. |
| CSIBayerGR10P          | Camera Serial Interface Packed BayerGR10. |
| CSIBayerRG10P          | Camera Serial Interface Packed BayerRG10. |
| CSIMono12P             | Camera Serial Interface Packed Mono12.    |
| CSIBayerGB12P          | Camera Serial Interface Packed BayerGB12. |
| CSIBayerBG12P          | Camera Serial Interface Packed BayerBG12. |
| CSIBayerGR12P          | Camera Serial Interface Packed BayerGR12. |
| CSIBayerRG12P          | Camera Serial Interface Packed BayerRG12. |
| CSIMono14P             | Camera Serial Interface Packed Mono14.    |
| CSIBayerGB14P          | Camera Serial Interface Packed BayerGB14. |
| CSIBayerBG14P          | Camera Serial Interface Packed BayerBG14. |
| CSIBayerGR14P          | Camera Serial Interface Packed BayerGR14. |
| CSIBayerRG14P          | Camera Serial Interface Packed BayerRG14. |
| JPEG                   | JPEG encoding format.                     |
| TofIR_FourGroup_Mono16 | TofIR_FourGroup_Mono16 encoding format.   |

Definition at line 495 of file TYFeatureList.h.

## 5.4.2.17 TY\_SRC\_SEL\_LIST

```
enum TY_SRC_SEL_LIST : uint32_t
```

## Enumerator

|                 |                         |
|-----------------|-------------------------|
| SRC_SEL_DEPTH   | Depth source.           |
| SRC_SEL_TEXTURE | Texture source.         |
| SRC_SEL_LEFT    | Left binocular source.  |
| SRC_SEL_RIGHT   | Right binocular source. |

Definition at line 779 of file TYFeatureList.h.

#### 5.4.2.18 TY\_TRIG\_ACT\_LIST

```
enum TY_TRIG_ACT_LIST : uint32_t
```

##### Enumerator

|                       |                                  |
|-----------------------|----------------------------------|
| TRIG_ACT_RISING_EDGE  | Rising edge trigger activation.  |
| TRIG_ACT_FALLING_EDGE | Falling edge trigger activation. |
| TRIG_ACT_ANY_EDGE     | Any edge trigger activation.     |
| TRIG_ACT_LEVEL_HIGH   | High level trigger activation.   |
| TRIG_ACT_LEVEL_LOW    | Low level trigger activation.    |

Definition at line 391 of file TYFeatureList.h.

#### 5.4.2.19 TY\_TRIG\_MODE\_LIST

```
enum TY_TRIG_MODE_LIST : uint32_t
```

##### Enumerator

|               |                                |
|---------------|--------------------------------|
| TRIG_MODE_OFF | Disables the selected trigger. |
| TRIG_MODE_ON  | Enable the selected trigger.   |

Definition at line 337 of file TYFeatureList.h.

#### 5.4.2.20 TY\_TRIG\_SEL\_LIST

```
enum TY_TRIG_SEL_LIST : uint32_t
```

##### Enumerator

|                                  |                                 |
|----------------------------------|---------------------------------|
| TRIG_SEL_FRAME_ACQUISITIONSTART  | Trigger for acquisition start.  |
| TRIG_SEL_FRAME_ACQUISITIONEND    | Trigger for acquisition end.    |
| TRIG_SEL_FRAME_ACQUISITIONACTIVE | Trigger for acquisition active. |
| TRIG_SEL_FRAME_FRAMESTART        | Trigger for frame start.        |
| TRIG_SEL_FRAME_FRAMEEND          | Trigger for frame end.          |
| TRIG_SEL_FRAME_FRAMEACTIVE       | Trigger for frame active.       |
| TRIG_SEL_FRAME_FRAMEBURSTSTART   | Trigger for frame burst start.  |
| TRIG_SEL_FRAME_FRAMEBURSTEND     | Trigger for frame burst end.    |

## Enumerator

|                                 |                                 |
|---------------------------------|---------------------------------|
| TRIG_SEL_FRAME_FRAMEBURSTACTIVE | Trigger for frame burst active. |
| TRIG_SEL_FRAME_LINESTART        | Trigger for line start.         |
| TRIG_SEL_FRAME_EXPOSURESTART    | Trigger for exposure start.     |
| TRIG_SEL_FRAME_EXPOSUREEND      | Trigger for exposure end.       |
| TRIG_SEL_FRAME_EXPOSUREACTIVE   | Trigger for exposure active.    |

Definition at line 296 of file TYFeatureList.h.

#### 5.4.2.21 TY\_TRIG\_SRC\_LIST

```
enum TY_TRIG_SRC_LIST : uint32_t
```

## Enumerator

|                   |                          |
|-------------------|--------------------------|
| TRIG_SRC_LINE0    | Trigger source line 0.   |
| TRIG_SRC_LINE1    | Trigger source line 1.   |
| TRIG_SRC_LINE2    | Trigger source line 2.   |
| TRIG_SRC_LINE3    | Trigger source line 3.   |
| TRIG_SRC_LINE4    | Trigger source line 4.   |
| TRIG_SRC_LINE5    | Trigger source line 5.   |
| TRIG_SRC_LINE6    | Trigger source line 6.   |
| TRIG_SRC_LINE7    | Trigger source line 7.   |
| TRIG_SRC_SOFTWARE | Software trigger source. |

Definition at line 358 of file TYFeatureList.h.

#### 5.4.2.22 TY\_USER\_SET\_SEL\_LIST

```
enum TY_USER_SET_SEL_LIST : uint32_t
```

## Enumerator

|                        |                     |
|------------------------|---------------------|
| USER_SET_SEL_DEFAULT0  | Default user set 0. |
| USER_SET_SEL_DEFAULT1  | Default user set 1. |
| USER_SET_SEL_DEFAULT2  | Default user set 2. |
| USER_SET_SEL_DEFAULT3  | Default user set 3. |
| USER_SET_SEL_DEFAULT4  | Default user set 4. |
| USER_SET_SEL_DEFAULT5  | Default user set 5. |
| USER_SET_SEL_DEFAULT6  | Default user set 6. |
| USER_SET_SEL_DEFAULT7  | Default user set 7. |
| USER_SET_SEL_USER_SET0 | User set 0.         |
| USER_SET_SEL_USER_SET1 | User set 1.         |

## Enumerator

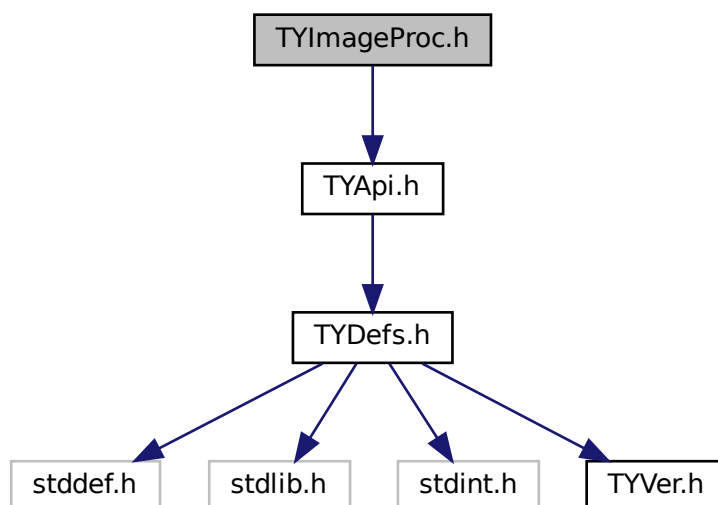
|                        |             |
|------------------------|-------------|
| USER_SET_SEL_USER_SET2 | User set 2. |
| USER_SET_SEL_USER_SET3 | User set 3. |
| USER_SET_SEL_USER_SET4 | User set 4. |
| USER_SET_SEL_USER_SET5 | User set 5. |
| USER_SET_SEL_USER_SET6 | User set 6. |
| USER_SET_SEL_USER_SET7 | User set 7. |

Definition at line 720 of file TYFeatureList.h.

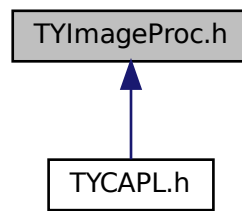
## 5.5 TYImageProc.h File Reference

```
#include "TYApi.h"
```

Include dependency graph for TYImageProc.h:



This graph shows which files directly or indirectly include this file:



## Classes

- struct [DepthSpeckleFilterParameters](#)
- struct [DepthInpainterParameters](#)
- struct [DepthEnhenceParameters](#)

## Macros

- #define **DepthSpeckleFilterParameters\_Initializer** {150, 64, 20}
- #define **DepthInpainterParameters\_Initializer** {5, 50}
- #define **DepthEnhenceParameters\_Initializer** {10, 20, 10, 0.1f}

## Functions

- TY\_CAPI [TYGetImageAlgorithmVersion](#) (TY\_VERSION\_INFO \*version)  
*Get current library version.*
- TY\_CAPI [TYImageProcesAcceEnable](#) (bool en)  
*Image processing acceleration switch.*
- TY\_CAPI [TYUndistortImage](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*srcCalibInfo, const [TY\\_IMAGE\\_DATA](#) \*srcImage, const [TY\\_CAMERA\\_INTRINSIC](#) \*cameraNewIntrinsic, [TY\\_IMAGE\\_DATA](#) \*dstImage, const TYLensOpticalType type=TY\_LENS\_PINHOLE)  
*Do image undistortion, only support TY\_PIXEL\_FORMAT\_MONO, TY\_PIXEL\_FORMAT\_RGB, TY\_PIXEL\_FORMAT\_RGBA, TY\_PIXEL\_FORMAT\_BGR.*
- TY\_CAPI [TYUndistortImage2](#) (const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*calib\_info, const [TY\\_IMAGE\\_DATA](#) \*srcImage, const [TY\\_CAMERA\\_ROTATION](#) \*cameraRotation, const [TY\\_CAMERA\\_INTRINSIC](#) \*cameraNewIntrinsic, [TY\\_IMAGE\\_DATA](#) \*dstImage, const TYLensOpticalType type=TY\_LENS\_PINHOLE)  
*Do image undistortion, only support TY\_PIXEL\_FORMAT\_MONO, TY\_PIXEL\_FORMAT\_RGB, TY\_PIXEL\_FORMAT\_RGBA, TY\_PIXEL\_FORMAT\_BGR.*
- TY\_CAPI [TYDepthSpeckleFilter](#) ([TY\\_IMAGE\\_DATA](#) \*depthImage, const [DepthSpeckleFilterParameters](#) \*param, const [TY\\_CAMERA\\_CALIB\\_INFO](#) \*calib\_data=NULLPTR, const float depth\_scale\_unit=1.f)  
*Remove speckles on depth image.*
- TY\_CAPI [TYDepthImageInpainter](#) ([TY\\_IMAGE\\_DATA](#) \*depth, const [DepthInpainterParameters](#) \*param)  
*Repair invalid pixels in depth image, filling small holes.*
- TY\_CAPI [TYDepthEnhenceFilter](#) (const [TY\\_IMAGE\\_DATA](#) \*depthImages, int imageNum, [TY\\_IMAGE\\_DATA](#) \*guide, [TY\\_IMAGE\\_DATA](#) \*output, const [DepthEnhenceParameters](#) \*param)  
*Remove speckles on depth image.*



### 5.5.1 Detailed Description

@breif Image post-process API

Copyright

Copyright(C)2016-2018 Percipio All Rights Reserved

### 5.5.2 Function Documentation

#### 5.5.2.1 TYDepthEnhenceFilter()

```
TY_CAPI TYDepthEnhenceFilter (
 const TY_IMAGE_DATA * depthImages,
 int imageNum,
 TY_IMAGE_DATA * guide,
 TY_IMAGE_DATA * output,
 const DepthEnhenceParameters * param)
```

Remove speckles on depth image.

Parameters

|         |                   |                               |
|---------|-------------------|-------------------------------|
| in      | <i>depthImage</i> | Pointer to depth image array. |
| in      | <i>imageNum</i>   | Depth image array size.       |
| in, out | <i>guide</i>      | Guide image.                  |
| out     | <i>output</i>     | Output depth image.           |
| in      | <i>param</i>      | Algorithm parameters.         |

Return values

|                                    |                                                          |
|------------------------------------|----------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                 |
| <i>TY_STATUS_NULL_POINTER</i>      | Any depthImage, param, output or output->buffer is NULL. |
| <i>TY_STATUS_INVALID_PARAMETER</i> | imageNum >= 11 or imageNum <= 0, or any image invalid    |
| <i>TY_STATUS_OUT_OF_MEMORY</i>     | Output image not suitable.                               |

#### 5.5.2.2 TYDepthImageInpainter()

```
TY_CAPI TYDepthImageInpainter (
 TY_IMAGE_DATA * depth,
 const DepthInpainterParameters * param)
```

Repair invalid pixels in depth image, filling small holes.

## Parameters

|         |              |                              |
|---------|--------------|------------------------------|
| in, out | <i>depth</i> | Depth image to be processed. |
| in      | <i>param</i> | Algorithm parameters.        |

## Return values

|                                    |                                                       |
|------------------------------------|-------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Success.                                              |
| <i>TY_STATUS_NULL_POINTER</i>      | depth, param or depth->buffer is NULL.                |
| <i>TY_STATUS_INVALID_PARAMETER</i> | param->kernel_size or param->kernel_size out of range |

## 5.5.2.3 TYDepthSpeckleFilter()

```

TY_CAPI TYDepthSpeckleFilter (
 TY_IMAGE_DATA * depthImage,
 const DepthSpeckleFilterParameters * param,
 const TY_CAMERA_CALIB_INFO * calib_data = nullptr,
 const float depth_scale_unit = 1.f)

```

Remove speckles on depth image.

## Parameters

|         |                   |                              |
|---------|-------------------|------------------------------|
| in, out | <i>depthImage</i> | Depth image to be processed. |
| in      | <i>param</i>      | Algorithm parameters.        |
| in      | <i>calib_data</i> | Image calibration data.      |

## Return values

|                                    |                                                              |
|------------------------------------|--------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                     |
| <i>TY_STATUS_NULL_POINTER</i>      | Any depth, param or depth->buffer is NULL.                   |
| <i>TY_STATUS_INVALID_PARAMETER</i> | param->max_speckle_size <= 0 or param->max_speckle_diff <= 0 |

## 5.5.2.4 TYGetImageAlgorithmVersion()

```

TY_CAPI TYGetImageAlgorithmVersion (
 TY_VERSION_INFO * version)

```

Get current library version.

## Parameters

|     |                |                                   |
|-----|----------------|-----------------------------------|
| out | <i>version</i> | Version information to be filled. |
|-----|----------------|-----------------------------------|

## Return values

|                               |                                                     |
|-------------------------------|-----------------------------------------------------|
| <i>TY_STATUS_OK</i>           | Succeed.                                            |
| <i>TY_STATUS_NULL_POINTER</i> | TYGetImageAlgorithmVersion called with NULL pointer |

## 5.5.2.5 TYImageProcesAcceEnable()

```
TY_CAPI TYImageProcesAcceEnable (
 bool en)
```

Image processing acceleration switch.

## Parameters

|    |           |                                          |
|----|-----------|------------------------------------------|
| in | <i>en</i> | Enable image process acceleration switch |
|----|-----------|------------------------------------------|

## 5.5.2.6 TYUndistortImage()

```
TY_CAPI TYUndistortImage (
 const TY_CAMERA_CALIB_INFO * srcCalibInfo,
 const TY_IMAGE_DATA * srcImage,
 const TY_CAMERA_INTRINSIC * cameraNewIntrinsic,
 TY_IMAGE_DATA * dstImage,
 const TYLensOpticalType type = TY_LENS_PINHOLE)
```

Do image undistortion, only support TY\_PIXEL\_FORMAT\_MONO ,TY\_PIXEL\_FORMAT\_RGB,TY\_PIXEL\_FORMAT\_BGR.

## Parameters

|     |                           |                                                                                             |
|-----|---------------------------|---------------------------------------------------------------------------------------------|
| in  | <i>srcCalibInfo</i>       | Image calibration data.                                                                     |
| in  | <i>srcImage</i>           | Source image.                                                                               |
| in  | <i>cameraNewIntrinsic</i> | Expected new image intrinsic, will use srcCalibInfo for new image intrinsic if set to NULL. |
| out | <i>dstImage</i>           | Output image.                                                                               |

## Return values

|                                    |                                                                                                           |
|------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                  |
| <i>TY_STATUS_NULL_POINTER</i>      | Any srcCalibInfo, srcImage, dstImage, srcImage->buffer, dstImage->buffer is NULL.                         |
| <i>TY_STATUS_INVALID_PARAMETER</i> | Invalid srcImage->width, srcImage->height, dstImage->width, dstImage->height or unsupported pixel format. |

### 5.5.2.7 TYUndistortImage2()

```

TY_CAPI TYUndistortImage2 (
 const TY_CAMERA_CALIB_INFO * calib_info,
 const TY_IMAGE_DATA * srcImage,
 const TY_CAMERA_ROTATION * cameraRotation,
 const TY_CAMERA_INTRINSIC * cameraNewIntrinsic,
 TY_IMAGE_DATA * dstImage,
 const TYLensOpticalType type = TY_LENS_PINHOLE)

```

Do image undistortion, only support TY\_PIXEL\_FORMAT\_MONO ,TY\_PIXEL\_FORMAT\_RGB,TY\_PIXEL\_FORMAT\_BGR.

#### Parameters

|     |                           |                                                                                             |
|-----|---------------------------|---------------------------------------------------------------------------------------------|
| in  | <i>srcCalibInfo</i>       | Image calibration data.                                                                     |
| in  | <i>srcImage</i>           | Source image.                                                                               |
| in  | <i>cameraRotation</i>     | Camera rotation parameter for image orientation adjustment.                                 |
| in  | <i>cameraNewIntrinsic</i> | Expected new image intrinsic, will use srcCalibInfo for new image intrinsic if set to NULL. |
| out | <i>dstImage</i>           | Output image.                                                                               |

#### Return values

|                                    |                                                                                                           |
|------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <i>TY_STATUS_OK</i>                | Succeed.                                                                                                  |
| <i>TY_STATUS_NULL_POINTER</i>      | Any srcCalibInfo, srcImage, dstImage, srcImage->buffer, dstImage->buffer is NULL.                         |
| <i>TY_STATUS_INVALID_PARAMETER</i> | Invalid srcImage->width, srcImage->height, dstImage->width, dstImage->height or unsupported pixel format. |

# Index

ACQ\_MODE\_CONTINUOUS  
    TYFeatureList.h, [211](#)  
ACQ\_MODE\_MULTI\_FRAME  
    TYFeatureList.h, [211](#)  
ACQ\_MODE\_SINGLE\_FRAME  
    TYFeatureList.h, [211](#)  
AlgVersion, [51](#)

BIN\_H1  
    TYFeatureList.h, [212](#)  
BIN\_H2  
    TYFeatureList.h, [212](#)  
BIN\_H3  
    TYFeatureList.h, [212](#)  
BIN\_H4  
    TYFeatureList.h, [212](#)  
BIN\_V1  
    TYFeatureList.h, [212](#)  
BIN\_V2  
    TYFeatureList.h, [212](#)  
BIN\_V3  
    TYFeatureList.h, [212](#)  
BIN\_V4  
    TYFeatureList.h, [212](#)

CSIBayerBG10P  
    TYFeatureList.h, [216](#)  
CSIBayerBG12P  
    TYFeatureList.h, [216](#)  
CSIBayerBG14P  
    TYFeatureList.h, [216](#)  
CSIBayerGB10P  
    TYFeatureList.h, [216](#)  
CSIBayerGB12P  
    TYFeatureList.h, [216](#)  
CSIBayerGB14P  
    TYFeatureList.h, [216](#)  
CSIBayerGR10P  
    TYFeatureList.h, [216](#)  
CSIBayerGR12P  
    TYFeatureList.h, [216](#)  
CSIBayerGR14P  
    TYFeatureList.h, [216](#)  
CSIBayerRG10P  
    TYFeatureList.h, [216](#)  
CSIBayerRG12P  
    TYFeatureList.h, [216](#)  
CSIBayerRG14P  
    TYFeatureList.h, [216](#)  
CSIMono10P

    TYFeatureList.h, [216](#)  
CSIMono12P  
    TYFeatureList.h, [216](#)  
CSIMono14P  
    TYFeatureList.h, [216](#)  
  
DEPTH\_TOF\_QUALITY\_HIGH  
    TYFeatureList.h, [212](#)  
DEPTH\_TOF\_QUALITY\_LOW  
    TYFeatureList.h, [212](#)  
DEPTH\_TOF\_QUALITY\_MEDIUM  
    TYFeatureList.h, [212](#)  
DepthEnhanceParameters, [51](#)  
DepthInpainterParameters, [52](#)  
DepthSpeckleFilterParameters, [52](#)  
DEV\_CHARSET\_ASCII  
    TYFeatureList.h, [212](#)  
DEV\_CHARSET\_UTF8  
    TYFeatureList.h, [212](#)  
DEV\_COMPRESS\_HW  
    TYFeatureList.h, [213](#)  
DEV\_COMPRESS\_NONE  
    TYFeatureList.h, [213](#)  
DEV\_LINK\_HB\_OFF  
    TYFeatureList.h, [213](#)  
DEV\_LINK\_HB\_ON  
    TYFeatureList.h, [213](#)  
DEV\_STREAM\_ASYNC\_ALL  
    TYFeatureList.h, [213](#)  
DEV\_STREAM\_ASYNC\_DEPTH  
    TYFeatureList.h, [213](#)  
DEV\_STREAM\_ASYNC\_DEPTHANDRGB  
    TYFeatureList.h, [213](#)  
DEV\_STREAM\_ASYNC\_OFF  
    TYFeatureList.h, [213](#)  
DEV\_STREAM\_ASYNC\_RGB  
    TYFeatureList.h, [213](#)  
DEV\_STREAM\_CH\_RECEIVER  
    TYFeatureList.h, [214](#)  
DEV\_STREAM\_CH\_TRANSMITTER  
    TYFeatureList.h, [214](#)  
DEV\_TEMP\_SEL\_CPUCORE  
    TYFeatureList.h, [214](#)  
DEV\_TEMP\_SEL\_IRFLASH  
    TYFeatureList.h, [214](#)  
DEV\_TEMP\_SEL\_LASER  
    TYFeatureList.h, [214](#)  
DEV\_TEMP\_SEL\_LEFT  
    TYFeatureList.h, [214](#)  
DEV\_TEMP\_SEL\_MAINBOARD

- TYFeatureList.h, [214](#)
- DEV\_TEMP\_SEL\_RIGHT
  - TYFeatureList.h, [214](#)
- DEV\_TEMP\_SEL\_TEXTURE
  - TYFeatureList.h, [214](#)
- DEV\_TEMP\_SEL\_TEXTURE\_FLASH
  - TYFeatureList.h, [214](#)
- DEV\_TIME\_SYNC\_CAN
  - TYFeatureList.h, [214](#)
- DEV\_TIME\_SYNC\_HOST
  - TYFeatureList.h, [214](#)
- DEV\_TIME\_SYNC\_NONE
  - TYFeatureList.h, [214](#)
- DEV\_TIME\_SYNC\_NTP
  - TYFeatureList.h, [214](#)
- DEV\_TIME\_SYNC\_PTP\_MASTER
  - TYFeatureList.h, [214](#)
- DEV\_TIME\_SYNC\_PTP\_SLAVE
  - TYFeatureList.h, [214](#)
- DEV\_TYPE\_PERIPHERAL
  - TYFeatureList.h, [215](#)
- DEV\_TYPE\_RECEIVER
  - TYFeatureList.h, [215](#)
- DEV\_TYPE\_TRANSCEIVER
  - TYFeatureList.h, [215](#)
- DEV\_TYPE\_TRANSMITTER
  - TYFeatureList.h, [215](#)
- f16, [52](#)
- GAIN\_SEL\_ANALOG\_ALL
  - TYFeatureList.h, [215](#)
- GAIN\_SEL\_DIGITAL\_ALL
  - TYFeatureList.h, [215](#)
- GAIN\_SEL\_DIGITALBLUE
  - TYFeatureList.h, [215](#)
- GAIN\_SEL\_DIGITALGREEN
  - TYFeatureList.h, [215](#)
- GAIN\_SEL\_DIGITALRED
  - TYFeatureList.h, [215](#)
- GEV\_CCP\_CONTROL
  - TYFeatureList.h, [215](#)
- GEV\_CCP\_EXCLUSIVE
  - TYFeatureList.h, [215](#)
- GEV\_CCP\_OPEN
  - TYFeatureList.h, [215](#)
- gz\_header\_s, [53](#)
- gzFile\_s, [53](#)
- JPEG
  - TYFeatureList.h, [216](#)
- LIGHT\_SEL\_FLOOD\_0
  - TYFeatureList.h, [216](#)
- LIGHT\_SEL\_LASER\_0
  - TYFeatureList.h, [216](#)
- LIGHT\_SEL\_TEXTURE\_FLOOD
  - TYFeatureList.h, [216](#)
- nsimd\_aarch64\_vf16, [54](#)
- nsimd\_avx2\_vf16, [54](#)
- nsimd\_avx2\_vlf16, [55](#)
- nsimd\_avx512\_knl\_vf16, [55](#)
- nsimd\_avx512\_knl\_vlf16, [55](#)
- nsimd\_avx512\_skylake\_vf16, [56](#)
- nsimd\_avx512\_skylake\_vlf16, [56](#)
- nsimd\_avx\_vf16, [56](#)
- nsimd\_avx\_vlf16, [57](#)
- nsimd\_cpu\_vf16, [57](#)
- nsimd\_cpu\_vf32, [57](#)
- nsimd\_cpu\_vf64, [58](#)
- nsimd\_cpu\_vi16, [58](#)
- nsimd\_cpu\_vi32, [59](#)
- nsimd\_cpu\_vi64, [59](#)
- nsimd\_cpu\_vi8, [59](#)
- nsimd\_cpu\_vlf16, [60](#)
- nsimd\_cpu\_vlf32, [60](#)
- nsimd\_cpu\_vlf64, [61](#)
- nsimd\_cpu\_vli16, [61](#)
- nsimd\_cpu\_vli32, [61](#)
- nsimd\_cpu\_vli64, [62](#)
- nsimd\_cpu\_vli8, [62](#)
- nsimd\_cpu\_vlu16, [63](#)
- nsimd\_cpu\_vlu32, [63](#)
- nsimd\_cpu\_vlu64, [64](#)
- nsimd\_cpu\_vlu8, [64](#)
- nsimd\_cpu\_vu16, [65](#)
- nsimd\_cpu\_vu32, [65](#)
- nsimd\_cpu\_vu64, [65](#)
- nsimd\_cpu\_vu8, [66](#)
- nsimd\_neon128\_vf16, [66](#)
- nsimd\_neon128\_vf64, [67](#)
- nsimd\_neon128\_vlf16, [67](#)
- nsimd\_neon128\_vlf64, [67](#)
- nsimd\_sse2\_vf16, [68](#)
- nsimd\_sse2\_vlf16, [68](#)
- nsimd\_sse42\_vf16, [68](#)
- nsimd\_sse42\_vlf16, [69](#)
- nsimd\_vmx\_vf16, [69](#)
- nsimd\_vmx\_vf64, [69](#)
- nsimd\_vmx\_vi64, [70](#)
- nsimd\_vmx\_vlf16, [70](#)
- nsimd\_vmx\_vlf64, [70](#)
- nsimd\_vmx\_vli64, [71](#)
- nsimd\_vmx\_vlu64, [71](#)
- nsimd\_vmx\_vu64, [71](#)
- nsimd\_vsx\_vf16, [72](#)
- nsimd\_vsx\_vlf16, [72](#)
- OperatorInfo, [73](#)
- ParamDetail, [73](#)
- pattern\_bin\_param, [74](#)
- pattern\_gray\_param, [74](#)
- pattern\_sine\_param, [74](#)
- SRC\_SEL\_DEPTH
  - TYFeatureList.h, [216](#)

- SRC\_SEL\_LEFT
  - TYFeatureList.h, [216](#)
- SRC\_SEL\_RIGHT
  - TYFeatureList.h, [216](#)
- SRC\_SEL\_TEXTURE
  - TYFeatureList.h, [216](#)
- TofIR\_FourGroup\_Mono16
  - TYFeatureList.h, [216](#)
- TRIG\_ACT\_ANY\_EDGE
  - TYFeatureList.h, [217](#)
- TRIG\_ACT\_FALLING\_EDGE
  - TYFeatureList.h, [217](#)
- TRIG\_ACT\_LEVEL\_HIGH
  - TYFeatureList.h, [217](#)
- TRIG\_ACT\_LEVEL\_LOW
  - TYFeatureList.h, [217](#)
- TRIG\_ACT\_RISING\_EDGE
  - TYFeatureList.h, [217](#)
- TRIG\_MODE\_OFF
  - TYFeatureList.h, [217](#)
- TRIG\_MODE\_ON
  - TYFeatureList.h, [217](#)
- TRIG\_SEL\_FRAME\_ACQUISITIONACTIVE
  - TYFeatureList.h, [217](#)
- TRIG\_SEL\_FRAME\_ACQUISITIONEND
  - TYFeatureList.h, [217](#)
- TRIG\_SEL\_FRAME\_ACQUISITIONSTART
  - TYFeatureList.h, [217](#)
- TRIG\_SEL\_FRAME\_EXPOSUREACTIVE
  - TYFeatureList.h, [218](#)
- TRIG\_SEL\_FRAME\_EXPOSUREEND
  - TYFeatureList.h, [218](#)
- TRIG\_SEL\_FRAME\_EXPOSURESTART
  - TYFeatureList.h, [218](#)
- TRIG\_SEL\_FRAME\_FRAMEACTIVE
  - TYFeatureList.h, [217](#)
- TRIG\_SEL\_FRAME\_FRAMEBURSTACTIVE
  - TYFeatureList.h, [218](#)
- TRIG\_SEL\_FRAME\_FRAMEBURSTEND
  - TYFeatureList.h, [217](#)
- TRIG\_SEL\_FRAME\_FRAMEBURSTSTART
  - TYFeatureList.h, [217](#)
- TRIG\_SEL\_FRAME\_FRAMEEND
  - TYFeatureList.h, [217](#)
- TRIG\_SEL\_FRAME\_FRAMESTART
  - TYFeatureList.h, [217](#)
- TRIG\_SEL\_FRAME\_LINESTART
  - TYFeatureList.h, [218](#)
- TRIG\_SRC\_LINE0
  - TYFeatureList.h, [218](#)
- TRIG\_SRC\_LINE1
  - TYFeatureList.h, [218](#)
- TRIG\_SRC\_LINE2
  - TYFeatureList.h, [218](#)
- TRIG\_SRC\_LINE3
  - TYFeatureList.h, [218](#)
- TRIG\_SRC\_LINE4
  - TYFeatureList.h, [218](#)
- TRIG\_SRC\_LINE5
  - TYFeatureList.h, [218](#)
- TRIG\_SRC\_LINE6
  - TYFeatureList.h, [218](#)
- TRIG\_SRC\_LINE7
  - TYFeatureList.h, [218](#)
- TRIG\_SRC\_SOFTWARE
  - TYFeatureList.h, [218](#)
- TY\_ACC\_BIAS, [75](#)
  - TYDefs.h, [190](#)
- TY\_ACC\_MISALIGNMENT, [76](#)
  - TYDefs.h, [190](#)
- TY\_ACC\_SCALE, [76](#)
  - TYDefs.h, [190](#)
- TY\_ACCESS\_MODE\_LIST
  - TYDefs.h, [190](#), [196](#)
- TY\_ACQ\_MODE\_LIST
  - TYFeatureList.h, [211](#)
- TY\_AEC\_ROI\_PARAM, [77](#)
- TY\_BIN\_H\_LIST
  - TYFeatureList.h, [211](#)
- TY\_BIN\_V\_LIST
  - TYFeatureList.h, [212](#)
- TY\_BOOL\_AUTO\_AWB
  - TYDefs.h, [198](#)
- TY\_BOOL\_AUTO\_EXPOSURE
  - TYDefs.h, [198](#)
- TY\_BOOL\_AUTO\_GAIN
  - TYDefs.h, [198](#)
- TY\_BOOL\_BRIGHTNESS\_HISTOGRAM
  - TYDefs.h, [198](#)
- TY\_BOOL\_CMOS\_SYNC
  - TYDefs.h, [198](#)
- TY\_BOOL\_DEPTH\_POSTPROC
  - TYDefs.h, [198](#)
- TY\_BOOL\_HDR
  - TYDefs.h, [199](#)
- TY\_BOOL\_IMU\_DATA\_ONOFF
  - TYDefs.h, [199](#)
- TY\_BOOL\_IR\_FLASHLIGHT
  - TYDefs.h, [198](#)
- TY\_BOOL\_KEEP\_ALIVE\_ONOFF
  - TYDefs.h, [198](#)
- TY\_BOOL\_LASER\_AUTO\_CTRL
  - TYDefs.h, [198](#)
- TY\_BOOL\_RGB\_FLASHLIGHT
  - TYDefs.h, [198](#)
- TY\_BOOL\_SGBM\_HFILTER\_HALF\_WIN
  - TYDefs.h, [199](#)
- TY\_BOOL\_SGBM\_LRC
  - TYDefs.h, [199](#)
- TY\_BOOL\_SGBM\_MEDFILTER
  - TYDefs.h, [199](#)
- TY\_BOOL\_SGPM\_EPI\_EN
  - TYDefs.h, [199](#)
- TY\_BOOL\_SGPM\_ORDER\_FILTER\_EN
  - TYDefs.h, [199](#)
- TY\_BOOL\_TIME\_SYNC\_READY

- TYDefs.h, [198](#)
- TY\_BOOL\_TOF\_ANTI\_INTERFERENCE
  - TYDefs.h, [200](#)
- TY\_BOOL\_TRIGGER\_OUT\_IO
  - TYDefs.h, [198](#)
- TY\_BOOL\_UNDISTORTION
  - TYDefs.h, [198](#)
- TY\_BYTEARRAY\_ATTR, [77](#)
  - TYDefs.h, [191](#)
  - unit\_size, [78](#)
  - valid\_size, [78](#)
- TY\_BYTEARRAY\_CUSTOM\_BLOCK
  - TYDefs.h, [197](#)
- TY\_BYTEARRAY\_HDR\_PARAMETER
  - TYDefs.h, [199](#)
- TY\_BYTEARRAY\_ISP\_BLOCK
  - TYDefs.h, [197](#)
- TY\_CAMERA\_CALIB\_INFO, [78](#)
  - TYDefs.h, [191](#)
- TY\_CAMERA\_DISTORTION, [79](#)
  - TYDefs.h, [191](#)
- TY\_CAMERA\_EXTRINSIC, [79](#)
  - TYDefs.h, [191](#)
- TY\_CAMERA\_INTRINSIC, [80](#)
  - TYDefs.h, [192](#)
- TY\_CAMERA\_ROTATION, [81](#)
  - TYDefs.h, [192](#)
- TY\_CAMERA\_STATISTICS, [81](#)
- TY\_CAMERA\_TO\_IMU, [82](#)
  - TYDefs.h, [192](#)
- TY\_COMPONENT\_BRIGHT\_HISTO
  - TYDefs.h, [197](#)
- TY\_COMPONENT\_DEPTH\_CAM
  - TYDefs.h, [197](#)
- TY\_COMPONENT\_DEVICE
  - TYDefs.h, [197](#)
- TY\_COMPONENT\_ID
  - TYDefs.h, [193](#)
- TY\_COMPONENT\_IMU
  - TYDefs.h, [197](#)
- TY\_COMPONENT\_IR\_CAM\_LEFT
  - TYDefs.h, [197](#)
- TY\_COMPONENT\_IR\_CAM\_RIGHT
  - TYDefs.h, [197](#)
- TY\_COMPONENT\_LASER
  - TYDefs.h, [197](#)
- TY\_COMPONENT\_RGB\_CAM
  - TYDefs.h, [197](#)
- TY\_COMPONENT\_RGB\_CAM\_LEFT
  - TYDefs.h, [197](#)
- TY\_COMPONENT\_RGB\_CAM\_RIGHT
  - TYDefs.h, [197](#)
- TY\_COMPONENT\_STORAGE
  - TYDefs.h, [197](#)
- TY\_DECLARE\_IMAGE\_MODE1
  - TYDefs.h, [189](#)
- TY\_DEPTH\_TOF\_QUALITY\_LIST
  - TYFeatureList.h, [212](#)
- TY\_DEV\_CHARSET\_LIST
  - TYFeatureList.h, [212](#)
- TY\_DEV\_COMPRESS\_LIST
  - TYFeatureList.h, [213](#)
- TY\_DEV\_LINK\_HB\_MODE\_LIST
  - TYFeatureList.h, [213](#)
- TY\_DEV\_STREAM\_ASYNC\_LIST
  - TYFeatureList.h, [213](#)
- TY\_DEV\_STREAM\_CH\_TYPE\_LIST
  - TYFeatureList.h, [213](#)
- TY\_DEV\_TEMP\_SEL\_LIST
  - TYFeatureList.h, [214](#)
- TY\_DEV\_TIME\_SYNC\_LIST
  - TYFeatureList.h, [214](#)
- TY\_DEV\_TYPE\_LIST
  - TYFeatureList.h, [214](#)
- TY\_DEVICE\_BASE\_INFO, [82](#)
  - TYDefs.h, [193](#)
- TY\_DEVICE\_COMPONENT\_LIST
  - TYDefs.h, [193](#), [196](#)
- TY\_DEVICE\_NET\_INFO, [83](#)
- TY\_DEVICE\_USB\_INFO, [84](#)
- TY\_DI\_WORKMODE, [84](#)
- TY\_DO\_WORKMODE, [85](#)
- TY\_ENUM\_DEPTH\_QUALITY
  - TYDefs.h, [200](#)
- TY\_ENUM\_ENTRY, [85](#)
  - TYDefs.h, [193](#)
- TY\_ENUM\_IMAGE\_MODE
  - TYDefs.h, [197](#)
- TY\_ENUM\_IMU\_FPS
  - TYDefs.h, [199](#)
- TY\_ENUM\_STREAM\_ASYNC
  - TYDefs.h, [198](#)
- TY\_ENUM\_TIME\_SYNC\_TYPE
  - TYDefs.h, [198](#)
- TY\_ENUM\_TRIGGER\_POL
  - TYDefs.h, [197](#)
- TY\_EVENT\_INFO, [86](#)
- TY\_FEATURE\_ID
  - TYDefs.h, [194](#)
- TY\_FEATURE\_ID\_LIST
  - TYDefs.h, [197](#)
- TY\_FEATURE\_INFO, [86](#)
- TY\_FLOAT\_EXPOSURE\_TIME\_US
  - TYDefs.h, [198](#)
- TY\_FLOAT\_RANGE, [87](#)
  - TYDefs.h, [194](#)
- TY\_FLOAT\_SCALE\_UNIT
  - TYDefs.h, [197](#)
- TY\_FLOAT\_SGPM\_EPI\_HS
  - TYDefs.h, [199](#)
- TY\_FRAME\_DATA, [87](#)
- TY\_GAIN\_SEL\_LIST
  - TYFeatureList.h, [215](#)
- TY\_GEV\_CCP\_LIST
  - TYFeatureList.h, [215](#)
- TY\_GYRO\_BIAS, [88](#)



- TYDefs.h, [194](#)
- TY\_GYRO\_MISALIGNMENT, [88](#)
  - TYDefs.h, [195](#)
- TY\_GYRO\_SCALE, [89](#)
  - TYDefs.h, [195](#)
- TY\_IMAGE\_DATA, [90](#)
- TY\_IMU\_DATA, [90](#)
- TY\_INT\_ANALOG\_GAIN
  - TYDefs.h, [198](#)
- TY\_INT\_B\_GAIN
  - TYDefs.h, [198](#)
- TY\_INT\_CAPTURE\_TIME\_US
  - TYDefs.h, [198](#)
- TY\_INT\_DEPTH\_MAX\_MM
  - TYDefs.h, [199](#)
- TY\_INT\_DEPTH\_MIN\_MM
  - TYDefs.h, [199](#)
- TY\_INT\_EXPOSURE\_TIME
  - TYDefs.h, [198](#)
- TY\_INT\_FILTER\_THRESHOLD
  - TYDefs.h, [200](#)
- TY\_INT\_FRAME\_PER\_TRIGGER
  - TYDefs.h, [197](#)
- TY\_INT\_G\_GAIN
  - TYDefs.h, [198](#)
- TY\_INT\_GAIN
  - TYDefs.h, [198](#)
- TY\_INT\_HEIGHT
  - TYDefs.h, [197](#)
- TY\_INT\_IR\_FLASHLIGHT\_INTENSITY
  - TYDefs.h, [198](#)
- TY\_INT\_KEEP\_ALIVE\_TIMEOUT
  - TYDefs.h, [198](#)
- TY\_INT\_LASER\_POWER
  - TYDefs.h, [198](#)
- TY\_INT\_LINK\_CMD\_TIMEOUT
  - TYDefs.h, [197](#)
- TY\_INT\_MAX\_SPECKLE\_DIFF
  - TYDefs.h, [200](#)
- TY\_INT\_MAX\_SPECKLE\_SIZE
  - TYDefs.h, [200](#)
- TY\_INT\_NTP\_SERVER\_IP
  - TYDefs.h, [197](#)
- TY\_INT\_PACKET\_DELAY
  - TYDefs.h, [197](#)
- TY\_INT\_R\_GAIN
  - TYDefs.h, [198](#)
- TY\_INT\_RANGE, [91](#)
- TY\_INT\_RGB\_FLASHLIGHT\_INTENSITY
  - TYDefs.h, [198](#)
- TY\_INT\_SGBM\_DISPARITY\_NUM
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_DISPARITY\_OFFSET
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_IMAGE\_NUM
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_LRC\_DIFF
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_MATCH\_WIN\_HEIGHT
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_MATCH\_WIN\_WIDTH
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_MEDFILTER\_THRESH
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_SEMI\_PARAM\_P1
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_SEMI\_PARAM\_P1\_SCALE
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_SEMI\_PARAM\_P2
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_TEXTURE\_OFFSET
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_TEXTURE\_THRESH
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_UNIQUE\_ABSDIFF
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_UNIQUE\_FACTOR
  - TYDefs.h, [199](#)
- TY\_INT\_SGBM\_UNIQUE\_MAX\_COST
  - TYDefs.h, [199](#)
- TY\_INT\_SGPM\_EPI\_CH0
  - TYDefs.h, [199](#)
- TY\_INT\_SGPM\_EPI\_CH1
  - TYDefs.h, [199](#)
- TY\_INT\_SGPM\_EPI\_HF
  - TYDefs.h, [199](#)
- TY\_INT\_SGPM\_EPI\_THRESH
  - TYDefs.h, [199](#)
- TY\_INT\_SGPM\_NORMAL\_PHASE\_OFFSET
  - TYDefs.h, [199](#)
- TY\_INT\_SGPM\_NORMAL\_PHASE\_SCALE
  - TYDefs.h, [199](#)
- TY\_INT\_SGPM\_ORDER\_FILTER\_CHN
  - TYDefs.h, [199](#)
- TY\_INT\_SGPM\_PHASE\_NUM
  - TYDefs.h, [199](#)
- TY\_INT\_SGPM\_REF\_PHASE\_OFFSET
  - TYDefs.h, [199](#)
- TY\_INT\_SGPM\_REF\_PHASE\_SCALE
  - TYDefs.h, [199](#)
- TY\_INT\_TOF\_ANTI\_SUNLIGHT\_INDEX
  - TYDefs.h, [200](#)
- TY\_INT\_TOF\_CHANNEL
  - TYDefs.h, [200](#)
- TY\_INT\_TOF\_HDR\_RATIO
  - TYDefs.h, [198](#)
- TY\_INT\_TOF\_JITTER\_THRESHOLD
  - TYDefs.h, [198](#)
- TY\_INT\_TOF\_MODULATION\_THRESHOLD
  - TYDefs.h, [200](#)
- TY\_INT\_TRIGGER\_DELAY\_US
  - TYDefs.h, [198](#)
- TY\_INT\_TRIGGER\_DURATION\_US
  - TYDefs.h, [198](#)
- TY\_INT\_WIDTH
  - TYDefs.h, [197](#)

TY\_INTERFACE\_INFO, [91](#)  
    TYDefs.h, [195](#)  
TY\_INTERFACE\_TYPE\_LIST  
    TYDefs.h, [195](#), [200](#)  
TY\_LASER\_PARAM, [92](#)  
TY\_LASER\_PATTERN\_PARAM, [92](#)  
TY\_LIGHT\_SEL\_LIST  
    TYFeatureList.h, [215](#)  
TY\_PHC\_GROUP\_ATTR, [93](#)  
TY\_PHC\_GROUP\_ATTR::phc\_group\_attr, [75](#)  
TY\_PIX\_FMT\_LIST  
    TYFeatureList.h, [216](#)  
TY\_PIXEL\_BITS\_LIST  
    TYDefs.h, [195](#), [200](#)  
TY\_PIXEL\_COLOR\_DESC, [94](#)  
TY\_PIXEL\_DESC, [94](#)  
TY\_PIXEL\_FORMAT\_BAYER8BG  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_BAYER8GB  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_BAYER8GR  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_BAYER8RG  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_BGR  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_BGR48  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_CSI\_BAYER10BGGR  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_CSI\_BAYER10GBRG  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_CSI\_BAYER10GRBG  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_CSI\_BAYER10RGGB  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_CSI\_BAYER12BGGR  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_CSI\_BAYER12GBRG  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_CSI\_BAYER12GRBG  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_CSI\_BAYER12RGGB  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_CSI\_MONO10  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_CSI\_MONO12  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_DEPTH16  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_JPEG  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_LIST  
    TYDefs.h, [200](#)  
TY\_PIXEL\_FORMAT\_MJPG  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_MONO  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_MONO16  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_RGB  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_RGB48  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_TOF\_IR\_MONO16  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_XYZ48  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_YUYV  
    TYDefs.h, [201](#)  
TY\_PIXEL\_FORMAT\_YVYU  
    TYDefs.h, [201](#)  
TY\_RESOLUTION\_MODE\_1280x720  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_1280x800  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_1280x960  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_1600x1200  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_160x100  
    TYDefs.h, [201](#)  
TY\_RESOLUTION\_MODE\_160x120  
    TYDefs.h, [201](#)  
TY\_RESOLUTION\_MODE\_1920x1080  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_1920x1440  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_2048x1536  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_240x320  
    TYDefs.h, [201](#)  
TY\_RESOLUTION\_MODE\_240x96  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_2560x1920  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_2592x1944  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_320x180  
    TYDefs.h, [201](#)  
TY\_RESOLUTION\_MODE\_320x200  
    TYDefs.h, [201](#)  
TY\_RESOLUTION\_MODE\_320x240  
    TYDefs.h, [201](#)  
TY\_RESOLUTION\_MODE\_480x640  
    TYDefs.h, [201](#)  
TY\_RESOLUTION\_MODE\_640x360  
    TYDefs.h, [201](#)  
TY\_RESOLUTION\_MODE\_640x400  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_640x480  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_800x600  
    TYDefs.h, [202](#)  
TY\_RESOLUTION\_MODE\_960x1280  
    TYDefs.h, [202](#)

- TY\_RESOLUTION\_MODE\_LIST
  - TYDefs.h, [201](#)
- TY\_SRC\_SEL\_LIST
  - TYFeatureList.h, [216](#)
- TY\_STRUCT\_AEC\_ROI
  - TYDefs.h, [198](#)
- TY\_STRUCT\_CAM\_CALIB\_DATA
  - TYDefs.h, [197](#)
- TY\_STRUCT\_CAM\_DISTORTION
  - TYDefs.h, [197](#)
- TY\_STRUCT\_CAM\_INTRINSIC
  - TYDefs.h, [197](#)
- TY\_STRUCT\_CAM\_RECTIFIED\_INTRI
  - TYDefs.h, [197](#)
- TY\_STRUCT\_CAM\_RECTIFIED\_ROTATION
  - TYDefs.h, [197](#)
- TY\_STRUCT\_CAM\_STATISTICS
  - TYDefs.h, [197](#)
- TY\_STRUCT\_DI0\_WORKMODE
  - TYDefs.h, [198](#)
- TY\_STRUCT\_DI1\_WORKMODE
  - TYDefs.h, [198](#)
- TY\_STRUCT\_DI2\_WORKMODE
  - TYDefs.h, [198](#)
- TY\_STRUCT\_DO0\_WORKMODE
  - TYDefs.h, [198](#)
- TY\_STRUCT\_DO1\_WORKMODE
  - TYDefs.h, [198](#)
- TY\_STRUCT\_DO2\_WORKMODE
  - TYDefs.h, [198](#)
- TY\_STRUCT\_EXTRINSIC\_TO\_DEPTH
  - TYDefs.h, [197](#)
- TY\_STRUCT\_EXTRINSIC\_TO\_IR\_LEFT
  - TYDefs.h, [197](#)
- TY\_STRUCT\_FLOOD\_ENABLE\_BY\_IDX
  - TYDefs.h, [198](#)
- TY\_STRUCT\_FLOOD\_POWER\_BY\_IDX
  - TYDefs.h, [198](#)
- TY\_STRUCT\_IMU\_ACC\_BIAS
  - TYDefs.h, [199](#)
- TY\_STRUCT\_IMU\_ACC\_MISALIGNMENT
  - TYDefs.h, [199](#)
- TY\_STRUCT\_IMU\_ACC\_SCALE
  - TYDefs.h, [199](#)
- TY\_STRUCT\_IMU\_CAM\_TO\_IMU
  - TYDefs.h, [199](#)
- TY\_STRUCT\_IMU\_GYRO\_BIAS
  - TYDefs.h, [199](#)
- TY\_STRUCT\_IMU\_GYRO\_MISALIGNMENT
  - TYDefs.h, [199](#)
- TY\_STRUCT\_IMU\_GYRO\_SCALE
  - TYDefs.h, [199](#)
- TY\_STRUCT\_LASER\_ENABLE\_BY\_IDX
  - TYDefs.h, [198](#)
- TY\_STRUCT\_LASER\_POWER\_BY\_IDX
  - TYDefs.h, [198](#)
- TY\_STRUCT\_PHC\_GROUP\_ATTR
  - TYDefs.h, [199](#)
- TY\_STRUCT\_TOF\_FREQ
  - TYDefs.h, [200](#)
- TY\_STRUCT\_TRIGGER\_PARAM
  - TYDefs.h, [198](#)
- TY\_STRUCT\_TRIGGER\_PARAM\_EX
  - TYDefs.h, [198](#)
- TY\_STRUCT\_TRIGGER\_TIMER\_LIST
  - TYDefs.h, [198](#)
- TY\_STRUCT\_TRIGGER\_TIMER\_PERIOD
  - TYDefs.h, [198](#)
- TY\_TEMP\_DATA, [95](#)
- TY\_TOF\_FREQ, [95](#)
- TY\_TRIG\_ACT\_LIST
  - TYFeatureList.h, [217](#)
- TY\_TRIG\_MODE\_LIST
  - TYFeatureList.h, [217](#)
- TY\_TRIG\_SEL\_LIST
  - TYFeatureList.h, [217](#)
- TY\_TRIG\_SRC\_LIST
  - TYFeatureList.h, [218](#)
- TY\_TRIGGER\_MODE\_LIST
  - TYDefs.h, [196](#), [202](#)
- TY\_TRIGGER\_MODE\_M\_PER
  - TYDefs.h, [202](#)
- TY\_TRIGGER\_MODE\_M\_SIG
  - TYDefs.h, [202](#)
- TY\_TRIGGER\_MODE\_OFF
  - TYDefs.h, [202](#)
- TY\_TRIGGER\_MODE\_PER\_PASS
  - TYDefs.h, [202](#)
- TY\_TRIGGER\_MODE\_PER\_PASS2
  - TYDefs.h, [202](#)
- TY\_TRIGGER\_MODE\_SIG\_PASS
  - TYDefs.h, [202](#)
- TY\_TRIGGER\_MODE\_SLAVE
  - TYDefs.h, [202](#)
- TY\_TRIGGER\_PARAM, [95](#)
- TY\_TRIGGER\_PARAM\_EX, [96](#)
- TY\_TRIGGER\_TIMER\_LIST, [96](#)
- TY\_TRIGGER\_TIMER\_PERIOD, [97](#)
- TY\_USER\_SET\_SEL\_LIST
  - TYFeatureList.h, [218](#)
- TY\_VECT\_3F, [97](#)
- TY\_VERSION\_INFO, [97](#)
- TYApi.h, [99](#)
  - TYCfgLogFile, [103](#)
  - TYCfgLogServer, [104](#)
  - TYClearBufferQueue, [105](#)
  - TYCloseDevice, [105](#)
  - TYCloseInterface, [106](#)
  - TYDeinitLib, [107](#)
  - TYDisableComponents, [107](#)
  - TYDisableLog, [108](#)
  - TYEnableComponents, [108](#)
  - TYEnableLog, [109](#)
  - TYEnqueueBuffer, [110](#)
  - TYErrorString, [111](#)
  - TYFetchFrame, [111](#)

- TYForceDeviceIP, [112](#)
- TYGetBool, [113](#)
- TYGetByteArray, [115](#)
- TYGetByteArrayAttr, [116](#)
- TYGetByteArraySize, [118](#)
- TYGetComponentIDs, [119](#)
- TYGetDeviceFeatureInfo, [120](#)
- TYGetDeviceFeatureNumber, [121](#)
- TYGetDeviceInfo, [122](#)
- TYGetDeviceInterface, [123](#)
- TYGetDeviceList, [124](#)
- TYGetDeviceNumber, [124](#)
- TYGetDeviceXML, [125](#)
- TYGetDeviceXMLSize, [126](#)
- TYGetEnabledComponents, [127](#)
- TYGetEnum, [128](#)
- TYGetEnumEntryCount, [129](#)
- TYGetEnumEntryInfo, [130](#)
- TYGetFeatureInfo, [131](#)
- TYGetFloat, [132](#)
- TYGetFloatRange, [134](#)
- TYGetFrameBufferSize, [135](#)
- TYGetInt, [136](#)
- TYGetInterfaceList, [137](#)
- TYGetInterfaceNumber, [138](#)
- TYGetIntRange, [139](#)
- TYGetString, [140](#)
- TYGetStringLength, [141](#)
- TYGetStruct, [143](#)
- TYHasDevice, [145](#)
- TYHasFeature, [146](#)
- TYHasInterface, [147](#)
- TYLibVersion, [147](#)
- TYOpenDevice, [148](#)
- TYOpenDeviceWithIP, [149](#)
- TYOpenInterface, [151](#)
- TYRegisterEventCallback, [151](#)
- TYRegisterImuCallback, [152](#)
- TYSendSoftTrigger, [153](#)
- TYSetBool, [154](#)
- TYSetByteArray, [155](#)
- TYSetEnum, [157](#)
- TYSetFloat, [158](#)
- TYSetInt, [160](#)
- TYSetLogLevel, [161](#)
- TYSetLogPrefix, [161](#)
- TYSetString, [162](#)
- TYSetStruct, [164](#)
- TYStartCapture, [165](#)
- TYStopCapture, [166](#)
- TYUpdateAllDeviceList, [167](#)
- TYUpdateDeviceList, [167](#)
- TYUpdateInterfaceList, [168](#)
- TYCfgLogFile
  - TYApi.h, [103](#)
- TYCfgLogServer
  - TYApi.h, [104](#)
- TYClearBufferQueue
  - TYApi.h, [105](#)
- TYCloseDevice
  - TYApi.h, [105](#)
- TYCloseInterface
  - TYApi.h, [106](#)
- TYCoordinateMapper.h, [168](#)
  - TYInvertExtrinsic, [170](#)
  - TYMapDepthImageToColorCoordinate, [171](#)
  - TYMapDepthImageToPoint3d, [172](#)
  - TYMapDepthToColorCoordinate, [172](#)
  - TYMapDepthToPoint3d, [173](#)
  - TYMapMono16ImageToDepthCoordinate, [173](#)
  - TYMapMono8ImageToDepthCoordinate, [174](#)
  - TYMapPoint3dToDepth, [175](#)
  - TYMapPoint3dToDepthImage, [175](#)
  - TYMapPoint3dToPoint3d, [176](#)
  - TYMapRGB48ImageToDepthCoordinate, [176](#)
  - TYMapRGBImageToDepthCoordinate, [177](#)
  - TYMapRGBPixelsToDepthCoordinate, [178](#)
- TYDefs.h, [179](#)
  - TY\_ACC\_BIAS, [190](#)
  - TY\_ACC\_MISALIGNMENT, [190](#)
  - TY\_ACC\_SCALE, [190](#)
  - TY\_ACCESS\_MODE\_LIST, [190](#), [196](#)
  - TY\_BOOL\_AUTO\_AWB, [198](#)
  - TY\_BOOL\_AUTO\_EXPOSURE, [198](#)
  - TY\_BOOL\_AUTO\_GAIN, [198](#)
  - TY\_BOOL\_BRIGHTNESS\_HISTOGRAM, [198](#)
  - TY\_BOOL\_CMOS\_SYNC, [198](#)
  - TY\_BOOL\_DEPTH\_POSTPROC, [198](#)
  - TY\_BOOL\_HDR, [199](#)
  - TY\_BOOL\_IMU\_DATA\_ONOFF, [199](#)
  - TY\_BOOL\_IR\_FLASHLIGHT, [198](#)
  - TY\_BOOL\_KEEP\_ALIVE\_ONOFF, [198](#)
  - TY\_BOOL\_LASER\_AUTO\_CTRL, [198](#)
  - TY\_BOOL\_RGB\_FLASHLIGHT, [198](#)
  - TY\_BOOL\_SGBM\_HFILTER\_HALF\_WIN, [199](#)
  - TY\_BOOL\_SGBM\_LRC, [199](#)
  - TY\_BOOL\_SGBM\_MEDFILTER, [199](#)
  - TY\_BOOL\_SGPM\_EPI\_EN, [199](#)
  - TY\_BOOL\_SGPM\_ORDER\_FILTER\_EN, [199](#)
  - TY\_BOOL\_TIME\_SYNC\_READY, [198](#)
  - TY\_BOOL\_TOF\_ANTI\_INTERFERENCE, [200](#)
  - TY\_BOOL\_TRIGGER\_OUT\_IO, [198](#)
  - TY\_BOOL\_UNDISTORTION, [198](#)
  - TY\_BYTEARRAY\_ATTR, [191](#)
  - TY\_BYTEARRAY\_CUSTOM\_BLOCK, [197](#)
  - TY\_BYTEARRAY\_HDR\_PARAMETER, [199](#)
  - TY\_BYTEARRAY\_ISP\_BLOCK, [197](#)
  - TY\_CAMERA\_CALIB\_INFO, [191](#)
  - TY\_CAMERA\_DISTORTION, [191](#)
  - TY\_CAMERA\_EXTRINSIC, [191](#)
  - TY\_CAMERA\_INTRINSIC, [192](#)
  - TY\_CAMERA\_ROTATION, [192](#)
  - TY\_CAMERA\_TO\_IMU, [192](#)
  - TY\_COMPONENT\_BRIGHT\_HISTO, [197](#)
  - TY\_COMPONENT\_DEPTH\_CAM, [197](#)
  - TY\_COMPONENT\_DEVICE, [197](#)

TY\_COMPONENT\_ID, 193  
TY\_COMPONENT\_IMU, 197  
TY\_COMPONENT\_IR\_CAM\_LEFT, 197  
TY\_COMPONENT\_IR\_CAM\_RIGHT, 197  
TY\_COMPONENT\_LASER, 197  
TY\_COMPONENT\_RGB\_CAM, 197  
TY\_COMPONENT\_RGB\_CAM\_LEFT, 197  
TY\_COMPONENT\_RGB\_CAM\_RIGHT, 197  
TY\_COMPONENT\_STORAGE, 197  
TY\_DECLARE\_IMAGE\_MODE1, 189  
TY\_DEVICE\_BASE\_INFO, 193  
TY\_DEVICE\_COMPONENT\_LIST, 193, 196  
TY\_ENUM\_DEPTH\_QUALITY, 200  
TY\_ENUM\_ENTRY, 193  
TY\_ENUM\_IMAGE\_MODE, 197  
TY\_ENUM\_IMU\_FPS, 199  
TY\_ENUM\_STREAM\_ASYNC, 198  
TY\_ENUM\_TIME\_SYNC\_TYPE, 198  
TY\_ENUM\_TRIGGER\_POL, 197  
TY\_FEATURE\_ID, 194  
TY\_FEATURE\_ID\_LIST, 197  
TY\_FLOAT\_EXPOSURE\_TIME\_US, 198  
TY\_FLOAT\_RANGE, 194  
TY\_FLOAT\_SCALE\_UNIT, 197  
TY\_FLOAT\_SGPM\_EPI\_HS, 199  
TY\_GYRO\_BIAS, 194  
TY\_GYRO\_MISALIGNMENT, 195  
TY\_GYRO\_SCALE, 195  
TY\_INT\_ANALOG\_GAIN, 198  
TY\_INT\_B\_GAIN, 198  
TY\_INT\_CAPTURE\_TIME\_US, 198  
TY\_INT\_DEPTH\_MAX\_MM, 199  
TY\_INT\_DEPTH\_MIN\_MM, 199  
TY\_INT\_EXPOSURE\_TIME, 198  
TY\_INT\_FILTER\_THRESHOLD, 200  
TY\_INT\_FRAME\_PER\_TRIGGER, 197  
TY\_INT\_G\_GAIN, 198  
TY\_INT\_GAIN, 198  
TY\_INT\_HEIGHT, 197  
TY\_INT\_IR\_FLASHLIGHT\_INTENSITY, 198  
TY\_INT\_KEEP\_ALIVE\_TIMEOUT, 198  
TY\_INT\_LASER\_POWER, 198  
TY\_INT\_LINK\_CMD\_TIMEOUT, 197  
TY\_INT\_MAX\_SPECKLE\_DIFF, 200  
TY\_INT\_MAX\_SPECKLE\_SIZE, 200  
TY\_INT\_NTP\_SERVER\_IP, 197  
TY\_INT\_PACKET\_DELAY, 197  
TY\_INT\_R\_GAIN, 198  
TY\_INT\_RGB\_FLASHLIGHT\_INTENSITY, 198  
TY\_INT\_SGBM\_DISPARITY\_NUM, 199  
TY\_INT\_SGBM\_DISPARITY\_OFFSET, 199  
TY\_INT\_SGBM\_IMAGE\_NUM, 199  
TY\_INT\_SGBM\_LRC\_DIFF, 199  
TY\_INT\_SGBM\_MATCH\_WIN\_HEIGHT, 199  
TY\_INT\_SGBM\_MATCH\_WIN\_WIDTH, 199  
TY\_INT\_SGBM\_MEDFILTER\_THRESH, 199  
TY\_INT\_SGBM\_SEMI\_PARAM\_P1, 199  
TY\_INT\_SGBM\_SEMI\_PARAM\_P1\_SCALE, 199  
TY\_INT\_SGBM\_SEMI\_PARAM\_P2, 199  
TY\_INT\_SGBM\_TEXTURE\_OFFSET, 199  
TY\_INT\_SGBM\_TEXTURE\_THRESH, 199  
TY\_INT\_SGBM\_UNIQUE\_ABSDIFF, 199  
TY\_INT\_SGBM\_UNIQUE\_FACTOR, 199  
TY\_INT\_SGBM\_UNIQUE\_MAX\_COST, 199  
TY\_INT\_SGPM\_EPI\_CH0, 199  
TY\_INT\_SGPM\_EPI\_CH1, 199  
TY\_INT\_SGPM\_EPI\_HF, 199  
TY\_INT\_SGPM\_EPI\_THRESH, 199  
TY\_INT\_SGPM\_NORMAL\_PHASE\_OFFSET, 199  
TY\_INT\_SGPM\_NORMAL\_PHASE\_SCALE, 199  
TY\_INT\_SGPM\_ORDER\_FILTER\_CHN, 199  
TY\_INT\_SGPM\_PHASE\_NUM, 199  
TY\_INT\_SGPM\_REF\_PHASE\_OFFSET, 199  
TY\_INT\_SGPM\_REF\_PHASE\_SCALE, 199  
TY\_INT\_TOF\_ANTI\_SUNLIGHT\_INDEX, 200  
TY\_INT\_TOF\_CHANNEL, 200  
TY\_INT\_TOF\_HDR\_RATIO, 198  
TY\_INT\_TOF\_JITTER\_THRESHOLD, 198  
TY\_INT\_TOF\_MODULATION\_THRESHOLD, 200  
TY\_INT\_TRIGGER\_DELAY\_US, 198  
TY\_INT\_TRIGGER\_DURATION\_US, 198  
TY\_INT\_WIDTH, 197  
TY\_INTERFACE\_INFO, 195  
TY\_INTERFACE\_TYPE\_LIST, 195, 200  
TY\_PIXEL\_BITS\_LIST, 195, 200  
TY\_PIXEL\_FORMAT\_BAYER8BG, 201  
TY\_PIXEL\_FORMAT\_BAYER8GB, 201  
TY\_PIXEL\_FORMAT\_BAYER8GR, 201  
TY\_PIXEL\_FORMAT\_BAYER8RG, 201  
TY\_PIXEL\_FORMAT\_BGR, 201  
TY\_PIXEL\_FORMAT\_BGR48, 201  
TY\_PIXEL\_FORMAT\_CSI\_BAYER10BGGR, 201  
TY\_PIXEL\_FORMAT\_CSI\_BAYER10GBRG, 201  
TY\_PIXEL\_FORMAT\_CSI\_BAYER10GRBG, 201  
TY\_PIXEL\_FORMAT\_CSI\_BAYER10RGGG, 201  
TY\_PIXEL\_FORMAT\_CSI\_BAYER12BGGR, 201  
TY\_PIXEL\_FORMAT\_CSI\_BAYER12GBRG, 201  
TY\_PIXEL\_FORMAT\_CSI\_BAYER12GRBG, 201  
TY\_PIXEL\_FORMAT\_CSI\_BAYER12RGGG, 201  
TY\_PIXEL\_FORMAT\_CSI\_MONO10, 201  
TY\_PIXEL\_FORMAT\_CSI\_MONO12, 201  
TY\_PIXEL\_FORMAT\_DEPTH16, 201  
TY\_PIXEL\_FORMAT\_JPEG, 201  
TY\_PIXEL\_FORMAT\_LIST, 200  
TY\_PIXEL\_FORMAT\_MJPEG, 201  
TY\_PIXEL\_FORMAT\_MONO, 201  
TY\_PIXEL\_FORMAT\_MONO16, 201  
TY\_PIXEL\_FORMAT\_RGB, 201  
TY\_PIXEL\_FORMAT\_RGB48, 201  
TY\_PIXEL\_FORMAT\_TOF\_IR\_MONO16, 201  
TY\_PIXEL\_FORMAT\_XYZ48, 201  
TY\_PIXEL\_FORMAT\_YUYV, 201  
TY\_PIXEL\_FORMAT\_YVYU, 201  
TY\_RESOLUTION\_MODE\_1280x720, 202  
TY\_RESOLUTION\_MODE\_1280x800, 202  
TY\_RESOLUTION\_MODE\_1280x960, 202

- TY\_RESOLUTION\_MODE\_1600x1200, [202](#)
- TY\_RESOLUTION\_MODE\_160x100, [201](#)
- TY\_RESOLUTION\_MODE\_160x120, [201](#)
- TY\_RESOLUTION\_MODE\_1920x1080, [202](#)
- TY\_RESOLUTION\_MODE\_1920x1440, [202](#)
- TY\_RESOLUTION\_MODE\_2048x1536, [202](#)
- TY\_RESOLUTION\_MODE\_240x320, [201](#)
- TY\_RESOLUTION\_MODE\_240x96, [202](#)
- TY\_RESOLUTION\_MODE\_2560x1920, [202](#)
- TY\_RESOLUTION\_MODE\_2592x1944, [202](#)
- TY\_RESOLUTION\_MODE\_320x180, [201](#)
- TY\_RESOLUTION\_MODE\_320x200, [201](#)
- TY\_RESOLUTION\_MODE\_320x240, [201](#)
- TY\_RESOLUTION\_MODE\_480x640, [201](#)
- TY\_RESOLUTION\_MODE\_640x360, [201](#)
- TY\_RESOLUTION\_MODE\_640x400, [202](#)
- TY\_RESOLUTION\_MODE\_640x480, [202](#)
- TY\_RESOLUTION\_MODE\_800x600, [202](#)
- TY\_RESOLUTION\_MODE\_960x1280, [202](#)
- TY\_RESOLUTION\_MODE\_LIST, [201](#)
- TY\_STRUCT\_AEC\_ROI, [198](#)
- TY\_STRUCT\_CAM\_CALIB\_DATA, [197](#)
- TY\_STRUCT\_CAM\_DISTORTION, [197](#)
- TY\_STRUCT\_CAM\_INTRINSIC, [197](#)
- TY\_STRUCT\_CAM\_RECTIFIED\_INTRI, [197](#)
- TY\_STRUCT\_CAM\_RECTIFIED\_ROTATION, [197](#)
- TY\_STRUCT\_CAM\_STATISTICS, [197](#)
- TY\_STRUCT\_DI0\_WORKMODE, [198](#)
- TY\_STRUCT\_DI1\_WORKMODE, [198](#)
- TY\_STRUCT\_DI2\_WORKMODE, [198](#)
- TY\_STRUCT\_DO0\_WORKMODE, [198](#)
- TY\_STRUCT\_DO1\_WORKMODE, [198](#)
- TY\_STRUCT\_DO2\_WORKMODE, [198](#)
- TY\_STRUCT\_EXTRINSIC\_TO\_DEPTH, [197](#)
- TY\_STRUCT\_EXTRINSIC\_TO\_IR\_LEFT, [197](#)
- TY\_STRUCT\_FLOOD\_ENABLE\_BY\_IDX, [198](#)
- TY\_STRUCT\_FLOOD\_POWER\_BY\_IDX, [198](#)
- TY\_STRUCT\_IMU\_ACC\_BIAS, [199](#)
- TY\_STRUCT\_IMU\_ACC\_MISALIGNMENT, [199](#)
- TY\_STRUCT\_IMU\_ACC\_SCALE, [199](#)
- TY\_STRUCT\_IMU\_CAM\_TO\_IMU, [199](#)
- TY\_STRUCT\_IMU\_GYRO\_BIAS, [199](#)
- TY\_STRUCT\_IMU\_GYRO\_MISALIGNMENT, [199](#)
- TY\_STRUCT\_IMU\_GYRO\_SCALE, [199](#)
- TY\_STRUCT\_LASER\_ENABLE\_BY\_IDX, [198](#)
- TY\_STRUCT\_LASER\_POWER\_BY\_IDX, [198](#)
- TY\_STRUCT\_PHC\_GROUP\_ATTR, [199](#)
- TY\_STRUCT\_TOF\_FREQ, [200](#)
- TY\_STRUCT\_TRIGGER\_PARAM, [198](#)
- TY\_STRUCT\_TRIGGER\_PARAM\_EX, [198](#)
- TY\_STRUCT\_TRIGGER\_TIMER\_LIST, [198](#)
- TY\_STRUCT\_TRIGGER\_TIMER\_PERIOD, [198](#)
- TY\_TRIGGER\_MODE\_LIST, [196](#), [202](#)
- TY\_TRIGGER\_MODE\_M\_PER, [202](#)
- TY\_TRIGGER\_MODE\_M\_SIG, [202](#)
- TY\_TRIGGER\_MODE\_OFF, [202](#)
- TY\_TRIGGER\_MODE\_PER\_PASS, [202](#)
- TY\_TRIGGER\_MODE\_PER\_PASS2, [202](#)
- TY\_TRIGGER\_MODE\_SIG\_PASS, [202](#)
- TY\_TRIGGER\_MODE\_SLAVE, [202](#)
- TYDeinitLib
  - TYApi.h, [107](#)
- TYDepthEnhanceFilter
  - TYImageProc.h, [221](#)
- TYDepthImageInpainter
  - TYImageProc.h, [221](#)
- TYDepthSpeckleFilter
  - TYImageProc.h, [222](#)
- TYDisableComponents
  - TYApi.h, [107](#)
- TYDisableLog
  - TYApi.h, [108](#)
- TYEnableComponents
  - TYApi.h, [108](#)
- TYEnableLog
  - TYApi.h, [109](#)
- TYEnqueueBuffer
  - TYApi.h, [110](#)
- TYEnumEntry, [98](#)
- TYErrorString
  - TYApi.h, [111](#)
- TYFeatureList.h, [202](#)
  - ACQ\_MODE\_CONTINUOUS, [211](#)
  - ACQ\_MODE\_MULTI\_FRAME, [211](#)
  - ACQ\_MODE\_SINGLE\_FRAME, [211](#)
  - BIN\_H1, [212](#)
  - BIN\_H2, [212](#)
  - BIN\_H3, [212](#)
  - BIN\_H4, [212](#)
  - BIN\_V1, [212](#)
  - BIN\_V2, [212](#)
  - BIN\_V3, [212](#)
  - BIN\_V4, [212](#)
  - CSIBayerBG10P, [216](#)
  - CSIBayerBG12P, [216](#)
  - CSIBayerBG14P, [216](#)
  - CSIBayerGB10P, [216](#)
  - CSIBayerGB12P, [216](#)
  - CSIBayerGB14P, [216](#)
  - CSIBayerGR10P, [216](#)
  - CSIBayerGR12P, [216](#)
  - CSIBayerGR14P, [216](#)
  - CSIBayerRG10P, [216](#)
  - CSIBayerRG12P, [216](#)
  - CSIBayerRG14P, [216](#)
  - CSIMono10P, [216](#)
  - CSIMono12P, [216](#)
  - CSIMono14P, [216](#)
  - DEPTH\_TOF\_QUALITY\_HIGH, [212](#)
  - DEPTH\_TOF\_QUALITY\_LOW, [212](#)
  - DEPTH\_TOF\_QUALITY\_MEDIUM, [212](#)
  - DEV\_CHARSET\_ASCII, [212](#)
  - DEV\_CHARSET\_UTF8, [212](#)
  - DEV\_COMPRESS\_HW, [213](#)
  - DEV\_COMPRESS\_NONE, [213](#)
  - DEV\_LINK\_HB\_OFF, [213](#)



- DEV\_LINK\_HB\_ON, [213](#)
- DEV\_STREAM\_ASYNC\_ALL, [213](#)
- DEV\_STREAM\_ASYNC\_DEPTH, [213](#)
- DEV\_STREAM\_ASYNC\_DEPTHANDRGB, [213](#)
- DEV\_STREAM\_ASYNC\_OFF, [213](#)
- DEV\_STREAM\_ASYNC\_RGB, [213](#)
- DEV\_STREAM\_CH\_RECEIVER, [214](#)
- DEV\_STREAM\_CH\_TRANSMITTER, [214](#)
- DEV\_TEMP\_SEL\_CPUCORE, [214](#)
- DEV\_TEMP\_SEL\_IRFLASH, [214](#)
- DEV\_TEMP\_SEL\_LASER, [214](#)
- DEV\_TEMP\_SEL\_LEFT, [214](#)
- DEV\_TEMP\_SEL\_MAINBOARD, [214](#)
- DEV\_TEMP\_SEL\_RIGHT, [214](#)
- DEV\_TEMP\_SEL\_TEXTURE, [214](#)
- DEV\_TEMP\_SEL\_TEXTURE\_FLASH, [214](#)
- DEV\_TIME\_SYNC\_CAN, [214](#)
- DEV\_TIME\_SYNC\_HOST, [214](#)
- DEV\_TIME\_SYNC\_NONE, [214](#)
- DEV\_TIME\_SYNC\_NTP, [214](#)
- DEV\_TIME\_SYNC\_PTP\_MASTER, [214](#)
- DEV\_TIME\_SYNC\_PTP\_SLAVE, [214](#)
- DEV\_TYPE\_PERIPHERAL, [215](#)
- DEV\_TYPE\_RECEIVER, [215](#)
- DEV\_TYPE\_TRANSCEIVER, [215](#)
- DEV\_TYPE\_TRANSMITTER, [215](#)
- GAIN\_SEL\_ANALOG\_ALL, [215](#)
- GAIN\_SEL\_DIGITAL\_ALL, [215](#)
- GAIN\_SEL\_DIGITALBLUE, [215](#)
- GAIN\_SEL\_DIGITALGREEN, [215](#)
- GAIN\_SEL\_DIGITALRED, [215](#)
- GEV\_CCP\_CONTROL, [215](#)
- GEV\_CCP\_EXCLUSIVE, [215](#)
- GEV\_CCP\_OPEN, [215](#)
- JPEG, [216](#)
- LIGHT\_SEL\_FLOOD\_0, [216](#)
- LIGHT\_SEL\_LASER\_0, [216](#)
- LIGHT\_SEL\_TEXTURE\_FLOOD, [216](#)
- SRC\_SEL\_DEPTH, [216](#)
- SRC\_SEL\_LEFT, [216](#)
- SRC\_SEL\_RIGHT, [216](#)
- SRC\_SEL\_TEXTURE, [216](#)
- TofIR\_FourGroup\_Mono16, [216](#)
- TRIG\_ACT\_ANY\_EDGE, [217](#)
- TRIG\_ACT\_FALLING\_EDGE, [217](#)
- TRIG\_ACT\_LEVEL\_HIGH, [217](#)
- TRIG\_ACT\_LEVEL\_LOW, [217](#)
- TRIG\_ACT\_RISING\_EDGE, [217](#)
- TRIG\_MODE\_OFF, [217](#)
- TRIG\_MODE\_ON, [217](#)
- TRIG\_SEL\_FRAME\_ACQUISITIONACTIVE, [217](#)
- TRIG\_SEL\_FRAME\_ACQUISITIONEND, [217](#)
- TRIG\_SEL\_FRAME\_ACQUISITIONSTART, [217](#)
- TRIG\_SEL\_FRAME\_EXPOSUREACTIVE, [218](#)
- TRIG\_SEL\_FRAME\_EXPOSUREEND, [218](#)
- TRIG\_SEL\_FRAME\_EXPOSURESTART, [218](#)
- TRIG\_SEL\_FRAME\_FRAMEACTIVE, [217](#)
- TRIG\_SEL\_FRAME\_FRAMEBURSTACTIVE, [218](#)
- TRIG\_SEL\_FRAME\_FRAMEBURSTEND, [217](#)
- TRIG\_SEL\_FRAME\_FRAMEBURSTSTART, [217](#)
- TRIG\_SEL\_FRAME\_FRAMEEND, [217](#)
- TRIG\_SEL\_FRAME\_FRAMESTART, [217](#)
- TRIG\_SEL\_FRAME\_LINESTART, [218](#)
- TRIG\_SRC\_LINE0, [218](#)
- TRIG\_SRC\_LINE1, [218](#)
- TRIG\_SRC\_LINE2, [218](#)
- TRIG\_SRC\_LINE3, [218](#)
- TRIG\_SRC\_LINE4, [218](#)
- TRIG\_SRC\_LINE5, [218](#)
- TRIG\_SRC\_LINE6, [218](#)
- TRIG\_SRC\_LINE7, [218](#)
- TRIG\_SRC\_SOFTWARE, [218](#)
- TY\_ACQ\_MODE\_LIST, [211](#)
- TY\_BIN\_H\_LIST, [211](#)
- TY\_BIN\_V\_LIST, [212](#)
- TY\_DEPTH\_TOF\_QUALITY\_LIST, [212](#)
- TY\_DEV\_CHARSET\_LIST, [212](#)
- TY\_DEV\_COMPRESS\_LIST, [213](#)
- TY\_DEV\_LINK\_HB\_MODE\_LIST, [213](#)
- TY\_DEV\_STREAM\_ASYNC\_LIST, [213](#)
- TY\_DEV\_STREAM\_CH\_TYPE\_LIST, [213](#)
- TY\_DEV\_TEMP\_SEL\_LIST, [214](#)
- TY\_DEV\_TIME\_SYNC\_LIST, [214](#)
- TY\_DEV\_TYPE\_LIST, [214](#)
- TY\_GAIN\_SEL\_LIST, [215](#)
- TY\_GEV\_CCP\_LIST, [215](#)
- TY\_LIGHT\_SEL\_LIST, [215](#)
- TY\_PIX\_FMT\_LIST, [216](#)
- TY\_SRC\_SEL\_LIST, [216](#)
- TY\_TRIG\_ACT\_LIST, [217](#)
- TY\_TRIG\_MODE\_LIST, [217](#)
- TY\_TRIG\_SEL\_LIST, [217](#)
- TY\_TRIG\_SRC\_LIST, [218](#)
- TY\_USER\_SET\_SEL\_LIST, [218](#)
- USER\_SET\_SEL\_DEFAULT0, [218](#)
- USER\_SET\_SEL\_DEFAULT1, [218](#)
- USER\_SET\_SEL\_DEFAULT2, [218](#)
- USER\_SET\_SEL\_DEFAULT3, [218](#)
- USER\_SET\_SEL\_DEFAULT4, [218](#)
- USER\_SET\_SEL\_DEFAULT5, [218](#)
- USER\_SET\_SEL\_DEFAULT6, [218](#)
- USER\_SET\_SEL\_DEFAULT7, [218](#)
- USER\_SET\_SEL\_USER\_SET0, [218](#)
- USER\_SET\_SEL\_USER\_SET1, [218](#)
- USER\_SET\_SEL\_USER\_SET2, [219](#)
- USER\_SET\_SEL\_USER\_SET3, [219](#)
- USER\_SET\_SEL\_USER\_SET4, [219](#)
- USER\_SET\_SEL\_USER\_SET5, [219](#)
- USER\_SET\_SEL\_USER\_SET6, [219](#)
- USER\_SET\_SEL\_USER\_SET7, [219](#)
- TYFetchFrame
  - TYApi.h, [111](#)
- TYForceDeviceIP
  - TYApi.h, [112](#)
- TYGetBool
  - TYApi.h, [113](#)

- TYGetByteArray
  - TYApi.h, [115](#)
- TYGetByteArrayAttr
  - TYApi.h, [116](#)
- TYGetByteArraySize
  - TYApi.h, [118](#)
- TYGetComponentIDs
  - TYApi.h, [119](#)
- TYGetDeviceFeatureInfo
  - TYApi.h, [120](#)
- TYGetDeviceFeatureNumber
  - TYApi.h, [121](#)
- TYGetDeviceInfo
  - TYApi.h, [122](#)
- TYGetDeviceInterface
  - TYApi.h, [123](#)
- TYGetDeviceList
  - TYApi.h, [124](#)
- TYGetDeviceNumber
  - TYApi.h, [124](#)
- TYGetDeviceXML
  - TYApi.h, [125](#)
- TYGetDeviceXMLSize
  - TYApi.h, [126](#)
- TYGetEnabledComponents
  - TYApi.h, [127](#)
- TYGetEnum
  - TYApi.h, [128](#)
- TYGetEnumEntryCount
  - TYApi.h, [129](#)
- TYGetEnumEntryInfo
  - TYApi.h, [130](#)
- TYGetFeatureInfo
  - TYApi.h, [131](#)
- TYGetFloat
  - TYApi.h, [132](#)
- TYGetFloatRange
  - TYApi.h, [134](#)
- TYGetFrameBufferSize
  - TYApi.h, [135](#)
- TYGetImageAlgorithmVersion
  - TYImageProc.h, [222](#)
- TYGetInt
  - TYApi.h, [136](#)
- TYGetInterfaceList
  - TYApi.h, [137](#)
- TYGetInterfaceNumber
  - TYApi.h, [138](#)
- TYGetIntRange
  - TYApi.h, [139](#)
- TYGetString
  - TYApi.h, [140](#)
- TYGetStringLength
  - TYApi.h, [141](#)
- TYGetStruct
  - TYApi.h, [143](#)
- TYHasDevice
  - TYApi.h, [145](#)
- TYHasFeature
  - TYApi.h, [146](#)
- TYHasInterface
  - TYApi.h, [147](#)
- TYImageProc.h, [219](#)
  - TYDepthEnhanceFilter, [221](#)
  - TYDepthImageInpainter, [221](#)
  - TYDepthSpeckleFilter, [222](#)
  - TYGetImageAlgorithmVersion, [222](#)
  - TYImageProcesAcceEnable, [223](#)
  - TYUndistortImage, [223](#)
  - TYUndistortImage2, [223](#)
- TYImageProcesAcceEnable
  - TYImageProc.h, [223](#)
- TYInvertExtrinsic
  - TYCoordinateMapper.h, [170](#)
- TYLibVersion
  - TYApi.h, [147](#)
- TYMapDepthImageToColorCoordinate
  - TYCoordinateMapper.h, [171](#)
- TYMapDepthImageToPoint3d
  - TYCoordinateMapper.h, [172](#)
- TYMapDepthToColorCoordinate
  - TYCoordinateMapper.h, [172](#)
- TYMapDepthToPoint3d
  - TYCoordinateMapper.h, [173](#)
- TYMapMono16ImageToDepthCoordinate
  - TYCoordinateMapper.h, [173](#)
- TYMapMono8ImageToDepthCoordinate
  - TYCoordinateMapper.h, [174](#)
- TYMapPoint3dToDepth
  - TYCoordinateMapper.h, [175](#)
- TYMapPoint3dToDepthImage
  - TYCoordinateMapper.h, [175](#)
- TYMapPoint3dToPoint3d
  - TYCoordinateMapper.h, [176](#)
- TYMapRGB48ImageToDepthCoordinate
  - TYCoordinateMapper.h, [176](#)
- TYMapRGBImageToDepthCoordinate
  - TYCoordinateMapper.h, [177](#)
- TYMapRGBPixelsToDepthCoordinate
  - TYCoordinateMapper.h, [178](#)
- TYOpenDevice
  - TYApi.h, [148](#)
- TYOpenDeviceWithIP
  - TYApi.h, [149](#)
- TYOpenInterface
  - TYApi.h, [151](#)
- TYRegisterEventCallback
  - TYApi.h, [151](#)
- TYRegisterImuCallback
  - TYApi.h, [152](#)
- TYSendSoftTrigger
  - TYApi.h, [153](#)
- TYSetBool
  - TYApi.h, [154](#)
- TYSetByteArray
  - TYApi.h, [155](#)



TYSetEnum  
    TYApi.h, [157](#)

TYSetFloat  
    TYApi.h, [158](#)

TYSetInt  
    TYApi.h, [160](#)

TYSetLogLevel  
    TYApi.h, [161](#)

TYSetLogPrefix  
    TYApi.h, [161](#)

TYSetString  
    TYApi.h, [162](#)

TYSetStruct  
    TYApi.h, [164](#)

TYStartCapture  
    TYApi.h, [165](#)

TYStopCapture  
    TYApi.h, [166](#)

TYUndistortImage  
    TYImageProc.h, [223](#)

TYUndistortImage2  
    TYImageProc.h, [223](#)

TYUpdateAllDeviceList  
    TYApi.h, [167](#)

TYUpdateDeviceList  
    TYApi.h, [167](#)

TYUpdateInterfaceList  
    TYApi.h, [168](#)

unit\_size  
    TY\_BYTEARRAY\_ATTR, [78](#)

USER\_SET\_SEL\_DEFAULT0  
    TYFeatureList.h, [218](#)

USER\_SET\_SEL\_DEFAULT1  
    TYFeatureList.h, [218](#)

USER\_SET\_SEL\_DEFAULT2  
    TYFeatureList.h, [218](#)

USER\_SET\_SEL\_DEFAULT3  
    TYFeatureList.h, [218](#)

USER\_SET\_SEL\_DEFAULT4  
    TYFeatureList.h, [218](#)

USER\_SET\_SEL\_DEFAULT5  
    TYFeatureList.h, [218](#)

USER\_SET\_SEL\_DEFAULT6  
    TYFeatureList.h, [218](#)

USER\_SET\_SEL\_DEFAULT7  
    TYFeatureList.h, [218](#)

USER\_SET\_SEL\_USER\_SET0  
    TYFeatureList.h, [218](#)

USER\_SET\_SEL\_USER\_SET1  
    TYFeatureList.h, [218](#)

USER\_SET\_SEL\_USER\_SET2  
    TYFeatureList.h, [219](#)

USER\_SET\_SEL\_USER\_SET3  
    TYFeatureList.h, [219](#)

USER\_SET\_SEL\_USER\_SET4  
    TYFeatureList.h, [219](#)

USER\_SET\_SEL\_USER\_SET5  
    TYFeatureList.h, [219](#)

USER\_SET\_SEL\_USER\_SET6  
    TYFeatureList.h, [219](#)

USER\_SET\_SEL\_USER\_SET7  
    TYFeatureList.h, [219](#)

valid\_size  
    TY\_BYTEARRAY\_ATTR, [78](#)

z\_stream\_s, [98](#)